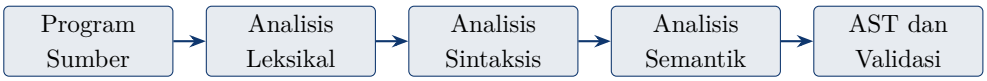
BUKU AJAR

TEKNIK KOMPILASI

Dari Teori Bahasa ke
Implementasi Front-End Kompilator



Buku Ajar Berbasis *Outcome-Based Education*

Cecep Suwanda, S.Si., M.Kom.

Program Studi Teknik Informatika
Fakultas Teknologi Informasi
Universitas Bale Bandung

Mata Kuliah Teknik Kompilasi (MWCA017)
Semester Ganjil — 2026

Teknik Kompilasi: Dari Teori Bahasa ke Implementasi Front-End Kompilator

Edisi Pertama

© 2026 Cecep Suwanda, S.Si., M.Kom.

Program Studi Teknik Informatika
Fakultas Teknologi Informasi
Universitas Bale Bandung

Hak cipta dilindungi undang-undang. Dilarang memperbanyak sebagian atau seluruh isi buku ini dalam bentuk apa pun tanpa izin tertulis dari penulis dan penerbit.

ISBN: dalam proses

Katalog dalam terbitan: dalam proses

Dicetak dan diterbitkan oleh
Program Studi Teknik Informatika, Fakultas Teknologi Informasi, Universitas Bale Bandung

Bandung, 2026

Kata Pengantar

Buku ajar ini disusun untuk mendukung pembelajaran mata kuliah **Teknik Kompilasi** pada Program Studi Teknik Informatika, Fakultas Teknologi Informasi, Universitas Bale Bandung. Mata kuliah ini membahas bagaimana sebuah program sumber berbahasa tingkat tinggi diterjemahkan menjadi representasi yang dapat diproses lebih lanjut oleh mesin. Penekanan buku ini adalah *front-end* kompilator: analisis leksikal, analisis sintaksis, analisis semantik, serta pembentukan dan validasi *Abstract Syntax Tree* (AST).

Penyusunan buku mengikuti Rencana Pembelajaran Semester (RPS) berbasis *Outcome-Based Education* (OBE). Setiap bab dimulai dengan outcome pembelajaran dan Sub-CPMK yang jelas, dilanjutkan dengan materi, contoh, praktikum, latihan, asesmen, daftar cek kompetensi, rangkuman, dan evaluasi. Dengan struktur ini, mahasiswa mengetahui sejak awal kompetensi yang harus dicapai dan bukti belajar yang harus ditunjukkan.

Buku ini terdiri atas tujuh belas bab. Bab I dan Bab II memberikan orientasi serta landasan teori. Bab III sampai Bab XVI merupakan empat belas bab inti yang mengikuti empat belas topik RPS secara berurutan. Bab XVII mengintegrasikan seluruh kompetensi melalui evaluasi, refleksi, dan proyek akhir. Empat lampiran memuat rubrik penilaian, contoh tugas dan portofolio, glosarium, serta sumber belajar tambahan.

Sepanjang semester, mahasiswa mengerjakan satu proyek berkelanjutan: mini-kompilator untuk bahasa **Subset-C**. Proyek ini tumbuh dari bab ke bab, mulai dari spesifikasi bahasa dan *lexer*, dilanjutkan dengan *parser*, AST, tabel simbol, pemeriksaan tipe, hingga validator dan diagnostik. Dengan cara ini, teori yang dipelajari langsung berubah menjadi artefak perangkat lunak yang dapat diuji.

Buku ini memanfaatkan sumber-sumber terbuka yang telah tersedia, antara lain karya Thain, Mogensen, Nystrom, Guntoro, dan catatan kuliah Asep Juarna, serta sejumlah sumber daring yang tercantum pada daftar pustaka. Penulis menyadari bahwa buku ajar ini masih dapat disempurnakan. Kritik dan saran dari dosen, mahasiswa, dan pembaca sangat diharapkan untuk perbaikan edisi berikutnya.

Bandung, 2026
Cecep Suwanda, S.Si., M.Kom.

Ucapan Terima Kasih

Penulis mengucapkan terima kasih kepada seluruh pihak yang telah membantu penyusunan buku ajar ini. Ucapan terima kasih khusus disampaikan kepada para penulis sumber terbuka yang karya-karyanya menjadi rujukan utama: Douglas Thain, Torben Ægidius Mogensen, Robert Nystrom, Guntoro, dan Asep Juarna. Karya mereka membuat materi teknik kompilasi dapat dipelajari secara luas dan terjangkau.

Terima kasih juga disampaikan kepada pimpinan Program Studi Teknik Informatika, Fakultas Teknologi Informasi, Universitas Bale Bandung, atas dukungan dalam penyusunan bahan ajar berbasis OBE. Ucapan terima kasih kepada rekan dosen atas diskusi mengenai capaian pembelajaran, metode asesmen, dan penyelarasan materi dengan RPS.

Penulis berterima kasih kepada mahasiswa yang telah mengikuti mata kuliah ini. Pertanyaan, kesulitan, dan umpan balik mereka menjadi bahan berharga untuk memperbaiki urutan materi, contoh, dan latihan. Semoga buku ini bermanfaat bagi pembelajaran Teknik Kompilasi dan menumbuhkan minat mahasiswa pada perancangan bahasa dan perangkat lunak sistem.

Capaian Pembelajaran dan Peta Kurikulum

Bagian ini merangkum capaian pembelajaran, peta Sub-CPMK, dan keterkaitan setiap bab dengan pertemuan pada RPS. Tabel-tabel di sini menjadi acuan untuk membaca seluruh buku dan memeriksa ketercapaian kompetensi.

Capaian Pembelajaran Lulusan (CPL)

Tabel 1. CPL yang didukung mata kuliah

Kode	Rumusan
CPL 02	Memiliki pengetahuan yang memadai terkait dengan cara kerja sistem komputer dan mampu merancang serta mengembangkan berbagai algoritma untuk memecahkan masalah.
CPL 03	Mampu merancang dan menganalisis algoritma untuk menyelesaikan permasalahan organisasi secara optimal, serta memilih dan menerapkannya pada bahasa pemrograman tertentu.

Capaian Pembelajaran Mata Kuliah (CPMK)

Tabel 2. CPMK dan fokus kompetensi

Kode	Rumusan
CPMK 01	Menjelaskan tahapan kompilasi: analisis leksikal, analisis sintaksis, dan analisis semantik.
CPMK 02	Memahami konsep otomata hingga (*finite automata*).
CPMK 03	Merancang tata bahasa bebas konteks untuk bahasa sederhana.
CPMK 04	Mengimplementasikan parser sederhana untuk validasi kode.

Peta Sub-CPMK

Tabel 3. Peta sembilan belas Sub-CPMK ke bab dan asesmen

Kode	Rumusan	Bab	Asesmen
1.1	Menjelaskan perbedaan interpreter dan compiler serta jenis translator	III	Kuis
1.2	Mengidentifikasi fase kompilator beserta peran tabel simbol dan penanganan kesalahan	III	Kuis
1.3	Menganalisis trade-off one-pass dan multi-pass serta pengaruh run-time environment	III	Kuis
1.4	Menjelaskan hierarki Chomsky dan hubungannya dengan automata	IV	Tugas Teori
1.5	Menjelaskan analisis semantik, attribute grammar, SDD, dan translation scheme	XIV	Praktikum 4
1.6	Mengimplementasikan symbol table bersarang dan type checking	XV	Praktikum 4
1.7	Menangani semantic error dengan pesan informatif	XVI	Praktikum 4, Proyek
2.1	Membuat ekspresi reguler untuk token spesifik bahasa	V	Tugas Teori
2.2	Mengimplementasikan NFA dan DFA untuk token recognition	VI	Praktikum 1
2.3	Menjelaskan ekivalensi NFA-DFA dan minimalisasi DFA	VI	Tugas Teori
2.4	Mengimplementasikan lexical analyzer handwritten dan generator-based	VII	Praktikum 1
3.1	Mendefinisikan CFG dengan BNF/EBNF serta membuat derivasi dan parse tree	VIII	Praktikum 2
3.2	Menganalisis ambiguitas, left recursion, dan left factoring	IX	Tugas Teori, UTS
3.3	Menghitung FIRST dan FOLLOW serta menjelaskan sifat LL(1) dan LR(1)	X	UTS, Praktikum 2
3.4	Merancang CFG untuk bahasa sederhana seperti subset C	VIII	Praktikum 2
4.1	Membangun recursive descent parser dan predictive parser LL(1)	X	Praktikum 2
4.2	Menerapkan bottom-up parsing dan menganalisis konflik	XI–XII	UTS, Praktikum 3
4.3	Menggunakan parser generator dan semantic actions	XIII	Praktikum 3
4.4	Membangun AST dan memvalidasi kode dengan pesan sintaksis yang informatif	XIII, XVI	Praktikum 3, Proyek

Peta Pertemuan dengan Bab

Tabel 4. Peta enam belas pertemuan RPS ke bab

Pert.	Topik	Bab	Sub-CPMK	Asesmen
1	Pengenalan kompilasi dan arsitektur kompilator	III	1.1–1.3	Kuis
2	Teori bahasa formal dan hierarki Chomsky	IV	1.4	Tugas Teori
3	Analisis leksikal: token, lexeme, pattern, regex	V	2.1	Tugas Teori
4	Finite automata: NFA, DFA, konversi, minimalisasi	VI	2.2–2.3	Praktikum 1
5	Implementasi lexical analyzer	VII	2.4	Praktikum 1
6	Context-free grammar: BNF, EBNF, derivasi, parse tree	VIII	3.1, 3.4	Praktikum 2
7	Ambiguitas, left recursion, left factoring	IX	3.2	Tugas Teori
8	Ujian Tengah Semester (UTS)	—	CPMK 01–03	UTS
9	Top-down parsing: recursive descent, LL(1), FIRST/FOLLOW	X	3.3, 4.1	Praktikum 2
10	Bottom-up parsing: shift-reduce, operator precedence	XI	4.2	Latihan
11	LR parsing: canonical LR, SLR, LALR, konflik	XII	4.2	UTS
12	Parser generator dan semantic actions	XIII	4.3–4.4	Praktikum 3
13	Analisis semantik: attribute grammar, SDD, translation scheme	XIV	1.5	Latihan
14	Symbol table, scope management, type checking	XV	1.6	Praktikum 4
15	AST, validasi kode, penanganan error	XVI	4.4, 1.7	Praktikum 4
16	Ujian Akhir Semester (UAS)	—	CPMK 01–04	UAS

Bobot Penilaian

Tabel 5. Komponen penilaian dan bobot

Komponen	Cakupan	Bobot
Kuis	Sub-CPMK 1.1–1.3	5%
Tugas Teori	Grammar, automata, parsing	8%
Praktikum 1	Lexer	8%
Praktikum 2	Parser	8%
Praktikum 3	Parser generator dan AST	7%
Praktikum 4	Symbol table dan analisis semantik	8%
UTS	CPMK 01–03	20%
Project Final	Mini compiler front-end	20%
UAS	CPMK 01–04	16%
Total		**100%**

Informasi

RPS mencantumkan Sub-CPMK 3.4 pada asesmen Praktikum 2, tetapi tidak menyebutnya pada tabel pertemuan. Buku ini menempatkan 3.4 pada Bab VIII dan Praktikum 2 agar seluruh Sub-CPMK memiliki bab utama dan asesmen yang jelas.

Cara Menggunakan Buku Ini

Anatomi Bab

Setiap bab inti memiliki urutan yang sama:

1. **Outcome Pembelajaran dan Sub-CPMK**: kompetensi yang harus dicapai dan kode Sub-CPMK yang terkait.
2. **Materi Pokok**: peta isi bab.
3. **Lima bagian materi**: uraian konsep, definisi, contoh, dan ilustrasi.
4. **Contoh**: soal berpenyelesaian lengkap.
5. **Aktivitas Pembelajaran dan Praktikum**: latihan terbimbing dan increment proyek.
6. **Latihan dan Refleksi**: soal latihan bertingkat dan pertanyaan refleksi.
7. **Asesmen**: instrumen dan rubrik yang mengukur Sub-CPMK.
8. **Checklist Pencapaian Kompetensi**: bukti yang harus ditunjukkan mahasiswa.
9. **Rangkuman**: ringkasan dan jembatan ke bab berikutnya.
10. **Evaluasi**: soal evaluasi akhir bab dan rubrik singkat.

Arti Kotak

Informasi

Kotak **Informasi** memuat konsep pendukung, konteks sejarah, atau keterkaitan dengan bidang lain.

Peringatan

Kotak **Peringatan** memuat kesalahan umum, jebakan konseptual, atau batasan yang perlu diperhatikan.

Tips

Kotak **Tips** memuat strategi praktis untuk belajar, menulis kode, atau mengerjakan ujian.

Refleksi

Kotak **Refleksi** mengajak mahasiswa menilai pemahaman dan kebiasaan belajarnya sendiri.

Proyek Mini-Kompilator

Proyek berkelanjutan pada buku ini adalah mini-kompilator untuk **Subset-C**. Spesifikasi bahasa, kode sumber, contoh program, dan tes tersedia di direktori `code/subsetc`. Proyek dibagi menjadi enam milestone (M0–M5) yang dikerjakan pada bab-bab tertentu. Setiap bab praktikum membuka dengan tabel artefak: berkas, status awal, pekerjaan yang harus dilakukan, dan definisi selesai.

Mahasiswa disarankan menyimpan proyek dalam repositori versi dan melakukan commit pada setiap milestone. Kode yang ditampilkan di buku adalah kutipan dari berkas nyata; nomor baris pada buku sesuai dengan nomor baris berkas.

Dua Jalur Belajar

Jalur teori menekankan definisi, pembuktian, dan analisis. Mahasiswa membaca bab secara berurutan, mengerjakan latihan teori, dan menggunakan praktikum sebagai ilustrasi. Jalur ini cocok bagi mahasiswa yang ingin memperdalam dasar formal.

Jalur praktik menekankan implementasi. Mahasiswa membaca bab secukupnya, langsung mengerjakan praktikum, lalu kembali ke teori ketika menemui kesulitan. Jalur ini cocok bagi mahasiswa yang belajar paling baik melalui kode dan eksperimen.

Kedua jalur bertemu pada proyek akhir dan UAS. Tidak ada jalur yang lebih mudah: keduanya menuntut pemahaman konsep dan kemampuan implementasi.

Konvensi Notasi

- Simbol nonterminal ditulis miring, misalnya E, T, dan F.
- Simbol terminal ditulis dengan huruf tebal atau dalam tanda kutip, misalnya `id`, `+`, dan `if`.
- String kosong dilambangkan dengan ε.
- Turunan ditulis dengan $\Rightarrow$, dan turunan nol atau lebih dengan $\Rightarrow^*$.
- Nama berkas dan perintah ditulis dengan huruf mesin, misalnya `lexer.c` dan `pdflatex`.
- Istilah Inggris dimiringkan pada kemunculan pertama, misalnya *parser*, lalu bentuk Indonesia dipakai konsisten.

Mengompilasi Bab Secara Mandiri

Setiap bab dapat dikompilasi secara mandiri. Dari direktori bab, jalankan:

```
pdflatex chapter.tex
pdflatex chapter.tex
```

Rujukan ke bab lain yang belum ditulis akan tampil sebagai `??`. Ini normal dan tidak mengganggu pemeriksaan tata letak. Untuk mengompilasi seluruh buku, gunakan `scripts/build-all.bat`.

Daftar Isi

Kata Pengantar . . . . . **ii**

Ucapan Terima Kasih . . . . . **iii**

Capaian Pembelajaran dan Peta Kurikulum . . . . . **iv**
Capaian Pembelajaran Lulusan (CPL) iv
Capaian Pembelajaran Mata Kuliah (CPMK) iv
Peta Sub-CPMK iv
Peta Pertemuan dengan Bab vi
Bobot Penilaian vii

Cara Menggunakan Buku Ini . . . . . **viii**
Anatomi Bab viii
Arti Kotak viii
Proyek Mini-Kompilator ix
Dua Jalur Belajar ix
Konvensi Notasi ix
Mengompilasi Bab Secara Mandiri x

I Orientasi Buku Ajar **1**
1.1 Mengapa Teknik Kompilasi Penting 1
1.2 Mata Kuliah dalam Kurikulum dan RPS Berbasis OBE 2
1.3 Arsitektur Buku dan Alur Belajar Tujuh Belas Bab 3
1.4 Anatomi Bab dan Fitur Pembelajaran 3
1.5 Proyek Mini-Kompilator dan Strategi Belajar 4
Contoh 5

Aktivitas Pembelajaran dan Praktikum . . . 5
Latihan dan Refleksi . . . 6
Asesmen . . . 6
Checklist Pencapaian Kompetensi . . . 7
Rangkuman . . . 7
Evaluasi . . . 8

II Landasan Teori dan Notasi Formal . . . 9
2.1 Himpunan, Relasi, Fungsi, dan Induksi Matematika . . . 9
2.1.1 Himpunan dan Operasi Himpunan . . . 10
2.1.2 Relasi dan Fungsi . . . 10
2.1.3 Induksi Matematika . . . 10
2.2 Alfabet, String, Bahasa, dan Operasi Bahasa . . . 11
2.2.1 Alfabet dan String . . . 11
2.2.2 Operasi atas Bahasa . . . 11
2.2.3 Bahasa dan Masalah Keputusan . . . 12
2.3 Tata Bahasa sebagai Sistem Penulisan Ulang . . . 12
2.3.1 Definisi Tata Bahasa . . . 12
2.3.2 Derivasi dan Bentuk Kalimat . . . 13
2.3.3 Pohon Urai . . . 13
2.3.4 Klasifikasi Tata Bahasa . . . 14
2.4 Mesin Abstrak dan Daya Komputasi . . . 14
2.4.1 Otomata sebagai Pengenal Bahasa . . . 14
2.4.2 Hierarki Chomsky . . . 15
2.4.3 Implikasi bagi Kompilator . . . 15
2.5 Peta Konsep: dari Teori Bahasa ke Front-End Kompilator . . . 16
Contoh . . . 17
Aktivitas Pembelajaran dan Praktikum . . . 19
Latihan dan Refleksi . . . 19
Asesmen . . . 20
Checklist Pencapaian Kompetensi . . . 20
Rangkuman . . . 21
Evaluasi . . . 21

III Pengenalan Kompilasi dan Arsitektur Kompilator . . . 23
3.1 Translator, Assembler, Interpreter, dan Compiler . . . 23
3.1.1 Keluarga Translator . . . 23
3.1.2 Compiler versus Interpreter . . . 24
3.1.3 Cross-Compiler dan Bootstrapping . . . 24
3.2 Fase-Fase Kompilasi: Analisis dan Sintesis . . . 25
3.2.1 Analisis Leksikal . . . 25
3.2.2 Analisis Sintaksis . . . 25

3.2.3 Analisis Semantik . . . 26
3.2.4 Sintesis . . . 26
3.3 Manajemen Tabel Simbol dan Penanganan Kesalahan . . . 27
3.3.1 Tabel Simbol . . . 27
3.3.2 Jenis Kesalahan . . . 27
3.3.3 Contoh Alur Tabel Simbol . . . 28
3.4 One-Pass versus Multi-Pass dan Organisasi Front-End/Back-End . . . 28
3.4.1 Kompilator Satu Lintasan . . . 28
3.4.2 Kompilator Banyak Lintasan . . . 29
3.4.3 Organisasi Front-End dan Back-End . . . 29
3.5 Lingkungan Run-Time: Static, Stack, dan Heap Allocation . . . 30
3.5.1 Tata Letak Memori Program . . . 30
3.5.2 Alokasi Statis . . . 30
3.5.3 Alokasi Stack . . . 30
3.5.4 Alokasi Heap . . . 31
Contoh . . . 31
Aktivitas Pembelajaran dan Praktikum . . . 32
Latihan dan Refleksi . . . 33
Asesmen . . . 34
Checklist Pencapaian Kompetensi . . . 34
Rangkuman . . . 35
Evaluasi . . . 35

IV Teori Bahasa Formal dan Hierarki Chomsky . . . 37
4.1 Alfabet, String, dan Bahasa Secara Formal . . . 37
4.1.1 Definisi Dasar . . . 37
4.1.2 Bahasa sebagai Masalah Keputusan . . . 38
4.1.3 Keterhitungan . . . 38
4.1.4 Relasi dan Homomorfisme . . . 38
4.2 Tata Bahasa dan Derivasi: Definisi Formal . . . 39
4.2.1 Derivasi Leftmost dan Rightmost . . . 39
4.2.2 Bentuk Normal . . . 40
4.2.3 Ekivalensi dan Penyederhanaan . . . 40
4.3 Hierarki Chomsky Tipe 3 sampai Tipe 0 . . . 40
4.3.1 Tipe 3: Tata Bahasa Regular . . . 41
4.3.2 Tipe 2: Tata Bahasa Bebas Konteks . . . 41
4.3.3 Tipe 1: Tata Bahasa Sensitif Konteks . . . 41
4.3.4 Tipe 0: Tata Bahasa Tanpa Batas . . . 42
4.4 Otomata dan Daya Ekspresif . . . 42
4.4.1 Otomata Hingga . . . 42
4.4.2 Otomata Pushdown . . . 42
4.4.3 Mesin Turing . . . 43

4.4.4 Ekivalensi dan Hubungan . . . 43
4.5 Implikasi bagi Desain Bahasa dan Kompilator . . . 44
4.5.1 Mengapa Leksikal Cukup Tipe 3 . . . 44
4.5.2 Mengapa Sintaksis Memerlukan Tipe 2 . . . 44
4.5.3 Mengapa Semantik Melampaui Tipe 2 . . . 44
4.5.4 Konsekuensi Desain . . . 45
Contoh . . . 45
Aktivitas Pembelajaran dan Praktikum . . . 46
Latihan dan Refleksi . . . 47
Asesmen . . . 47
Checklist Pencapaian Kompetensi . . . 48
Rangkuman . . . 48
Evaluasi . . . 49

V Analisis Leksikal . . . 50
5.1 Peran dan Antarmuka Lexer . . . 50
5.1.1 Posisi Lexer dalam Pipeline . . . 50
5.1.2 Antarmuka dengan Parser . . . 51
5.1.3 Kontrak Lexer . . . 51
5.2 Token, Lexeme, Pattern, dan Atribut . . . 52
5.2.1 Tiga Konsep yang Sering Tertukar . . . 52
5.2.2 Atribut Token . . . 52
5.2.3 Kata Kunci dan Identifier . . . 53
5.3 Ekspresi Reguler: Definisi dan Operasi . . . 53
5.3.1 Definisi . . . 53
5.3.2 Prioritas Operator . . . 54
5.3.3 Identitas Aljabar . . . 54
5.3.4 Ekspresi Reguler untuk Token . . . 54
5.4 Notasi, Prioritas, dan Spesifikasi Token . . . 55
5.4.1 Maximal Munch . . . 55
5.4.2 Notasi di Flex . . . 55
5.4.3 Aturan Spesifikasi . . . 56
5.4.4 Kesalahan Leksikal . . . 56
5.5 Spesifikasi Token Subset-C . . . 57
5.5.1 Kategori Token . . . 57
5.5.2 Header Token . . . 57
5.5.3 Program Uji . . . 58
5.5.4 Latihan Spesifikasi . . . 58
Contoh . . . 59
Aktivitas Pembelajaran dan Praktikum . . . 60
Latihan dan Refleksi . . . 61
Asesmen . . . 61

Checklist Pencapaian Kompetensi . . . 62
Rangkuman . . . 62
Evaluasi . . . 63

VI Finite Automata . . . 64
6.1 DFA: Definisi, Tabel Transisi, dan Simulasi . . . 64
6.1.1 Definisi . . . 64
6.1.2 Contoh: Bilangan a Genap . . . 65
6.1.3 Fungsi Transisi Diperluas . . . 65
6.1.4 Simulasi . . . 65
6.2 NFA dan Epsilon Closure . . . 66
6.2.1 Definisi . . . 66
6.2.2 Epsilon Closure . . . 66
6.2.3 Simulasi NFA . . . 66
6.3 Konversi NFA ke DFA: Subset Construction . . . 67
6.3.1 Ide Dasar . . . 67
6.3.2 Algoritma . . . 67
6.3.3 Contoh . . . 68
6.3.4 Implementasi . . . 68
6.4 Minimalisasi DFA . . . 69
6.4.1 State yang Ekuivalen . . . 69
6.4.2 Penyempurnaan Partisi . . . 69
6.4.3 Kompleksitas . . . 70
6.4.4 Implementasi . . . 70
6.5 Dari Ekspresi Reguler ke Automata dan Token Recognition . . . 71
6.5.1 Konstruksi Thompson . . . 71
6.5.2 Epsilon Closure dalam Kode . . . 71
6.5.3 Menggabungkan Beberapa Token . . . 72
6.5.4 Maximal Munch pada DFA . . . 72
6.5.5 Dari Otomata ke Lexer . . . 72
Contoh . . . 73
Aktivitas Pembelajaran dan Praktikum . . . 74
Latihan dan Refleksi . . . 75
Asesmen . . . 75
Checklist Pencapaian Kompetensi . . . 76
Rangkuman . . . 76
Evaluasi . . . 76

VII Implementasi Lexical Analyzer . . . 78
7.1 Arsitektur Scanner: Buffer, Sentinel, dan Lookahead . . . 78
7.1.1 Mengapa Buffer Diperlukan . . . 78
7.1.2 Skema Dua Buffer . . . 79

7.1.3 Lexeme Begin dan Forward . . . 79
7.1.4 Lookahead . . . 79
7.1.5 Kompleksitas . . . 79
7.2 Scanner Hand-Written dalam C . . . 80
7.2.1 Struktur Data . . . 80
7.2.2 Operasi Dasar . . . 80
7.2.3 Melewati Spasi dan Komentar . . . 81
7.2.4 Fungsi Utama `lexer_next` . . . 82
7.2.5 Operator dan Tanda Baca . . . 83
7.2.6 Kata Kunci . . . 85
7.3 Scanner Generator-Based dengan Flex . . . 85
7.3.1 Struktur Berkas Flex . . . 85
7.3.2 Definisi dan Makro . . . 85
7.3.3 Aturan Literal dan Identifier . . . 86
7.3.4 Membangun Lexer Flex . . . 87
7.3.5 Perbandingan dengan Lexer Tangan . . . 88
7.4 Penanganan Kesalahan Leksikal . . . 88
7.4.1 Jenis Kesalahan . . . 88
7.4.2 Strategi Pemulihan . . . 88
7.4.3 Pelaporan Kesalahan . . . 89
7.4.4 Contoh Diagnostik . . . 90
7.5 Integrasi ke Proyek: Token Stream dan Driver . . . 90
7.5.1 Driver . . . 90
7.5.2 Format Keluaran . . . 92
7.5.3 Pengujian . . . 92
7.5.4 M1: Definisi Selesai . . . 92
7.5.5 Catatan Verifikasi . . . 93
Contoh . . . 93
Aktivitas Pembelajaran dan Praktikum . . . 94
Latihan dan Refleksi . . . 95
Asesmen . . . 96
Checklist Pencapaian Kompetensi . . . 96
Rangkuman . . . 97
Evaluasi . . . 97

VIII Context-Free Grammar . . . 98
8.1 Definisi CFG dan Notasi BNF . . . 98
8.1.1 Definisi . . . 98
8.1.2 Notasi BNF . . . 99
8.1.3 Bahasa yang Dibangkitkan . . . 99
8.1.4 Mengapa CFG Cocok untuk Sintaksis . . . 99
8.2 EBNF dan Konversi ke BNF . . . 100

8.2.1 Notasi EBNF . . . 100
8.2.2 Aturan Konversi . . . 100
8.2.3 Tata Bahasa Subset-C dalam EBNF . . . 101
8.2.4 Hasil Konversi ke BNF . . . 102
8.3 Derivasi Leftmost dan Rightmost . . . 102
8.3.1 Derivasi . . . 102
8.3.2 Bentuk Kalimat . . . 103
8.3.3 Pohon Derivasi . . . 103
8.3.4 Derivasi dan Ambiguitas . . . 104
8.4 Parse Tree dan Hubungannya dengan AST . . . 104
8.4.1 Parse Tree . . . 104
8.4.2 Abstract Syntax Tree . . . 104
8.4.3 Perbedaan Penting . . . 105
8.4.4 Contoh AST . . . 105
8.4.5 Dari Pohon Urai ke AST . . . 105
8.5 Merancang Grammar Subset-C . . . 106
8.5.1 Prinsip Perancangan . . . 106
8.5.2 Struktur Nonterminal . . . 106
8.5.3 Deklarasi dan Fungsi . . . 106
8.5.4 Pernyataan . . . 106
8.5.5 Ekspresi Berprioritas . . . 107
8.5.6 Contoh Program . . . 107
Contoh . . . 108
Aktivitas Pembelajaran dan Praktikum . . . 109
Latihan dan Refleksi . . . 110
Asesmen . . . 110
Checklist Pencapaian Kompetensi . . . 111
Rangkuman . . . 111
Evaluasi . . . 111

IX Ambiguitas dan Transformasi Grammar . . . 113
9.1 Ambiguitas Grammar dan Dangling Else . . . 113
9.1.1 Definisi Ambiguitas . . . 113
9.1.2 Solusi Ambiguitas . . . 114
9.1.3 Dangling Else . . . 114
9.2 Precedence dan Associativity . . . 115
9.2.1 Precedence . . . 115
9.2.2 Associativity . . . 115
9.2.3 Menyatakan Precedence dalam Grammar . . . 116
9.2.4 Precedence pada Parser Generator . . . 116
9.3 Penghapusan Left Recursion . . . 117
9.3.1 Mengapa Left Recursion Bermasalah . . . 117

9.3.2 Left Recursion Langsung . . . 117
9.3.3 Left Recursion Tidak Langsung . . . 118
9.3.4 Kondisi Nullable . . . 118
9.4 Left Factoring . . . 118
9.4.1 Masalah Faktor Umum . . . 118
9.4.2 Transformasi . . . 118
9.4.3 Kapan Left Factoring Diperlukan . . . 119
9.5 Transformasi Grammar Subset-C . . . 120
9.5.1 Dari Grammar Awal ke Grammar LL(1) . . . 120
9.5.2 Grammar Hasil . . . 120
9.5.3 Analisis Dampak . . . 121
9.5.4 Pemeriksaan Kesetaraan Bahasa . . . 121
Contoh . . . 122
Aktivitas Pembelajaran dan Praktikum . . . 123
Latihan dan Refleksi . . . 124
Asesmen . . . 124
Checklist Pencapaian Kompetensi . . . 125
Rangkuman . . . 125
Evaluasi . . . 125

X Top-Down Parsing . . . 127
10.1 Recursive Descent: Satu Fungsi per Nonterminal . . . 127
10.1.1 Prinsip . . . 127
10.1.2 Struktur Parser . . . 128
10.1.3 Satu Fungsi per Nonterminal . . . 129
10.1.4 Keuntungan dan Keterbatasan . . . 130
10.2 Himpunan FIRST dan FOLLOW . . . 130
10.2.1 Definisi FIRST . . . 130
10.2.2 Definisi FOLLOW . . . 131
10.2.3 Contoh . . . 131
10.2.4 Alat Bantu . . . 131
10.2.5 Kondisi LL(1) . . . 132
10.3 Tabel Parsing LL(1) dan Predictive Parser . . . 133
10.3.1 Konstruksi Tabel . . . 133
10.3.2 Algoritma Predictive Parser . . . 133
10.3.3 Contoh Penelusuran . . . 134
10.4 Konstruksi Recursive Descent yang Sinkron dengan Tabel . . . 134
10.4.1 Dari Tabel ke Fungsi . . . 135
10.4.2 Pemilihan Berdasarkan Token . . . 135
10.4.3 Sinkronisasi . . . 136
10.4.4 Verifikasi dengan Tabel . . . 137
10.5 Penanganan Kesalahan Sintaksis . . . 137

10.5.1 Jenis Kesalahan . . . 137
10.5.2 Strategi Pemulihan . . . 138
10.5.3 Pesan Kesalahan yang Baik . . . 138
10.5.4 Batas Pemulihan . . . 139
Contoh . . . 139
Aktivitas Pembelajaran dan Praktikum . . . 140
Latihan dan Refleksi . . . 141
Asesmen . . . 142
Checklist Pencapaian Kompetensi . . . 142
Rangkuman . . . 143
Evaluasi . . . 143

XI Bottom-Up Parsing . . . 144
11.1 Model Shift-Reduce, Stack, dan Handle . . . 144
11.1.1 Ide Dasar . . . 144
11.1.2 Contoh Tata Bahasa . . . 145
11.1.3 Handle . . . 145
11.1.4 Penelusuran Shift-Reduce . . . 145
11.1.5 Keuntungan . . . 146
11.2 Viable Prefix dan Konflik . . . 146
11.2.1 Viable Prefix . . . 146
11.2.2 Konflik Shift-Reduce . . . 147
11.2.3 Konflik Reduce-Reduce . . . 147
11.2.4 Menyelesaikan Konflik . . . 147
11.3 Operator Precedence Grammar . . . 148
11.3.1 Definisi . . . 148
11.3.2 Relasi Precedence . . . 148
11.3.3 Tabel Operator Precedence . . . 148
11.3.4 Contoh Relasi . . . 149
11.3.5 Keterbatasan . . . 149
11.4 Tabel dan Algoritma Operator Precedence . . . 149
11.4.1 Algoritma . . . 149
11.4.2 Contoh Penelusuran . . . 150
11.4.3 Implementasi Sederhana . . . 150
11.4.4 Deteksi Kesalahan . . . 151
11.5 Perbandingan Top-Down dan Bottom-Up . . . 151
11.5.1 Perbandingan . . . 151
11.5.2 Kapan Memilih Top-Down . . . 151
11.5.3 Kapan Memilih Bottom-Up . . . 152
11.5.4 Jembatan ke LR . . . 152
Contoh . . . 152
Aktivitas Pembelajaran dan Praktikum . . . 153

Latihan dan Refleksi . . . 154
Asesmen . . . 154
Checklist Pencapaian Kompetensi . . . 155
Rangkuman . . . 155
Evaluasi . . . 155

XII LR Parsing . . . 157
12.1 Item LR(0), Closure, dan Goto . . . 157
12.1.1 Item LR(0) . . . 157
12.1.2 Closure . . . 158
12.1.3 Goto . . . 158
12.1.4 Koleksi Kanonik . . . 158
12.2 Automaton LR(0) dan SLR(1) . . . 159
12.2.1 Automaton LR(0) . . . 159
12.2.2 Tabel SLR(1) . . . 159
12.2.3 Alat Bantu . . . 160
12.3 Canonical LR(1) dan LALR(1) . . . 161
12.3.1 Item LR(1) . . . 162
12.3.2 Closure LR(1) . . . 162
12.3.3 Canonical LR(1) . . . 162
12.3.4 LALR(1) . . . 162
12.3.5 Perbandingan . . . 163
12.4 Konflik Shift-Reduce dan Reduce-Reduce . . . 163
12.4.1 Konflik Shift-Reduce . . . 163
12.4.2 Konflik Reduce-Reduce . . . 163
12.4.3 Resolusi . . . 163
12.4.4 Membaca Laporan Konflik . . . 164
12.5 LL(k) versus LR(k) dan Pemilihan Strategi . . . 164
12.5.1 Perbandingan . . . 165
12.5.2 Pemilihan . . . 165
12.5.3 Di Balik Layar Bison . . . 165
Contoh . . . 166
Aktivitas Pembelajaran dan Praktikum . . . 166
Latihan dan Refleksi . . . 167
Asesmen . . . 168
Checklist Pencapaian Kompetensi . . . 168
Rangkuman . . . 169
Evaluasi . . . 169

XIII Parser Generator dan Semantic Actions . . . 170
13.1 Arsitektur Parser Generator . . . 170
13.1.1 Ide Dasar . . . 170

13.1.2 Keuntungan . . . 171
13.1.3 Alat yang Umum . . . 171
13.2 Bison: Deklarasi, Aturan, dan Konflik . . . 171
13.2.1 Struktur Berkas Bison . . . 171
13.2.2 Deklarasi . . . 172
13.2.3 Aturan dan Aksi . . . 173
13.2.4 Konflik . . . 174
13.3 Semantic Actions dan Konstruksi AST . . . 174
13.3.1 Nilai Semantik . . . 174
13.3.2 Membangun AST . . . 175
13.3.3 Contoh AST . . . 176
13.3.4 Location Tracking . . . 176
13.4 Integrasi Flex–Bison dan Build . . . 177
13.4.1 Antarmuka `yylex` . . . 177
13.4.2 Langkah Build . . . 177
13.4.3 Driver . . . 177
13.4.4 Catatan Lingkungan . . . 178
13.5 ANTLR dan Perbandingan Pendekatan . . . 178
13.5.1 ANTLR . . . 178
13.5.2 Perbandingan . . . 179
13.5.3 Kapan Memilih . . . 179
Contoh . . . 179
Aktivitas Pembelajaran dan Praktikum . . . 180
Latihan dan Refleksi . . . 181
Asesmen . . . 182
Checklist Pencapaian Kompetensi . . . 182
Rangkuman . . . 183
Evaluasi . . . 183

XIV Analisis Semantik . . . 185
14.1 Tugas dan Ruang Lingkup Analisis Semantik . . . 185
14.1.1 Di Luar Sintaksis . . . 185
14.1.2 Informasi yang Dibutuhkan . . . 186
14.1.3 Urutan Analisis . . . 186
14.2 Attribute Grammar: Synthesized dan Inherited . . . 187
14.2.1 Atribut . . . 187
14.2.2 Atribut Synthesized . . . 187
14.2.3 Atribut Inherited . . . 187
14.2.4 Dependency Graph . . . 188
14.3 Syntax-Directed Definition . . . 188
14.3.1 Definisi . . . 188
14.3.2 Contoh SDD untuk Ekspresi . . . 188

14.3.3 S-Attributed dan L-Attributed . . . 188
14.3.4 Evaluasi . . . 189
14.4 Translation Schemes . . . 189
14.4.1 Definisi . . . 189
14.4.2 Contoh . . . 189
14.4.3 Dari SDD ke Translation Scheme . . . 190
14.4.4 Contoh pada Bison . . . 190
14.4.5 Kapan Memilih . . . 190
14.5 Studi Kasus: Inferensi Tipe Subset-C . . . 190
14.5.1 Modul Tipe . . . 191
14.5.2 Aturan Tipe . . . 192
14.5.3 Contoh Inferensi . . . 192
14.5.4 Propagasi Error . . . 192
Contoh . . . 192
Aktivitas Pembelajaran dan Praktikum . . . 193
Latihan dan Refleksi . . . 194
Asesmen . . . 195
Checklist Pencapaian Kompetensi . . . 195
Rangkuman . . . 195
Evaluasi . . . 196

XV Symbol Table dan Type Checking . . . 197
15.1 Struktur dan Operasi Symbol Table . . . 197
15.1.1 Atribut Simbol . . . 197
15.1.2 Operasi Dasar . . . 198
15.1.3 Representasi . . . 198
15.1.4 Resolusi Nama . . . 199
15.2 Implementasi Hash Table di C . . . 199
15.2.1 Mengapa Hash Table . . . 199
15.2.2 Fungsi Hash . . . 199
15.2.3 Chaining . . . 199
15.2.4 Implementasi Proyek . . . 200
15.3 Nested Scopes dan Shadowing . . . 201
15.3.1 Scope Bersarang . . . 201
15.3.2 Shadowing . . . 202
15.3.3 Lookup . . . 202
15.3.4 Deklarasi Ganda . . . 203
15.4 Sistem Tipe dan Pemeriksaan Tipe . . . 203
15.4.1 Sistem Tipe . . . 204
15.4.2 Aturan Kompatibilitas . . . 204
15.4.3 Traversal AST . . . 204
15.4.4 Blok dan Deklarasi . . . 207

15.4.5 Fungsi dan Parameter . . . 208
15.5 Konversi Tipe dan Pelaporan Kesalahan . . . 208
15.5.1 Konversi Implisit . . . 208
15.5.2 Konversi Eksplisit . . . 208
15.5.3 Pesan Kesalahan . . . 209
15.5.4 Propagasi Error . . . 209
Contoh . . . 210
Aktivitas Pembelajaran dan Praktikum . . . 211
Latihan dan Refleksi . . . 212
Asesmen . . . 213
Checklist Pencapaian Kompetensi . . . 213
Rangkuman . . . 213
Evaluasi . . . 214

XVI AST, Validasi Kode, dan Penanganan Error . . . 215
16.1 Desain dan Konstruksi AST . . . 215
16.1.1 Prinsip Desain . . . 215
16.1.2 Struktur Generik . . . 216
16.1.3 Generik versus Spesifik . . . 218
16.1.4 Membangun AST . . . 218
16.2 Traversal AST dan Visitor di C . . . 218
16.2.1 Traversal Rekursif . . . 218
16.2.2 Pola Visitor . . . 219
16.2.3 Contoh Traversal . . . 219
16.2.4 Konteks Traversal . . . 219
16.3 Validasi Kode: Aturan dan Multi-Pass . . . 220
16.3.1 Aturan Validasi . . . 220
16.3.2 Multi-Pass . . . 220
16.3.3 Implementasi Validator . . . 220
16.3.4 Validasi Fungsi . . . 222
16.4 Klasifikasi, Pemulihan, dan Pelaporan Kesalahan . . . 223
16.4.1 Klasifikasi Kesalahan . . . 223
16.4.2 Pemulihan . . . 223
16.4.3 Format Diagnostik . . . 224
16.4.4 Batas Pelaporan . . . 224
16.5 Diagnostik Informatif dan Pengujian . . . 224
16.5.1 Kualitas Pesan . . . 224
16.5.2 Kode Diagnostik . . . 225
16.5.3 Pengujian . . . 225
16.5.4 Driver Final . . . 225
16.5.5 M5: Proyek Akhir . . . 226
Contoh . . . 227

Aktivitas Pembelajaran dan Praktikum . . . 228
Latihan dan Refleksi . . . 229
Asesmen . . . 230
Checklist Pencapaian Kompetensi . . . 230
Rangkuman . . . 230
Evaluasi . . . 231

XVII Evaluasi dan Integrasi Kompetensi . . . 232
17.1 Peta Pencapaian 19 Sub-CPMK . . . 232
17.1.1 Rekapitulasi . . . 232
17.1.2 Analisis Ketercapaian . . . 234
17.2 Blueprint Kuis, UTS, UAS, dan Proyek Akhir . . . 234
17.2.1 Komponen dan Bobot . . . 234
17.2.2 UTS . . . 234
17.2.3 UAS . . . 235
17.2.4 Proyek Akhir . . . 235
17.2.5 Contoh Soal UTS . . . 235
17.2.6 Contoh Soal UAS . . . 236
17.3 Rubrik Komprehensif dan Penilaian . . . 236
17.3.1 Dimensi Penilaian . . . 236
17.3.2 Level Kinerja . . . 236
17.3.3 Konversi Nilai . . . 237
17.3.4 Menghindari Perhitungan Ganda . . . 237
17.4 Refleksi, Jurnal, dan Portofolio . . . 237
17.4.1 Siklus Refleksi . . . 237
17.4.2 Jurnal Mingguan . . . 238
17.4.3 Portofolio . . . 238
17.4.4 Evaluasi Diri . . . 238
17.5 Tindak Lanjut dan Topik Lanjutan . . . 239
17.5.1 Setelah Front-End . . . 239
17.5.2 Jalur Belajar . . . 239
17.5.3 Karier . . . 240
17.5.4 Saran Akhir . . . 240
Contoh . . . 240
Aktivitas Pembelajaran dan Praktikum . . . 242
Latihan dan Refleksi . . . 242
Asesmen . . . 243
Checklist Pencapaian Kompetensi . . . 244
Rangkuman . . . 244
Evaluasi . . . 244

A Rubrik Penilaian Lengkap . . . 246

A.1 Bobot Komponen Penilaian 246
A.2 Rubrik Praktikum 246
A.3 Konversi Nilai 247

B Contoh Tugas, Portofolio, dan Rubrik Refleksi 248
B.1 Lembar Tugas Praktikum 248
B.2 Template Laporan Proyek 248
B.3 Template Jurnal Refleksi 249
B.4 Rubrik Refleksi 249
B.5 Contoh Entri Portofolio 249

C Glosarium 250

D Referensi Tambahan dan Sumber Belajar 253
D.1 Referensi RPS 253
D.2 Karya Lokal 254
D.3 Alat dan Sumber Daring 254
D.4 Jalur Bacaan 254
D.5 Peta Sitasi 255

Tentang Penulis 258

Daftar Gambar

1.1 Alur besar buku: dari orientasi menuju integrasi kompetensi 3

2.1 Konkatenasi dua bahasa menghasilkan bahasa baru 12
2.2 Pohon urai untuk `id` + `id` ∗ `id` 14
2.3 Tangga hierarki Chomsky: semakin ke bawah, semakin besar daya ekspresif 15
2.4 Peta konsep dari teori bahasa menuju front-end kompilator 16

3.1 Keluarga translator dan tiga anggotanya yang utama 24
3.2 Pipeline kompilasi dari program sumber ke program target 25
3.3 Perbandingan alur satu lintasan dan banyak lintasan 29
3.4 Tata letak memori program pada lingkungan run-time 30
3.5 Struktur khas sebuah activation record 31

4.1 Hierarki Chomsky sebagai kelas bahasa yang bersarang 40
4.2 PDA untuk $\{a^n b^n \mid n \geq 1\}$ 43

5.1 Antarmuka lexer, token stream, parser, dan tabel simbol 51
5.2 Anatomi sebuah token beserta atributnya 53
5.3 Komposisi ekspresi reguler $(a \mid b)^* abb$ 54
5.4 Maximal munch memilih `<=` sebagai satu token 55

6.1 DFA untuk bilangan a genap 65
6.2 NFA untuk $(a \mid b)^* abb$ 67
6.3 DFA hasil subset construction dari NFA pada Gambar 6.2 68
6.4 Penyempurnaan partisi pada DFA $(a \mid b)^* abb$ 70
6.5 Menggabungkan NFA token dengan state awal baru 72

7.1 Skema dua buffer dengan sentinel di akhir setiap buffer 79
7.2 Alur pemulihan kesalahan leksikal 89

8.1 Konversi pengulangan EBNF menjadi nonterminal rekursif BNF 101
8.2 Pohon urai untuk `id` + `id` ∗ `id` 104
8.3 Pohon urai (kiri) dan AST (kanan) untuk ekspresi yang sama 105
8.4 Hierarki nonterminal utama Subset-C 106

9.1 Dua pohon urai untuk dangling else 114
9.2 Tingkat nonterminal ekspresi: semakin ke bawah, semakin tinggi precedence 116

9.3 Transformasi left recursion langsung . . . 117
9.4 Left factoring mengeluarkan faktor umum . . . 119

10.1 Rantai pemanggilan utama recursive descent parser . . . 128
10.2 Alur panic mode pada parser . . . 138

11.1 Model shift-reduce dengan stack dan masukan . . . 145
11.2 Tiga kemungkinan aksi pada parser shift-reduce . . . 147

12.1 Item dan transisi goto pada simbol E . . . 158
12.2 Bagian automaton LR(0) untuk grammar ekspresi . . . 159
12.3 Penggabungan state LR(1) dengan inti sama menjadi LALR(1) . . . 162
12.4 Konflik shift-reduce: dua aksi untuk satu sel tabel . . . 164

13.1 Alur kerja parser generator . . . 171
13.2 AST untuk $a + b * c$ . . . 176
13.3 Integrasi Flex dan Bison . . . 177

14.1 Atribut synthesized mengalir dari anak ke orang tua . . . 187
14.2 Dari SDD ke translation scheme ke implementasi . . . 190

15.1 Stack scope: pencarian bergerak dari dalam ke luar . . . 198
15.2 Hash table dengan chaining . . . 199
15.3 Hierarki tipe numerik Subset-C . . . 204

16.1 Urutan pre-order: $+$, a, $*$, b, c . . . 219
16.2 Pipeline validasi multi-pass . . . 220
16.3 Keputusan pemulihan setelah kesalahan . . . 223

17.1 Tiga tingkat ketercapaian kompetensi . . . 234
17.2 Alur asesmen dari teori menuju integrasi . . . 235

Daftar Tabel

1 CPL yang didukung mata kuliah iv
2 CPMK dan fokus kompetensi iv
3 Peta sembilan belas Sub-CPMK ke bab dan asesmen v
4 Peta enam belas pertemuan RPS ke bab vi
5 Komponen penilaian dan bobot vii

1.1 Keterkaitan CPL, CPMK, dan fokus buku 2
1.2 Milestone proyek dan bab pendukungnya 4
1.3 Rencana aktivitas Bab I 5
1.4 Instrumen asesmen dan indikator 7

2.1 Hierarki Chomsky dan mesin pengenal 15
2.2 Konsep teori dan penerapannya pada front-end 16
2.3 Rencana aktivitas Bab II 19
2.4 Instrumen asesmen Bab II 20

3.1 Perbandingan compiler dan interpreter 24
3.2 Masukan dan keluaran setiap fase 26
3.3 Contoh pesan kesalahan yang informatif 28
3.4 Trade-off one-pass dan multi-pass 29
3.5 Rencana aktivitas Bab III 32
3.6 Instrumen asesmen Bab III 34

4.1 Ringkasan hierarki Chomsky 42
4.2 Hubungan tata bahasa, bahasa, dan mesin 43
4.3 Pemilihan formalisme untuk setiap fase 45
4.4 Rencana aktivitas Bab IV 46
4.5 Instrumen asesmen Bab IV 48

5.1 Contoh token, lexeme, dan pattern 52
5.2 Prioritas aturan token Subset-C 56
5.3 Ringkasan kategori token Subset-C 57
5.4 Rencana aktivitas Bab V 60
5.5 Instrumen asesmen Bab V 62

6.1 Tabel transisi DFA pada Gambar 6.1 65

6.2 Aturan konstruksi Thompson . . . 71
6.3 Rencana aktivitas Bab VI . . . 74
6.4 Instrumen asesmen Bab VI . . . 75

7.1 Perbandingan lexer tangan dan lexer Flex . . . 88
7.2 Contoh pesan kesalahan leksikal . . . 90
7.3 Artefak M1 dan statusnya . . . 93
7.4 Rencana aktivitas Bab VII . . . 94
7.5 Rubrik Praktikum 1 . . . 95
7.6 Instrumen asesmen Bab VII . . . 96

8.1 Operator EBNF dan artinya . . . 100
8.2 Perbedaan pohon urai dan AST . . . 105
8.3 Rencana aktivitas Bab VIII . . . 109
8.4 Instrumen asesmen Bab VIII . . . 110

9.1 Tingkat precedence operator Subset-C . . . 115
9.2 Perubahan utama pada grammar Subset-C . . . 121
9.3 Rencana aktivitas Bab IX . . . 123
9.4 Instrumen asesmen Bab IX . . . 124

10.1 FIRST dan FOLLOW grammar ekspresi . . . 131
10.2 Tabel LL(1) untuk grammar ekspresi . . . 133
10.3 Penelusuran predictive parser . . . 134
10.4 Pemetaan nonterminal ke fungsi parser . . . 135
10.5 Contoh pesan kesalahan sintaksis . . . 139
10.6 Rencana aktivitas Bab X . . . 140
10.7 Rubrik Praktikum 2 . . . 141
10.8 Instrumen asesmen Bab X . . . 142

11.1 Penelusuran shift-reduce . . . 146
11.2 Tabel operator precedence sederhana . . . 148
11.3 Perbandingan parsing top-down dan bottom-up . . . 151
11.4 Rencana aktivitas Bab XI . . . 153
11.5 Instrumen asesmen Bab XI . . . 154

12.1 Potongan tabel SLR(1) untuk grammar ekspresi . . . 159
12.2 Perbandingan SLR, LR(1), dan LALR . . . 163
12.3 Perbandingan keluarga LL(k) dan LR(k) . . . 165
12.4 Rencana aktivitas Bab XII . . . 166
12.5 Instrumen asesmen Bab XII . . . 168

13.1 Perbandingan Bison, ANTLR, dan parser tangan . . . 179
13.2 Rencana aktivitas Bab XIII . . . 180

13.3 Rubrik Praktikum 3 . . . 181
13.4 Instrumen asesmen Bab XIII . . . 182

14.1 Tugas utama analisis semantik . . . 186
14.2 SDD sederhana untuk tipe ekspresi . . . 188
14.3 Perbandingan S-attributed dan L-attributed . . . 189
14.4 Aturan tipe untuk operator Subset-C . . . 192
14.5 Rencana aktivitas Bab XIV . . . 193
14.6 Instrumen asesmen Bab XIV . . . 195

15.1 Operasi tabel simbol . . . 198
15.2 Aturan konversi Subset-C v1.0 . . . 209
15.3 Contoh pesan kesalahan tipe . . . 209
15.4 Rencana aktivitas Bab XV . . . 211
15.5 Rubrik Praktikum 4 . . . 212
15.6 Instrumen asesmen Bab XV . . . 213

16.1 Klasifikasi kesalahan dan fase pendeteksinya . . . 223
16.2 Contoh pesan dari setiap fase . . . 224
16.3 Artefak M5 . . . 227
16.4 Rencana aktivitas Bab XVI . . . 228
16.5 Rubrik Project Final . . . 229
16.6 Instrumen asesmen Bab XVI . . . 230

17.1 Peta pencapaian Sub-CPMK . . . 233
17.2 Blueprint asesmen dan cakupan CPMK . . . 234
17.3 Rubrik komprehensif . . . 236
17.4 Konversi nilai akhir . . . 237
17.5 Rubrik portofolio . . . 238
17.6 Jalur belajar lanjutan . . . 239
17.7 Rencana aktivitas Bab XVII . . . 242
17.8 Instrumen asesmen integratif . . . 243

A.1 Komponen penilaian dan bobotnya . . . 246
A.2 Rubrik umum Praktikum 1–4 . . . 246
A.3 Konversi nilai akhir . . . 247

B.1 Format lembar tugas praktikum . . . 248
B.2 Template jurnal mingguan . . . 249
B.3 Rubrik penilaian refleksi . . . 249

C.1 Istilah penting dalam buku ini . . . 250

D.1 Jalur bacaan menurut minat . . . 254
D.2 Kunci bibliografi dan nomor RPS . . . 255

Daftar Kode Program

1.1 Program Subset-C pertama: penjumlahan dua bilangan 4
5.1 Bagian awal `token.h`: enum jenis token 57
5.2 Potongan program uji `lex_demo.sc` 58
6.1 Fungsi `to_dfa` pada `fa_tool.py` 68
6.2 Awal fungsi `minimize` pada `fa_tool.py` 70
6.3 Fungsi `epsilon_closure` pada `fa_tool.py` 71
7.1 Antarmuka `lexer.h` 80
7.2 Fungsi `skip_space` pada `lexer.c` 81
7.3 Awal `lexer_next`: EOF dan identifier 82
7.4 Penanganan operator dan karakter ilegal 83
7.5 Definisi dan aturan kata kunci pada `lexer_flex.l` 86
7.6 Aturan operator, literal, identifier, dan kesalahan 86
7.7 Fungsi pelaporan `error.c` 89
7.8 Driver `main_lex.c` 90
8.1 Bagian awal `grammar.ebnf` 101
8.2 Bagian awal `grammar.bnf` 102
9.1 Grammar Subset-C yang diarahkan ke LL(1) 120
10.1 Antarmuka `parser.h` 128
10.2 Fungsi `parse_primary` 129
10.3 Fungsi `compute_first` pada `first_follow.py` 131
10.4 Bagian awal `parse_stmt` 135
10.5 Fungsi `synchronize` 136
11.1 Fungsi `reduce_top` pada `shift_reduce.c` 150
12.1 Fungsi `closure` dan `goto` 160
12.2 Fungsi `build_slr_table` 160
13.1 Deklarasi `parser.y` 172
13.2 Aturan `program` dan `stmt_list` 173
13.3 Aksi untuk assignment, call, dan block 175
13.4 Aksi untuk `primary` 176
14.1 Antarmuka `types.h` 191
15.1 Antarmuka `symtab.h` 200
15.2 Mendorong dan melepas scope 201
15.3 Lookup pada scope bersarang 202
15.4 Fungsi `check_expr` 204
15.5 Fungsi `check_block` 207
16.1 Antarmuka `ast.h` 216
16.2 Fungsi `visit` pada `validate.c` 221

16.3 Fungsi `validate_program` 222
16.4 Driver final `main.c` 225

BAB I

Orientasi: Buku Ajar, OBE, dan Peta Perjalanan

Outcome Pembelajaran dan Sub-CPMK

CPMK yang menjadi konteks: CPMK 01, CPMK 02, CPMK 03, CPMK 04

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan tujuan, cakupan, dan struktur buku ajar;
2. menghubungkan setiap bab dengan RPS berbasis OBE;
3. menggunakan fitur pembelajaran, proyek mini-kompilator, dan sumber belajar yang tersedia;
4. menyusun rencana belajar pribadi untuk satu semester.

Materi Pokok

- Mengapa Teknik Kompilasi penting bagi sarjana informatika.
- Posisi mata kuliah dalam kurikulum dan RPS berbasis OBE.
- Arsitektur buku dan alur belajar tujuh belas bab.
- Anatomi bab, kotak pembelajaran, dan komponen asesmen.
- Proyek mini-kompilator Subset-C dan strategi belajar.

1.1 Mengapa Teknik Kompilasi Penting

Teknik Kompilasi adalah mata kuliah yang menjelaskan bagaimana sebuah program sumber berbahasa tingkat tinggi diterjemahkan menjadi bentuk yang dapat dijalankan oleh mesin. Pemahaman ini penting bukan hanya untuk mahasiswa yang ingin membangun kompilator, tetapi juga untuk memahami alat bantu pengembangan, bahasa domain khusus, analisis kode, dan sistem kecerdasan buatan yang memproses bahasa.

Pekerjaan seorang insinyur perangkat lunak sangat sering bersinggungan dengan teknologi yang lahir dari teknik kompilasi: *parser* pada editor, *linter* dan *static analyzer*, mesin

template, *query optimizer* basis data, hingga penerjemah bahasa alami. Kemampuan membaca spesifikasi bahasa, memodelkan tata bahasa, dan menganalisis kesalahan secara sistematis adalah kompetensi yang dapat dipakai di banyak bidang.

Buku ini mengikuti pendekatan *Outcome-Based Education* (OBE). Setiap bab dimulai dengan outcome yang harus dicapai, dilanjutkan dengan materi, contoh, praktikum, latihan, asesmen, dan daftar cek kompetensi. Dengan cara ini, mahasiswa tidak hanya membaca teori, tetapi juga dapat menunjukkan bukti pencapaian kompetensi [16].

Informasi

Mata kuliah ini menekankan **front-end** kompilator: analisis leksikal, analisis sintaksis, analisis semantik, serta pembentukan dan validasi AST. Optimasi dan pembangkitan kode dibahas sebagai konteks pada Bab III dan Bab XVII, bukan sebagai bab tersendiri.

1.2 Mata Kuliah dalam Kurikulum dan RPS Berbasis OBE

Mata kuliah Teknik Kompilasi (MWCA017) diselenggarakan pada Semester Ganjil dengan bobot 3 SKS. Mata kuliah ini mendukung dua Capaian Pembelajaran Lulusan (CPL), yaitu CPL 02 tentang cara kerja sistem komputer dan CPL 03 tentang perancangan serta analisis algoritma. Turunan kedua CPL tersebut dinyatakan dalam empat CPMK.

Tabel 1.1. Keterkaitan CPL, CPMK, dan fokus buku

Komponen	Rumusan dan fokus
CPL 02	Memahami cara kerja sistem komputer serta mampu merancang dan mengembangkan algoritma untuk memecahkan masalah.
CPL 03	Mampu merancang dan menganalisis algoritma secara optimal serta menerapkannya pada bahasa pemrograman tertentu.
CPMK 01	Menjelaskan tahapan kompilasi: leksikal, sintaksis, dan semantik.
CPMK 02	Memahami konsep otomata hingga.
CPMK 03	Merancang tata bahasa bebas konteks untuk bahasa sederhana.
CPMK 04	Mengimplementasikan parser sederhana untuk validasi kode.

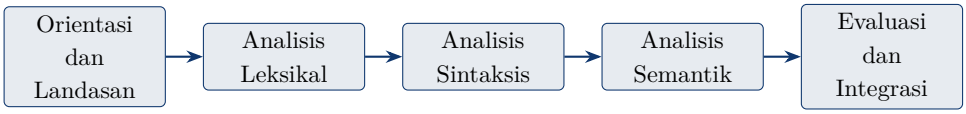
RPS membagi 16 pertemuan menjadi 14 pertemuan materi, satu Ujian Tengah Semester (UTS), dan satu Ujian Akhir Semester (UAS). Buku ini memetakan keempat belas topik tersebut ke empat belas bab inti, sehingga setiap pertemuan memiliki rujukan bab yang jelas.

Peringatan

Buku ini menggunakan data identitas dari ketentuan program studi: Universitas Bale Bandung, Fakultas Teknologi Informasi, kode MWCA017, dan Semester Ganjil. Data institusi pada berkas RPS asli merupakan placeholder dan tidak dipakai.

1.3 Arsitektur Buku dan Alur Belajar Tujuh Belas Bab

Buku ini disusun dalam tiga lapis. Lapis pertama adalah orientasi dan landasan teori (Bab I–II). Lapis kedua adalah empat belas bab inti (Bab III–XVI) yang mengikuti urutan topik RPS. Lapis ketiga adalah evaluasi dan integrasi kompetensi (Bab XVII). Lampiran A–D memuat rubrik, contoh tugas, glosarium, dan sumber belajar tambahan.



Gambar 1.1. Alur besar buku: dari orientasi menuju integrasi kompetensi

Urutan bab dirancang agar setiap bab memakai hasil bab sebelumnya. Sebagai contoh, Bab V memperkenalkan token, Bab VI memodelkan pengenal token dengan otomata, dan Bab VII mengimplementasikan keduanya menjadi *lexer* yang bekerja. Pola yang sama berlanjut pada parsing, analisis semantik, dan validasi AST.

Setiap bab inti memuat lima bagian materi, contoh berpenyelesaian, praktikum, latihan, asesmen, daftar cek, rangkuman, dan evaluasi. Komponen tersebut bukan pelengkap, melainkan bukti belajar yang dapat dinilai.

1.4 Anatomi Bab dan Fitur Pembelajaran

Setiap bab inti memiliki urutan yang tetap. Kotak **Outcome Pembelajaran dan Sub-CPMK** menyatakan kompetensi yang harus dicapai. Bagian **Materi Pokok** memberikan peta isi. Setelah itu pembaca menemukan uraian materi, contoh, praktikum, latihan, asesmen, daftar cek, rangkuman, dan evaluasi.

Tiga jenis kotak digunakan di seluruh buku:

- **Informasi** untuk konsep pendukung atau konteks tambahan;
- **Peringatan** untuk kesalahan umum dan jebakan konseptual;
- **Tips** untuk strategi praktis saat menulis kode atau mengerjakan ujian.

Kode program ditampilkan sebagai **Kode Program** dengan nomor baris. Berkas lengkap proyek tersedia di direktori `code/subsetc`. Contoh program pertama ditunjukkan pada Kode Program 1.1.

```
1 // Program Subset-C pertama: penjumlahan dua bilangan
2 int main() {
3     int a = 5;
4     int b = 7;
5     int hasil = a + b;
6     print(hasil);
7     return 0;
8 }
```

Kode Program 1.1. Program Subset-C pertama: penjumlahan dua bilangan

Program pada Kode Program 1.1 masih sangat sederhana, tetapi sudah mengandung seluruh unsur yang akan diolah oleh front-end: deklarasi fungsi, deklarasi variabel, ekspresi, pemanggilan `print`, dan pernyataan `return`.

1.5 Proyek Mini-Kompilator dan Strategi Belajar

Sepanjang semester, mahasiswa membangun satu proyek berkelanjutan: mini-kompilator untuk bahasa **Subset-C** (SC). Proyek ini tumbuh bersama bab. Pada Bab VII mahasiswa menyelesaikan *lexer*; pada Bab X–XIII menyelesaikan *parser* dan AST; pada Bab XIV–XVI menambahkan analisis semantik, tabel simbol, pemeriksaan tipe, dan validator.

Tabel 1.2. Milestone proyek dan bab pendukungnya

Milestone	Bab	Keluaran
M0	III	Struktur repo, spesifikasi bahasa v0.1, program contoh
M1	VII	Lexer tulisan tangan dan lexer Flex, 30 tes
M2	X	Recursive descent parser, AST minimal, tes
M3	XIII	Grammar Bison, AST lengkap, integrasi build
M4	XV	Tabel simbol bersarang dan pemeriksaan tipe
M5	XVI	Validator, diagnostik, dan proyek akhir

Strategi belajar yang disarankan adalah *baca, kerjakan, jelaskan*. Baca materi sebelum kuliah, kerjakan praktikum dan latihan secara bertahap, lalu jelaskan hasilnya kepada teman atau tuliskan dalam jurnal belajar. Pola ini melatih pemahaman konsep sekaligus kemampuan komunikasi teknis.

Tips

Jangan menunda praktikum sampai akhir semester. Setiap milestone hanya memerlukan tambahan kecil di atas milestone sebelumnya. Jika proyek dikerjakan konsisten, proyek akhir pada Bab XVI akan terasa sebagai penyelesaian alami, bukan pekerjaan mendadak.

Contoh

Contoh 1.1 (Membaca anatomi satu bab)**.** Perhatikan sebuah bab inti, misalnya Bab VII tentang implementasi lexer. Kotak outcome menyebut Sub-CPMK 2.4, yaitu mengimplementasikan lexical analyzer untuk bahasa sederhana. Bagian materi menjelaskan buffer, sentinel, dan *maximal munch*. Contoh menunjukkan potongan kode `next_token`. Praktikum meminta mahasiswa melengkapi lexer dan menjalankan 30 tes. Asesmen menilai kebenaran, kualitas kode, kelengkapan, dan pengujian. Daftar cek menyatakan bukti yang harus ditunjukkan.

Dengan membaca anatomi ini, mahasiswa mengetahui sejak awal apa yang harus dihasilkan, bukan hanya apa yang harus dibaca.

Contoh 1.2 (Dari program sumber ke token)**.** Untuk program pada Kode Program 1.1, urutan token yang dihasilkan lexer kira-kira adalah:

`int`, `main`, `(`, `)`, `{`, `int`, `a`, `=`, `5`, `;`, `int`, `b`, `=`, `7`, `;`, `print`, `(`, `a`, `+`, `b`, `)`, `;`, `return`, `0`, `;`, `}`, `EOF`.

Perhatikan bahwa spasi dan komentar tidak menghasilkan token, sedangkan setiap tanda baca dan literal menghasilkan token tersendiri.

Aktivitas Pembelajaran dan Praktikum

Tabel 1.3. Rencana aktivitas Bab I

Sesi	Bentuk	Kegiatan
1	Ceramah	Membaca peta buku dan RPS; diskusi capaian pembelajaran
2	Latihan	Menyusun rencana belajar pribadi dan memeriksa prasyarat
3	Praktikum 0	Menyiapkan lingkungan kerja dan menjalankan program contoh

Praktikum 0: Persiapan Lingkungan

1. Pastikan `pdflatex` dan `bibtex` tersedia.

2. Unduh atau salin direktori proyek `code/subsetc`.

3. Baca `code/subsetc/README.md` dan spesifikasi bahasa.

4. Jalankan program contoh `hello.sc` melalui alat yang disediakan dosen.

5. Tuliskan satu paragraf refleksi tentang kesulitan yang dihadapi.

Praktikum ini tidak dinilai secara sumatif, tetapi menjadi prasyarat administratif untuk Praktikum 1.

Latihan dan Refleksi

Latihan

1. Jelaskan perbedaan tujuan buku ajar ini dengan buku teks kompilator pada umumnya.

2. Sebutkan empat CPMK dan hubungkan masing-masing dengan minimal satu bab inti.

3. Mengapa buku ini memisahkan bab orientasi dan landasan teori dari empat belas bab inti?

4. Gambarkan alur dari program sumber sampai AST dalam bentuk diagram sendiri.

5. Baca `code/subsetc/examples/hello.sc`, lalu tuliskan token-token yang dihasilkan secara manual.

Refleksi

Refleksi

Jawab dengan jujur:

1. Kompetensi apa yang paling ingin Anda capai pada mata kuliah ini?

2. Bagian mana dari proses kompilasi yang paling sulit Anda bayangkan?

3. Kebiasaan belajar apa yang akan Anda ubah setelah membaca bab ini?

Asesmen

Asesmen Bab I bersifat formatif dan tidak memiliki bobot tersendiri. Nilainya menjadi bagian dari komponen **Kuis** (5%) dan **Tugas Teori** (8%) pada RPS.

Tabel 1.4. Instrumen asesmen dan indikator

Instrumen	Indikator	Bobot
Kuis awal	Menjelaskan struktur buku, CPMK, dan alur belajar	5% (bagian)
Tugas rencana belajar	Menyusun rencana studi yang realistis dan terukur	8% (bagian)
Praktikum 0	Menyiapkan lingkungan dan menjalankan program contoh	kelulusan prasyarat

Kriteria kelulusan Bab I: mahasiswa dapat menjelaskan peta buku, menghubungkan bab dengan CPMK, dan menunjukkan bahwa lingkungan kerja siap dipakai.

Checklist Pencapaian Kompetensi

Centang setiap butir yang sudah dapat Anda tunjukkan dengan bukti.

☐ Saya dapat menjelaskan tujuan dan cakupan buku ajar.

☐ Saya dapat menyebutkan empat CPMK dan kaitannya dengan bab.

☐ Saya dapat menjelaskan alur tujuh belas bab.

☐ Saya dapat menemukan komponen OBE di setiap bab.

☐ Saya sudah menyiapkan lingkungan kerja untuk praktikum.

☐ Saya sudah menjalankan program contoh `hello.sc`.

☐ Saya sudah menuliskan rencana belajar pribadi.

☐ Saya dapat menjelaskan perbedaan front-end dan back-end.

Rangkuman

Pada bab ini telah dibahas:

- pentingnya Teknik Kompilasi bagi sarjana informatika;
- posisi mata kuliah dalam kurikulum dan RPS berbasis OBE;
- arsitektur buku yang terdiri atas orientasi, landasan teori, empat belas bab inti, dan bab evaluasi;
- anatomi bab serta fungsi kotak pembelajaran dan kode program;
- proyek mini-kompilator Subset-C dan milestone-nya.

Bab berikutnya, Bab II, menyajikan landasan teori dan notasi formal yang dipakai di seluruh buku: himpunan, relasi, bahasa, tata bahasa, dan mesin abstrak. Materi tersebut menjadi bahasa bersama untuk memahami analisis leksikal, sintaksis, dan semantik.

Evaluasi

1. Uraikan tiga alasan mengapa pemahaman teknik kompilasi berguna bagi seorang insinyur perangkat lunak.
2. Buat tabel yang memetakan empat CPMK ke minimal dua bab inti masing-masing.
3. Jelaskan perbedaan peran Bab I, Bab II, dan Bab XVII dalam struktur buku.
4. Diberikan potongan program Subset-C, tuliskan urutan token dan jelaskan aturan yang Anda pakai.
5. Rancang rencana belajar empat belas minggu yang memuat bacaan, praktikum, dan evaluasi mandiri.

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan konsep (60%), kelengkapan (25%), kejelasan bahasa (15%).
- Soal 4: kebenaran token (70%), penjelasan aturan (30%).
- Soal 5: kelayakan jadwal (50%), keterukuran target (30%), keseimbangan teori-praktik (20%).

BAB II

Landasan Teori dan Notasi Formal

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK pendukung: Sub-CPMK 1.4, Sub-CPMK 3.1

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menggunakan notasi himpunan, relasi, fungsi, dan induksi matematika secara tepat;
2. mendefinisikan alfabet, string, bahasa, dan operasi atas bahasa;
3. menjelaskan tata bahasa sebagai sistem penulisan ulang dan menghasilkan derivasi;
4. membedakan kelas bahasa pada hierarki Chomsky beserta mesin abstrak yang bersesuaian;
5. memetakan konsep teori bahasa ke fase-fase front-end kompilator.

Materi Pokok

- Himpunan, relasi, fungsi, dan induksi matematika.
- Alfabet, string, bahasa, serta operasi gabungan, konkatenasi, dan bintang Kleene.
- Tata bahasa sebagai sistem penulisan ulang, derivasi, dan pohon urai.
- Mesin abstrak: otomata hingga, otomata pushdown, dan mesin Turing.
- Peta konsep dari teori bahasa ke analisis leksikal, sintaksis, dan semantik.

2.1 Himpunan, Relasi, Fungsi, dan Induksi Matematika

Teori kompilasi dibangun di atas bahasa matematika yang ringkas dan presisi. Sebelum membahas token, tata bahasa, atau otomata, kita perlu menyepakati notasi dasar. Bagian ini merangkum konsep himpunan, relasi, fungsi, dan induksi matematika yang akan dipakai berulang kali.

2.1.1 Himpunan dan Operasi Himpunan

Definisi 2.1 (Himpunan dan keanggotaan)**.** Himpunan adalah kumpulan objek yang terdefinisi dengan baik. Jika x merupakan anggota himpunan A, kita menulis $x \in A$; jika tidak, $x \notin A$. Himpunan yang tidak memiliki anggota disebut himpunan kosong dan dilambangkan $\varnothing$.

Himpunan dapat dinyatakan dengan mendaftar anggotanya, misalnya $A = \{0, 1, 2\}$, atau dengan aturan keanggotaan, misalnya $B = \{n \in \mathbb{N} \mid n \text{ genap}\}$. Himpunan A merupakan himpunan bagian dari B, ditulis $A \subseteq B$, jika setiap anggota A juga anggota B.

Operasi dasar himpunan adalah gabungan $A \cup B$, irisan $A \cap B$, selisih $A \setminus B$, dan komplemen A^c terhadap semesta pembicaraan. Himpunan kuasa $\mathcal{P}(A)$ adalah himpunan seluruh himpunan bagian A. Jika A memiliki n anggota, maka $\mathcal{P}(A)$ memiliki 2^n anggota.

Produk Cartesian $A \times B$ adalah himpunan seluruh pasangan terurut (a, b) dengan $a \in A$ dan $b \in B$. Konsep ini menjadi dasar untuk mendefinisikan relasi dan fungsi.

2.1.2 Relasi dan Fungsi

Definisi 2.2 (Relasi dan fungsi)**.** Relasi dari A ke B adalah himpunan bagian dari $A \times B$. Fungsi $f : A \to B$ adalah relasi khusus yang memasangkan setiap $a \in A$ dengan tepat satu $b \in B$. Himpunan A disebut domain, dan himpunan nilai $f(a)$ disebut range.

Fungsi f bersifat *injektif* jika $f(a_1) = f(a_2)$ mengakibatkan $a_1 = a_2$, *surjektif* jika setiap anggota B merupakan nilai f, dan *bijektif* jika kedua sifat tersebut dimiliki. Dalam teori bahasa, fungsi sering dipakai untuk memetakan string ke string lain, misalnya fungsi pembalik $w \mapsto w^R$.

Relasi ekuivalensi adalah relasi yang bersifat reflektif, simetris, dan transitif. Relasi ini mempartisi himpunan menjadi kelas-kelas ekuivalensi. Konsep partisi akan muncul kembali saat minimalisasi DFA: state yang berperilaku sama dikelompokkan ke dalam kelas ekuivalensi.

2.1.3 Induksi Matematika

Definisi 2.3 (Prinsip induksi sederhana)**.** Untuk membuktikan bahwa pernyataan $P(n)$ benar untuk setiap bilangan asli $n \geq n_0$, cukup dibuktikan:

(i) **basis**: $P(n_0)$ benar;

(ii) **langkah induksi**: jika $P(k)$ benar untuk suatu $k \geq n_0$, maka $P(k+1)$ benar.

Induksi kuat mengizinkan asumsi bahwa $P(n_0), P(n_0+1), \ldots, P(k)$ semuanya benar untuk membuktikan $P(k+1)$. Induksi struktural adalah varian yang dipakai untuk membuktikan sifat string, pohon, atau ekspresi yang didefinisikan secara rekursif.

Sebagai contoh, kita buktikan bahwa panjang konkatenasi memenuhi $|uv| = |u|+|v|$ untuk semua string u dan v. Basis: jika $v = \varepsilon$, maka $uv = u$ dan $|uv| = |u| = |u|+0 = |u|+|v|$. Langkah induksi: misalkan sifat berlaku untuk v. Untuk $v' = va$ dengan a sebuah simbol, berlaku $uv' = (uv)a$, sehingga

$$|uv'| = |uv| + 1 = |u| + |v| + 1 = |u| + |v'|.$$

Dengan induksi struktural, sifat terbukti untuk semua string v.

Induksi juga dipakai untuk membuktikan bahwa sebuah tata bahasa membangkitkan bahasa tertentu. Misalnya, untuk membuktikan bahwa G membangkitkan $L(G) = \{a^n b^n \mid n \geq 0\}$, kita membuktikan dua arah: setiap string yang diturunkan memiliki bentuk tersebut, dan setiap string berbentuk tersebut dapat diturunkan.

2.2 Alfabet, String, Bahasa, dan Operasi Bahasa

2.2.1 Alfabet dan String

Definisi 2.4 (Alfabet, string, dan bahasa)**.** Alfabet Σ adalah himpunan berhingga yang tidak kosong dari simbol. String adalah barisan berhingga simbol dari Σ. String kosong dilambangkan ε dan memiliki panjang nol. Bahasa L atas Σ adalah himpunan string atas Σ, yaitu $L \subseteq \Sigma^*$.

Himpunan seluruh string atas Σ dilambangkan Σ^*; himpunan string tidak kosong dilambangkan Σ^+. Dengan demikian $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$.

Panjang string w dilambangkan $|w|$. Konkatenasi string u dan v dilambangkan uv. Pembalik string w dilambangkan w^R. String u adalah prefiks dari w jika terdapat v sehingga $w = uv$; sufiks didefinisikan secara simetris; substring adalah potongan yang mungkin berada di tengah.

Contoh 2.5 (String dan operasinya)**.** Misalkan $\Sigma = \{a, b\}$ dan $w = abba$. Maka $|w| = 4$, $w^R = abba$, prefiks w adalah $\varepsilon, a, ab, abb, abba$, dan sufiksnya adalah $\varepsilon, a, ba, bba, abba$. String bb merupakan substring w, sedangkan bab bukan.

2.2.2 Operasi atas Bahasa

Karena bahasa adalah himpunan string, operasi himpunan berlaku: gabungan $L_1 \cup L_2$, irisan $L_1 \cap L_2$, dan selisih $L_1 \setminus L_2$. Selain itu, terdapat operasi khas bahasa: konkatenasi dan bintang Kleene.

Definisi 2.6 (Konkatenasi dan bintang Kleene)**.** Konkatenasi dua bahasa didefinisikan sebagai

$$L_1 L_2 = \{uv \mid u \in L_1,\ v \in L_2\}.$$

Bintang Kleene L^* adalah himpunan konkatenasi nol atau lebih string dari L:

$$L^* = \bigcup_{k \geq 0} L^k, \qquad L^0 = \{\varepsilon\}.$$

Operasi $L^+ = \bigcup_{k \geq 1} L^k$ disebut *positive closure*.

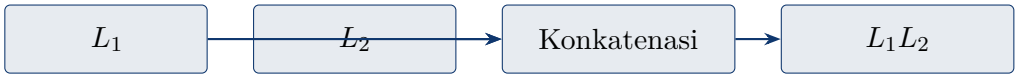
Contoh 2.7 (Operasi bahasa)**.** Misalkan $L_1 = \{a, ab\}$ dan $L_2 = \{b, bb\}$. Maka

$$L_1 L_2 = \{ab, abb, abb, abbb\} = \{ab, abb, abbb\},$$

karena $ab \cdot b = abb$ dan $ab \cdot bb = abbb$, sedangkan $a \cdot b = ab$. Selanjutnya

$$L_1^* = \{\varepsilon, a, ab, aa, aab, aba, abab, \ldots\}.$$

Perhatikan bahwa L_1^* memuat string dengan jumlah b yang sebanding dengan jumlah a, tetapi tidak semua kombinasinya.



Gambar 2.1. Konkatenasi dua bahasa menghasilkan bahasa baru

Operasi bahasa memenuhi beberapa identitas yang berguna. Salah satunya adalah $(L_1 \cup L_2)L_3 = L_1L_3 \cup L_2L_3$. Identitas lain adalah $(L^*)^* = L^*$ dan $\varnothing^* = \{\varepsilon\}$. Identitas-identitas ini akan dipakai saat mengubah ekspresi reguler menjadi otomata dan saat membuktikan ekuivalensi dua definisi bahasa.

2.2.3 Bahasa dan Masalah Keputusan

Setiap bahasa L dapat dipandang sebagai masalah keputusan: diberikan string w, tentukan apakah $w \in L$. Sebuah *lexer* menjawab pertanyaan ini untuk bahasa token tertentu; sebuah *parser* menjawabnya untuk bahasa program yang lebih kompleks. Klasifikasi bahasa menurut daya ekspresifnya, yang dibahas pada Bagian 2.4, menentukan jenis mesin yang diperlukan untuk menjawab pertanyaan tersebut.

2.3 Tata Bahasa sebagai Sistem Penulisan Ulang

2.3.1 Definisi Tata Bahasa

Definisi 2.8 (Tata bahasa)**.** Tata bahasa G didefinisikan sebagai empat-tupel

$$G = (V, T, P, S),$$

dengan V himpunan berhingga simbol nonterminal, T himpunan berhingga simbol terminal, P himpunan aturan produksi, dan $S \in V$ simbol awal. Himpunan V dan T saling lepas.

Aturan produksi berbentuk $\alpha \to \beta$, dengan α memuat paling sedikit satu nonterminal. Jika α hanya berupa satu nonterminal, aturan disebut *context-free*. Bentuk khusus $A \to \alpha$ inilah yang menjadi fokus Bab VIII dan seterusnya.

Notasi BNF (*Backus-Naur Form*) menulis aturan $A \to \alpha_1 \mid \alpha_2 \mid \cdots$ sebagai satu aturan dengan alternatif. Sebagai contoh, tata bahasa aritmetika sederhana dapat ditulis:

$$\begin{aligned} E &\to E + T \mid T, \\ T &\to T * F \mid F, \\ F &\to (E) \mid \texttt{id}. \end{aligned}$$

2.3.2 Derivasi dan Bentuk Kalimat

Definisi 2.9 (Derivasi)**.** Jika $A \to \gamma$ adalah aturan produksi dan $\alpha A \beta$ adalah string simbol, maka $\alpha A \beta \Rightarrow \alpha \gamma \beta$ adalah satu langkah derivasi. Notasi $\Rightarrow^*$ menyatakan nol atau lebih langkah derivasi. String yang diperoleh dari S melalui derivasi disebut bentuk kalimat; bentuk kalimat yang seluruhnya terminal disebut string dalam bahasa.

Bahasa yang dibangkitkan oleh G adalah

$$L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}.$$

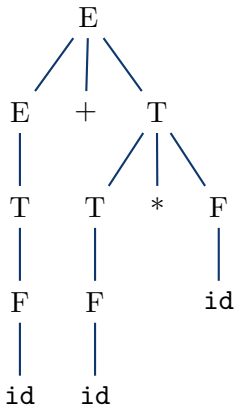
Derivasi *leftmost* selalu mengganti nonterminal paling kiri; derivasi *rightmost* mengganti nonterminal paling kanan. Untuk tata bahasa yang tidak ambigu, kedua derivasi menghasilkan pohon urai yang sama.

Contoh 2.10 (Derivasi leftmost)**.** Untuk string $\texttt{id} + \texttt{id} * \texttt{id}$, derivasi leftmost pada tata bahasa aritmetika adalah:

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow T + T \\ &\Rightarrow F + T \\ &\Rightarrow \texttt{id} + T \\ &\Rightarrow \texttt{id} + T * F \\ &\Rightarrow \texttt{id} + F * F \\ &\Rightarrow \texttt{id} + \texttt{id} * F \\ &\Rightarrow \texttt{id} + \texttt{id} * \texttt{id}. \end{aligned}$$

2.3.3 Pohon Urai

Derivasi dapat direpresentasikan sebagai pohon urai (*parse tree*). Akar pohon adalah simbol awal S. Setiap simpul dalam adalah nonterminal, dan anak-anaknya adalah simbol pada ruas kanan aturan produksi yang dipakai. Daun-daun pohon, dibaca dari kiri ke kanan, membentuk string yang diturunkan.



Gambar 2.2. Pohon urai untuk `id` + `id` ∗ `id`

Pohon urai menunjukkan struktur hierarkis ekspresi: perkalian lebih dalam daripada penjumlahan, sehingga dikerjakan lebih dahulu. Struktur inilah yang nantinya diringkas menjadi *Abstract Syntax Tree* (AST) pada Bab XVI.

2.3.4 Klasifikasi Tata Bahasa

Tata bahasa dapat diklasifikasikan menurut bentuk aturan produksinya. Tata bahasa regular memiliki aturan berbentuk $A \to aB$ atau $A \to a$, tata bahasa bebas konteks memiliki aturan $A \to \alpha$, tata bahasa sensitif konteks memiliki aturan $\alpha A \beta \to \alpha \gamma \beta$ dengan $\gamma \neq \varepsilon$, dan tata bahasa tanpa batas tidak membatasi bentuk aturan. Klasifikasi ini bersesuaian dengan hierarki Chomsky yang dibahas pada bagian berikutnya.

2.4 Mesin Abstrak dan Daya Komputasi

2.4.1 Otomata sebagai Pengenal Bahasa

Otomata adalah model matematika dari mesin yang membaca string simbol satu per satu dan berpindah antarstate. Secara informal, otomata memiliki himpunan state, state awal, himpunan state penerima, dan fungsi transisi. String diterima jika setelah seluruh simbol dibaca, mesin berada pada state penerima.

Otomata hingga (*finite automaton*) memiliki memori terbatas: hanya state saat ini yang disimpan. Karena itu, otomata hingga hanya mengenali bahasa regular. Otomata pushdown menambahkan stack, sehingga dapat mengenali bahasa bebas konteks. Mesin Turing menambahkan pita yang dapat dibaca dan ditulis tanpa batas, sehingga dapat mengenali bahasa yang lebih luas.

Definisi 2.11 (DFA)**.** Deterministic Finite Automaton (DFA) adalah lima-tupel

$$M = (Q, \Sigma, \delta, q_0, F),$$

dengan Q himpunan state berhingga, Σ alfabet, q_0 state awal, $F \subseteq Q$ himpunan state penerima, dan $\delta : Q \times \Sigma \to Q$ fungsi transisi total.

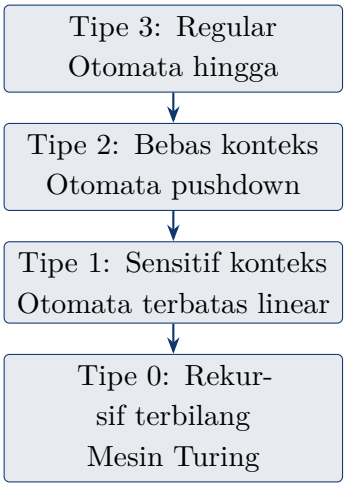
DFA bersifat deterministik: untuk setiap state dan simbol, terdapat tepat satu state tujuan. NFA (*nondeterministic finite automaton*) mengizinkan nol, satu, atau beberapa transisi untuk pasangan yang sama, termasuk transisi ε. Meskipun tampak lebih kuat, NFA dan DFA mengenali kelas bahasa yang sama; buktinya diberikan melalui konstruksi subset pada Bab VI.

2.4.2 Hierarki Chomsky

Tabel 2.1. Hierarki Chomsky dan mesin pengenal

Tipe	Kelas bahasa	Bentuk aturan	Mesin
3	Regular	$A \to aB$, $A \to a$, $A \to \varepsilon$	Otomata hingga
2	Bebas konteks	$A \to \alpha$	Otomata pushdown
1	Sensitif konteks	$\alpha A \beta \to \alpha \gamma \beta$	Otomata terbatas linear
0	Rekursif terbilang	$\alpha \to \beta$	Mesin Turing

Setiap kelas bahasa merupakan himpunan bagian sejati dari kelas di atasnya. Bahasa $\{a^n b^n \mid n \geq 0\}$ adalah contoh bahasa bebas konteks yang bukan regular. Bahasa $\{a^n b^n c^n \mid n \geq 0\}$ adalah contoh bahasa sensitif konteks yang bukan bebas konteks.



Gambar 2.3. Tangga hierarki Chomsky: semakin ke bawah, semakin besar daya ekspresif

2.4.3 Implikasi bagi Kompilator

Front-end kompilator memanfaatkan dua tingkat hierarki. Analisis leksikal berada pada Tipe 3 karena token dapat dikenali dengan memori terbatas. Analisis sintaksis berada

pada Tipe 2 karena struktur program membutuhkan stack, misalnya untuk menangani kurung bersarang atau pemanggilan fungsi. Analisis semantik melampaui Tipe 2 karena memerlukan informasi konteks seperti tipe dan scope.

Pemahaman hierarki ini mencegah kesalahan Desain: ekspresi reguler tidak cukup untuk memeriksa kurung seimbang, dan tata bahasa bebas konteks tidak cukup untuk memeriksa kesesuaian tipe. Setiap tingkat memerlukan alat dan teknik yang berbeda.

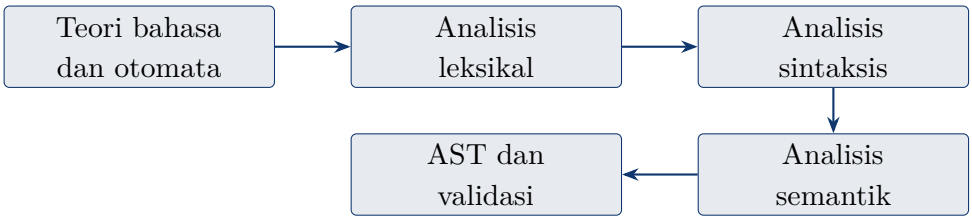
2.5 Peta Konsep: dari Teori Bahasa ke Front-End Kompilator

Konsep-konsep pada bab ini bukan materi hafalan yang berdiri sendiri. Mereka membentuk rantai yang menghubungkan teori bahasa dengan implementasi kompilator. Rantai tersebut dapat diringkas sebagai berikut:

$$\text{ekspresi reguler} \rightarrow \text{NFA} \rightarrow \text{DFA} \rightarrow \text{lexer},$$

$$\text{CFG} \rightarrow \text{derivasi} \rightarrow \text{pohon urai} \rightarrow \text{parser} \rightarrow \text{AST},$$

$$\text{atribut} \rightarrow \text{SDD} \rightarrow \text{analisis semantik} \rightarrow \text{validasi}.$$



Gambar 2.4. Peta konsep dari teori bahasa menuju front-end kompilator

Tabel 2.2. Konsep teori dan penerapannya pada front-end

Konsep	**Penerapan**	**Bab**
Ekspresi reguler	Spesifikasi token: identifier, literal, operator	V
NFA dan DFA	Pengenalan token dan konstruksi scanner	VI–VII
CFG dan derivasi	Spesifikasi struktur program dan parser	VIII–X
FIRST dan FOLLOW	Konstruksi tabel LL(1)	X
LR item dan tabel	Konstruksi parser bottom-up	XI–XII
Atribut dan SDD	Pemeriksaan tipe dan analisis semantik	XIV
Pohon dan AST	Representasi program untuk validasi	VIII, XVI

Peta ini menjelaskan mengapa urutan bab mengikuti urutan fase kompilasi. Setiap bab menambahkan satu lapisan representasi: dari karakter menjadi token, dari token menjadi

pohon, dari pohon menjadi struktur bermakna, dan akhirnya menjadi program yang tervalidasi.

Bab III akan memperlihatkan keseluruhan peta ini dalam konteks arsitektur kompilator. Bab IV kemudian memperdalam hierarki Chomsky, yang menjadi dasar pemilihan alat pada setiap fase. Dengan memahami peta konsep ini, mahasiswa dapat melihat bahwa setiap teknik yang dipelajari memiliki tempat yang jelas dalam sistem yang lebih besar.

Contoh

Contoh 2.12 (Pembuktian dengan induksi)**.** Buktikan bahwa untuk setiap $n \geq 1$,

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Basis. Untuk $n = 1$, ruas kiri bernilai 1 dan ruas kanan bernilai $1(2)/2 = 1$. Basis benar.

Langkah induksi. Asumsikan sifat benar untuk k:

$$\sum_{i=1}^{k} i = \frac{k(k+1)}{2}.$$

Maka

$$\sum_{i=1}^{k+1} i = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}.$$

Bentuk terakhir adalah rumus untuk $n = k+1$. Dengan induksi, sifat terbukti untuk semua $n \geq 1$.

Contoh 2.13 (Membuktikan bahasa)**.** Misalkan $G = (\{S\}, \{a, b\}, P, S)$ dengan aturan $S \to aSb \mid \varepsilon$. Tunjukkan bahwa $L(G) = \{a^n b^n \mid n \geq 0\}$.

Arah pertama. Setiap derivasi yang mungkin adalah

$$S \Rightarrow aSb \Rightarrow a^2Sb^2 \Rightarrow \cdots \Rightarrow a^nSb^n \Rightarrow a^nb^n.$$

Dengan induksi pada jumlah langkah, setiap string yang diturunkan berbentuk a^nb^n.

Arah kedua. Untuk setiap $n \geq 0$, string a^nb^n dapat diturunkan dengan menerapkan aturan $S \to aSb$ sebanyak n kali, lalu $S \to \varepsilon$. Jadi setiap string berbentuk tersebut berada di $L(G)$.

Kedua arah membuktikan $L(G) = \{a^nb^n \mid n \geq 0\}$.

Contoh 2.14 (Derivasi dan pohon urai)**.** Diberikan tata bahasa

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to (E) \mid \texttt{id}.$$

Tentukan derivasi leftmost dan pohon urai untuk $\texttt{id} * (\texttt{id} + \texttt{id})$.

Derivasi leftmost:

$$\begin{aligned} E &\Rightarrow T \\ &\Rightarrow T * F \\ &\Rightarrow F * F \\ &\Rightarrow \texttt{id} * F \\ &\Rightarrow \texttt{id} * (E) \\ &\Rightarrow \texttt{id} * (E + T) \\ &\Rightarrow \texttt{id} * (T + T) \\ &\Rightarrow \texttt{id} * (F + T) \\ &\Rightarrow \texttt{id} * (\texttt{id} + T) \\ &\Rightarrow \texttt{id} * (\texttt{id} + F) \\ &\Rightarrow \texttt{id} * (\texttt{id} + \texttt{id}). \end{aligned}$$

Pohon uraiya memiliki akar E, cabang kiri T yang menurun ke $\texttt{id}$, operator $*$, dan cabang kanan F yang menurun ke (E). Di dalam kurung, ekspresi $E + T$ diurai dengan cara yang sama seperti contoh sebelumnya.

Contoh 2.15 (Klasifikasi bahasa)**.** Tentukan kelas bahasa berikut pada hierarki Chomsky.

(a) $L_1 = \{w \in \{a, b\}^* \mid w \text{ memuat } aba\}$.

(b) $L_2 = \{a^n b^n \mid n \geq 0\}$.

(c) $L_3 = \{a^n b^n c^n \mid n \geq 0\}$.

Jawaban.

(a) L_1 regular. Bahasa ini dikenali oleh DFA dengan empat state yang mengingat apakah prefiks terakhir adalah ε, a, ab, atau aba.

(b) L_2 bebas konteks, tetapi bukan regular. Tata bahasa $S \to aSb \mid \varepsilon$ membangkitkannya, sedangkan lemma pompa untuk bahasa regular dapat dipakai untuk membuktikan bahwa bahasa ini tidak regular.

(c) L_3 sensitif konteks, tetapi bukan bebas konteks. Bahasa ini memerlukan pencocokan tiga kelompok simbol, yang tidak dapat dilakukan oleh satu stack.

Aktivitas Pembelajaran dan Praktikum

Tabel 2.3. Rencana aktivitas Bab II

Sesi	Bentuk	Kegiatan
1	Ceramah	Himpunan, relasi, fungsi, dan induksi
2	Latihan	Membuktikan sifat string dan bahasa
3	Lokakarya	Menulis tata bahasa dan derivasi untuk bahasa sederhana
4	Diskusi	Mengklasifikasikan bahasa pada hierarki Chomsky

Lokakarya Notasi

1. Diberikan bahasa $L = \{w \in \{0,1\}^* \mid w \text{ berakhir dengan } 01\}$, tuliskan lima string anggota dan lima string bukan anggota.
2. Tuliskan tata bahasa bebas konteks untuk L.
3. Buat derivasi leftmost untuk string 00101.
4. Gambarkan pohon urai untuk derivasi tersebut.
5. Tentukan kelas bahasa L pada hierarki Chomsky dan jelaskan alasannya.

Bab ini tidak menambahkan increment proyek. Milestone M0 dimulai pada Bab III setelah arsitektur kompilator dibahas. Namun, mahasiswa disarankan menyimpan catatan notasi dan contoh derivasi sebagai rujukan saat menulis spesifikasi Subset-C.

Latihan dan Refleksi

Latihan

1. Buktikan dengan induksi bahwa $|w^R| = |w|$ untuk setiap string w.
2. Diberikan $L_1 = \{a, aa\}$ dan $L_2 = \{b, ab\}$. Tentukan $L_1 L_2$, L_1^*, dan $L_1 \cup L_2$.
3. Tuliskan tata bahasa bebas konteks untuk bahasa $\{a^i b^j \mid i \geq j \geq 0\}$.
4. Buat derivasi leftmost dan pohon urai untuk string `id` + `id` ∗ `id` pada tata bahasa aritmetika.
5. Jelaskan mengapa bahasa $\{a^n b^n \mid n \geq 0\}$ tidak dapat dikenali oleh DFA.
6. Berikan contoh bahasa sensitif konteks yang bukan bebas konteks, lalu jelaskan mengapa satu stack tidak cukup.

7. Mengapa analisis leksikal cukup memakai otomata hingga, sedangkan analisis sintaksis memerlukan otomata pushdown?

8. Jelaskan hubungan antara pohon urai dan AST. Mengapa AST lebih ringkas?

Refleksi

Refleksi

1. Konsep formal mana yang paling sulit Anda pahami? Mengapa?

2. Apakah Anda dapat menjelaskan perbedaan bahasa regular dan bebas konteks tanpa membaca catatan?

3. Bagaimana notasi formal membantu Anda menulis spesifikasi bahasa yang lebih tepat?

Asesmen

Asesmen Bab II bersifat formatif dan menjadi bagian dari Tugas Teori (8%). Fokus asesmen adalah ketepatan notasi dan kemampuan menghubungkan teori dengan fase kompilasi.

Tabel 2.4. Instrumen asesmen Bab II

Instrumen	Indikator	Bobot
Kuis notasi	Ketepatan penggunaan himpunan, string, dan derivasi	formatif
Tugas derivasi	Derivasi dan pohon urai benar dan lengkap	formatif
Tugas klasifikasi	Klasifikasi bahasa dan alasan tepat	formatif
Partisipasi	Keaktifan diskusi dan lokakarya	formatif

Kriteria kelulusan: mahasiswa dapat menulis definisi formal tanpa kesalahan notasi, menghasilkan derivasi yang benar, dan menjelaskan hubungan antara kelas bahasa dan mesin pengenal.

Checklist Pencapaian Kompetensi

☐ Saya dapat menggunakan notasi himpunan, relasi, dan fungsi.

☐ Saya dapat membuktikan sifat sederhana dengan induksi.

☐ Saya dapat mendefinisikan alfabet, string, dan bahasa.

☐ Saya dapat melakukan operasi gabungan, konkatenasi, dan bintang Kleene.

☐ Saya dapat menulis tata bahasa dan menghasilkan derivasi.

☐ Saya dapat menggambar pohon urai dari suatu derivasi.

☐ Saya dapat membedakan bahasa regular, bebas konteks, sensitif konteks, dan rekursif terbilang.

☐ Saya dapat menjelaskan hubungan teori bahasa dengan fase front-end kompilator.

Rangkuman

Pada bab ini telah dibahas:

- notasi himpunan, relasi, fungsi, dan induksi matematika;
- alfabet, string, bahasa, serta operasi konkatenasi dan bintang Kleene;
- tata bahasa sebagai sistem penulisan ulang, derivasi, dan pohon urai;
- hierarki Chomsky dan mesin pengenal yang bersesuaian;
- peta konsep dari teori bahasa ke analisis leksikal, sintaksis, dan semantik.

Bab III akan memakai seluruh notasi ini untuk menjelaskan arsitektur kompilator. Kita akan melihat bagaimana string sumber berubah menjadi token, pohon, dan representasi semantik, serta di mana setiap teknik yang dipelajari pada bab-bab berikutnya digunakan.

Evaluasi

1. Definisikan tata bahasa $G = (V, T, P, S)$ dan jelaskan arti setiap komponen.
2. Diberikan $L = \{w \in \{a, b\}^* \mid |w| \text{ genap}\}$. Tuliskan tata bahasa regular untuk L.
3. Buktikan bahwa $L = \{a^n b^n \mid n \geq 0\}$ bukan bahasa regular. Jelaskan langkah pembuktiannya.
4. Buat derivasi leftmost, derivasi rightmost, dan pohon urai untuk string $a + b * c$ pada tata bahasa aritmetika.
5. Jelaskan perbedaan daya ekspresif otomata hingga, otomata pushdown, dan mesin Turing.
6. Mengapa analisis semantik tidak dapat dilakukan hanya dengan tata bahasa bebas konteks? Berikan satu contoh.

Kunci dan Rubrik Singkat

- Soal 1–2: ketepatan definisi dan notasi (70%), kelengkapan (30%).
- Soal 3: struktur pembuktian (50%), ketepatan argumen (50%).
- Soal 4: kebenaran derivasi (50%), kebenaran pohon (50%).
- Soal 5–6: ketepatan konsep (60%), contoh (40%).

BAB III

Pengenalan Kompilasi dan Arsitektur Kompilator

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 1.1, Sub-CPMK 1.2, Sub-CPMK 1.3

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. membedakan translator, assembler, interpreter, dan compiler;
2. menjelaskan fase-fase kompilasi beserta peran tabel simbol dan penanganan kesalahan;
3. menganalisis trade-off kompilator satu lintasan dan banyak lintasan;
4. menjelaskan pengaruh lingkungan run-time terhadap organisasi memori program;
5. menyiapkan struktur proyek mini-kompilator Subset-C (M0).

Materi Pokok

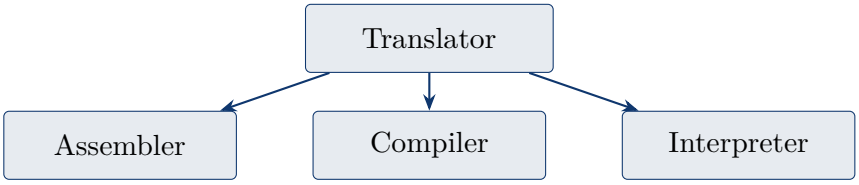
- Keluarga translator: assembler, compiler, interpreter, dan cross-compiler.
- Fase analisis dan sintesis pada kompilator.
- Manajemen tabel simbol dan penanganan kesalahan.
- Kompilator satu lintasan versus banyak lintasan.
- Lingkungan run-time: alokasi statis, stack, dan heap.

3.1 Translator, Assembler, Interpreter, dan Compiler

3.1.1 Keluarga Translator

Definisi 3.1 (Translator)**.** Translator adalah program yang menerjemahkan program sumber dari satu bahasa ke bahasa lain. Keluaran translator dapat berupa bahasa assembly, kode mesin, bytecode, atau bahasa tingkat tinggi lain.

Bahasa sumber dan bahasa target menentukan jenis translator. Assembler menerjemahkan bahasa assembly ke kode mesin. Compiler menerjemahkan bahasa tingkat tinggi ke bahasa tingkat rendah, biasanya assembly atau kode mesin. Interpreter tidak menghasilkan kode target yang berdiri sendiri; ia membaca program sumber dan langsung menjalankan aksinya. Cross-compiler menghasilkan kode untuk platform yang berbeda dari platform tempat compiler berjalan.



Gambar 3.1. Keluarga translator dan tiga anggotanya yang utama

3.1.2 Compiler versus Interpreter

Compiler dan interpreter sama-sama membaca program sumber, tetapi memiliki model eksekusi yang berbeda. Compiler menghasilkan program target yang dapat dijalankan berulang kali tanpa kehadiran compiler. Interpreter mengeksekusi program sumber secara langsung, biasanya dengan bantuan representasi antara seperti bytecode.

Tabel 3.1. Perbandingan compiler dan interpreter

Aspek	**Compiler**	**Interpreter**
Keluaran	Kode target yang berdiri sendiri	Tidak ada kode target permanen
Waktu eksekusi	Cepat setelah kompilasi	Lebih lambat karena penerjemahan saat runtime
Waktu mulai	Lebih lama karena kompilasi	Lebih cepat
Portabilitas	Kode target terikat platform	Selama interpreter tersedia
Analisis	Dapat melakukan analisis menyeluruh	Analisis terbatas pada saat eksekusi
Contoh	GCC, Clang, javac	Python, Ruby, interpreter JavaScript

Batas antara compiler dan interpreter tidak selalu tegas. Java mengompilasi sumber ke bytecode, lalu bytecode dijalankan oleh mesin virtual yang dapat melakukan kompilasi just-in-time. Pendekatan hybrid ini menggabungkan kelebihan keduanya.

3.1.3 Cross-Compiler dan Bootstrapping

Cross-compiler berguna ketika program dikembangkan pada satu platform tetapi dijalankan pada platform lain, misalnya kompilasi untuk mikrokontroler dari komputer

desktop. Compiler seperti ini harus mengetahui konvensi pemanggilan, ukuran kata, dan tata letak memori platform target.

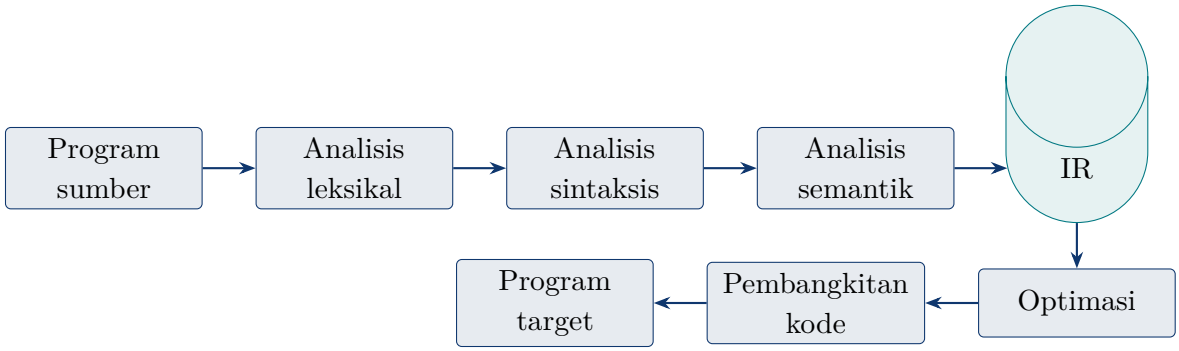
Bootstrapping adalah proses membangun compiler untuk suatu bahasa menggunakan compiler bahasa itu sendiri. Proses ini biasanya dimulai dengan compiler sederhana dalam bahasa lain, lalu ditingkatkan secara bertahap. Banyak compiler modern, termasuk GCC dan Clang, memiliki sejarah bootstrapping yang panjang.

Informasi

Istilah *compiler* berasal dari gagasan mengompilasi kumpulan rutin menjadi satu program. Pada praktik modern, compiler juga melakukan analisis, optimasi, dan pembangkitan kode, sehingga jauh lebih kompleks daripada sekadar penggabung rutin.

3.2 Fase-Fase Kompilasi: Analisis dan Sintesis

Kompilator modern biasanya dibagi menjadi dua bagian besar: analisis (front-end) dan sintesis (back-end). Bagian analisis memahami struktur program sumber; bagian sintesis menghasilkan program target. Batas keduanya adalah representasi antara (*intermediate representation*, IR).



Gambar 3.2. Pipeline kompilasi dari program sumber ke program target

3.2.1 Analisis Leksikal

Analisis leksikal membaca karakter demi karakter dan mengelompokkannya menjadi token. Token memiliki jenis dan atribut, misalnya `INT_LIT` dengan nilai 42. Spasi, baris baru, dan komentar dibuang. Keluaran fase ini adalah aliran token yang siap diproses parser.

3.2.2 Analisis Sintaksis

Analisis sintaksis memeriksa apakah aliran token membentuk struktur yang sesuai dengan tata bahasa. Hasilnya adalah pohon urai atau AST. Parser juga dapat membangun tabel

simbol awal dan melakukan pemulihan kesalahan sintaksis.

3.2.3 Analisis Semantik

Analisis semantik memeriksa makna program: apakah identifier sudah dideklarasikan, apakah tipe operand cocok, apakah fungsi dipanggil dengan jumlah argumen yang benar, dan apakah kontrol alur valid. Keluaran fase ini adalah AST yang telah dianotasi dengan informasi tipe dan referensi simbol.

3.2.4 Sintesis

Setelah IR terbentuk, fase sintesis melakukan optimasi dan pembangkitan kode. Optimasi dapat bersifat lokal (dalam satu blok dasar) atau global (lintas blok). Pembangkitan kode memilih instruksi target, mengalokasikan register, dan menghasilkan program assembly atau kode mesin.

Tabel 3.2. Masukan dan keluaran setiap fase

Fase	Masukan	Keluaran
Leksikal	Karakter sumber	Aliran token
Sintaksis	Aliran token	Pohon urai / AST
Semantik	AST dan tabel simbol	AST teranotasi
IR	AST teranotasi	Kode antara
Optimasi	Kode antara	Kode antara yang lebih efisien
Pembangkitan kode	Kode antara	Assembly atau kode mesin

Contoh 3.2 (Perjalanan program sederhana)**.** Untuk program

```
int main() {
    int a = 5;
    int b = 7;
    print(a + b);
    return 0;
}
```

fase leksikal menghasilkan token `int`, `main`, `(`, `)`, `{`, dan seterusnya. Fase sintaksis membentuk pohon deklarasi fungsi dan pernyataan. Fase semantik mencatat bahwa `a` dan `b` bertipe `int`, serta bahwa `print` menerima satu argumen. Fase IR dapat menghasilkan instruksi tiga-alamat seperti `t1 = a + b` dan `print t1`.

3.3 Manajemen Tabel Simbol dan Penanganan Kesalahan

3.3.1 Tabel Simbol

Definisi 3.3 (Tabel simbol)**.** Tabel simbol adalah struktur data yang memetakan identifier ke informasi tentang identifier tersebut, seperti tipe, scope, alamat memori, dan kategori (variabel, fungsi, parameter).

Tabel simbol mendukung operasi `insert`, `lookup`, dan `delete`. Pada bahasa dengan scope bersarang, tabel dapat diimplementasikan sebagai stack tabel atau satu tabel dengan rantai scope. Informasi yang disimpan biasanya mencakup nama, tipe, ukuran, alamat, dan referensi ke deklarasi.

Tabel simbol dipakai oleh hampir semua fase. Parser menggunakannya untuk membedakan identifier dan kata kunci; analisis semantik menggunakannya untuk memeriksa deklarasi dan tipe; pembangkitan kode menggunakannya untuk menentukan alamat variabel. Karena itu, tabel simbol sering disebut sebagai pusat informasi kompilator.

3.3.2 Jenis Kesalahan

Kesalahan program dapat diklasifikasikan menurut fase yang mendeteksinya:

- **kesalahan leksikal**: karakter ilegal, string tidak ditutup, komentar tidak ditutup;
- **kesalahan sintaksis**: token tidak sesuai tata bahasa, kurung tidak seimbang, titik koma hilang;
- **kesalahan semantik**: identifier tidak dideklarasikan, tipe tidak cocok, argumen fungsi salah;
- **kesalahan run-time**: pembagian nol, indeks di luar batas, stack overflow.

Compiler yang baik tidak berhenti pada kesalahan pertama. Ia melaporkan beberapa kesalahan, berusaha pulih, dan melanjutkan analisis. Strategi pemulihan yang umum adalah *panic mode*, *phrase-level recovery*, dan *error production*.

Tabel 3.3. Contoh pesan kesalahan yang informatif

Jenis	Pesan buruk	Pesan informatif
Leksikal	`error`	`baris 4, kolom 12: karakter '@' tidak dikenal`
Sintaksis	`syntax error`	`baris 7: diharapkan ';' setelah ekspresi`
Semantik	`type error`	`baris 9: tidak dapat menetapkan float ke int tanpa cast`

Pesan yang baik memuat lokasi (baris dan kolom), penyebab, dan saran perbaikan bila memungkinkan. Informasi lokasi sangat membantu karena mahasiswa dan pengguna dapat langsung menuju sumber masalah.

3.3.3 Contoh Alur Tabel Simbol

Perhatikan potongan program:

```
int x = 1;
{
    int x = 2;
    print(x);
}
print(x);
```

Saat memasuki blok, tabel simbol baru didorong ke stack. Deklarasi `int x = 2` berada pada scope dalam dan menutupi (*shadow*) `x` luar. Saat blok selesai, tabel scope dalam dibuang, sehingga `print(x)` terakhir merujuk ke `x` luar. Perilaku ini akan diimplementasikan pada Bab XV.

3.4 One-Pass versus Multi-Pass dan Organisasi Front-End/Back-End

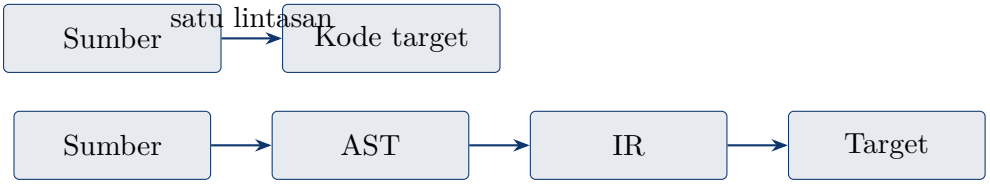
3.4.1 Kompilator Satu Lintasan

Kompilator satu lintasan (*one-pass compiler*) membaca program sumber satu kali dan langsung menghasilkan kode target. Setiap konstruksi bahasa harus dapat diproses tanpa menunggu informasi dari bagian program yang belum dibaca. Pendekatan ini cocok untuk bahasa yang dirancang agar dapat dikompilasi satu lintasan, seperti Pascal klasik.

Keuntungan one-pass adalah kebutuhan memori kecil dan waktu kompilasi singkat. Kerugiannya, optimasi terbatas dan dukungan bahasa menjadi kaku. Misalnya, pemanggilan fungsi sebelum deklarasinya sulit ditangani, dan analisis lintas prosedur tidak mungkin dilakukan.

3.4.2 Kompilator Banyak Lintasan

Kompilator banyak lintasan (*multi-pass compiler*) membaca representasi program beberapa kali. Lintasan pertama dapat membangun tabel simbol dan AST; lintasan berikutnya melakukan pemeriksaan tipe, optimasi, dan pembangkitan kode. Pendekatan ini memungkinkan analisis yang lebih dalam dan optimasi yang lebih baik.



Gambar 3.3. Perbandingan alur satu lintasan dan banyak lintasan

3.4.3 Organisasi Front-End dan Back-End

Pemisahan front-end dan back-end memungkinkan satu front-end dipakai untuk beberapa target. Front-end memahami bahasa sumber dan menghasilkan IR; back-end memahami IR dan menghasilkan kode untuk arsitektur tertentu. Dengan desain ini, penambahan target baru tidak memerlukan perubahan front-end, dan penambahan bahasa baru tidak memerlukan perubahan back-end.

Tabel 3.4. Trade-off one-pass dan multi-pass

Aspek	One-pass	Multi-pass
Memori	Kecil	Lebih besar
Kecepatan kompilasi	Cepat	Lebih lambat
Kedalaman analisis	Terbatas	Mendalam
Optimasi	Minimal	Luas
Fleksibilitas bahasa	Rendah	Tinggi
Contoh	Pascal klasik	GCC, Clang, LLVM

Peringatan

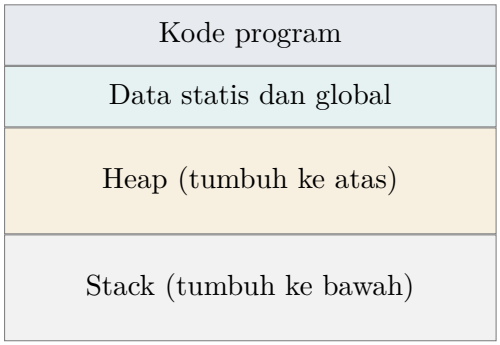
One-pass bukan berarti sederhana. Banyak kompilator satu lintasan tetap memerlukan tabel simbol dan mekanisme *backpatching* untuk menangani alamat yang belum diketahui. Trade-off utama tetap antara memori, kecepatan, dan kualitas

kode.

3.5 Lingkungan Run-Time: Static, Stack, dan Heap Allocation

3.5.1 Tata Letak Memori Program

Ketika program dijalankan, sistem operasi menyediakan ruang alamat yang biasanya dibagi menjadi beberapa segmen: kode, data statis, stack, dan heap. Segmen kode menyimpan instruksi. Segmen data statis menyimpan variabel global dan konstanta. Stack menyimpan frame aktivasi fungsi. Heap menyediakan memori untuk alokasi dinamis.



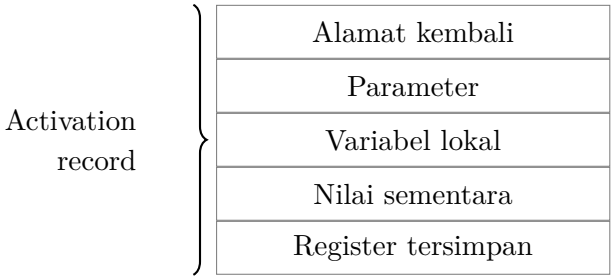
Gambar 3.4. Tata letak memori program pada lingkungan run-time

3.5.2 Alokasi Statis

Alokasi statis menempatkan variabel pada alamat tetap yang ditentukan saat kompilasi. Variabel global, konstanta, dan variabel dengan `static` biasanya dialokasikan secara statis. Keuntungannya, akses cepat dan tidak ada biaya alokasi saat runtime. Kerugiannya, ukuran harus diketahui saat kompilasi dan tidak mendukung rekursi untuk variabel lokal.

3.5.3 Alokasi Stack

Alokasi stack dipakai untuk variabel lokal dan parameter fungsi. Setiap pemanggilan fungsi mendorong satu *activation record* (frame) ke stack. Frame berisi parameter, variabel lokal, alamat kembali, dan informasi lain yang diperlukan. Saat fungsi selesai, frame dilepas. Model ini mendukung pemanggilan rekursif dan sangat efisien.



Gambar 3.5. Struktur khas sebuah activation record

3.5.4 Alokasi Heap

Heap dipakai untuk objek yang ukurannya tidak diketahui saat kompilasi atau yang harus hidup melewati masa hidup fungsi. Alokasi dan dealokasi dilakukan secara eksplisit (`malloc`/`free`) atau otomatis melalui pengumpul sampah (*garbage collector*). Heap lebih fleksibel, tetapi memiliki biaya: fragmentasi, waktu alokasi, dan kemungkinan kebocoran memori.

Contoh 3.4 (Activation record faktorial)**.** Untuk fungsi rekursif

```
int faktorial(int n) {
    if (n <= 1) return 1;
    return n * faktorial(n - 1);
}
```

setiap pemanggilan `faktorial` mendorong frame baru yang memuat parameter `n` dan alamat kembali. Pemanggilan `faktorial(3)` menghasilkan tiga frame bersarang sebelum kembali. Setelah nilai kembali dihitung, frame teratas dilepas satu per satu.

Pemahaman lingkungan run-time penting meskipun buku ini berfokus pada front-end. Tanpa pemahaman ini, sulit menjelaskan mengapa informasi scope, tipe, dan alamat perlu dipelihara sejak analisis semantik.

Contoh

Contoh 3.5 (Aliran token program Subset-C)**.** Diberikan program

```
int main() {
    int a = 5;
    print(a);
    return 0;
}
```

Aliran token yang dihasilkan secara konseptual adalah:

```
INT ID(main) LPAREN RPAREN LBRACE INT ID(a) ASSIGN INT_LIT(5) SEMI PRINT LPAREN
ID(a) RPAREN SEMI RETURN INT_LIT(0) SEMI RBRACE EOF.
```

Setiap token membawa lokasi (baris, kolom) untuk keperluan diagnostik.

Contoh 3.6 (Compiler versus interpreter)**.** Program yang sama dijalankan pada compiler dan interpreter. Pada compiler, program sumber dikompilasi menjadi executable, lalu executable dijalankan berulang kali. Pada interpreter, program sumber dibaca dan dieksekusi setiap kali dijalankan. Jika program dijalankan 100 kali, compiler membayar biaya kompilasi sekali, sedangkan interpreter membayar biaya parsing dan evaluasi 100 kali. Namun, untuk program yang hanya dijalankan sekali, interpreter sering lebih cepat karena tidak perlu menghasilkan kode target.

Contoh 3.7 (Activation record)**.** Misalkan fungsi `faktorial(3)` dipanggil. Frame pertama memuat `n = 3` dan alamat kembali ke pemanggil. Karena `n > 1`, fungsi memanggil `faktorial(2)`, mendorong frame kedua dengan `n = 2`. Proses berlanjut hingga `faktorial(1)`. Setelah basis tercapai, nilai 1 dikembalikan, lalu frame dilepas satu per satu sambil mengalikan hasilnya: $1 \times 2 \times 3 = 6$.

Contoh 3.8 (Memilih strategi kompilasi)**.** Sebuah tim mengembangkan bahasa domain khusus untuk perangkat mikrokontroler dengan memori sangat terbatas. Mereka memilih kompilator satu lintasan yang langsung menghasilkan kode mesin. Keputusan ini masuk akal karena memori kecil dan optimasi agresif tidak menjadi prioritas. Sebaliknya, tim yang mengembangkan bahasa umum untuk banyak platform memilih front-end bersama dan beberapa back-end, sehingga investasi pada analisis dan IR terbayar.

Aktivitas Pembelajaran dan Praktikum

Tabel 3.5. Rencana aktivitas Bab III

Sesi	Bentuk	Kegiatan
1	Ceramah	Keluarga translator dan perbedaan compiler-interpreter
2	Diskusi	Fase kompilasi dan aliran informasi
3	Latihan	Membaca tabel simbol dan pesan kesalahan
4	Praktikum	M0: menyiapkan repositori dan spesifikasi Subset-C

M0: Repositori dan Spesifikasi Bahasa

Tujuan. Menyiapkan struktur proyek dan spesifikasi awal Subset-C sebagai fondasi seluruh milestone berikutnya.

Langkah kerja.

1. Salin direktori `code/subsetc` ke repositori pribadi.
2. Baca `code/subsetc/spec/subset-c.md` dan `code/subsetc/README.md`.
3. Jalankan `build.bat` atau `Makefile` untuk memeriksa ketersediaan toolchain. Jika toolchain belum ada, catat langkah instalasinya.
4. Tambahkan minimal dua program contoh baru ke `examples/`: satu program aritmetika dan satu program dengan kontrol alur.
5. Tuliskan satu halaman catatan tentang keputusan desain bahasa yang Anda ambil.

Definisi selesai.

☐ Repositori memiliki struktur `spec`, `include`, `src`, `examples`, `tests`.

☐ Spesifikasi v0.1 terbaca dan dipahami.

☐ Minimal tiga program contoh tersedia.

☐ Catatan keputusan desain diserahkan.

Penilaian. M0 tidak memiliki bobot tersendiri, tetapi menjadi prasyarat Praktikum 1 (8%). Rubrik yang dipakai adalah rubrik umum praktikum pada Lampiran A.

Latihan dan Refleksi

Latihan

1. Jelaskan perbedaan assembler, compiler, interpreter, dan cross-compiler. Berikan satu contoh untuk masing-masing.
2. Sebutkan enam fase kompilasi dan jelaskan masukan serta keluaran setiap fase.
3. Mengapa tabel simbol disebut pusat informasi kompilator?
4. Berikan tiga contoh kesalahan leksikal, tiga kesalahan sintaksis, dan tiga kesalahan semantik.
5. Bandingkan kompilator satu lintasan dan banyak lintasan dari sisi memori, kecepatan, dan kualitas kode.
6. Jelaskan mengapa pemisahan front-end dan back-end memudahkan penambahan target baru.
7. Gambarkan tata letak memori program dan jelaskan fungsi setiap segmen.
8. Buat sketsa activation record untuk fungsi dengan dua parameter dan tiga variabel lokal.

Refleksi

Refleksi

1. Setelah membaca bab ini, fase mana yang paling menarik bagi Anda? Mengapa?
2. Apakah Anda dapat menjelaskan mengapa compiler dan interpreter dapat menghasilkan perilaku yang berbeda untuk program yang sama?
3. Apa satu hal yang akan Anda lakukan berbeda saat menulis program setelah memahami lingkungan run-time?

Asesmen

Asesmen Bab III mencakup Kuis (5%) dan bagian awal Tugas Teori (8%). Fokusnya adalah pemahaman arsitektur dan aliran informasi antar-fase.

Tabel 3.6. Instrumen asesmen Bab III

Instrumen	Indikator	Bobot
Kuis	Membedakan translator, compiler, interpreter, dan assembler	5%
Kuis	Menjelaskan fase kompilasi dan peran tabel simbol	5%
Tugas Teori	Menganalisis trade-off one-pass dan multi-pass	8%
Praktikum M0	Repositori dan spesifikasi v0.1 lengkap	prasyarat

Kriteria kelulusan: mahasiswa dapat menjelaskan pipeline kompilasi, menempatkan tabel simbol dan penanganan kesalahan pada pipeline, serta memilih strategi kompilasi yang sesuai dengan kebutuhan.

Checklist Pencapaian Kompetensi

☐ Saya dapat membedakan translator, assembler, compiler, dan interpreter.

☐ Saya dapat menjelaskan perbedaan compiler dan interpreter.

☐ Saya dapat menyebutkan fase analisis dan sintesis.

☐ Saya dapat menjelaskan peran tabel simbol pada setiap fase.

☐ Saya dapat mengklasifikasikan kesalahan program.

☐ Saya dapat menulis pesan kesalahan yang informatif.

☐ Saya dapat membandingkan one-pass dan multi-pass.

☐ Saya dapat menjelaskan organisasi front-end dan back-end.

☐ Saya dapat menggambarkan tata letak memori dan activation record.

☐ Saya sudah menyelesaikan M0: repositori dan spesifikasi v0.1.

Rangkuman

Pada bab ini telah dibahas:

- keluarga translator dan perbedaan compiler, interpreter, assembler, serta cross-compiler;
- fase analisis (leksikal, sintaksis, semantik) dan sintesis (IR, optimasi, pembangkitan kode);
- peran tabel simbol dan strategi penanganan kesalahan;
- trade-off kompilator satu lintasan dan banyak lintasan;
- lingkungan run-time dengan alokasi statis, stack, dan heap.

Bab IV memperdalam landasan formal yang dipakai pada setiap fase, khususnya hierarki Chomsky dan hubungan antara kelas bahasa dengan mesin pengenal. Pemahaman ini akan menjelaskan mengapa lexer cukup memakai otomata hingga, sedangkan parser memerlukan otomata pushdown.

Evaluasi

1. Jelaskan perbedaan mendasar antara compiler dan interpreter, lalu berikan satu situasi ketika interpreter lebih tepat dipilih.
2. Gambarkan pipeline kompilasi dan jelaskan perubahan representasi pada setiap fase.
3. Mengapa analisis semantik memerlukan tabel simbol, sedangkan analisis leksikal tidak?
4. Diberikan program dengan kesalahan pada baris 5, 8, dan 12, rancang strategi pelaporan agar pengguna dapat memperbaiki ketiganya sekaligus.
5. Bandingkan one-pass dan multi-pass untuk bahasa yang mendukung fungsi rekursif dan pemanggilan sebelum deklarasi.
6. Jelaskan bagaimana pemisahan front-end dan back-end membantu portabilitas compiler.
7. Gambarkan activation record untuk fungsi `int f(int a, int b) { int x; int y; return a + b + x + y; }`.

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan konsep (60%), contoh (40%).
- Soal 4: kelengkapan strategi (50%), kualitas pesan (50%).
- Soal 5–6: analisis trade-off (70%), kesimpulan (30%).
- Soal 7: kebenaran struktur frame (80%), kelengkapan (20%).

BAB IV

Teori Bahasa Formal dan Hierarki Chomsky

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 1.4

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. mendefinisikan alfabet, string, bahasa, dan tata bahasa secara formal;
2. membedakan tata bahasa regular, bebas konteks, sensitif konteks, dan tanpa batas;
3. menjelaskan hubungan setiap kelas bahasa dengan mesin pengenal yang bersesuaian;
4. membuktikan bahwa suatu bahasa tidak regular dengan lemma pompa;
5. menjelaskan implikasi hierarki Chomsky bagi desain lexer, parser, dan analisis semantik.

Materi Pokok

- Definisi formal alfabet, string, bahasa, dan operasi bahasa.
- Tata bahasa, derivasi, bentuk kalimat, dan ekivalensi.
- Hierarki Chomsky Tipe 3 sampai Tipe 0.
- Otomata hingga, otomata pushdown, dan mesin Turing.
- Lemma pompa dan implikasi bagi front-end kompilator.

4.1 Alfabet, String, dan Bahasa Secara Formal

4.1.1 Definisi Dasar

Definisi 4.1 (Alfabet dan string)**.** Alfabet Σ adalah himpunan berhingga dan tidak kosong dari simbol. String adalah barisan berhingga simbol dari Σ. Panjang string w

dilambangkan $|w|$. String kosong ε memiliki panjang nol. Himpunan seluruh string atas Σ dilambangkan Σ^*.

Operasi dasar pada string adalah konkatenasi, pembalik, dan pengambilan prefiks atau sufiks. Konkatenasi bersifat asosiatif dan memiliki identitas ε. Monoid bebas atas Σ adalah $(\Sigma^*, \cdot, \varepsilon)$.

Definisi 4.2 (Bahasa)**.** Bahasa L atas alfabet Σ adalah himpunan bagian dari Σ^*, yaitu $L \subseteq \Sigma^*$. Anggota L disebut string yang diterima atau kata dalam bahasa.

Karena bahasa adalah himpunan, operasi himpunan berlaku. Selain itu, terdapat operasi konkatenasi L_1L_2, bintang Kleene L^*, dan positive closure L^+. Operasi-operasi ini mempertahankan sifat tertentu, yang dibahas pada Bagian 4.3.

4.1.2 Bahasa sebagai Masalah Keputusan

Setiap bahasa L mendefinisikan masalah keputusan: diberikan $w \in \Sigma^*$, tentukan apakah $w \in L$. Dalam konteks kompilasi, *lexer* menjawab masalah ini untuk bahasa token, dan *parser* menjawabnya untuk bahasa program.

Kelas bahasa dibedakan oleh jenis mesin yang diperlukan untuk menjawab masalah tersebut. Bahasa regular dapat diputuskan oleh otomata hingga; bahasa bebas konteks oleh otomata pushdown; bahasa sensitif konteks oleh otomata terbatas linear; dan bahasa rekursif terbilang oleh mesin Turing.

4.1.3 Keterhitungan

Himpunan Σ^* untuk alfabet berhingga bersifat terbilang (*countably infinite*). Namun, himpunan semua bahasa atas Σ, yaitu $\mathcal{P}(\Sigma^*)$, bersifat tidak terbilang. Akibatnya, sebagian besar bahasa tidak dapat dikenali oleh mesin apa pun yang dapat dinyatakan secara berhingga. Hierarki Chomsky membatasi perhatian pada kelas-kelas bahasa yang memiliki model pengenal berhingga dan berguna secara praktis [5].

4.1.4 Relasi dan Homomorfisme

Homomorfisme $h : \Sigma^* \to \Gamma^*$ adalah fungsi yang memetakan setiap simbol ke string dan diperluas ke seluruh string dengan $h(\varepsilon) = \varepsilon$ dan $h(xa) = h(x)h(a)$. Homomorfisme berguna untuk mentransformasi bahasa, misalnya menghapus komentar dari program sumber.

Sifat tertutup (*closure properties*) suatu kelas bahasa menentukan operasi yang dapat dilakukan tanpa keluar dari kelas tersebut. Kelas regular tertutup terhadap gabungan, irisan, komplemen, konkatenasi, dan bintang Kleene. Kelas bebas konteks tertutup terhadap gabungan, konkatenasi, dan bintang Kleene, tetapi tidak tertutup terhadap irisan dan komplemen. Sifat ini penting saat menggabungkan spesifikasi token atau tata bahasa.

4.2 Tata Bahasa dan Derivasi: Definisi Formal

Definisi 4.3 (Tata bahasa)**.** Tata bahasa G adalah empat-tupel

$$G = (V, T, P, S),$$

dengan V himpunan berhingga nonterminal, T himpunan berhingga terminal, P himpunan berhingga aturan produksi, dan $S \in V$ simbol awal. Himpunan V dan T saling lepas.

Aturan produksi berbentuk $\alpha \to \beta$, dengan $\alpha \in (V \cup T)^+$, $\beta \in (V \cup T)^*$, dan α memuat paling sedikit satu nonterminal. Jika $\alpha = A \in V$, aturan disebut bebas konteks.

Definisi 4.4 (Derivasi)**.** Jika $\alpha A \beta$ adalah string dan $A \to \gamma \in P$, maka $\alpha A \beta \Rightarrow \alpha \gamma \beta$ adalah satu langkah derivasi. Notasi $\Rightarrow^*$ menyatakan nol atau lebih langkah, dan $\Rightarrow^+$ menyatakan satu atau lebih langkah.

Bentuk kalimat adalah string yang dapat diturunkan dari S. Bahasa yang dibangkitkan G adalah

$$L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}.$$

Dua tata bahasa G_1 dan G_2 ekivalen jika $L(G_1) = L(G_2)$.

4.2.1 Derivasi Leftmost dan Rightmost

Derivasi leftmost selalu mengganti nonterminal paling kiri. Derivasi rightmost mengganti nonterminal paling kanan. Untuk tata bahasa yang tidak ambigu, keduanya menghasilkan pohon urai yang sama. Ambiguitas terjadi jika satu string memiliki dua pohon urai berbeda; hal ini dibahas pada Bab IX.

Contoh 4.5 (Derivasi leftmost dan rightmost)**.** Diberikan tata bahasa

$$S \to AB, \quad A \to aA \mid a, \quad B \to bB \mid b.$$

String $aabb$ dapat diturunkan secara leftmost:

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb.$$

Derivasi rightmost:

$$S \Rightarrow AB \Rightarrow AbB \Rightarrow Abb \Rightarrow aAbb \Rightarrow aabb.$$

Kedua derivasi menghasilkan pohon urai yang sama.

4.2.2 Bentuk Normal

Bentuk normal menyederhanakan aturan produksi tanpa mengubah bahasa. Dua bentuk normal yang penting adalah bentuk normal Chomsky (CNF) dan bentuk normal Greibach (GNF). Pada CNF, setiap aturan berbentuk $A \to BC$ atau $A \to a$. Pada GNF, setiap aturan berbentuk $A \to a\alpha$, dengan a terminal.

CNF berguna untuk pembuktian dan algoritma parsing dinamis seperti CYK. GNF berguna untuk konstruksi parser top-down karena setiap langkah langsung menghasilkan terminal di depan. Transformasi ke bentuk normal tidak selalu diperlukan dalam implementasi praktis, tetapi membantu memahami sifat tata bahasa.

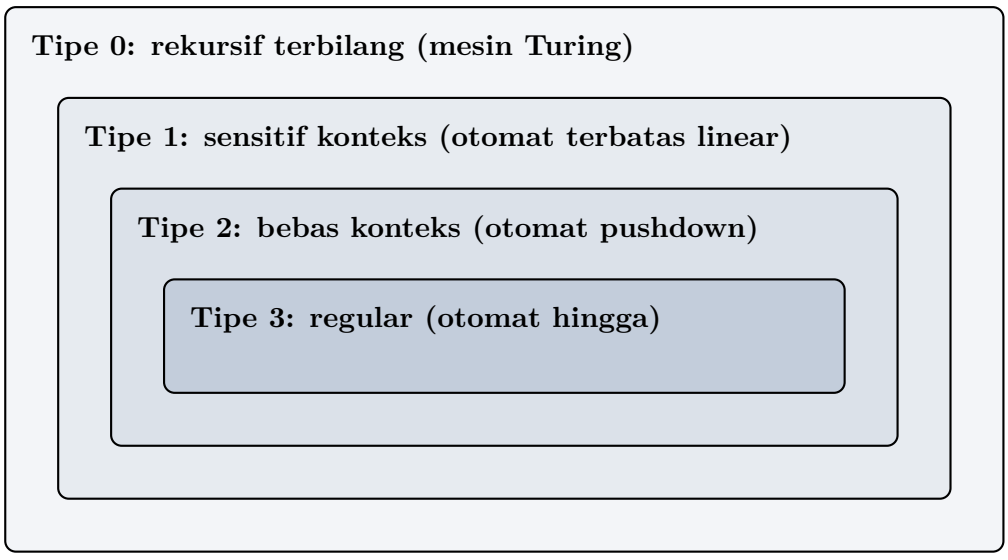
4.2.3 Ekivalensi dan Penyederhanaan

Tata bahasa dapat disederhanakan dengan menghapus produksi ε, produksi unit $A \to B$, dan simbol yang tidak berguna. Penyederhanaan ini mempertahankan bahasa dan mempermudah analisis. Sebagai contoh, produksi unit dapat dihilangkan dengan menyalin aturan dari nonterminal tujuan.

Penyederhanaan juga membantu mendeteksi masalah desain. Jika sebuah nonterminal tidak pernah mencapai string terminal, ia tidak berguna. Jika sebuah nonterminal tidak dapat dicapai dari simbol awal, ia juga tidak berguna. Kedua kondisi ini sering muncul saat mahasiswa menulis tata bahasa untuk bahasa yang kompleks.

4.3 Hierarki Chomsky Tipe 3 sampai Tipe 0

Noam Chomsky mengklasifikasikan tata bahasa berdasarkan bentuk aturan produksinya. Setiap tipe bersesuaian dengan kelas bahasa dan mesin pengenal tertentu. Semakin longgar bentuk aturan, semakin besar daya ekspresif bahasa yang dihasilkan.



Gambar 4.1. Hierarki Chomsky sebagai kelas bahasa yang bersarang

4.3.1 Tipe 3: Tata Bahasa Regular

Tata bahasa regular memiliki aturan berbentuk

$$A \to aB, \quad A \to a, \quad A \to \varepsilon,$$

dengan $A, B \in V$ dan $a \in T$. Bentuk ini disebut *right-linear*. Bentuk *left-linear* $A \to Ba$ juga menghasilkan kelas bahasa yang sama. Tata bahasa regular ekivalen dengan ekspresi reguler dan otomata hingga.

Contoh tata bahasa regular untuk identifier:

$$I \to aI \mid bI \mid \ldots \mid zI \mid A \mid \ldots \mid Z \mid _I,$$
$$A \to aA \mid \ldots \mid zA \mid 0A \mid \ldots \mid 9A \mid \varepsilon.$$

Tata bahasa ini membangkitkan identifier yang dimulai dengan huruf atau garis bawah, diikuti huruf, angka, atau garis bawah.

4.3.2 Tipe 2: Tata Bahasa Bebas Konteks

Tata bahasa bebas konteks (CFG) memiliki aturan $A \to \alpha$, dengan $A \in V$ dan $\alpha \in (V \cup T)^*$. CFG membangkitkan bahasa bebas konteks, yang dikenali oleh otomata pushdown.

Contoh CFG untuk ekspresi aritmetika:

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to (E) \mid \texttt{id}.$$

CFG juga dapat membangkitkan bahasa dengan struktur bersarang, misalnya $S \to aSb \mid \varepsilon$ untuk $\{a^n b^n \mid n \geq 0\}$.

Kelas CFG tertutup terhadap gabungan, konkatenasi, dan bintang Kleene, tetapi tidak tertutup terhadap irisan dan komplemen. Sebagai contoh, irisan $\{a^n b^n c^m\}$ dan $\{a^m b^n c^n\}$ adalah $\{a^n b^n c^n\}$, yang bukan bebas konteks.

4.3.3 Tipe 1: Tata Bahasa Sensitif Konteks

Tata bahasa sensitif konteks memiliki aturan

$$\alpha A \beta \to \alpha \gamma \beta,$$

dengan $\gamma \neq \varepsilon$. Aturan hanya boleh mengganti nonterminal A dalam konteks α dan β, dan panjang string tidak boleh berkurang. Kelas ini dikenali oleh otomata terbatas linear (*linear bounded automaton*).

Contoh bahasa sensitif konteks adalah $\{a^n b^n c^n \mid n \geq 1\}$. Bahasa ini memerlukan pencocokan tiga kelompok dengan jumlah sama, yang tidak dapat dilakukan oleh satu stack.

4.3.4 Tipe 0: Tata Bahasa Tanpa Batas

Tata bahasa tanpa batas (*unrestricted grammar*) memiliki aturan $\alpha \to \beta$ tanpa batasan bentuk, asalkan α memuat paling sedikit satu nonterminal. Kelas ini ekivalen dengan mesin Turing dan mencakup bahasa yang dapat dikenali secara rekursif terbilang.

Sebagian bahasa pada kelas ini tidak dapat diputuskan. Contoh klasiknya adalah masalah penghentian (*halting problem*): tidak ada algoritma yang dapat menentukan apakah program sembarang berhenti untuk semua masukan.

Tabel 4.1. Ringkasan hierarki Chomsky

Tipe	Bahasa	Bentuk aturan	Mesin
3	Regular	$A \to aB$, $A \to a$	Otomata hingga
2	Bebas konteks	$A \to \alpha$	Otomata pushdown
1	Sensitif konteks	$\alpha A \beta \to \alpha \gamma \beta$	Otomata terbatas linear
0	Rekursif terbilang	$\alpha \to \beta$	Mesin Turing

4.4 Otomata dan Daya Ekspresif

4.4.1 Otomata Hingga

Definisi 4.6 (DFA)**.** DFA adalah lima-tupel $M = (Q, \Sigma, \delta, q_0, F)$, dengan Q himpunan state berhingga, Σ alfabet, $q_0 \in Q$ state awal, $F \subseteq Q$ himpunan state penerima, dan $\delta : Q \times \Sigma \to Q$ fungsi transisi total.

DFA membaca string dari kiri ke kanan. Pada setiap langkah, ia menggunakan state saat ini dan simbol masukan untuk menentukan state berikutnya. String diterima jika setelah seluruh simbol dibaca, state berada di F.

NFA mengizinkan transisi ε dan beberapa transisi untuk pasangan state-simbol yang sama. Meskipun tampak lebih kuat, NFA dan DFA ekivalen. Setiap NFA dapat diubah menjadi DFA melalui konstruksi subset, yang dibahas pada Bab VI.

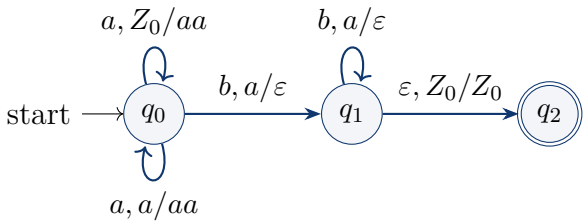
4.4.2 Otomata Pushdown

Definisi 4.7 (PDA)**.** Otomata pushdown (PDA) adalah tujuh-tupel

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

dengan Q himpunan state, Σ alfabet masukan, Γ alfabet stack, δ fungsi transisi, q_0 state awal, Z_0 simbol awal stack, dan F himpunan state penerima.

PDA memiliki stack yang dapat didorong (*push*) dan dilepas (*pop*). Stack memberikan memori tak terbatas tetapi hanya dapat diakses dari atas, sehingga cocok untuk struktur bersarang.



Gambar 4.2. PDA untuk $\{a^n b^n \mid n \geq 1\}$

PDA pada Gambar 4.2 mendorong satu simbol a ke stack untuk setiap a yang dibaca, lalu melepasnya untuk setiap b. Transisi ke state penerima hanya dapat dilakukan ketika stack kembali ke simbol awal Z_0.

4.4.3 Mesin Turing

Mesin Turing memiliki pita tak terbatas, kepala baca-tulis, dan fungsi transisi yang dapat bergerak ke kiri atau ke kanan serta menulis simbol. Mesin Turing menjadi model komputasi umum; menurut tesis Church-Turing, setiap fungsi yang dapat dihitung secara efektif dapat dihitung oleh mesin Turing.

Mesin Turing mengenali bahasa rekursif terbilang. Sebagian bahasa tersebut tidak dapat diputuskan, artinya tidak ada algoritma yang selalu berhenti dan memberikan jawaban benar untuk setiap masukan.

4.4.4 Ekivalensi dan Hubungan

Tabel 4.2. Hubungan tata bahasa, bahasa, dan mesin

Tata bahasa	**Bahasa**	**Mesin**
Regular	Regular	DFA/NFA
Bebas konteks	Bebas konteks	PDA
Sensitif konteks	Sensitif konteks	Otomata terbatas linear
Tanpa batas	Rekursif terbilang	Mesin Turing

Ekivalensi ini bukan sekadar hasil teoretis. Ia menjadi dasar pemilihan alat: ekspresi reguler dan *lexer generator* untuk token, CFG dan *parser generator* untuk struktur, serta analisis atribut untuk semantik. Memaksakan alat yang lebih kuat dari yang diperlukan biasanya menghasilkan solusi yang lebih lambat dan lebih sulit dipelihara.

4.5 Implikasi bagi Desain Bahasa dan Kompilator

4.5.1 Mengapa Leksikal Cukup Tipe 3

Token memiliki struktur yang sederhana: identifier adalah urutan huruf dan angka, bilangan adalah urutan digit, dan operator adalah simbol tunggal atau ganda. Struktur seperti ini dapat dinyatakan dengan ekspresi reguler dan dikenali dengan otomata hingga. Tidak ada kebutuhan untuk menyimpan konteks bersarang pada level token.

Penggunaan otomata hingga memberi keuntungan efisiensi. Lexer dapat berjalan dalam waktu linear terhadap panjang masukan dan menggunakan memori konstan. Alat seperti Flex menghasilkan DFA yang sangat cepat dari spesifikasi ekspresi reguler.

4.5.2 Mengapa Sintaksis Memerlukan Tipe 2

Struktur program memiliki sarang: kurung, blok, dan pemanggilan fungsi. Bahasa seperti $\{a^n b^n\}$ tidak regular karena memerlukan penghitungan yang tidak terbatas. CFG dan otomata pushdown menyediakan stack untuk menangani sarang tersebut.

Namun, CFG juga memiliki batas. CFG tidak dapat memeriksa bahwa sebuah variabel telah dideklarasikan sebelum dipakai, atau bahwa tipe operand cocok. Pemeriksaan seperti ini memerlukan informasi konteks dan tabel simbol, sehingga berada di luar daya ekspresif CFG.

4.5.3 Mengapa Semantik Melampaui Tipe 2

Analisis semantik memerlukan atribut, lingkungan (*environment*), dan aturan tipe. Formalisme yang lebih sesuai adalah *attribute grammar* atau *syntax-directed definition* (SDD). Pada formalisme ini, setiap simpul pohon memiliki atribut yang dihitung dengan aturan. Sebagian atribut bersifat *synthesized* (dihitung dari anak), sebagian *inherited* (diwarisi dari orang tua atau saudara).

Dengan attribute grammar, pemeriksaan tipe dapat dinyatakan secara deklaratif. Sebagai contoh, atribut `type` pada simpul ekspresi dihitung dari tipe operand. Jika operand tidak kompatibel, atribut bernilai `error` dan diagnostik dilaporkan.

4.5.4 Konsekuensi Desain

Tabel 4.3. Pemilihan formalisme untuk setiap fase

Fase	Formalisme	Alasan
Leksikal	Regular/DFA	Struktur token sederhana, efisien
Sintaksis	CFG/PDA	Struktur bersarang memerlukan stack
Semantik	Attribute grammar	Memerlukan konteks dan tipe
Validasi	Aturan dan traversal	Memerlukan analisis lintas simpul

Pemilihan formalisme yang tepat memiliki konsekuensi praktis. Jika sebuah bahasa dirancang dengan tata bahasa yang terlalu kompleks, parser menjadi sulit dibangun dan dipelihara. Jika tata bahasa terlalu sederhana, informasi penting harus diperiksa pada fase berikutnya. Desain bahasa yang baik menempatkan setiap kekangan pada tingkat yang paling sesuai.

Peringatan

Jangan menggunakan ekspresi reguler untuk memeriksa struktur bersarang seperti kurung seimbang. Ekspresi reguler tidak memiliki memori tak terbatas, sehingga akan gagal pada sarang yang dalam. Gunakan parser atau PDA untuk keperluan tersebut.

Contoh

Contoh 4.8 (Mengklasifikasikan tata bahasa)**.** Tentukan tipe setiap tata bahasa berikut.

(a) $S \to aS \mid bS \mid \varepsilon$.

(b) $S \to aSb \mid \varepsilon$.

(c) $S \to aSBC \mid aBC$, $CB \to BC$, $aB \to ab$, $bB \to bb$, $bC \to bc$, $cC \to cc$.

(d) $S \to AB$, $AB \to BA$, $A \to a$, $B \to b$.

Jawaban.

(a) Tipe 3 (regular), karena semua aturan berbentuk $A \to aB$ atau $A \to \varepsilon$.

(b) Tipe 2 (bebas konteks), karena aturan berbentuk $A \to \alpha$ dan menghasilkan $\{a^n b^n\}$.

(c) Tipe 1 (sensitif konteks), karena aturan seperti $CB \to BC$ bergantung pada konteks dan panjang string tidak berkurang. Tata bahasa ini membangkitkan $\{a^n b^n c^n\}$.

(d) Tipe 0 (tanpa batas), karena aturan $AB \to BA$ tidak memenuhi batasan sensitif konteks.

Contoh 4.9 (Lemma pompa untuk bahasa regular)**.** Buktikan bahwa $L = \{a^n b^n \mid n \geq 0\}$ bukan bahasa regular. Misalkan L regular dengan panjang pompa p. Pilih string $w = a^p b^p \in L$. Menurut lemma pompa, w dapat ditulis $w = xyz$ dengan $|xy| \leq p$, $|y| \geq 1$, dan $xy^i z \in L$ untuk semua $i \geq 0$.

Karena $|xy| \leq p$, string y hanya terdiri atas a. Jika $y = a^k$ dengan $k \geq 1$, maka $xy^2z = a^{p+k}b^p$, yang bukan anggota L karena jumlah a dan b tidak sama. Kontradiksi. Jadi L bukan bahasa regular.

Contoh 4.10 (PDA untuk $a^n b^n$)**.** PDA pada Gambar 4.2 bekerja sebagai berikut. Pada state q_0, setiap a didorong ke stack. Setelah membaca b pertama, PDA pindah ke q_1 dan melepas satu simbol a untuk setiap b. Jika stack kembali ke Z_0 tepat setelah b terakhir, PDA pindah ke q_2 dan menerima string. String dengan jumlah a dan b berbeda akan gagal karena stack tidak kembali ke Z_0 pada waktu yang tepat.

Contoh 4.11 (Dari ekspresi reguler ke otomata)**.** Ekspresi reguler $(a|b)^*abb$ menyatakan bahasa seluruh string atas $\{a, b\}$ yang berakhir dengan abb. Otomata hingga untuk bahasa ini memiliki empat state yang mengingat prefiks terpanjang yang cocok: ε, a, ab, dan abb. State terakhir adalah state penerima. Setiap simbol baru memperbarui state sesuai prefiks terpanjang, sehingga otomata hanya memerlukan memori konstan.

Aktivitas Pembelajaran dan Praktikum

Tabel 4.4. Rencana aktivitas Bab IV

Sesi	Bentuk	Kegiatan
1	Ceramah	Definisi formal bahasa dan tata bahasa
2	Latihan	Klasifikasi tata bahasa pada hierarki Chomsky
3	Diskusi	Hubungan tata bahasa, automata, dan fase kompilasi
4	Lokakarya	Membuktikan bahasa tidak regular dan merancang PDA

Lokakarya Klasifikasi

1. Diberikan lima tata bahasa, tentukan tipenya dan jelaskan alasannya.
2. Pilih satu bahasa regular dan satu bahasa bebas konteks. Tuliskan tata bahasa, automata, dan contoh string untuk masing-masing.
3. Buktikan dengan lemma pompa bahwa $\{a^n b^n \mid n \geq 0\}$ bukan regular.

4. Rancang PDA untuk $\{a^n b^{2n} \mid n \geq 0\}$.

5. Diskusikan mengapa analisis semantik tidak cukup dilakukan dengan CFG.

Bab ini tidak menambahkan increment proyek. Materi digunakan sebagai dasar teoretis untuk Bab V–VII (leksikal) dan Bab VIII–XIII (sintaksis).

Latihan dan Refleksi

Latihan

1. Definisikan tata bahasa regular dan berikan dua contoh.

2. Jelaskan perbedaan tata bahasa bebas konteks dan sensitif konteks.

3. Tuliskan tata bahasa untuk $\{a^n b^{2n} \mid n \geq 0\}$.

4. Buktikan bahwa $\{a^n b^n c^n \mid n \geq 1\}$ bukan bahasa bebas konteks.

5. Gambarkan PDA untuk palindrome atas $\{a, b\}$.

6. Jelaskan mengapa mesin Turing lebih kuat daripada PDA.

7. Sebutkan tiga sifat tertutup bahasa regular dan dua sifat yang tidak dimiliki bahasa bebas konteks.

8. Mengapa lexer tidak memerlukan stack, sedangkan parser memerlukannya?

Refleksi

Refleksi

1. Konsep hierarki Chomsky mana yang paling jelas bagi Anda?

2. Apakah Anda dapat menjelaskan mengapa sebuah bahasa tidak regular tanpa menghafal lemma pompa?

3. Bagaimana pemahaman hierarki ini mengubah cara Anda memandang token dan pohon sintaksis?

Asesmen

Asesmen Bab IV menjadi bagian Tugas Teori (8%). Fokusnya adalah ketepatan klasifikasi, pembuktian, dan hubungan teori dengan fase kompilasi.

Tabel 4.5. Instrumen asesmen Bab IV

Instrumen	Indikator	Bobot
Tugas klasifikasi	Tipe tata bahasa dan alasan tepat	8%
Tugas pembuktian	Lemma pompa digunakan dengan benar	8%
Lokakarya	PDA dan tata bahasa dirancang dengan benar	formatif
Kuis	Definisi formal dan hubungan automata	formatif

Kriteria kelulusan: mahasiswa dapat mengklasifikasikan tata bahasa, membuktikan bahasa tidak regular, dan menjelaskan mengapa setiap fase kompilator memerlukan tingkat formalisme yang berbeda.

Checklist Pencapaian Kompetensi

☐ Saya dapat mendefinisikan alfabet, string, dan bahasa.

☐ Saya dapat menulis tata bahasa dalam bentuk empat-tupel.

☐ Saya dapat menghasilkan derivasi leftmost dan rightmost.

☐ Saya dapat membedakan tipe 3, 2, 1, dan 0.

☐ Saya dapat menghubungkan setiap tipe dengan mesin pengenal.

☐ Saya dapat menggunakan lemma pompa untuk membuktikan bahasa tidak regular.

☐ Saya dapat merancang PDA sederhana.

☐ Saya dapat menjelaskan implikasi hierarki bagi lexer, parser, dan analisis semantik.

Rangkuman

Pada bab ini telah dibahas:

- definisi formal alfabet, string, bahasa, dan tata bahasa;
- derivasi, bentuk kalimat, ekivalensi, dan bentuk normal;
- hierarki Chomsky Tipe 3 sampai Tipe 0;
- otomata hingga, otomata pushdown, dan mesin Turing;
- lemma pompa dan implikasi hierarki bagi front-end kompilator.

Bab V memulai perjalanan implementasi dengan analisis leksikal. Kita akan melihat bagaimana ekspresi reguler dipakai untuk mendefinisikan token, dan bagaimana teori Tipe 3 berubah menjadi *lexer* yang bekerja.

Evaluasi

1. Tuliskan definisi formal tata bahasa dan jelaskan setiap komponennya.
2. Klasifikasikan tata bahasa $S \to aSb \mid \varepsilon$ dan jelaskan alasannya.
3. Buktikan bahwa $\{a^n b^n \mid n \geq 0\}$ bukan bahasa regular.
4. Jelaskan mengapa $\{a^n b^n c^n \mid n \geq 1\}$ bukan bahasa bebas konteks.
5. Gambarkan PDA untuk $\{a^n b^{2n} \mid n \geq 0\}$.
6. Bandingkan daya ekspresif DFA, PDA, dan mesin Turing.
7. Mengapa lexer cukup memakai otomata hingga, sedangkan parser memerlukan otomata pushdown?
8. Berikan satu contoh kesalahan yang tidak dapat dideteksi oleh tata bahasa bebas konteks.

Kunci dan Rubrik Singkat

- Soal 1–2: ketepatan definisi dan klasifikasi (70%), penjelasan (30%).
- Soal 3–4: struktur pembuktian (60%), ketepatan argumen (40%).
- Soal 5: kebenaran transisi dan stack (80%), kelengkapan (20%).
- Soal 6–8: ketepatan konsep (60%), contoh (40%).

BAB V

Analisis Leksikal: Token, Lexeme, Pattern, dan Ekspresi Reguler

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 2.1

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan peran dan antarmuka lexer dalam pipeline kompilasi;
2. membedakan token, lexeme, dan pattern beserta atributnya;
3. mendefinisikan dan mengoperasikan ekspresi reguler;
4. menulis spesifikasi token dengan prioritas dan aturan *maximal munch*;
5. menyusun spesifikasi token Subset-C dan header `token.h`.

Materi Pokok

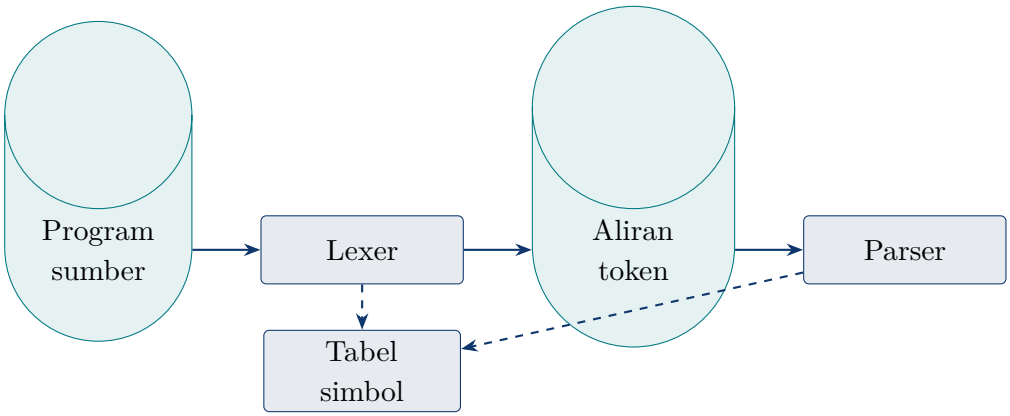
- Peran lexer dan antarmukanya dengan parser.
- Token, lexeme, pattern, dan atribut token.
- Ekspresi reguler: definisi, operasi, dan identitas.
- Notasi, prioritas aturan, dan maximal munch.
- Spesifikasi token Subset-C dan penanganan kesalahan leksikal.

5.1 Peran dan Antarmuka Lexer

5.1.1 Posisi Lexer dalam Pipeline

Lexer, atau *scanner*, adalah fase pertama kompilator. Ia membaca aliran karakter dari program sumber dan mengelompokkannya menjadi token. Token adalah unit bermakna

terkecil yang dikenali parser. Lexer membuang spasi, baris baru, dan komentar, sehingga parser tidak perlu menangani detail leksikal.



Gambar 5.1. Antarmuka lexer, token stream, parser, dan tabel simbol

5.1.2 Antarmuka dengan Parser

Antarmuka yang paling umum adalah *pull model*: parser memanggil fungsi `next_token()` setiap kali membutuhkan token berikutnya. Lexer mempertahankan posisi baca dan mengembalikan token satu per satu. Beberapa lexer menyediakan `peek()` untuk melihat token berikutnya tanpa mengonsumsinya.

Pada *push model*, lexer aktif memanggil parser atau mengisi buffer token. Model ini jarang dipakai pada kompilator sederhana, tetapi muncul pada sistem *streaming* atau *event-based*. Untuk proyek Subset-C, kita memakai pull model karena lebih mudah diuji dan di-debug.

5.1.3 Kontrak Lexer

Lexer memiliki kontrak berikut:

- mengembalikan token berikutnya, atau `TOK_EOF` pada akhir masukan;
- mengisi atribut token: jenis, lexeme, baris, kolom, dan nilai bila ada;
- melewati spasi, tab, baris baru, dan komentar;
- menghasilkan `TOK_ERROR` untuk karakter ilegal tanpa menghentikan seluruh proses;
- tidak mengubah program sumber.

Kontrak ini penting karena parser dan pengujian bergantung pada perilaku yang konsisten. Jika lexer diam-diam menghentikan proses pada kesalahan pertama, parser tidak dapat melakukan pemulihan.

Informasi

Lexer tidak perlu memahami makna program. Tugasnya hanya mengenali bentuk token. Keputusan seperti "apakah variabel sudah dideklarasikan?" adalah tugas analisis semantik, bukan lexer.

5.2 Token, Lexeme, Pattern, dan Atribut

5.2.1 Tiga Konsep yang Sering Tertukar

Definisi 5.1 (Token, lexeme, dan pattern)**. Token** adalah kategori leksikal, misalnya `IDENT`, `INT_LIT`, atau `PLUS`. **Lexeme** adalah string konkret yang cocok dengan kategori tersebut, misalnya `hasil`, `42`, atau `+`. **Pattern** adalah aturan atau ekspresi reguler yang mendefinisikan bentuk lexeme untuk suatu token.

Tabel 5.1. Contoh token, lexeme, dan pattern

Token	Lexeme	Pattern (informal)
`IDENT`	`hasil`, `_x1`	huruf atau garis bawah, diikuti huruf/angka/garis bawah
`INT_LIT`	`0`, `42`	satu atau lebih digit
`FLOAT_LIT`	`3.14`	digit, titik, digit
`PLUS`	`+`	karakter `+`
`LE`	`<=`	karakter `<` diikuti `=`
`STRING_LIT`	`"halo"`	kutip ganda, isi, kutip ganda

Satu token dapat memiliki banyak lexeme. Token `IDENT` memiliki lexeme `a`, `total`, dan `_tmp`. Sebaliknya, satu lexeme biasanya dipetakan ke satu token, kecuali kata kunci yang tampak seperti identifier.

5.2.2 Atribut Token

Token tidak hanya membawa jenis. Ia juga membawa atribut yang dibutuhkan fase berikutnya:

- **lexeme**: string asli;
- **baris** dan **kolom**: lokasi untuk diagnostik;
- **nilai**: bilangan bulat, bilangan pecahan, atau kode karakter;
- **indeks simbol**: referensi ke tabel simbol bila diperlukan.

Token
jenis: `TOK_IDENT` lexeme: `hasil` baris: 4 kolom: 9
nilai: — referensi simbol: `sym#17`

Gambar 5.2. Anatomi sebuah token beserta atributnya

5.2.3 Kata Kunci dan Identifier

Kata kunci seperti `if`, `while`, dan `return` memiliki bentuk yang sama dengan identifier. Ada dua strategi umum. Pertama, lexer mengenali semua kata sebagai identifier, lalu memeriksa tabel kata kunci. Kedua, pattern kata kunci ditempatkan sebelum pattern identifier sehingga aturan kata kunci menang.

Strategi pertama lebih mudah dipelihara karena menambah kata kunci cukup mengubah tabel. Strategi kedua lebih cepat karena keputusan diambil langsung oleh DFA. Pada proyek Subset-C, kita memakai strategi pertama agar tabel kata kunci dapat diuji secara terpisah.

Contoh 5.2 (Token dan lexeme)**.** Untuk potongan program

```
int total = 42;
total = total + 1;
```

token yang dihasilkan adalah `INT`, `IDENT(total)`, `ASSIGN`, `INT_LIT(42)`, `SEMI`, `IDENT(total)`, `ASSIGN`, `IDENT(total)`, `PLUS`, `INT_LIT(1)`, `SEMI`, `EOF`. Lexeme `total` muncul tiga kali, tetapi setiap kemunculan adalah token tersendiri dengan lokasi berbeda.

5.3 Ekspresi Reguler: Definisi dan Operasi

5.3.1 Definisi

Definisi 5.3 (Ekspresi reguler)**.** Diberikan alfabet Σ. Ekspresi reguler atas Σ didefinisikan secara rekursif:

(i) $\varnothing$, ε, dan setiap $a \in \Sigma$ adalah ekspresi reguler;

(ii) jika r dan s ekspresi reguler, maka $r \mid s$, rs, dan r^* juga ekspresi reguler;

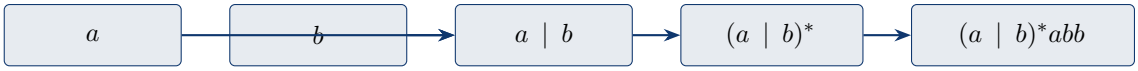
(iii) tidak ada bentuk lain.

Bahasa yang dinyatakan r, ditulis $L(r)$, didefinisikan sebagai himpunan string yang cocok dengan r.

Operator utama adalah gabungan (*union*) $r \mid s$, konkatenasi rs, dan bintang Kleene r^*. Operator r^+ adalah singkatan rr^*, dan $r?$ adalah $r \mid \varepsilon$.

5.3.2 Prioritas Operator

Prioritas operator dari tertinggi ke terendah adalah bintang Kleene, konkatenasi, lalu gabungan. Dengan demikian $ab^* \mid c$ dibaca $(a(b^*)) \mid c$, bukan $a(b^* \mid c)$ atau $(ab)^* \mid c$.



Gambar 5.3. Komposisi ekspresi reguler $(a \mid b)^*abb$

5.3.3 Identitas Aljabar

Ekspresi reguler memenuhi sejumlah identitas yang berguna untuk penyederhanaan:

$$\begin{aligned} r \mid s &= s \mid r, & (rs)t &= r(st), \\ r \mid r &= r, & r\varepsilon &= \varepsilon r = r, \\ \varnothing \mid r &= r, & \varnothing r &= r\varnothing = \varnothing, \\ (r \mid s)t &= rt \mid st, & r(s \mid t) &= rs \mid rt, \\ (r^*)^* &= r^*, & \varepsilon^* &= \varepsilon, \\ \varnothing^* &= \varepsilon, & (r \mid \varepsilon)^* &= r^*. \end{aligned}$$

Identitas ini dipakai oleh alat seperti Flex untuk menyederhanakan otomata dan mempercepat pencocokan. Namun, identitas tidak selalu mempermudah pembacaan manusia. Untuk spesifikasi token, kejelasan lebih penting daripada bentuk paling ringkas.

5.3.4 Ekspresi Reguler untuk Token

Berikut pola umum token:

- identifier: `[A-Za-z_][A-Za-z0-9_]*`
- integer: `[0-9]+`
- float: `[0-9]+\.[0-9]+`
- string: `"([^"\\]|\\.)*"`
- komentar baris: `//[^\n]*`
- komentar blok: `/\*([^*]|\*[^/])*\*/`

Notasi di atas memakai kelas karakter dan escape. Dalam implementasi, detail seperti escape sequence dan karakter Unicode perlu diperhatikan. Untuk Subset-C, kita membatasi diri pada ASCII agar spesifikasi dan pengujian lebih sederhana.

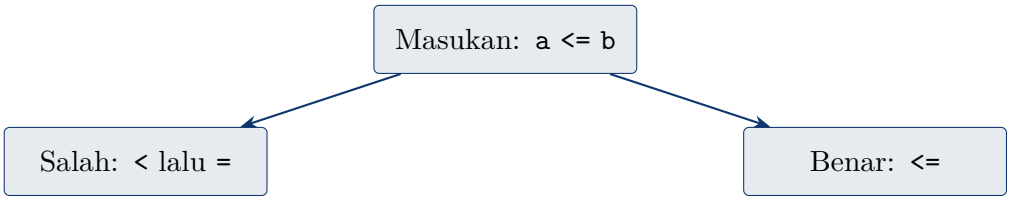
Peringatan

Ekspresi reguler untuk komentar blok di atas adalah penyederhanaan. Implementasi harus menangani kasus seperti `/**/` dan komentar yang tidak ditutup. Uji selalu dengan kasus batas.

5.4 Notasi, Prioritas, dan Spesifikasi Token

5.4.1 Maximal Munch

Ketika beberapa pattern cocok pada posisi yang sama, lexer memilih lexeme terpanjang. Aturan ini disebut *maximal munch* atau *longest match*. Contohnya, pada masukan `<=`, lexer harus memilih token `LE`, bukan token `LT` diikuti `ASSIGN`. Pada masukan `==`, lexer memilih `EQ`, bukan dua kali `ASSIGN`.



Gambar 5.4. Maximal munch memilih `<=` sebagai satu token

Jika dua pattern menghasilkan panjang yang sama, aturan yang muncul lebih dahulu dalam spesifikasi menang. Karena itu, urutan aturan harus dirancang dengan hati-hati. Kata kunci biasanya ditempatkan sebelum identifier, atau identifier diperiksa terhadap tabel kata kunci setelah dicocokkan.

5.4.2 Notasi di Flex

Flex memakai notasi yang mirip ekspresi reguler dengan beberapa tambahan:

- `[A-Za-z]` menyatakan kelas karakter;
- `[^0-9]` menyatakan komplemen kelas;
- `.` mencocokkan karakter apa pun kecuali baris baru;
- `\n`, `\t` adalah escape;
- `"..."` mencocokkan string literal;
- `{m,n}` menyatakan pengulangan;
- `^` dan `$` bergantung pada mode.

Flex juga menyediakan *start condition* dan *trailing context*. Fitur ini memungkinkan lexer menangani mode khusus, misalnya di dalam komentar atau string. Untuk proyek Subset-C, kita membatasi diri pada fitur dasar agar lexer tangan dan lexer Flex dapat dibandingkan secara adil.

5.4.3 Aturan Spesifikasi

Spesifikasi token yang baik memenuhi aturan berikut:

1. setiap token memiliki pattern yang jelas;
2. operator yang lebih panjang ditempatkan sebelum yang lebih pendek;
3. literal float ditempatkan sebelum integer;
4. komentar dan whitespace ditangani lebih dahulu;
5. setiap karakter ilegal memiliki perilaku yang terdefinisi;
6. setiap aturan memiliki contoh dan tes.

Tabel 5.2. Prioritas aturan token Subset-C

Urutan	Kategori	Contoh
1	whitespace dan komentar	spasi, `//`, `/* */`
2	operator dua karakter	`<=`, `>=`, `==`, `!=`, `&&`, `||`
3	operator satu karakter	`+`, `-`, `*`, `/`, `<`, `=`
4	literal float	`3.14`, `0.5`
5	literal integer	`0`, `42`
6	identifier dan kata kunci	`hasil`, `if`
7	literal char dan string	`'a'`, `"halo"`
8	karakter ilegal	`@`, `#`

5.4.4 Kesalahan Leksikal

Kesalahan leksikal tidak boleh membuat lexer berhenti total. Lexer harus menghasilkan token kesalahan, mencatat lokasi, lalu melanjutkan ke karakter berikutnya. Strategi pemulihan sederhana adalah *skip one character*: buang karakter ilegal dan lanjut.

Untuk string atau komentar yang tidak ditutup, lexer dapat mencari akhir baris terdekat, menghasilkan token kesalahan, lalu melanjutkan. Pesan harus menyebut lokasi awal dan jenis masalah, misalnya `baris 7, kolom 3: string tidak ditutup`.

5.5 Spesifikasi Token Subset-C

5.5.1 Kategori Token

Subset-C memiliki tujuh kategori token: kata kunci, identifier, literal, operator, tanda baca, akhir masukan, dan kesalahan. Daftar lengkap terdapat pada `code/subsetc/spec/tokens.md`. Tabel berikut merangkum kategori dan contohnya.

Tabel 5.3. Ringkasan kategori token Subset-C

Kategori	Contoh token	Contoh lexeme
Kata kunci	`TOK_INT`, `TOK_IF`, `TOK_WHILE`	`int`, `if`, `while`
Identifier	`TOK_IDENT`	`hasil`, `_x1`
Literal	`TOK_INT_LIT`, `TOK_STRING_LIT`	`42`, `"halo"`
Operator	`TOK_LE`, `TOK_AND`	`<=`, `&&`
Tanda baca	`TOK_LPAREN`, `TOK_SEMI`	`(`, `;`
Akhir	`TOK_EOF`	—
Kesalahan	`TOK_ERROR`	`@`

5.5.2 Header Token

Header `token.h` mendefinisikan enum `TokenType` dan struktur `Token`. Enum memberi nama simbolik pada setiap jenis token, sedangkan struktur menyimpan lexeme, lokasi, dan nilai. Bagian awal header ditunjukkan pada Kode Program 5.1.

```
8       TOK_EOF = 0,
9       TOK_ERROR,
10
11      /* Kata kunci */
12      TOK_INT,
13      TOK_FLOAT,
14      TOK_CHAR,
15      TOK_VOID,
16      TOK_IF,
17      TOK_ELSE,
18      TOK_WHILE,
19      TOK_FOR,
20      TOK_RETURN,
21      TOK_BREAK,
22      TOK_CONTINUE,
23      TOK_PRINT,
24
```

```
    /* Identifier dan literal */
    TOK_IDENT,
    TOK_INT_LIT,
    TOK_FLOAT_LIT,
    TOK_CHAR_LIT,
    TOK_STRING_LIT,

    /* Operator */
```

Kode Program 5.1. Bagian awal `token.h`: enum jenis token

Setiap token memiliki nama yang dapat dicetak melalui `token_type_name`. Fungsi ini berguna untuk keluaran tes dan pesan diagnostik. Struktur `Token` menyimpan `lexeme` sebagai pointer ke buffer yang dimiliki lexer; pemanggil tidak boleh membebaskannya.

5.5.3 Program Uji

Program `lex_demo.sc` dirancang untuk menguji seluruh kategori token, termasuk komentar, literal, dan operator dua karakter. Potongan awalnya ditunjukkan pada Kode Program 5.2.

```
// Berkas: examples/lex_demo.sc
// Program contoh untuk pengujian lexer.

// Identifier, literal, dan operator
int hitung(int a, int b) {
    int hasil = a + b * 2;
    float rasio = 3.14;
    char tanda = 'x';
    string pesan = "hasil = ";
    print(pesan);
    print(hasil);
    print(rasio);
```

Kode Program 5.2. Potongan program uji `lex_demo.sc`

Program ini akan dipakai pada Bab VII untuk membandingkan keluaran lexer tangan dan lexer Flex. Hasil yang diharapkan adalah aliran token yang identik, kecuali perbedaan lokasi yang disebabkan oleh penanganan baris baru.

5.5.4 Latihan Spesifikasi

Sebelum menulis kode, lengkapi spesifikasi token dengan:

- tabel pattern untuk setiap token;
- daftar kata kunci;

- aturan escape untuk char dan string;
- perilaku untuk karakter ilegal;
- minimal 30 kasus uji, termasuk kasus batas.

Spesifikasi yang lengkap akan mempermudah implementasi dan pengujian. Sebaliknya, spesifikasi yang ambigu biasanya berujung pada lexer yang berperilaku berbeda dari yang diharapkan.

Contoh

Contoh 5.4 (Menokenisasi secara manual)**.** Diberikan program

```
int x = 10;
x = x + 1;
```

Token yang dihasilkan:

Baris:Kolom	Token	Lexeme
1:1	TOK_INT	int
1:5	TOK_IDENT	x
1:7	TOK_ASSIGN	=
1:9	TOK_INT_LIT	10
1:11	TOK_SEMI	;
2:1	TOK_IDENT	x
2:3	TOK_ASSIGN	=
2:5	TOK_IDENT	x
2:7	TOK_PLUS	+
2:9	TOK_INT_LIT	1
2:10	TOK_SEMI	;
3:1	TOK_EOF	—

Perhatikan bahwa spasi tidak menghasilkan token, dan setiap token memiliki lokasi awal.

Contoh 5.5 (Token, lexeme, dan pattern)**.** Untuk token `FLOAT_LIT`, lexeme `3.14` cocok dengan pattern `[0-9]+\.[0-9]+`. Lexeme `42` bukan anggota token ini karena tidak memiliki titik. Sebaliknya, `42` adalah lexeme untuk token `INT_LIT`. Pattern mendefinisikan himpunan lexeme; token adalah nama himpunan tersebut.

Contoh 5.6 (Kata kunci versus identifier)**.** Misalkan tabel kata kunci berisi `int`, `if`, dan `while`. Masukan `int x;` menghasilkan token `INT`, `IDENT(x)`, `SEMI`. Masukan `integer x;` menghasilkan `IDENT(integer)`, `IDENT(x)`, `SEMI`, karena `integer` tidak ada dalam

tabel kata kunci. Lexer tidak menebak berdasarkan awalan; ia mencocokkan seluruh lexeme.

Contoh 5.7 (Maximal munch)**.** Pada masukan `a<=b`, lexer membaca `<` lalu melihat karakter berikutnya `=`. Karena `<=` cocok dan lebih panjang daripada `<`, lexer memilih `LE`. Pada masukan `a<<b`, tidak ada operator `<<` dalam Subset-C, sehingga lexer memilih `LT` dua kali. Aturan maximal munch bekerja pada pattern yang tersedia, bukan pada operator yang mungkin ada dalam bahasa lain.

Contoh 5.8 (Kesalahan leksikal)**.** Masukan `int a = 3.;` mengandung float tanpa digit setelah titik. Lexer menghasilkan `TOK_ERROR` pada posisi titik, mencatat pesan `baris 1, kolom 10: digit diharapkan setelah titik`, lalu melanjutkan. Parser menerima token kesalahan dan dapat melaporkan masalah tanpa menghentikan seluruh kompilasi.

Aktivitas Pembelajaran dan Praktikum

Tabel 5.4. Rencana aktivitas Bab V

Sesi	Bentuk	Kegiatan
1	Ceramah	Peran lexer dan antarmuka token stream
2	Latihan	Menokenisasi program secara manual
3	Diskusi	Ekspresi reguler dan prioritas aturan
4	Praktikum	Menyusun spesifikasi token dan kasus uji

P1-A: Spesifikasi Token dan 30 Kasus Uji

Tujuan. Menyusun spesifikasi token Subset-C yang lengkap sebelum implementasi lexer.

Langkah kerja.

1. Lengkapi `spec/tokens.md` dengan tabel pattern untuk setiap token.
2. Tuliskan daftar kata kunci dan aturan escape untuk char serta string.
3. Buat 30 kasus uji di `tests/lexer/`: 10 kasus normal, 10 kasus batas, dan 10 kasus kesalahan.
4. Untuk setiap kasus, tuliskan berkas `.sc` dan berkas `.expected` dengan format `BARIS:KOLOM JENIS LEXEME`.
5. Periksa bahwa urutan aturan memenuhi maximal munch.

Definisi selesai.

☐ Semua token memiliki pattern.

☐ 30 kasus uji tersedia dan terdokumentasi.

☐ Format keluaran tes konsisten.

☐ Tidak ada aturan yang saling bertentangan.

P1-A belum dinilai secara sumatif. Hasilnya menjadi dasar Praktikum 1 (8%) pada Bab VII.

Latihan dan Refleksi

Latihan

1. Jelaskan perbedaan token, lexeme, dan pattern dengan contoh.
2. Tuliskan pattern untuk identifier yang boleh mengandung garis bawah dan angka, tetapi tidak boleh diawali angka.
3. Tuliskan pattern untuk bilangan pecahan tanpa bagian eksponen.
4. Mengapa operator dua karakter harus ditempatkan sebelum operator satu karakter?
5. Berikan contoh di mana maximal munch mencegah kesalahan parsing.
6. Rancang aturan untuk komentar blok yang tidak bersarang.
7. Jelaskan perilaku lexer ketika menemukan string yang tidak ditutup.
8. Bandingkan strategi kata kunci sebagai identifier yang diperiksa tabel dengan strategi kata kunci sebagai pattern tersendiri.

Refleksi

Refleksi

1. Bagian mana dari spesifikasi token yang paling sulit Anda tulis? Mengapa?
2. Apakah Anda dapat menjelaskan mengapa lexer tidak perlu mengetahui tipe variabel?
3. Kasus batas apa yang sering Anda lupakan saat menulis spesifikasi?

Asesmen

Asesmen Bab V menjadi bagian Tugas Teori (8%) dan prasyarat Praktikum 1 (8%). Fokusnya adalah ketepatan spesifikasi token dan pemahaman maximal munch.

Tabel 5.5. Instrumen asesmen Bab V

Instrumen	Indikator	Bobot
Tugas Teori	Pattern token, prioritas, dan contoh benar	8%
P1-A	Spesifikasi token dan 30 kasus uji lengkap	prasyarat
Kuis	Token, lexeme, pattern, dan maximal munch	formatif

Kriteria kelulusan: mahasiswa dapat menulis spesifikasi token yang tidak ambigu, menjelaskan prioritas aturan, dan menghasilkan kasus uji yang mencakup kasus normal, batas, dan kesalahan.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan peran lexer dalam pipeline.

☐ Saya dapat membedakan token, lexeme, dan pattern.

☐ Saya dapat menyebutkan atribut yang dibawa token.

☐ Saya dapat menulis ekspresi reguler untuk token umum.

☐ Saya dapat menjelaskan prioritas operator ekspresi reguler.

☐ Saya dapat menerapkan aturan maximal munch.

☐ Saya dapat menyusun aturan prioritas token.

☐ Saya dapat menangani kesalahan leksikal tanpa menghentikan proses.

☐ Saya sudah menyelesaikan P1-A: spesifikasi dan 30 kasus uji.

Rangkuman

Pada bab ini telah dibahas:

- peran lexer dan antarmuka pull model dengan parser;
- perbedaan token, lexeme, pattern, dan atribut token;
- definisi, operasi, prioritas, dan identitas ekspresi reguler;
- aturan maximal munch dan prioritas spesifikasi token;
- spesifikasi token Subset-C, header `token.h`, dan penanganan kesalahan leksikal.

Bab VI akan menghubungkan ekspresi reguler dengan mesin pengenal: NFA, DFA, konversi NFA ke DFA, dan minimalisasi DFA. Teori ini menjadi dasar implementasi lexer pada Bab VII.

Evaluasi

1. Definisikan token, lexeme, dan pattern. Berikan satu contoh untuk masing-masing.
2. Tuliskan ekspresi reguler untuk identifier Subset-C.
3. Tuliskan ekspresi reguler untuk bilangan pecahan sederhana.
4. Jelaskan aturan maximal munch dan berikan dua contoh.
5. Mengapa urutan aturan dalam spesifikasi token penting?
6. Rancang aturan untuk komentar baris dan komentar blok.
7. Diberikan masukan `a<=b==c`, tuliskan aliran token yang dihasilkan.
8. Jelaskan bagaimana lexer harus menangani karakter ilegal.

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan definisi dan pattern (70%), contoh (30%).
- Soal 4–6: pemahaman aturan dan prioritas (70%), kelengkapan (30%).
- Soal 7–8: kebenaran token dan perilaku (80%), penjelasan (20%).

BAB VI

Finite Automata: NFA, DFA, Konversi, dan Minimalisasi

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 2.2, Sub-CPMK 2.3

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. mendefinisikan DFA dan NFA beserta cara kerjanya;
2. menghitung epsilon closure dan menyimulasikan NFA;
3. mengonversi NFA menjadi DFA dengan subset construction;
4. meminimalkan DFA dengan penyempurnaan partisi;
5. menghubungkan otomata hingga dengan pengenalan token.

Materi Pokok

- DFA: definisi, tabel transisi, dan simulasi.
- NFA: nondeterminisme dan epsilon closure.
- Konversi NFA ke DFA dengan subset construction.
- Minimalisasi DFA dan partisi ekuivalensi.
- Dari ekspresi reguler ke otomata dan token recognition.

6.1 DFA: Definisi, Tabel Transisi, dan Simulasi

6.1.1 Definisi

Definisi 6.1 (Deterministic Finite Automaton)**.** DFA adalah lima-tupel

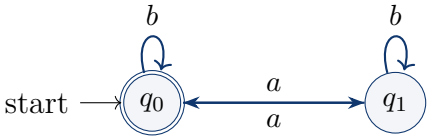
$$M = (Q, \Sigma, \delta, q_0, F),$$

dengan Q himpunan state berhingga, Σ alfabet berhingga, $\delta : Q \times \Sigma \to Q$ fungsi transisi total, $q_0 \in Q$ state awal, dan $F \subseteq Q$ himpunan state penerima.

Sifat deterministik berarti untuk setiap pasangan state dan simbol terdapat tepat satu state tujuan. DFA tidak memiliki transisi ε dan tidak memiliki pilihan ganda. Karena itu, DFA mudah disimulasikan: cukup menyimpan satu state saat ini.

6.1.2 Contoh: Bilangan a Genap

Misalkan $L = \{w \in \{a, b\}^* \mid w \text{ memuat jumlah } a \text{ genap}\}$. DFA untuk bahasa ini memiliki dua state: q_0 untuk jumlah a genap dan q_1 untuk jumlah a ganjil. State awal dan penerima adalah q_0.



Gambar 6.1. DFA untuk bilangan a genap

Tabel 6.1. Tabel transisi DFA pada Gambar 6.1

State	a	b
$\rightarrow *q_0$	q_1	q_0
q_1	q_0	q_1

6.1.3 Fungsi Transisi Diperluas

Fungsi transisi δ didefinisikan untuk satu simbol. Kita memperluasnya menjadi $\hat{\delta} : Q \times \Sigma^* \to Q$ dengan

$$\hat{\delta}(q, \varepsilon) = q, \qquad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a).$$

String w diterima jika $\hat{\delta}(q_0, w) \in F$.

6.1.4 Simulasi

Simulasi DFA sangat sederhana:

1. mulai dari q_0;
2. untuk setiap simbol masukan, pindah ke $\delta(q, a)$;
3. jika transisi tidak terdefinisi, tolak;
4. setelah masukan habis, terima jika state berada di F.

Kompleksitas waktu simulasi adalah $O(n)$, dengan n panjang masukan, dan kompleksitas ruang $O(1)$ selain tabel transisi. Sifat inilah yang membuat DFA cocok untuk lexer berkecepatan tinggi.

6.2 NFA dan Epsilon Closure

6.2.1 Definisi

Definisi 6.2 (Nondeterministic Finite Automaton)**.** NFA adalah lima-tupel

$$M = (Q, \Sigma, \delta, q_0, F),$$

dengan $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. Untuk setiap state dan simbol, NFA dapat memiliki nol, satu, atau beberapa state tujuan.

Nondeterminisme berarti mesin dapat "menebak" langkah yang benar. String diterima jika terdapat setidaknya satu jalur dari q_0 ke state penerima. NFA tidak lebih kuat daripada DFA, tetapi sering lebih mudah ditulis dan dikonstruksi dari ekspresi reguler.

6.2.2 Epsilon Closure

Definisi 6.3 (Epsilon closure)**.** Epsilon closure dari himpunan state S, ditulis ε-closure(S), adalah himpunan state yang dapat dicapai dari S melalui nol atau lebih transisi ε.

Epsilon closure dihitung dengan penelusuran graf. Mulai dari S, ikuti semua transisi ε secara rekursif, dan tambahkan setiap state yang ditemukan. Kompleksitasnya linear terhadap jumlah transisi.

Contoh 6.4 (Epsilon closure)**.** Misalkan NFA memiliki transisi $q_0 \xrightarrow{\varepsilon} q_1$, $q_1 \xrightarrow{\varepsilon} q_2$, dan $q_2 \xrightarrow{a} q_3$. Maka

$$\varepsilon\text{-closure}(\{q_0\}) = \{q_0, q_1, q_2\}.$$

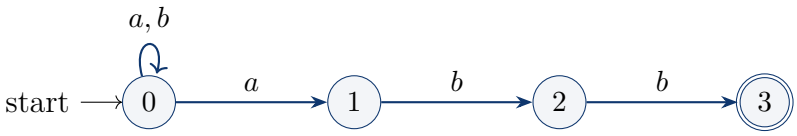
Transisi a dari q_0 harus mempertimbangkan seluruh state dalam closure, sehingga tujuan q_3 dapat dicapai.

6.2.3 Simulasi NFA

Simulasi NFA menyimpan himpunan state, bukan satu state:

1. mulai dengan ε-closure$(\{q_0\})$;
2. untuk setiap simbol a, hitung himpunan state tujuan dari semua state saat ini, lalu ambil epsilon closure-nya;
3. jika himpunan state kosong, tolak;
4. setelah masukan habis, terima jika himpunan state beririsan dengan F.

Kompleksitas waktu simulasi NFA adalah $O(n \cdot |Q|^2)$ pada implementasi sederhana, karena setiap langkah dapat melibatkan banyak state. Untuk lexer, NFA biasanya dikonversi menjadi DFA agar simulasi tetap $O(n)$.



Gambar 6.2. NFA untuk $(a \mid b)^*abb$

NFA pada Gambar 6.2 menerima string yang berakhir dengan *abb*. Pada state 0, mesin dapat tetap di state 0 atau berpindah ke state 1 ketika membaca a. Pilihan ini adalah inti nondeterminisme.

6.3 Konversi NFA ke DFA: Subset Construction

6.3.1 Ide Dasar

Setiap state DFA merepresentasikan himpunan state NFA yang mungkin dicapai setelah membaca prefiks tertentu. Jika NFA berada pada himpunan state S dan membaca simbol a, state DFA berikutnya adalah epsilon closure dari semua tujuan transisi a dari S. Konstruksi ini disebut *subset construction* atau *powerset construction*.

6.3.2 Algoritma

1. State awal DFA adalah ε-closure$(\{q_0\})$.
2. Untuk setiap state DFA S yang belum diproses dan setiap simbol $a \in \Sigma$:
   (a) hitung $T = \varepsilon\text{-closure}(\bigcup_{q \in S} \delta(q, a))$;
   (b) jika T belum ada, tambahkan sebagai state DFA;
   (c) tetapkan transisi $\delta_D(S, a) = T$.
3. State DFA menerima jika $S \cap F \neq \varnothing$.

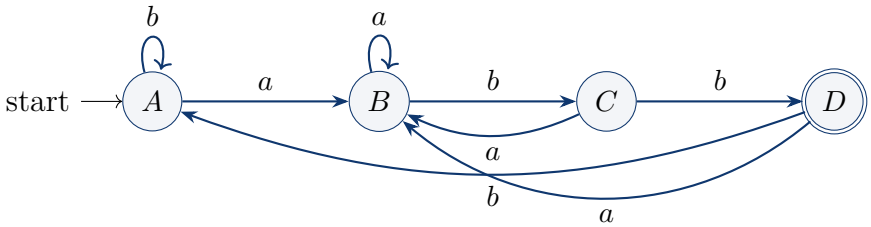
Jumlah state DFA paling banyak $2^{|Q|}$, sehingga konversi dapat meledak secara eksponensial. Dalam praktik, banyak himpunan state tidak dapat dicapai, sehingga DFA hasil konversi biasanya jauh lebih kecil.

6.3.3 Contoh

Untuk NFA pada Gambar 6.2, state awal adalah $A = \{0\}$. Transisinya:

$$
\begin{aligned}
A \xrightarrow{a} \{0,1\} = B, \quad & A \xrightarrow{b} \{0\} = A, \\
B \xrightarrow{a} \{0,1\} = B, \quad & B \xrightarrow{b} \{0,2\} = C, \\
C \xrightarrow{a} \{0,1\} = B, \quad & C \xrightarrow{b} \{0,3\} = D, \\
D \xrightarrow{a} \{0,1\} = B, \quad & D \xrightarrow{b} \{0\} = A.
\end{aligned}
$$

State penerima adalah D, karena memuat state NFA 3.



Gambar 6.3. DFA hasil subset construction dari NFA pada Gambar 6.2

6.3.4 Implementasi

Algoritma subset construction dapat diimplementasikan dengan antrean state DFA dan kamus transisi. Kunci state DFA adalah `frozenset` dari state NFA. Potongan implementasi Python ditunjukkan pada Kode Program 6.1.

```
150     def to_dfa(self):
151         start = self.epsilon_closure({self.start})
152         states = {start}
153         queue = deque([start])
154         delta = {}
155 
156         while queue:
157             current = queue.popleft()
158             for symbol in self.alphabet:
159                 nxt = self.epsilon_closure(self.move(current, symbol))
160                 delta[(current, symbol)] = nxt
161                 if nxt not in states:
162                     states.add(nxt)
163                     queue.append(nxt)
164 
165         accepting = {
166             state for state in states if state & self.accepting
167         }
168         return DFA(states, self.alphabet, delta, start, accepting)
```

Kode Program 6.1. Fungsi `to_dfa` pada `fa_tool.py`

Implementasi ini menghasilkan DFA yang mungkin masih memiliki state tidak terjangkau. Minimalisasi dan penghapusan state tidak terjangkau dilakukan pada tahap berikutnya.

6.4 Minimalisasi DFA

6.4.1 State yang Ekuivalen

Definisi 6.5 (Ekuivalensi state)**.** Dua state p dan q ekuivalen jika untuk setiap string w, $\hat{\delta}(p, w) \in F$ tepat ketika $\hat{\delta}(q, w) \in F$. State yang tidak ekuivalen disebut terbedakan.

Minimalisasi DFA berarti menggabungkan state-state ekuivalen sehingga diperoleh DFA dengan jumlah state paling sedikit yang mengenali bahasa yang sama. DFA minimal bersifat unik untuk suatu bahasa regular (hingga penamaan state).

6.4.2 Penyempurnaan Partisi

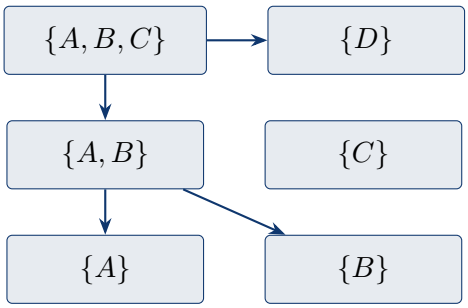
Algoritma yang umum adalah penyempurnaan partisi:

1. partisi awal: $\{F, Q \setminus F\}$;
2. untuk setiap grup, kelompokkan state berdasarkan vektor tujuan transisinya terhadap grup saat ini;
3. ulangi sampai tidak ada grup yang terpecah;
4. gabungkan state dalam grup yang sama.

Contoh 6.6 (Minimalisasi DFA *abb*)**.** Untuk DFA pada Gambar 6.3, partisi awal adalah $\{A, B, C\}$ dan $\{D\}$. Pada iterasi pertama:

- A dan B memiliki transisi b ke grup $\{A, B, C\}$, sedangkan C bertransisi b ke D. Jadi C terpisah.
- A dan B dipisahkan karena $A \xrightarrow{b} A$ (grup $\{A, B\}$) sedangkan $B \xrightarrow{b} C$ (grup $\{C\}$).

Hasil akhirnya adalah empat grup $\{A\}, \{B\}, \{C\}, \{D\}$. DFA tersebut sudah minimal.



Gambar 6.4. Penyempurnaan partisi pada DFA $(a \mid b)^*abb$

6.4.3 Kompleksitas

Implementasi sederhana penyempurnaan partisi memiliki kompleksitas $O(|Q|^2|\Sigma|)$ untuk DFA dengan Q state. Algoritma Hopcroft menurunkan kompleksitas menjadi $O(|Q| \log |Q| \cdot |\Sigma|)$. Untuk lexer praktis, ukuran DFA biasanya kecil, sehingga implementasi sederhana sudah memadai.

6.4.4 Implementasi

Fungsi `minimize` pada `fa_tool.py` melengkapi DFA dengan sink state, membuang state tidak terjangkau, lalu menyempurnakan partisi. Potongan awalnya ditunjukkan pada Kode Program 6.2.

```
36      def minimize(self):
37          """Minimalisasi DFA dengan penyempurnaan partisi."""
38          sink = "__sink__"
39          states = set(self.states) | {sink}
40          delta = {}
41          for state in states:
42              for symbol in self.alphabet:
43                  delta[(state, symbol)] = self.delta.get(
44                      (state, symbol), sink
45                  )
46
47          # Buang state yang tidak dapat dicapai.
48          reachable = {self.start}
49          queue = deque([self.start])
50          while queue:
51              state = queue.popleft()
52              for symbol in self.alphabet:
```

Kode Program 6.2. Awal fungsi `minimize` pada `fa_tool.py`

Sink state diperlukan agar setiap state memiliki transisi untuk semua simbol. Tanpa sink state, dua state yang sama-sama menolak suatu simbol dapat dianggap berbeda

secara keliru.

6.5 Dari Ekspresi Reguler ke Automata dan Token Recognition

6.5.1 Konstruksi Thompson

Konstruksi Thompson mengubah ekspresi reguler menjadi NFA dengan transisi epsilon. Setiap ekspresi reguler memiliki NFA dengan satu state awal dan satu state penerima. Aturan dasarnya:

Tabel 6.2. Aturan konstruksi Thompson

Ekspresi	Konstruksi
$\varnothing$	State awal dan penerima tanpa jalur
ε	Transisi epsilon dari awal ke penerima
a	Transisi a dari awal ke penerima
$r \mid s$	State awal baru bercabang ke NFA r dan s
rs	State penerima r dihubungkan ke awal s
r^*	Loop epsilon di sekitar NFA r

NFA hasil konstruksi mungkin memiliki banyak transisi epsilon. Setelah itu, subset construction menghasilkan DFA, dan minimalisasi menghilangkan state yang ekuivalen. Alur lengkapnya adalah:

$$\text{regex} \rightarrow \text{NFA Thompson} \rightarrow \text{DFA} \rightarrow \text{DFA minimal}.$$

6.5.2 Epsilon Closure dalam Kode

Epsilon closure adalah inti simulasi NFA dan subset construction. Implementasinya menggunakan penelusuran graf. Potongan kode Python ditunjukkan pada Kode Program 6.3.

```
127     def epsilon_closure(self, states):
128         closure = set(states)
129         stack = list(states)
130         while stack:
131             state = stack.pop()
132             for nxt in self.delta.get((state, EPSILON), set()):
133                 if nxt not in closure:
134                     closure.add(nxt)
135                     stack.append(nxt)
```

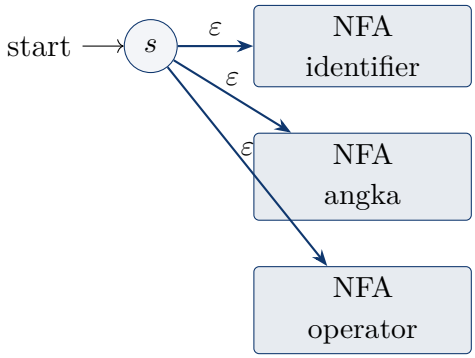
```
136         return frozenset(closure)
```

Kode Program 6.3. Fungsi `epsilon_closure` pada `fa_tool.py`

Fungsi ini mengembalikan `frozenset` agar dapat dipakai sebagai kunci kamus transisi DFA. Penggunaan himpunan beku juga mencegah modifikasi tidak sengaja.

6.5.3 Menggabungkan Beberapa Token

Lexer biasanya mengenali banyak token sekaligus. Caranya adalah membuat NFA terpisah untuk setiap pattern, lalu menambahkan state awal baru dengan transisi epsilon ke setiap NFA. State penerima setiap NFA diberi label jenis token. Jika beberapa token cocok pada panjang yang sama, label dengan prioritas tertinggi yang dipilih.



Gambar 6.5. Menggabungkan NFA token dengan state awal baru

6.5.4 Maximal Munch pada DFA

DFA gabungan mengenali seluruh token. Untuk menerapkan maximal munch, lexer menjalankan DFA selama masih ada transisi yang mungkin, sambil mengingat posisi terakhir yang berada di state penerima. Ketika transisi berikutnya tidak ada, lexer kembali ke posisi terakhir tersebut dan mengeluarkan token. Sisa karakter diproses pada pemanggilan berikutnya.

Pendekatan ini memerlukan *rollback* pada buffer masukan. Karena itu, lexer biasanya menyimpan seluruh berkas atau blok besar di memori, bukan membaca karakter satu per satu dari disk.

6.5.5 Dari Otomata ke Lexer

DFA gabungan dapat direpresentasikan sebagai tabel transisi dua dimensi: baris adalah state, kolom adalah simbol, dan isi sel adalah state tujuan. Tabel ini dapat dikompilasi menjadi array atau *switch* besar. Lexer tangan biasanya memakai *switch* untuk keterbacaan, sedangkan lexer generator memakai tabel untuk kecepatan.

Bab VII akan mengimplementasikan kedua pendekatan: lexer tangan dalam C dan lexer Flex. Perbandingan keduanya menunjukkan trade-off antara kontrol dan produktivitas.

Contoh

Contoh 6.7 (Menelusuri DFA)**.** Gunakan DFA pada Gambar 6.1 untuk memeriksa string *abba*. State awal q_0. Membaca a: pindah ke q_1. Membaca b: tetap di q_1. Membaca b: tetap di q_1. Membaca a: pindah ke q_0. State akhir q_0 adalah state penerima, sehingga *abba* diterima. Jumlah a pada string adalah dua (genap), sesuai definisi bahasa.

Contoh 6.8 (Epsilon closure bertingkat)**.** Misalkan NFA memiliki transisi $q_0 \xrightarrow{\varepsilon} q_1$, $q_1 \xrightarrow{\varepsilon} q_2$, $q_2 \xrightarrow{a} q_3$, $q_3 \xrightarrow{\varepsilon} q_4$. Maka

$$\varepsilon\text{-closure}(\{q_0\}) = \{q_0, q_1, q_2\}.$$

Setelah membaca a, himpunan state menjadi $\{q_3\}$, lalu epsilon closure-nya menjadi $\{q_3, q_4\}$. Jika q_4 adalah state penerima, string a diterima.

Contoh 6.9 (Subset construction)**.** Untuk NFA dengan transisi $q_0 \xrightarrow{a} \{q_0, q_1\}$, $q_0 \xrightarrow{b} \{q_0\}$, $q_1 \xrightarrow{b} \{q_2\}$, dan q_2 penerima:

- state awal $S_0 = \{q_0\}$;
- $S_0 \xrightarrow{a} \{q_0, q_1\} = S_1$;
- $S_0 \xrightarrow{b} \{q_0\} = S_0$;
- $S_1 \xrightarrow{a} \{q_0, q_1\} = S_1$;
- $S_1 \xrightarrow{b} \{q_0, q_2\} = S_2$;
- $S_2 \xrightarrow{a} \{q_0, q_1\} = S_1$;
- $S_2 \xrightarrow{b} \{q_0\} = S_0$.

State penerima adalah S_2, karena memuat q_2. DFA hasilnya memiliki tiga state.

Contoh 6.10 (Minimalisasi)**.** Diberikan DFA dengan state $\{0, 1, 2, 3\}$, state penerima $\{3\}$, dan transisi:

$$0 \xrightarrow{a} 1, \quad 0 \xrightarrow{b} 2, \quad 1 \xrightarrow{a} 1, \quad 1 \xrightarrow{b} 3, \quad 2 \xrightarrow{a} 1, \quad 2 \xrightarrow{b} 2, \quad 3 \xrightarrow{a} 1, \quad 3 \xrightarrow{b} 2.$$

Partisi awal $\{0, 1, 2\}$ dan $\{3\}$. State 1 dan 2 memiliki transisi b ke grup berbeda (1 ke 3, 2 ke 2), sehingga terpisah. State 0 dan 1 juga terpisah karena transisi b-nya berbeda. Hasil akhirnya empat grup; DFA sudah minimal.

Contoh 6.11 (Token recognition)**.** Misalkan DFA gabungan memiliki state penerima untuk identifier dan angka. Pada masukan `abc123`, DFA membaca seluruh string dan berhenti di state penerima identifier. Pada masukan `123abc`, DFA membaca `123` sebagai angka, lalu berhenti karena transisi berikutnya tidak ada. Lexer mengeluarkan token angka `123` dan memproses `abc` pada pemanggilan berikutnya. Maximal munch memastikan `123` tidak dipecah menjadi tiga token digit.

Aktivitas Pembelajaran dan Praktikum

Tabel 6.3. Rencana aktivitas Bab VI

Sesi	Bentuk	Kegiatan
1	Ceramah	DFA, NFA, dan epsilon closure
2	Latihan	Menelusuri string pada DFA dan NFA
3	Lokakarya	Subset construction dan minimalisasi
4	Praktikum	P1-B: DFA token dan prototipe scanner

P1-B: DFA Token dan Prototipe Scanner

Tujuan. Mengubah spesifikasi token menjadi DFA dan prototipe scanner berbasis tabel.

Langkah kerja.

1. Untuk setiap kategori token pada `spec/tokens.md`, gambar NFA atau DFA yang mengenali kategori tersebut.
2. Gabungkan DFA token menjadi satu DFA dengan state awal baru dan prioritas yang jelas.
3. Implementasikan simulator DFA berbasis tabel di Python atau C.
4. Uji simulator dengan 30 kasus uji dari P1-A.
5. Bandingkan hasilnya dengan `fa_tool.py`; catat perbedaan perilaku.

Definisi selesai.

☐ Setiap token memiliki DFA atau NFA.

☐ DFA gabungan menangani maximal munch.

☐ Simulator lulus 30 kasus uji.

☐ Perbedaan dengan `fa_tool.py` terdokumentasi.

P1-B menjadi bagian dari Praktikum 1 (8%) yang dilaporkan pada Bab VII.

Latihan dan Refleksi

Latihan

1. Gambarkan DFA untuk bahasa string atas $\{a, b\}$ yang memuat jumlah b kelipatan tiga.
2. Hitung epsilon closure dari $\{q_0\}$ pada NFA dengan transisi $q_0 \xrightarrow{\varepsilon} q_1$, $q_1 \xrightarrow{\varepsilon} q_2$, dan $q_2 \xrightarrow{a} q_3$.
3. Lakukan subset construction untuk NFA pada Latihan 2.
4. Minimalisasi DFA yang dihasilkan.
5. Jelaskan mengapa jumlah state DFA hasil subset construction paling banyak $2^{|Q|}$.
6. Rancang NFA untuk $(ab \mid ba)^*$.
7. Terapkan konstruksi Thompson pada $(a \mid b)^*abb$.
8. Mengapa lexer memerlukan rollback pada maximal munch?

Refleksi

Refleksi

1. Apakah Anda dapat menjelaskan perbedaan DFA dan NFA tanpa melihat catatan?
2. Bagian mana dari subset construction yang paling sulit dikerjakan manual?
3. Mengapa minimalisasi penting untuk ukuran tabel lexer?

Asesmen

Asesmen Bab VI mencakup Tugas Teori (8%) dan bagian P1-B. Fokusnya adalah kemampuan mengonversi dan meminimalkan otomata.

Tabel 6.4. Instrumen asesmen Bab VI

Instrumen	Indikator	Bobot
Tugas Teori	DFA/NFA, subset construction, minimalisasi	8%
P1-B	DFA token dan prototipe scanner	bagian P1
Kuis	Epsilon closure dan simulasi	formatif

Kriteria kelulusan: mahasiswa dapat mengonversi NFA menjadi DFA secara manual, meminimalkan DFA, dan menjelaskan hubungannya dengan pengenalan token.

Checklist Pencapaian Kompetensi

☐ Saya dapat mendefinisikan DFA dan NFA.

☐ Saya dapat membuat tabel transisi DFA.

☐ Saya dapat menghitung epsilon closure.

☐ Saya dapat menyimulasikan NFA dengan himpunan state.

☐ Saya dapat melakukan subset construction.

☐ Saya dapat meminimalkan DFA dengan partisi.

☐ Saya dapat menjelaskan konstruksi Thompson.

☐ Saya dapat menggabungkan beberapa token menjadi satu DFA.

☐ Saya sudah menyelesaikan P1-B.

Rangkuman

Pada bab ini telah dibahas:

- DFA, tabel transisi, fungsi transisi diperluas, dan simulasi;
- NFA, nondeterminisme, epsilon closure, dan simulasi;
- subset construction untuk mengonversi NFA ke DFA;
- minimalisasi DFA dengan penyempurnaan partisi;
- konstruksi Thompson dan penggabungan token untuk lexer.

Bab VII akan mengimplementasikan seluruh teori ini menjadi lexer yang bekerja. Kita akan menulis lexer tangan dalam C, lexer Flex, dan driver yang menghasilkan aliran token untuk parser.

Evaluasi

1. Definisikan DFA dan NFA, lalu sebutkan perbedaannya.
2. Gambarkan DFA untuk $(a \mid b)^* abb$.
3. Hitung epsilon closure dari $\{q_0\}$ pada NFA yang diberikan.

4. Lakukan subset construction pada NFA tersebut.
5. Minimalisasi DFA hasil konversi.
6. Jelaskan konstruksi Thompson untuk r^*.
7. Mengapa lexer perlu menyimpan posisi terakhir yang cocok?
8. Jelaskan hubungan antara DFA minimal dan ukuran tabel lexer.

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan definisi dan perhitungan (70%), penjelasan (30%).
- Soal 4–5: kebenaran konversi (60%), minimalisasi (40%).
- Soal 6–8: ketepatan konsep (60%), contoh (40%).

BAB VII

Implementasi Lexical Analyzer

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 2.4

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan arsitektur scanner dengan buffer dan lookahead;
2. mengimplementasikan lexer tangan dalam C;
3. mengimplementasikan lexer dengan Flex;
4. menangani kesalahan leksikal dan menghasilkan diagnostik;
5. mengintegrasikan lexer ke dalam proyek Subset-C (M1).

Materi Pokok

- Buffer masukan, sentinel, lexeme begin, dan forward pointer.
- Struktur dan implementasi lexer tangan dalam C.
- Lexer generator-based dengan Flex.
- Penanganan kesalahan leksikal dan pemulihan.
- Driver, pengujian, dan integrasi proyek.

7.1 Arsitektur Scanner: Buffer, Sentinel, dan Lookahead

7.1.1 Mengapa Buffer Diperlukan

Lexer membaca program sumber karakter demi karakter. Membaca satu karakter dari disk pada setiap langkah sangat lambat. Karena itu, lexer biasanya membaca blok besar ke memori, lalu memprosesnya dari buffer. Pendekatan ini mengurangi biaya I/O dan memungkinkan lookahead beberapa karakter tanpa akses disk.

7.1.2 Skema Dua Buffer

Skema klasik menggunakan dua buffer setengah ukuran. Buffer pertama diisi, lalu buffer kedua, dan seterusnya. Di akhir setiap buffer ditempatkan sentinel `EOF`. Ketika pointer mencapai sentinel, lexer mengisi buffer berikutnya dan mengatur ulang pointer.



Gambar 7.1. Skema dua buffer dengan sentinel di akhir setiap buffer

7.1.3 Lexeme Begin dan Forward

Dua pointer digunakan:

- `lexeme_begin` menandai awal lexeme yang sedang dibaca;
- `forward` bergerak ke depan selama karakter masih cocok dengan pattern.

Ketika `forward` mencapai karakter yang tidak cocok, lexer menghitung panjang lexeme, membuat token, lalu memindahkan `lexeme_begin` ke posisi `forward`. Jika pattern tidak cocok sama sekali, lexer menghasilkan token kesalahan dan memajukan `forward` satu karakter.

7.1.4 Lookahead

Lookahead adalah kemampuan melihat karakter berikutnya tanpa mengonsumsinya. Banyak token memerlukan lookahead satu karakter, misalnya `<` versus `<=`. Lookahead yang lebih panjang diperlukan untuk kasus seperti komentar blok dan string escape. Implementasi sederhana menyediakan `peek()` dan `peek_next()`.

Informasi

Pada proyek Subset-C, seluruh berkas dibaca ke memori. Ini menyederhanakan implementasi dan cukup untuk ukuran program perkuliahan. Untuk compiler produksi, skema dua buffer atau *memory-mapped file* lebih hemat memori.

7.1.5 Kompleksitas

Dengan buffer, setiap karakter diproses paling banyak dua kali: sekali saat dibaca ke buffer dan sekali saat dipindai. Kompleksitas waktu lexer adalah $O(n)$, dengan n panjang masukan. Kompleksitas ruang adalah $O(b)$, dengan b ukuran buffer, ditambah tabel transisi DFA.

7.2 Scanner Hand-Written dalam C

7.2.1 Struktur Data

Lexer tangan menggunakan struktur `Lexer` yang menyimpan sumber, posisi, baris, kolom, dan status kesalahan. Struktur ini didefinisikan pada `lexer.h`, seperti ditunjukkan pada Kode Program 7.1.

```c
// Berkas: include/lexer.h
// Antarmuka lexer tangan untuk Subset-C.

#ifndef SUBSETC_LEXER_H
#define SUBSETC_LEXER_H

#include <stddef.h>
#include <stdio.h>
#include "token.h"

typedef struct {
    const char *source;
    size_t length;
    size_t pos;
    int line;
    int column;
    const char *filename;
    int had_error;
} Lexer;

void lexer_init(Lexer *lexer, const char *source, const char *filename);
Token lexer_next(Lexer *lexer);
void token_print(FILE *out, const Token *token);

#endif /* SUBSETC_LEXER_H */
```

Kode Program 7.1. Antarmuka `lexer.h`

Pemisahan header dan implementasi memudahkan pengujian dan memungkinkan driver lain memakai lexer tanpa mengetahui detail internal.

7.2.2 Operasi Dasar

Tiga operasi dasar menjadi fondasi scanner:

- `peek()` mengembalikan karakter saat ini tanpa memajukan posisi;
- `peek_next()` mengembalikan karakter berikutnya;

- `advance()` memajukan posisi dan memperbarui baris serta kolom.

Ketiga operasi ini menangani batas akhir masukan dengan mengembalikan `EOF`. Semua operasi lain dibangun di atas ketiganya, sehingga logika pelacakan lokasi hanya ditulis sekali.

7.2.3 Melewati Spasi dan Komentar

Fungsi `skip_space` membuang spasi, tab, baris baru, komentar baris, dan komentar blok. Untuk komentar blok yang tidak ditutup, fungsi melaporkan kesalahan dan menandai lexer berada dalam keadaan error. Potongan implementasinya ditunjukkan pada Kode Program 7.2.

```
51 static void skip_space(Lexer *lexer) {
52     for (;;) {
53         int ch = peek(lexer);
54 
55         if (ch == ' ' || ch == '\t' || ch == '\r' || ch == '\n') {
56             advance(lexer);
57             continue;
58         }
59 
60         if (ch == '/' && peek_next(lexer) == '/') {
61             advance(lexer);
62             advance(lexer);
63             while (peek(lexer) != EOF && peek(lexer) != '\n') {
64                 advance(lexer);
65             }
66             continue;
67         }
68 
69         if (ch == '/' && peek_next(lexer) == '*') {
70             int start_line = lexer->line;
71             int start_column = lexer->column;
72             int closed = 0;
73 
74             advance(lexer);
75             advance(lexer);
76             while (peek(lexer) != EOF) {
77                 if (peek(lexer) == '*' && peek_next(lexer) == '/') {
78                     advance(lexer);
79                     advance(lexer);
80                     closed = 1;
81                     break;
82                 }
```

```
83                  advance(lexer);
84              }
85              if (!closed) {
86                  error_at(lexer->filename, start_line, start_column,
87                           "komentar blok tidak ditutup");
88                  lexer->had_error = 1;
89              }
90              continue;
91          }
92
93          break;
94      }
95  }
```

Kode Program 7.2. Fungsi `skip_space` pada `lexer.c`

Perhatikan bahwa komentar tidak menghasilkan token. Lexer hanya memperbarui posisi dan melanjutkan ke karakter berikutnya.

7.2.4 Fungsi Utama `lexer_next`

Fungsi `lexer_next` adalah inti scanner. Alurnya:

1. lewati spasi dan komentar;
2. catat baris, kolom, dan posisi awal;
3. jika akhir masukan, kembalikan `TOK_EOF`;
4. jika huruf atau garis bawah, pindai identifier lalu periksa tabel kata kunci;
5. jika digit, pindai bilangan bulat atau pecahan;
6. jika kutip, pindai char atau string;
7. jika operator, periksa operator dua karakter lalu satu karakter;
8. jika tidak dikenal, hasilkan `TOK_ERROR`.

Bagian awal fungsi, termasuk penanganan identifier, ditunjukkan pada Kode Program 7.3.

```
149 Token lexer_next(Lexer *lexer) {
150     int line;
151     int column;
152     size_t start;
153     int ch;
154
155     skip_space(lexer);
```

```
156
157     line = lexer->line;
158     column = lexer->column;
159     start = lexer->pos;
160     ch = peek(lexer);
161
162     if (ch == EOF) {
163         return make_token(lexer, TOK_EOF, start, line, column);
164     }
165
166     if (isalpha((unsigned char)ch) || ch == '_') {
167         TokenType type;
168
169         advance(lexer);
170         while (isalnum((unsigned char)peek(lexer)) ||
171                peek(lexer) == '_') {
172             advance(lexer);
173         }
174         type = keyword_type(lexer->source + start,
175                             (int)(lexer->pos - start));
176         return make_token(lexer, type, start, line, column);
177     }
```

Kode Program 7.3. Awal `lexer_next`: EOF dan identifier

Penanganan bilangan memisahkan integer dan float berdasarkan ada tidaknya titik yang diikuti digit. Jika titik tidak diikuti digit, titik tidak dikonsumsi sebagai bagian bilangan, sehingga dapat dilaporkan sebagai kesalahan pada langkah berikutnya.

7.2.5 Operator dan Tanda Baca

Operator dua karakter diperiksa lebih dahulu. Potongan kode untuk operator dan penanganan kesalahan ditunjukkan pada Kode Program 7.4.

```
274     advance(lexer);
275
276     switch (ch) {
277     case '+': return make_token(lexer, TOK_PLUS, start, line, column);
278     case '-': return make_token(lexer, TOK_MINUS, start, line, column);
279     case '*': return make_token(lexer, TOK_STAR, start, line, column);
280     case '/': return make_token(lexer, TOK_SLASH, start, line, column);
281     case '%': return make_token(lexer, TOK_PERCENT, start, line, column);
282     case '(': return make_token(lexer, TOK_LPAREN, start, line, column);
283     case ')': return make_token(lexer, TOK_RPAREN, start, line, column);
284     case '{': return make_token(lexer, TOK_LBRACE, start, line, column);
285     case '}': return make_token(lexer, TOK_RBRACE, start, line, column);
```

```
286     case ';': return make_token(lexer, TOK_SEMI, start, line, column);
287     case ',': return make_token(lexer, TOK_COMMA, start, line, column);
288 
289     case '<':
290         if (peek(lexer) == '=') {
291             advance(lexer);
292             return make_token(lexer, TOK_LE, start, line, column);
293         }
294         return make_token(lexer, TOK_LT, start, line, column);
295 
296     case '>':
297         if (peek(lexer) == '=') {
298             advance(lexer);
299             return make_token(lexer, TOK_GE, start, line, column);
300         }
301         return make_token(lexer, TOK_GT, start, line, column);
302 
303     case '=':
304         if (peek(lexer) == '=') {
305             advance(lexer);
306             return make_token(lexer, TOK_EQ, start, line, column);
307         }
308         return make_token(lexer, TOK_ASSIGN, start, line, column);
309 
310     case '!':
311         if (peek(lexer) == '=') {
312             advance(lexer);
313             return make_token(lexer, TOK_NE, start, line, column);
314         }
315         return make_token(lexer, TOK_NOT, start, line, column);
316 
317     case '&':
318         if (peek(lexer) == '&') {
319             advance(lexer);
320             return make_token(lexer, TOK_AND, start, line, column);
321         }
322         error_at(lexer->filename, line, column,
323                  "operator '&' tidak dikenal");
324         lexer->had_error = 1;
325         return make_token(lexer, TOK_ERROR, start, line, column);
326 
327     case '|':
328         if (peek(lexer) == '|') {
329             advance(lexer);
```

```
            return make_token(lexer, TOK_OR, start, line, column);
        }
        error_at(lexer->filename, line, column,
                 "operator '|' tidak dikenal");
        lexer->had_error = 1;
        return make_token(lexer, TOK_ERROR, start, line, column);

    default:
        error_at(lexer->filename, line, column,
                 "karakter tidak dikenal");
        lexer->had_error = 1;
        return make_token(lexer, TOK_ERROR, start, line, column);
```

Kode Program 7.4. Penanganan operator dan karakter ilegal

Setiap cabang mengembalikan token dengan panjang yang tepat. Karena `advance()` sudah dipanggil sebelum `switch`, panjang token mencakup karakter operator. Untuk operator dua karakter, `advance()` dipanggil sekali lagi.

7.2.6 Kata Kunci

Fungsi `keyword_type` membandingkan lexeme dengan tabel kata kunci. Jika tidak ada yang cocok, lexeme dikembalikan sebagai `TOK_IDENT`. Pendekatan ini memisahkan daftar kata kunci dari logika scanning, sehingga menambah kata kunci cukup mengubah tabel.

7.3 Scanner Generator-Based dengan Flex

7.3.1 Struktur Berkas Flex

Berkas Flex (`.l`) terdiri atas tiga bagian yang dipisahkan `%%`:

1. **definisi**: deklarasi, makro, dan kode C;
2. **aturan**: pasangan pattern dan aksi;
3. **kode pengguna**: fungsi tambahan dan `main`.

Flex mengubah aturan menjadi DFA dan menghasilkan fungsi `yylex()` dalam berkas C. Pattern ditulis dengan ekspresi reguler, sedangkan aksi ditulis dalam C.

7.3.2 Definisi dan Makro

Bagian definisi mendefinisikan makro seperti `DIGIT` dan `ID`. Makro ini membuat aturan lebih ringkas dan konsisten. Potongan bagian definisi dan aturan kata kunci ditunjukkan pada Kode Program 7.5.

```
DIGIT   [0-9]
ID      [A-Za-z_][A-Za-z0-9_]*

%%

[ \t\r\n]+              { /* abaikan */ }

"//"[^\n]*              { /* komentar baris */ }

"/*"([^*]|\*+[^*/])*\*+"/" { /* komentar blok */ }

"int"                   { return TOK_INT; }
"float"                 { return TOK_FLOAT; }
"char"                  { return TOK_CHAR; }
"void"                  { return TOK_VOID; }
"if"                    { return TOK_IF; }
"else"                  { return TOK_ELSE; }
"while"                 { return TOK_WHILE; }
"for"                   { return TOK_FOR; }
"return"                { return TOK_RETURN; }
"break"                 { return TOK_BREAK; }
"continue"              { return TOK_CONTINUE; }
"print"                 { return TOK_PRINT; }

"<="                    { return TOK_LE; }
">="                    { return TOK_GE; }
"=="                    { return TOK_EQ; }
"!="                    { return TOK_NE; }
"&&"                    { return TOK_AND; }
"||"                    { return TOK_OR; }
```

Kode Program 7.5. Definisi dan aturan kata kunci pada `lexer_flex.l`

Aturan operator dua karakter ditempatkan sebelum operator satu karakter. Flex memilih pattern terpanjang; jika panjangnya sama, aturan yang lebih awal menang [6]. Urutan ini penting untuk memastikan `<=` tidak dipecah menjadi `<` dan `=`.

7.3.3 Aturan Literal dan Identifier

Aturan untuk literal, identifier, dan karakter ilegal ditunjukkan pada Kode Program 7.6.

```

"+"                     { return TOK_PLUS; }
"-"                     { return TOK_MINUS; }
"*"                     { return TOK_STAR; }
```

```
"/"                     { return TOK_SLASH; }
"%"                     { return TOK_PERCENT; }
"<"                     { return TOK_LT; }
">"                     { return TOK_GT; }
"!"                     { return TOK_NOT; }
"="                     { return TOK_ASSIGN; }
"("                     { return TOK_LPAREN; }
")"                     { return TOK_RPAREN; }
"{"                     { return TOK_LBRACE; }
"}"                     { return TOK_RBRACE; }
";"                     { return TOK_SEMI; }
","                     { return TOK_COMMA; }

{DIGIT}+"."{DIGIT}+     { return TOK_FLOAT_LIT; }
{DIGIT}+                { return TOK_INT_LIT; }

"'"([^'\\]|\\.)"'"     { return TOK_CHAR_LIT; }

"\""([^"\\]|\\.)*"\""   { return TOK_STRING_LIT; }

{ID}                    { return TOK_IDENT; }

.                       { report("karakter tidak dikenal");
                          return TOK_ERROR; }
```

Kode Program 7.6. Aturan operator, literal, identifier, dan kesalahan

Aksi dapat berupa `return TOKEN` atau blok C. Pada lexer ini, aksi hanya mengembalikan jenis token. Nilai literal dan lexeme diambil dari `yytext` oleh driver.

7.3.4 Membangun Lexer Flex

Langkah build:

```
flex -o build/lexer_flex.c src/lexer_flex.l
gcc -Iinclude build/lexer_flex.c src/token.c src/error.c \
    -o build/lexer_flex
```

Perintah pertama menghasilkan kode C dari spesifikasi Flex. Perintah kedua mengompilasi kode tersebut bersama modul token dan error. Langkah ini memerlukan Flex dan GCC yang terpasang.

7.3.5 Perbandingan dengan Lexer Tangan

Tabel 7.1. Perbandingan lexer tangan dan lexer Flex

Aspek	Lexer tangan	Lexer Flex
Kontrol	Penuh atas setiap detail	Terbatas pada aksi
Keterbacaan	Panjang, tetapi eksplisit	Ringkas dan deklaratif
Kualitas kesalahan	Dapat disesuaikan	Perlu kode tambahan
Perubahan pattern	Mengubah kode dan menguji ulang	Mengubah aturan
Ketergantungan	C standar	Flex dan generator
Kinerja	Bergantung implementasi	DFA yang dioptimalkan

Lexer tangan cocok untuk pembelajaran dan sistem yang membutuhkan diagnostik sangat spesifik. Lexer Flex cocok untuk prototipe cepat dan bahasa dengan banyak token. Pada proyek Subset-C, keduanya diimplementasikan agar mahasiswa memahami trade-off-nya.

7.4 Penanganan Kesalahan Leksikal

7.4.1 Jenis Kesalahan

Kesalahan leksikal terjadi ketika masukan tidak sesuai dengan pattern token mana pun. Contoh umum:

- karakter ilegal, misalnya `@` atau `#`;
- string yang tidak ditutup;
- literal karakter kosong atau terlalu panjang;
- komentar blok yang tidak ditutup;
- float tanpa digit setelah titik.

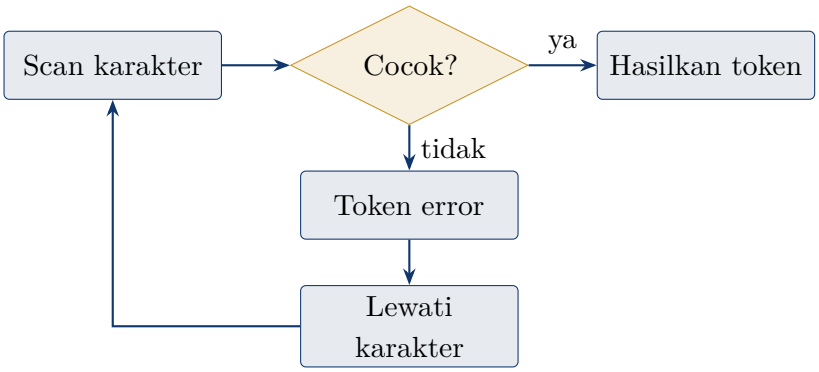
Kesalahan leksikal berbeda dari kesalahan sintaksis. Lexer tidak mengetahui struktur program, sehingga tidak dapat memberikan saran perbaikan yang bergantung pada konteks. Namun, lexer dapat memberikan lokasi yang tepat dan jenis masalah.

7.4.2 Strategi Pemulihan

Lexer yang baik tidak berhenti pada kesalahan pertama. Strategi yang umum:

1. hasilkan token `TOK_ERROR`;
2. catat lokasi dan pesan;
3. lewati satu karakter atau sampai batas aman;
4. lanjutkan scanning.

Untuk string yang tidak ditutup, lexer dapat melewati sampai akhir baris. Untuk komentar blok yang tidak ditutup, lexer dapat melewati sampai akhir berkas. Dalam kedua kasus, lexer harus tetap menghasilkan token agar parser dapat melanjutkan analisis.



Gambar 7.2. Alur pemulihan kesalahan leksikal

7.4.3 Pelaporan Kesalahan

Fungsi `error_at` mencetak pesan dengan format `berkas:baris:kolom: error: pesan`. Implementasinya ditunjukkan pada Kode Program 7.7.

```c
// Berkas: src/error.c
#include <stdio.h>
#include "error.h"

void error_at(const char *filename, int line, int column,
              const char *message) {
    fprintf(stderr, "%s:%d:%d: error: %s\n",
            filename ? filename : "<input>", line, column, message);
}
```

Kode Program 7.7. Fungsi pelaporan `error.c`

Format ini mudah dibaca manusia dan mudah diurai oleh alat. Banyak editor dan IDE dapat melompat ke lokasi kesalahan berdasarkan format tersebut.

7.4.4 Contoh Diagnostik

Tabel 7.2. Contoh pesan kesalahan leksikal

Masukan	Diagnostik
`@`	`baris 1, kolom 1: karakter tidak dikenal`
`"abc`	`baris 1, kolom 1: string tidak ditutup`
`'ab'`	`baris 1, kolom 1: literal karakter terlalu panjang`
`/* abc`	`baris 1, kolom 1: komentar blok tidak ditutup`
`3.`	`baris 1, kolom 2: digit diharapkan setelah titik`

Pesan yang baik menyebut lokasi awal, jenis masalah, dan bila mungkin tindakan yang diharapkan. Hindari pesan generik seperti `error` atau `invalid token`.

7.5 Integrasi ke Proyek: Token Stream dan Driver

7.5.1 Driver

Driver membaca berkas, menginisialisasi lexer, lalu mencetak token sampai `TOK_EOF`. Implementasinya ditunjukkan pada Kode Program 7.8.

```c
// Berkas: src/main_lex.c
// Driver untuk menguji lexer tangan.

#include <stdio.h>
#include <stdlib.h>
#include "lexer.h"
#include "token.h"

static char *read_file(const char *path) {
    FILE *file;
    long size;
    char *buffer;

    file = fopen(path, "rb");
    if (!file) {
        return NULL;
    }
    if (fseek(file, 0, SEEK_END) != 0) {
        fclose(file);
        return NULL;
    }
```

```
    size = ftell(file);
    if (size < 0) {
        fclose(file);
        return NULL;
    }
    rewind(file);

    buffer = (char *)malloc((size_t)size + 1);
    if (!buffer) {
        fclose(file);
        return NULL;
    }
    if (fread(buffer, 1, (size_t)size, file) != (size_t)size) {
        free(buffer);
        fclose(file);
        return NULL;
    }
    buffer[size] = '\0';
    fclose(file);
    return buffer;
}

int main(int argc, char **argv) {
    char *source;
    Lexer lexer;
    Token token;

    if (argc != 2) {
        fprintf(stderr, "penggunaan: %s berkas.sc\n", argv[0]);
        return 2;
    }

    source = read_file(argv[1]);
    if (!source) {
        fprintf(stderr, "tidak dapat membaca %s\n", argv[1]);
        return 2;
    }

    lexer_init(&lexer, source, argv[1]);
```

Kode Program 7.8. Driver `main_lex.c`

Driver ini adalah alat uji, bukan bagian dari compiler final. Pada milestone berikutnya, driver akan diganti oleh parser yang memanggil `lexer_next()` secara langsung.

7.5.2 Format Keluaran

Setiap baris keluaran memiliki format:

$$\text{baris} : \text{kolom} \quad \text{jenis token} \quad \text{lexeme}.$$

Format ini memudahkan perbandingan dengan berkas harapan (`.expected`). Contoh keluaran untuk `basic.sc`:

```
1:1 INT int
1:5 IDENT main
1:9 LPAREN (
1:10 RPAREN )
1:12 LBRACE {
```

7.5.3 Pengujian

Direktori `tests/lexer` memuat empat kelompok tes:

- `basic.sc`: program minimal;
- `operators.sc`: operator dan tanda baca;
- `literals.sc`: literal integer, float, char, dan string;
- `errors.sc`: karakter ilegal, float tidak lengkap, dan string tidak ditutup.

Setiap berkas `.sc` memiliki berkas `.expected`. Skrip `run-tests.bat` menjalankan lexer, membandingkan keluaran, dan melaporkan tes yang gagal.

7.5.4 M1: Definisi Selesai

☐ Lexer tangan dapat dikompilasi tanpa peringatan.

☐ Lexer Flex dapat dihasilkan dan dikompilasi.

☐ Empat kelompok tes tersedia dan lulus.

☐ Kesalahan leksikal menghasilkan diagnostik yang informatif.

☐ Laporan singkat berisi desain, hasil tes, dan kendala.

Tabel 7.3. Artefak M1 dan statusnya

Artefak	Status	Keterangan
`token.h`	selesai	enum dan struktur token
`lexer.h`	selesai	antarmuka lexer
`lexer.c`	selesai	scanner tangan
`lexer_flex.l`	selesai	scanner Flex
`error.c`	selesai	pelaporan kesalahan
`main_lex.c`	selesai	driver uji
`tests/lexer`	selesai	empat kelompok tes

7.5.5 Catatan Verifikasi

Lingkungan pengembangan buku ini tidak memiliki GCC, Flex, dan Bison. Kode pada bab ini disusun dan ditinjau secara statis, sedangkan keluaran yang diharapkan didokumentasikan pada berkas `.expected`. Mahasiswa harus menjalankan build dan tes pada lingkungan yang memiliki toolchain C. Jika hasilnya berbeda, periksa perbedaan versi Flex, penanganan baris baru, dan urutan aturan.

Contoh

Contoh 7.1 (Menelusuri lexer)**.** Untuk masukan `a <= b;`, lexer memanggil `skip_space`, mencatat posisi 1:1, lalu memindai identifier `a`. Token pertama adalah `IDENT` dengan lexeme `a`. Setelah spasi dilewati, lexer membaca `<` dan melihat `=` melalui `peek`. Karena `<=` cocok, lexer memajukan posisi sekali lagi dan menghasilkan `LE`. Berikutnya `b` menghasilkan `IDENT`, lalu `;` menghasilkan `SEMI`. Akhirnya `EOF` dikembalikan pada baris berikutnya.

Contoh 7.2 (Pemulihan kesalahan)**.** Untuk masukan `int a = 3.;`, lexer menghasilkan token `INT`, `IDENT(a)`, `ASSIGN`, dan `INT_LIT(3)`. Ketika melihat titik tanpa digit berikutnya, lexer tidak memasukkannya ke bilangan. Pada pemanggilan berikutnya, titik menghasilkan `TOK_ERROR`. Lexer mencatat `baris 1, kolom 10: karakter tidak dikenal`, lalu melanjutkan ke `;`. Parser menerima token kesalahan dan dapat melaporkan masalah tanpa kehilangan sisa program.

Contoh 7.3 (Prioritas aturan Flex)**.** Pada masukan `a <= b`, aturan `"<="` dan aturan `"<"` sama-sama cocok pada posisi `<`. Flex memilih pattern terpanjang, yaitu `"<="`. Jika kedua pattern memiliki panjang sama, aturan yang lebih awal dalam berkas yang menang. Karena itu, operator dua karakter harus ditulis sebelum operator satu karakter.

Contoh 7.4 (Perbandingan ukuran kode)**.** Lexer tangan pada proyek ini sekitar 350 baris C, sedangkan lexer Flex sekitar 85 baris. Lexer tangan memberikan kontrol penuh

atas lokasi, pesan kesalahan, dan penanganan kasus khusus. Lexer Flex lebih ringkas dan mudah diubah, tetapi memerlukan generator dan sedikit lebih sulit memberikan diagnostik yang sangat spesifik.

Contoh 7.5 (Membaca keluaran tes)**.** Berkas `basic.expected` memuat baris `1:5 IDENT main`. Artinya, pada baris 1 kolom 5, lexer menghasilkan token `IDENT` dengan lexeme `main`. Jika keluaran aktual berbeda, periksa apakah lexer menghitung kolom mulai dari 1 dan apakah baris baru menaikkan nomor baris.

Aktivitas Pembelajaran dan Praktikum

Tabel 7.4. Rencana aktivitas Bab VII

Sesi	Bentuk	Kegiatan
1	Ceramah	Buffer, sentinel, dan alur scanner
2	Live coding	Lexer tangan: identifier, angka, operator
3	Live coding	Lexer Flex dan perbandingan aturan
4	Praktikum	M1: implementasi, pengujian, dan laporan

M1 / Praktikum 1: Lexer

Tujuan. Menghasilkan lexer Subset-C yang bekerja, diuji, dan terdokumentasi.

Keluaran.

1. Lexer tangan dalam C (`lexer.c`) yang lulus tes.
2. Lexer Flex (`lexer_flex.l`) yang menghasilkan token setara.
3. Minimal 30 kasus uji dengan berkas harapan.
4. Laporan singkat: desain, hasil tes, kendala, dan refleksi.

Langkah kerja.

1. Lengkapi `include/token.h` dan `include/lexer.h`.
2. Implementasikan `src/lexer.c` dan `src/error.c`.
3. Implementasikan `src/main_lex.c`.
4. Implementasikan `src/lexer_flex.l`.
5. Jalankan `build.bat` dan `run-tests.bat`.

6. Bandingkan keluaran lexer tangan dan Flex.

7. Tulis laporan dan serahkan.

Rubrik.

Tabel 7.5. Rubrik Praktikum 1

Aspek	Indikator	Bobot
Kebenaran	Semua token dikenali, tes lulus	40%
Kualitas kode	Struktur, penamaan, komentar, tanpa peringatan	30%
Kelengkapan	Tangan dan Flex, 30 tes, laporan	20%
Pengujian	Kasus normal, batas, dan kesalahan	10%

Definisi selesai.

☐ Lexer tangan dan Flex berjalan.

☐ Minimal 30 tes lulus.

☐ Diagnostik menyebut lokasi dan penyebab.

☐ Laporan diserahkan.

Latihan dan Refleksi

Latihan

1. Jelaskan fungsi `lexeme_begin` dan `forward`.

2. Mengapa sentinel diperlukan pada buffer?

3. Tuliskan alur `lexer_next` untuk identifier.

4. Bagaimana lexer membedakan integer dan float?

5. Mengapa operator dua karakter harus diperiksa lebih dahulu?

6. Rancang pemulihan untuk komentar blok yang tidak ditutup.

7. Bandingkan lexer tangan dan Flex dari sisi kontrol dan produktivitas.

8. Tambahkan dukungan komentar bersarang pada lexer Flex.

Refleksi

Refleksi

1. Bagian implementasi mana yang paling menantang? Mengapa?
2. Apakah tes Anda mencakup kasus batas yang cukup?
3. Jika Anda harus memilih satu pendekatan untuk proyek nyata, mana yang Anda pilih dan mengapa?

Asesmen

Bab VII memuat Praktikum 1 dengan bobot 8% sesuai RPS. Asesmen mencakup kebenaran, kualitas kode, kelengkapan, dan pengujian.

Tabel 7.6. Instrumen asesmen Bab VII

Instrumen	Indikator	Bobot
Praktikum 1	Lexer tangan dan Flex, 30 tes, laporan	8%
Kuis	Buffer, maximal munch, pemulihan	formatif
Review kode	Keterbacaan, struktur, penanganan kesalahan	bagian P1

Kriteria kelulusan: lexer mengenali seluruh token Subset-C, lulus minimal 30 tes, menghasilkan diagnostik yang informatif, dan terdokumentasi dengan baik.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan skema buffer dan sentinel.

☐ Saya dapat menjelaskan peran lexeme begin dan forward.

☐ Saya dapat mengimplementasikan lexer tangan dalam C.

☐ Saya dapat mengimplementasikan lexer dengan Flex.

☐ Saya dapat menangani kesalahan leksikal tanpa berhenti.

☐ Saya dapat menghasilkan diagnostik dengan lokasi.

☐ Saya dapat menulis tes dan berkas harapan.

☐ Saya sudah menyelesaikan M1/Praktikum 1.

Rangkuman

Pada bab ini telah dibahas:

- buffer masukan, sentinel, lexeme begin, dan forward pointer;
- implementasi lexer tangan dalam C;
- implementasi lexer dengan Flex dan perbandingannya;
- strategi penanganan dan pelaporan kesalahan leksikal;
- driver, pengujian, dan integrasi proyek M1.

Bab VIII memulai analisis sintaksis. Kita akan beralih dari token ke tata bahasa bebas konteks, mempelajari derivasi, pohon urai, dan cara merancang tata bahasa untuk Subset-C.

Evaluasi

1. Jelaskan skema dua buffer dan fungsi sentinel.
2. Mengapa lexer memerlukan lookahead? Berikan dua contoh.
3. Tuliskan pseudokode `lexer_next` untuk identifier.
4. Jelaskan perbedaan penanganan integer dan float.
5. Rancang aturan Flex untuk operator dua karakter.
6. Bagaimana lexer memulihkan diri dari karakter ilegal?
7. Mengapa format diagnostik `berkas:baris:kolom` penting?
8. Bandingkan lexer tangan dan Flex untuk proyek berukuran besar.

Kunci dan Rubrik Singkat

- Soal 1–4: ketepatan konsep (70%), contoh (30%).
- Soal 5–6: kebenaran aturan dan pemulihan (70%), penjelasan (30%).
- Soal 7–8: analisis (60%), argumentasi (40%).

BAB VIII

Context-Free Grammar: BNF, EBNF, Derivasi, dan Parse Tree

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 3.1, Sub-CPMK 3.4

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. mendefinisikan CFG dan menulis tata bahasa dalam BNF;
2. mengonversi EBNF menjadi BNF;
3. menghasilkan derivasi leftmost dan rightmost;
4. membangun pohon urai dan membedakannya dari AST;
5. merancang tata bahasa bebas konteks untuk Subset-C.

Materi Pokok

- Definisi CFG, nonterminal, terminal, dan aturan produksi.
- Notasi BNF dan EBNF beserta konversinya.
- Derivasi leftmost, rightmost, dan pohon urai.
- Hubungan pohon urai dengan Abstract Syntax Tree.
- Perancangan tata bahasa Subset-C.

8.1 Definisi CFG dan Notasi BNF

8.1.1 Definisi

Definisi 8.1 (Context-Free Grammar)**.** Tata bahasa bebas konteks (CFG) adalah empat-tupel

$$G = (V, T, P, S),$$

dengan V himpunan nonterminal, T himpunan terminal, P himpunan aturan produksi berbentuk $A \to \alpha$ dengan $A \in V$ dan $\alpha \in (V \cup T)^*$, serta $S \in V$ simbol awal.

Aturan CFG mengganti satu nonterminal tanpa mempedulikan konteks di sekitarnya. Sifat ini membuat CFG cukup kuat untuk struktur program dan cukup sederhana untuk diimplementasikan dengan parser.

Contoh 8.2 (CFG aritmetika)**.**

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to (E) \mid \texttt{id}.$$

Nonterminal E, T, dan F masing-masing menyatakan ekspresi, term, dan faktor. Simbol `id`, `+`, `*`, `(`, dan `)` adalah terminal.

8.1.2 Notasi BNF

BNF (*Backus-Naur Form*) menulis aturan dengan simbol $\to$ dan alternatif dengan $|$. Aturan $A \to \alpha_1 \mid \alpha_2$ adalah singkatan dari dua aturan $A \to \alpha_1$ dan $A \to \alpha_2$.

Konvensi penulisan yang dipakai dalam buku ini:

- nonterminal ditulis miring: E, T, F;
- terminal ditulis dalam tanda kutip atau huruf mesin: `id`, `+`;
- simbol awal adalah nonterminal pertama yang didefinisikan;
- produksi ε ditulis eksplisit.

8.1.3 Bahasa yang Dibangkitkan

Derivasi $S \Rightarrow^* w$ menghasilkan string terminal w. Bahasa CFG adalah

$$L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}.$$

Sebagai contoh, tata bahasa $S \to aSb \mid \varepsilon$ membangkitkan $\{a^n b^n \mid n \geq 0\}$.

8.1.4 Mengapa CFG Cocok untuk Sintaksis

CFG memiliki tiga sifat yang dibutuhkan analisis sintaksis:

1. dapat menyatakan struktur bersarang;
2. dapat diurai secara efisien dengan LL atau LR;
3. memiliki teori yang matang untuk analisis dan transformasi.

CFG tidak dapat memeriksa tipe atau deklarasi, tetapi dapat memastikan bahwa program memiliki bentuk yang benar. Pemeriksaan makna dilakukan pada fase semantik.

8.2 EBNF dan Konversi ke BNF

8.2.1 Notasi EBNF

EBNF (*Extended Backus-Naur Form*) menambahkan operator untuk pengulangan, pilihan, dan pengelompokan:

Tabel 8.1. Operator EBNF dan artinya

Notasi	Arti	Contoh
$X \mid Y$	pilihan	`int | float`
$[X]$	opsional	`[else stmt]`
$\{X\}$	nol atau lebih	`{stmt}`
(X)	pengelompokan	`(a | b) c`

EBNF lebih ringkas dan mudah dibaca, tetapi parser generator seperti Yacc/Bison memakai BNF. Karena itu, EBNF biasanya dikonversi ke BNF sebelum implementasi.

8.2.2 Aturan Konversi

Konversi EBNF ke BNF dapat dilakukan secara sistematis:

1. pilihan $X \mid Y$ menjadi dua aturan terpisah;
2. opsional $[X]$ menjadi X atau ε;
3. pengulangan $\{X\}$ menjadi nonterminal bantu rekursif;
4. pengelompokan (X) diperlakukan sebagai nonterminal baru bila diperlukan.

Sebagai contoh, aturan EBNF

$$\text{params} = \text{param}\{\text{","}\ \text{param}\}$$

dikonversi menjadi

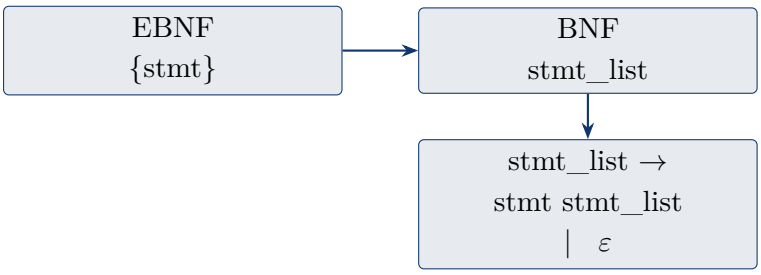
$$\text{params} \rightarrow \text{param}\ \text{params_tail},$$
$$\text{params_tail} \rightarrow \text{","}\ \text{param}\ \text{params_tail} \mid \varepsilon.$$

Aturan opsional

$$\text{returnstmt} = \text{"return"}\ [\text{expr}]\ \text{";"}$$

menjadi

$$\text{returnstmt} \rightarrow \text{"return"}\ \text{opt_expr}\ \text{";"}, \quad \text{opt_expr} \rightarrow \text{expr} \mid \varepsilon.$$



Gambar 8.1. Konversi pengulangan EBNF menjadi nonterminal rekursif BNF

8.2.3 Tata Bahasa Subset-C dalam EBNF

Tata bahasa Subset-C ditulis dalam EBNF pada `spec/grammar.ebnf`. Potongan awalnya ditunjukkan pada Kode Program 8.1.

```
// Berkas: spec/grammar.ebnf
// Tata bahasa EBNF Subset-C v0.4 (Bab VIII).

program     = { topdecl } EOF ;
topdecl     = type ID "(" [ params ] ")" ( block | ";" ) ;
params      = param { "," param } ;
param       = type ID ;
type        = "int" | "float" | "char" | "void" ;
block       = "{" { stmt } "}" ;
stmt        = decl ";" | block | ifstmt | whilestmt | forstmt
            | returnstmt | breakstmt | continuestmt
            | expr ";" | printstmt | ";" ;
decl        = type ID [ "=" expr ] ;
ifstmt      = "if" "(" expr ")" stmt [ "else" stmt ] ;
whilestmt   = "while" "(" expr ")" stmt ;
forstmt     = "for" "(" [ expr ] ";" [ expr ] ";" [ expr ] ")" stmt ;
returnstmt  = "return" [ expr ] ";" ;
breakstmt   = "break" ";" ;
continuestmt = "continue" ";" ;
printstmt   = "print" "(" expr ")" ";" ;
expr        = assign ;
assign      = ID "=" assign | logor ;
```

Kode Program 8.1. Bagian awal `grammar.ebnf`

Perhatikan penggunaan {} untuk daftar pernyataan dan [] untuk bagian opsional. Notasi ini memudahkan pembacaan, tetapi harus dikonversi sebelum dipakai oleh parser generator.

8.2.4 Hasil Konversi ke BNF

Hasil konversi lengkap ada pada `spec/grammar.bnf`. Potongannya ditunjukkan pada Kode Program 8.2.

```
// Berkas: spec/grammar.bnf
// Konversi EBNF ke BNF. Simbol bantu bernama *_tail dan opt_*.

program        ::= topdecl_list EOF
topdecl_list   ::= topdecl topdecl_list | epsilon
topdecl        ::= type ID "(" opt_params ")" topdecl_tail
opt_params     ::= params | epsilon
params         ::= param params_tail
params_tail    ::= "," param params_tail | epsilon
param          ::= type ID
type           ::= "int" | "float" | "char" | "void"
topdecl_tail   ::= block | ";"
block          ::= "{" stmt_list "}"
stmt_list      ::= stmt stmt_list | epsilon
stmt           ::= decl ";" | block | ifstmt | whilestmt | forstmt
                 | returnstmt | breakstmt | continuestmt
                 | expr ";" | printstmt | ";"
decl           ::= type ID opt_init
opt_init       ::= " = " expr | epsilon
ifstmt         ::= "if" "(" expr ")" stmt opt_else
opt_else       ::= "else" stmt | epsilon
whilestmt      ::= "while" "(" expr ")" stmt
forstmt        ::= "for" "(" opt_expr ";" opt_expr ";" opt_expr ")" stmt
opt_expr       ::= expr | epsilon
```

Kode Program 8.2. Bagian awal `grammar.bnf`

Nama nonterminal bantu diberi akhiran `_tail` atau awalan `opt_`. Konvensi ini memudahkan penelusuran asal aturan dan membantu saat mentransformasi tata bahasa pada Bab IX.

8.3 Derivasi Leftmost dan Rightmost

8.3.1 Derivasi

Derivasi adalah urutan penggantian nonterminal hingga diperoleh string terminal. Pada setiap langkah, satu nonterminal diganti dengan ruas kanan aturan produksi. Derivasi leftmost selalu mengganti nonterminal paling kiri; derivasi rightmost mengganti nonterminal paling kanan.

Contoh 8.3 (Derivasi leftmost)**.** Untuk tata bahasa aritmetika dan string $\texttt{id} + \texttt{id} * \texttt{id}$:

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow T + T \\ &\Rightarrow F + T \\ &\Rightarrow \texttt{id} + T \\ &\Rightarrow \texttt{id} + T * F \\ &\Rightarrow \texttt{id} + F * F \\ &\Rightarrow \texttt{id} + \texttt{id} * F \\ &\Rightarrow \texttt{id} + \texttt{id} * \texttt{id}. \end{aligned}$$

Contoh 8.4 (Derivasi rightmost)**.** Dengan tata bahasa yang sama:

$$\begin{aligned} E &\Rightarrow E + T \\ &\Rightarrow E + T * F \\ &\Rightarrow E + T * \texttt{id} \\ &\Rightarrow E + F * \texttt{id} \\ &\Rightarrow E + \texttt{id} * \texttt{id} \\ &\Rightarrow T + \texttt{id} * \texttt{id} \\ &\Rightarrow F + \texttt{id} * \texttt{id} \\ &\Rightarrow \texttt{id} + \texttt{id} * \texttt{id}. \end{aligned}$$

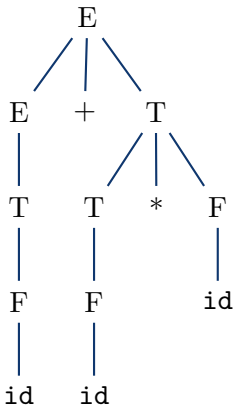
8.3.2 Bentuk Kalimat

Setiap string yang muncul selama derivasi disebut bentuk kalimat (*sentential form*). Bentuk kalimat dapat memuat campuran terminal dan nonterminal. String terminal yang dihasilkan disebut kalimat (*sentence*).

Sebagai contoh, pada derivasi di atas, $E + T * F$ adalah bentuk kalimat, sedangkan $\texttt{id} + \texttt{id} * \texttt{id}$ adalah kalimat.

8.3.3 Pohon Derivasi

Derivasi dapat direpresentasikan sebagai pohon. Akar adalah simbol awal; setiap penerapan aturan menambahkan anak-anak pada simpul nonterminal. Daun pohon, dibaca dari kiri ke kanan, membentuk string yang diturunkan.



Gambar 8.2. Pohon urai untuk `id` + `id` ∗ `id`

8.3.4 Derivasi dan Ambiguitas

Satu string dapat memiliki lebih dari satu derivasi leftmost jika tata bahasa ambigu. Ambiguitas tidak selalu terlihat dari string akhir; ia terlihat dari pohon urai yang berbeda. Bab IX membahas ambiguitas, dangling else, dan cara mengatasinya.

Informasi

Derivasi leftmost dan rightmost pada tata bahasa yang tidak ambigu menghasilkan pohon urai yang sama. Perbedaan urutan penggantian tidak mengubah struktur pohon.

8.4 Parse Tree dan Hubungannya dengan AST

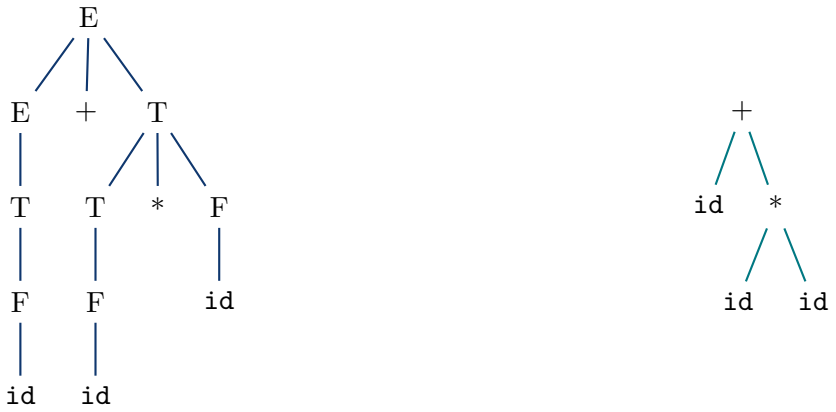
8.4.1 Parse Tree

Pohon urai (*parse tree*) merepresentasikan derivasi secara lengkap. Setiap simpul dalam adalah nonterminal, setiap daun adalah terminal atau ε, dan anak-anak simpul sesuai dengan ruas kanan aturan yang dipakai.

Pohon urai memuat semua detail sintaksis, termasuk tanda kurung, titik koma, dan kata kunci. Detail ini penting untuk validasi, tetapi membuat pohon besar dan sulit diproses pada fase berikutnya.

8.4.2 Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah penyederhanaan pohon urai. AST menghilangkan detail sintaksis yang tidak diperlukan untuk analisis semantik, seperti tanda kurung dan titik koma. Setiap simpul AST merepresentasikan konstruksi bahasa: ekspresi, pernyataan, deklarasi, atau fungsi.



Gambar 8.3. Pohon urai (kiri) dan AST (kanan) untuk ekspresi yang sama

8.4.3 Perbedaan Penting

Tabel 8.2. Perbedaan pohon urai dan AST

Aspek	**Pohon urai**	**AST**
Isi	Semua simbol tata bahasa	Konstruksi bermakna
Ukuran	Lebih besar	Lebih ringkas
Detail	Kurung, titik koma, kata kunci	Hanya operand dan operator
Penggunaan	Analisis sintaksis	Analisis semantik dan kode

8.4.4 Contoh AST

Untuk program `arith.sc`, AST tingkat atas memuat deklarasi fungsi `main`, deklarasi variabel `a` dan `b`, serta pernyataan `print`. Ekspresi `2 + 3 * 4` menjadi AST dengan akar `+`, anak kiri `2`, dan anak kanan `*` dengan anak `3` dan `4`. Struktur ini langsung mencerminkan prioritas operator.

8.4.5 Dari Pohon Urai ke AST

Konversi dapat dilakukan saat parsing. Ketika parser mengurangi (*reduce*) aturan produksi, ia membangun simpul AST yang sesuai. Aturan yang hanya mengatur sintaksis, seperti $F \to (E)$, tidak menghasilkan simpul baru; ia hanya meneruskan simpul E. Aturan operator menghasilkan simpul operator dengan anak operand.

Pendekatan ini menghindari pembentukan pohon urai lengkap, sehingga lebih efisien. Namun, untuk keperluan pengajaran, pohon urai tetap digambar agar mahasiswa memahami hubungan antara derivasi dan AST.

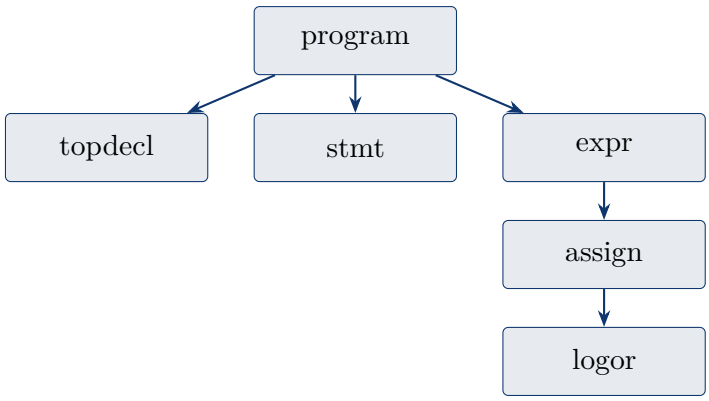
8.5 Merancang Grammar Subset-C

8.5.1 Prinsip Perancangan

Tata bahasa Subset-C dirancang dengan prinsip berikut:

1. satu nonterminal untuk satu tingkat prioritas ekspresi;
2. operator dengan prioritas lebih tinggi berada lebih dalam pohon;
3. pernyataan dan deklarasi dipisahkan;
4. bagian opsional ditandai dengan nonterminal `opt_*`;
5. pengulangan ditandai dengan nonterminal `*_tail`.

8.5.2 Struktur Nonterminal



Gambar 8.4. Hierarki nonterminal utama Subset-C

8.5.3 Deklarasi dan Fungsi

Aturan untuk deklarasi fungsi:

$$\text{topdecl} \rightarrow \text{type } \texttt{ID} \text{ "(" opt_params ")" topdecl_tail},$$
$$\text{topdecl_tail} \rightarrow \text{block} \mid \text{";"}.$$

Aturan ini memungkinkan deklarasi fungsi dengan blok tubuh atau deklarasi prototipe tanpa tubuh. Parameter opsional ditangani oleh `opt_params`.

8.5.4 Pernyataan

Aturan pernyataan mengelompokkan konstruksi kontrol:

$$\begin{aligned}\text{stmt} \rightarrow\ & \text{decl ";"} \mid \text{block} \mid \text{ifstmt} \mid \text{whilestmt} \\ & \mid \text{forstmt} \mid \text{returnstmt} \mid \text{breakstmt} \mid \text{continuestmt} \\ & \mid \text{expr ";"} \mid \text{printstmt} \mid \text{";"}.\end{aligned}$$

Pemisahan ini memudahkan parser memilih cabang berdasarkan token awal. Sebagai contoh, token `if` langsung memilih `ifstmt`, sedangkan token `while` memilih `whilestmt`.

8.5.5 Ekspresi Berprioritas

Ekspresi dipecah menjadi beberapa tingkat:

$$
\begin{aligned}
\text{expr} &\rightarrow \text{assign},\\
\text{assign} &\rightarrow \texttt{ID}\ \texttt{"="}\ \text{assign} \mid \text{logor},\\
\text{logor} &\rightarrow \text{logand}\ \text{logor_tail},\\
\text{logand} &\rightarrow \text{equality}\ \text{logand_tail},\\
\text{equality} &\rightarrow \text{relation}\ \text{equality_tail},\\
\text{relation} &\rightarrow \text{additive}\ \text{relation_tail},\\
\text{additive} &\rightarrow \text{term}\ \text{additive_tail},\\
\text{term} &\rightarrow \text{unary}\ \text{term_tail},\\
\text{unary} &\rightarrow \texttt{"!"}\ \text{unary} \mid \texttt{"-"}\ \text{unary} \mid \texttt{"+"}\ \text{unary} \mid \text{primary}.
\end{aligned}
$$

Setiap tingkat menangani satu kelompok operator. Prioritas naik dari `logor` ke `term`; unary dan primary berada di paling dalam.

8.5.6 Contoh Program

Program `arith.sc` menguji prioritas dan tanda kurung:

```
int main() {
    int a = 2 + 3 * 4;
    int b = (2 + 3) * 4;
    print(a);
    print(b);
    return 0;
}
```

Ekspresi pertama menghasilkan AST dengan akar `+` dan anak kanan `*`; ekspresi kedua menghasilkan akar `*` dengan anak kiri `+`. Perbedaan ini berasal dari tanda kurung dan tercermin dalam pohon urai.

Peringatan

Tata bahasa pada bab ini masih memuat rekursi kiri tidak langsung pada beberapa aturan dan belum tentu LL(1). Bab IX akan menghapus rekursi kiri dan melakukan left factoring sebelum parser top-down dibangun.

Contoh

Contoh 8.5 (Derivasi dengan tanda kurung)**.** Untuk string $(\texttt{id} + \texttt{id}) * \texttt{id}$, derivasi leftmost pada tata bahasa aritmetika adalah:

$$\begin{aligned} E &\Rightarrow T \\ &\Rightarrow T * F \\ &\Rightarrow F * F \\ &\Rightarrow (E) * F \\ &\Rightarrow (E + T) * F \\ &\Rightarrow (T + T) * F \\ &\Rightarrow (F + T) * F \\ &\Rightarrow (\texttt{id} + T) * F \\ &\Rightarrow (\texttt{id} + F) * F \\ &\Rightarrow (\texttt{id} + \texttt{id}) * F \\ &\Rightarrow (\texttt{id} + \texttt{id}) * \texttt{id}. \end{aligned}$$

Contoh 8.6 (Pohon urai if-else)**.** Diberikan pernyataan `if (x) y; else z;`. Pohon urai tingkat atas memiliki simpul `ifstmt` dengan anak `"if"`, `"("`, ekspresi `x`, `")"`, pernyataan `y;`, dan bagian `else z;`. Jika tata bahasa tidak membedakan pasangan `else` terdekat, pohon alternatif dapat terbentuk. Ambiguitas ini dibahas pada Bab IX.

Contoh 8.7 (Konversi EBNF)**.** Aturan EBNF

$$\text{args} = \text{expr}\{\text{","}\ \text{expr}\}$$

dikonversi menjadi:

$$\text{args} \rightarrow \text{expr}\ \text{args_tail},$$
$$\text{args_tail} \rightarrow \text{","}\ \text{expr}\ \text{args_tail} \mid \varepsilon.$$

Aturan ini right-recursive, sehingga cocok untuk parser top-down.

Contoh 8.8 (AST ekspresi)**.** Untuk $a + b * c$, AST memiliki akar `+`, anak kiri `a`, dan anak kanan `*` dengan anak `b` dan `c`. Jika ditulis dalam notasi prefix:

$$\texttt{(+ a (* b c))}.$$

Notasi ini menghilangkan tanda kurung sintaksis dan langsung menunjukkan struktur operator.

Contoh 8.9 (Merancang aturan fungsi)**.** Deklarasi

```
int tambah(int a, int b) {
    return a + b;
}
```

diurai oleh aturan `topdecl`: tipe `int`, identifier `tambah`, daftar parameter, lalu blok. Daftar parameter menggunakan `params` dan `params_tail`. Blok berisi satu pernyataan `return`. Setiap bagian memiliki nonterminal sendiri, sehingga parser dapat memeriksa struktur secara bertahap.

Aktivitas Pembelajaran dan Praktikum

Tabel 8.3. Rencana aktivitas Bab VIII

Sesi	Bentuk	Kegiatan
1	Ceramah	CFG, BNF, dan derivasi
2	Latihan	Menurunkan string dan menggambar pohon urai
3	Lokakarya	Konversi EBNF ke BNF
4	Praktikum	P2-A: tata bahasa Subset-C dan peer review

P2-A: Tata Bahasa Subset-C

Tujuan. Menulis tata bahasa EBNF untuk Subset-C, lalu mengonversinya ke BNF.

Langkah kerja.

1. Lengkapi `spec/grammar.ebnf` dengan aturan untuk seluruh konstruksi yang didukung.
2. Turunkan lima string contoh: ekspresi, deklarasi, if, while, dan fungsi.
3. Konversi EBNF ke BNF pada `spec/grammar.bnf`.
4. Gambar pohon urai untuk dua string.
5. Tukar berkas dengan teman untuk peer review.

Definisi selesai.

☐ EBNF mencakup semua konstruksi.

☐ Lima derivasi benar.

☐ BNF konsisten dengan EBNF.

☐ Dua pohon urai benar.

☐ Catatan peer review lengkap.

P2-A menjadi bagian Praktikum 2 (8%) yang dilaporkan pada Bab X.

Latihan dan Refleksi

Latihan

1. Definisikan CFG dan jelaskan perbedaan nonterminal dan terminal.
2. Tuliskan BNF untuk bahasa $\{a^n b^n \mid n \geq 0\}$.
3. Konversi aturan EBNF $[X]\ Y$ ke BNF.
4. Konversi aturan EBNF $\{X\}\ Y$ ke BNF.
5. Buat derivasi leftmost dan rightmost untuk $(\texttt{id} + \texttt{id}) * \texttt{id}$.
6. Gambarkan pohon urai untuk string pada Latihan 5.
7. Jelaskan perbedaan pohon urai dan AST.
8. Rancang aturan untuk pernyataan `for`.

Refleksi

Refleksi

1. Mengapa tata bahasa dengan tingkat prioritas terpisah lebih mudah diurai?
2. Apakah Anda dapat menjelaskan mengapa AST lebih kecil daripada pohon urai?
3. Bagian mana dari perancangan tata bahasa yang paling menantang?

Asesmen

Asesmen Bab VIII menjadi bagian Praktikum 2 (8%) dan Tugas Teori (8%). Fokusnya adalah ketepatan tata bahasa, derivasi, dan pohon urai.

Tabel 8.4. Instrumen asesmen Bab VIII

Instrumen	Indikator	Bobot
P2-A	EBNF, BNF, lima derivasi, dua pohon urai	bagian P2
Tugas Teori	Derivasi dan konversi EBNF-BNF	8%
Kuis	Definisi CFG dan pohon urai	formatif

Kriteria kelulusan: tata bahasa mencakup seluruh konstruksi, derivasi benar, dan pohon urai konsisten dengan tata bahasa.

Checklist Pencapaian Kompetensi

☐ Saya dapat mendefinisikan CFG.

☐ Saya dapat menulis aturan dalam BNF.

☐ Saya dapat membaca dan menulis EBNF.

☐ Saya dapat mengonversi EBNF ke BNF.

☐ Saya dapat menghasilkan derivasi leftmost dan rightmost.

☐ Saya dapat menggambar pohon urai.

☐ Saya dapat menjelaskan perbedaan pohon urai dan AST.

☐ Saya dapat merancang tata bahasa untuk Subset-C.

☐ Saya sudah menyelesaikan P2-A.

Rangkuman

Pada bab ini telah dibahas:

- definisi CFG dan notasi BNF;
- EBNF dan aturan konversinya ke BNF;
- derivasi leftmost, rightmost, dan bentuk kalimat;
- pohon urai dan hubungannya dengan AST;
- perancangan tata bahasa Subset-C.

Bab IX akan membersihkan tata bahasa dari ambiguitas, rekursi kiri, dan faktor umum. Transformasi ini diperlukan agar tata bahasa dapat diurai secara top-down pada Bab X.

Evaluasi

1. Definisikan CFG dan berikan satu contoh.
2. Tuliskan BNF untuk ekspresi aritmetika dengan prioritas.
3. Konversi aturan EBNF `params = param {"," param}` ke BNF.
4. Buat derivasi leftmost untuk `id * (id + id)`.
5. Gambarkan pohon urai untuk string pada Soal 4.
6. Jelaskan perbedaan pohon urai dan AST dengan contoh.

7. Rancang aturan untuk deklarasi variabel dengan inisialisasi opsional.
8. Mengapa tata bahasa Subset-C perlu diubah sebelum parsing top-down?

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan definisi dan konversi (70%), kelengkapan (30%).
- Soal 4–5: kebenaran derivasi dan pohon (70%), konsistensi (30%).
- Soal 6–8: ketepatan konsep (60%), contoh (40%).

BAB IX

Ambiguitas Grammar, Left Recursion, dan Left Factoring

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 3.2

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan ambiguitas dan dampaknya pada pohon urai;
2. menganalisis dangling else dan menentukan aturan resolusi;
3. menjelaskan precedence dan associativity;
4. menghapus rekursi kiri langsung dan tidak langsung;
5. melakukan left factoring dan mentransformasi grammar Subset-C.

Materi Pokok

- Ambiguitas grammar dan dangling else.
- Precedence dan associativity.
- Penghapusan left recursion langsung dan tidak langsung.
- Left factoring.
- Transformasi grammar Subset-C menjadi bentuk LL(1).

9.1 Ambiguitas Grammar dan Dangling Else

9.1.1 Definisi Ambiguitas

Definisi 9.1 (Tata bahasa ambigu)**.** Tata bahasa G disebut ambigu jika terdapat string $w \in L(G)$ yang memiliki dua pohon urai berbeda.

Ambiguitas berarti parser tidak dapat menentukan struktur yang dimaksud tanpa aturan tambahan. Pada bahasa pemrograman, ambiguitas harus dihilangkan atau diselesaikan dengan aturan eksplisit.

Contoh 9.2 (Grammar ambigu)**.** Tata bahasa

$$E \to E + E \mid E * E \mid \texttt{id}$$

ambigu. String $\texttt{id} + \texttt{id} * \texttt{id}$ memiliki dua pohon urai: satu mengutamakan penjumlahan, satu mengutamakan perkalian. Tanpa aturan prioritas, makna ekspresi tidak jelas.

9.1.2 Solusi Ambiguitas

Ada tiga cara umum menangani ambiguitas:

1. **restrukturisasi grammar**: membuat tingkat prioritas terpisah;
2. **aturan precedence dan associativity**: dipakai oleh parser generator;
3. **aturan semantik**: memilih pohon berdasarkan konteks, misalnya else terdekat.

Restrukturisasi grammar adalah solusi paling bersih karena menghilangkan ambiguitas pada tingkat sintaksis. Grammar aritmetika pada Bab VIII adalah contoh restrukturisasi: E, T, dan F memisahkan tingkat prioritas.

9.1.3 Dangling Else

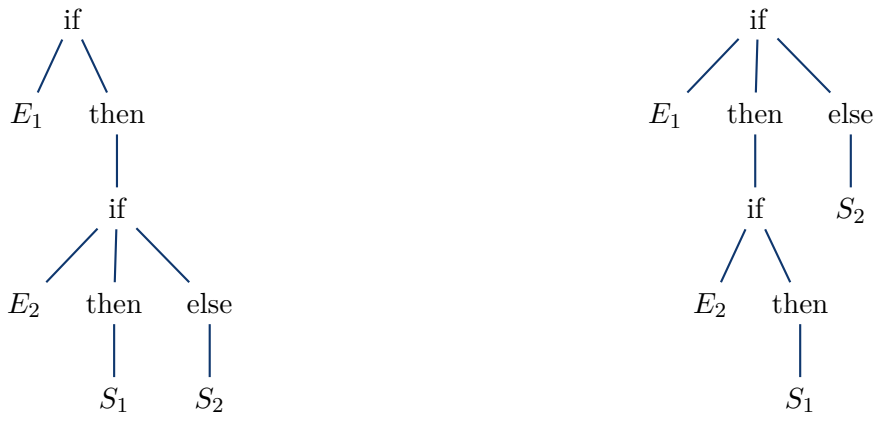
Ambiguitas klasik lain adalah *dangling else*:

$$\text{stmt} \to \text{if } E \text{ then stmt} \mid \text{if } E \text{ then stmt else stmt} \mid \text{other}.$$

String

$$\text{if } E_1 \text{ then if } E_2 \text{ then } S_1 \text{ else } S_2$$

dapat diurai dengan dua cara: `else` milik `if` luar atau `if` dalam.



Gambar 9.1. Dua pohon urai untuk dangling else

Aturan yang dipakai hampir semua bahasa adalah **else terdekat**: `else` dipasangkan dengan `if` terdekat yang belum memiliki `else`. Aturan ini dapat dinyatakan dengan grammar `matched`/`unmatched` atau diselesaikan oleh parser dengan prioritas shift.

Peringatan

Ambiguitas bukan hanya masalah teoretis. Parser yang memilih pohon secara acak dapat menghasilkan program yang berperilaku berbeda dari yang diharapkan. Selalu tetapkan aturan resolusi dan uji dengan kasus dangling else.

9.2 Precedence dan Associativity

9.2.1 Precedence

Precedence menentukan operator mana yang diikat lebih kuat. Pada ekspresi `id+id*id`, perkalian memiliki precedence lebih tinggi daripada penjumlahan, sehingga pohonnya memiliki `*` lebih dalam daripada `+`.

Tabel 9.1. Tingkat precedence operator Subset-C

Tingkat	Operator	Associativity
1 (terendah)	`||`	kiri
2	`&&`	kiri
3	`==`, `!=`	kiri
4	`<`, `<=`, `>`, `>=`	kiri
5	`+`, `-`	kiri
6	`*`, `/`, `%`	kiri
7	`!`, unary `-`, unary `+`	kanan
8 (tertinggi)	`()`, call, literal, identifier	—

9.2.2 Associativity

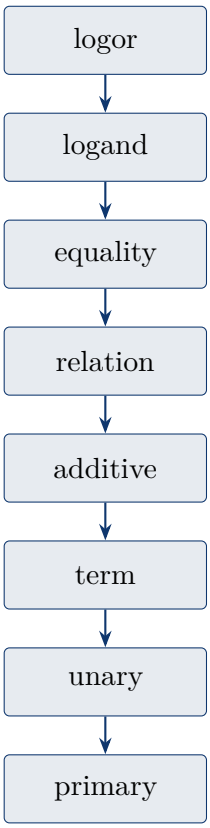
Associativity menentukan pengelompokan operator dengan precedence sama. Operator kiri-asosiatif dikelompokkan dari kiri:

$$a - b - c = (a - b) - c.$$

Operator kanan-asosiatif dikelompokkan dari kanan:

$$a = b = c = a = (b = c).$$

Operator non-asosiatif tidak boleh dirangkai, misalnya $a < b < c$ harus ditulis dengan tanda kurung.



Gambar 9.2. Tingkat nonterminal ekspresi: semakin ke bawah, semakin tinggi precedence

9.2.3 Menyatakan Precedence dalam Grammar

Precedence dinyatakan dengan membuat satu nonterminal per tingkat. Aturan tingkat bawah muncul sebagai anak dari tingkat atas. Dengan cara ini, pohon urai otomatis mencerminkan prioritas tanpa aturan tambahan.

Sebagai contoh, aturan

$$\text{additive} \rightarrow \text{term } \{(\text{"+"} \mid \text{"-"})\ \text{term}\}$$

menempatkan `term` di bawah `additive`. Karena `term` menangani perkalian, perkalian selalu lebih dalam daripada penjumlahan.

9.2.4 Precedence pada Parser Generator

Parser generator seperti Bison mengizinkan deklarasi precedence:

```
%left OR
%left AND
%left EQ NE
%left LT LE GT GE
%left PLUS MINUS
%left STAR SLASH PERCENT
```

```
%right NOT UMINUS
```

Deklarasi ini menyelesaikan konflik shift-reduce tanpa mengubah grammar. Namun, grammar yang sudah memiliki tingkat precedence terpisah biasanya lebih mudah dipahami dan dipelihara.

9.3 Penghapusan Left Recursion

9.3.1 Mengapa Left Recursion Bermasalah

Parser top-down seperti recursive descent memanggil fungsi untuk nonterminal. Jika aturan pertama berbentuk $A \to A\alpha$, fungsi `parseA` akan memanggil dirinya sendiri sebelum mengonsumsi token, sehingga terjadi rekursi tak berujung. Left recursion harus dihapus sebelum parsing top-down.

9.3.2 Left Recursion Langsung

Aturan berbentuk

$$A \to A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2$$

dapat diubah menjadi

$$A \to \beta_1 A' \mid \beta_2 A', \qquad A' \to \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon.$$

Nonterminal A' menangani pengulangan, sedangkan A memilih alternatif dasar.

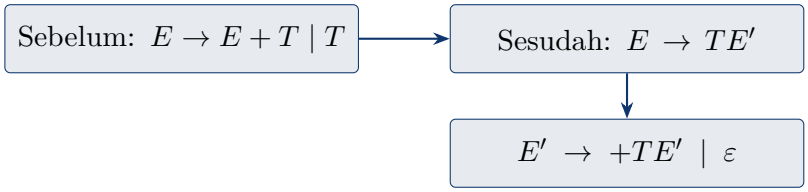
Contoh 9.3 (Ekspresi aritmetika)**.** Aturan

$$E \to E + T \mid T$$

diubah menjadi

$$E \to TE', \qquad E' \to +TE' \mid \varepsilon.$$

Pohon urai tetap sama, tetapi parser dapat memutuskan berdasarkan token pertama.



Gambar 9.3. Transformasi left recursion langsung

9.3.3 Left Recursion Tidak Langsung

Left recursion tidak langsung terjadi ketika A dapat menurunkan A melalui beberapa nonterminal:

$$A \to B\alpha, \qquad B \to A\beta.$$

Algoritma umum:

1. urutkan nonterminal $A_1, \ldots, A_n$;
2. untuk setiap A_i, ganti aturan yang dimulai dengan A_j $(j < i)$ menggunakan aturan A_j;
3. hapus left recursion langsung pada A_i.

Proses ini dapat mengubah bentuk grammar secara signifikan. Setelah transformasi, periksa kembali bahasa yang dibangkitkan dan uji dengan beberapa string.

9.3.4 Kondisi Nullable

Jika α dapat menurunkan ε, transformasi harus hati-hati. Aturan $A \to A\alpha$ dengan α nullable dapat menyebabkan loop epsilon. Pastikan tidak ada siklus ε dan periksa FIRST/FOLLOW setelah transformasi.

Informasi

Left recursion bukan kesalahan sintaksis. Banyak grammar alami menggunakan left recursion karena mencerminkan associativity kiri. Transformasi hanya diperlukan untuk parser top-down; parser bottom-up seperti LR justru dapat menanganinya secara langsung.

9.4 Left Factoring

9.4.1 Masalah Faktor Umum

Dua alternatif yang dimulai dengan simbol yang sama menyulitkan parser top-down. Contoh:

$$\text{stmt} \to \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S.$$

Parser tidak dapat memilih alternatif hanya dengan melihat token pertama, karena keduanya dimulai dengan `if`.

9.4.2 Transformasi

Left factoring mengeluarkan faktor umum:

$$A \to \alpha\beta_1 \mid \alpha\beta_2$$

menjadi

$$A \to \alpha A', \qquad A' \to \beta_1 \mid \beta_2.$$

Jika salah satu alternatif berakhir tepat setelah α, tambahkan ε pada A'.

Contoh 9.4 (If-else)**.** Aturan di atas menjadi:

$$\text{stmt} \to \text{if } E \text{ then } S \text{ stmt}'$$
$$\text{stmt}' \to \text{else } S \mid \varepsilon.$$

Sekarang parser dapat memutuskan setelah membaca bagian `then S`: jika token berikutnya `else`, pilih alternatif pertama; jika tidak, pilih ε.

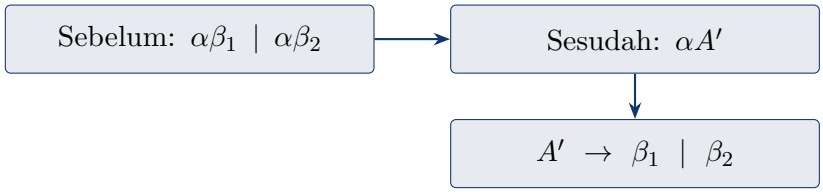
Contoh 9.5 (Identifier dan pemanggilan)**.** Aturan

$$\text{primary} \to \texttt{ID} \mid \texttt{ID "("} \text{ args } \texttt{")"}$$

difaktorkan menjadi

$$\text{primary} \to \texttt{ID} \text{ primary}'$$
$$\text{primary}' \to \texttt{"("} \text{ args } \texttt{")"} \mid \varepsilon.$$

Dengan bentuk ini, parser membaca `ID` sekali, lalu memutuskan berdasarkan token berikutnya.



Gambar 9.4. Left factoring mengeluarkan faktor umum

9.4.3 Kapan Left Factoring Diperlukan

Left factoring diperlukan jika dua alternatif memiliki prefiks umum yang tidak dapat dibedakan dengan satu token lookahead. Pada grammar Subset-C, faktor umum muncul pada:

- `primary`: `ID` versus `ID ( args )`;
- `stmt`: `ID = expr ;` versus `ID ( args ) ;`;
- `topdecl`: deklarasi fungsi dengan dan tanpa tubuh.

Setelah factoring, grammar lebih cocok untuk parser LL(1). Namun, factoring dapat membuat grammar lebih sulit dibaca manusia. Karena itu, grammar asli sering dipertahankan untuk dokumentasi, sedangkan grammar hasil transformasi dipakai untuk implementasi.

Peringatan

Left factoring tidak menghilangkan ambiguitas. Ia hanya mengurangi konflik FIRST. Dangling else tetap memerlukan aturan resolusi tambahan meskipun grammar sudah difaktorkan.

9.5 Transformasi Grammar Subset-C

9.5.1 Dari Grammar Awal ke Grammar LL(1)

Grammar Subset-C pada Bab VIII masih memiliki dua masalah untuk parsing top-down:

1. `primary` memiliki prefiks `ID` yang sama dengan pemanggilan fungsi;
2. `stmt` memiliki prefiks `ID` yang sama antara assignment dan expression statement.

Transformasi yang dilakukan:

- memfaktorkan `primary` menjadi `ID [ "(" [args] ")" ]`;
- memfaktorkan `stmt` menjadi `ID assign_or_call ";"`;
- memisahkan assignment dari ekspresi, sehingga `expr` tidak memuat `ID "=" ...`;
- mempertahankan tingkat precedence ekspresi.

9.5.2 Grammar Hasil

Grammar hasil disimpan pada `spec/grammar-ll1.ebnf`. Potongannya ditunjukkan pada Kode Program 9.1.

```
// Berkas: spec/grammar-ll1.ebnf
// Tata bahasa Subset-C yang diarahkan ke LL(1). Bab IX.
// Transformasi: assignment difaktorkan ke assign_or_call,
// ekspresi tidak memuat assignment, dan primary memakai optional call.

program         = { topdecl } EOF ;
topdecl         = type ID "(" [ params ] ")" ( block | ";" ) ;
params          = param { "," param } ;
param           = type ID ;
type            = "int" | "float" | "char" | "void" ;
block           = "{" { stmt } "}" ;
stmt            = decl ";" | block | ifstmt | whilestmt | forstmt
                | returnstmt | breakstmt | continuestmt | printstmt
                | ";" | ID assign_or_call ";" ;
assign_or_call  = "=" expr | "(" [ args ] ")" ;
```

```
decl          = type ID [ "=" expr ] ;
ifstmt        = "if" "(" expr ")" stmt [ "else" stmt ] ;
whilestmt     = "while" "(" expr ")" stmt ;
forstmt       = "for" "(" [ expr ] ";" [ expr ] ";" [ expr ] ")" stmt ;
returnstmt    = "return" [ expr ] ";" ;
breakstmt     = "break" ";" ;
continuestmt  = "continue" ";" ;
printstmt     = "print" "(" expr ")" ";" ;
expr          = logor ;
logor         = logand { "||" logand } ;
logand        = equality { "&&" equality } ;
equality      = relation { ( "==" | "!=" ) relation } ;
relation      = additive { ( "<" | "<=" | ">" | ">=" ) additive } ;
additive      = term { ( "+" | "-" ) term } ;
term          = unary { ( "*" | "/" | "%" ) unary } ;
```

Kode Program 9.1. Grammar Subset-C yang diarahkan ke LL(1)

Aturan `assign_or_call` menangani dua bentuk setelah `ID`: assignment `= expr` atau pemanggilan `( [args] )`. Dengan bentuk ini, parser hanya perlu membaca `ID` sekali.

9.5.3 Analisis Dampak

Tabel 9.2. Perubahan utama pada grammar Subset-C

Aturan	Perubahan	Alasan
`primary`	`ID [ "(" [args] ")" ]`	menghilangkan prefiks `ID`
`stmt`	`ID assign_or_call ";"`	memisahkan assignment dan call
`expr`	tanpa assignment	menghindari konflik FIRST
`ifstmt`	else opsional	dangling else diselesaikan parser

9.5.4 Pemeriksaan Kesetaraan Bahasa

Setelah transformasi, penting untuk memastikan bahasa yang dibangkitkan tidak berubah. Langkah pemeriksaan:

1. ambil beberapa string dari grammar lama;
2. turunkan dengan grammar baru;
3. bandingkan pohon atau AST yang dihasilkan;
4. uji string batas: assignment, call, if tanpa else, if dengan else, dan ekspresi campuran.

Jika ada string yang dahulu valid tetapi sekarang tidak, transformasi terlalu agresif. Jika ada string baru yang valid, transformasi memperluas bahasa secara tidak sengaja. Keduanya harus diperbaiki sebelum parser ditulis.

Informasi

Grammar LL(1) bukan tujuan akhir. Tujuannya adalah parser yang benar, mudah dipelihara, dan memberikan pesan kesalahan yang baik. Jika grammar LL(1) membuat pesan kesalahan memburuk, pertimbangkan parser dengan lookahead lebih panjang atau pendekatan bottom-up.

Contoh

Contoh 9.6 (Dua pohon urai)**.** Untuk grammar $E \to E + E \mid E * E \mid \texttt{id}$ dan string $\texttt{id}+\texttt{id}*\texttt{id}$, pohon pertama memiliki akar `+` dengan anak kanan `*`; pohon kedua memiliki akar `*` dengan anak kiri `+`. Keduanya valid menurut grammar, tetapi hanya pohon pertama yang sesuai dengan aturan precedence perkalian lebih tinggi.

Contoh 9.7 (Menghapus left recursion langsung)**.** Aturan

$$A \to Aa \mid Ab \mid c \mid d$$

diubah menjadi

$$A \to cA' \mid dA', \qquad A' \to aA' \mid bA' \mid \varepsilon.$$

String cab diturunkan sebagai $A \Rightarrow cA' \Rightarrow caA' \Rightarrow cabA' \Rightarrow cab$.

Contoh 9.8 (Left recursion tidak langsung)**.** Diberikan

$$A \to Ba \mid c, \qquad B \to Ab \mid d.$$

Substitusi B ke aturan A menghasilkan

$$A \to Aba \mid da \mid c.$$

Setelah itu, left recursion langsung dihapus:

$$A \to daA' \mid cA', \qquad A' \to baA' \mid \varepsilon.$$

Contoh 9.9 (Left factoring if-else)**.** Aturan

$$S \to \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S$$

difaktorkan menjadi

$$S \to \text{if } E \text{ then } S\ S', \qquad S' \to \text{else } S \mid \varepsilon.$$

Parser memutuskan `else` setelah selesai memproses S.

Contoh 9.10 (Dangling else)**.** Pada string `if (a) if (b) x; else y;`, aturan else terdekat memasangkan `else y;` dengan `if (b)`. Pohon yang dihasilkan adalah pohon kiri pada Gambar 9.1. Jika diinginkan pasangan luar, program harus ditulis dengan blok eksplisit: `if (a) { if (b) x; } else y;`.

Aktivitas Pembelajaran dan Praktikum

Tabel 9.3. Rencana aktivitas Bab IX

Sesi	Bentuk	Kegiatan
1	Ceramah	Ambiguitas dan dangling else
2	Latihan	Menghapus left recursion langsung
3	Latihan	Left factoring dan analisis FIRST
4	Praktikum	P2-B: transformasi grammar Subset-C

P2-B: Transformasi Grammar

Tujuan. Mentransformasi grammar Subset-C menjadi bentuk yang siap untuk analisis FIRST/FOLLOW dan parsing top-down.

Langkah kerja.

1. Mulai dari `spec/grammar.ebnf`.
2. Identifikasi prefiks umum dan left recursion.
3. Lakukan left factoring dan hapus left recursion.
4. Tulis hasilnya ke `spec/grammar-ll1.ebnf`.
5. Buktikan kesetaraan bahasa dengan lima string uji.
6. Catat konflik yang masih tersisa, misalnya dangling else.

Definisi selesai.

☐ Tidak ada left recursion langsung.

☐ Prefiks umum sudah difaktorkan.

☐ Lima string uji tetap valid.

☐ Konflik yang tersisa terdokumentasi.

☐ Grammar siap dianalisis FIRST/FOLLOW.

P2-B menjadi bagian Praktikum 2 (8%) yang dilaporkan pada Bab X.

Latihan dan Refleksi

Latihan

1. Jelaskan mengapa grammar $E \to E + E \mid E * E \mid \texttt{id}$ ambigu.
2. Hilangkan left recursion dari $A \to Aa \mid b$.
3. Hilangkan left recursion tidak langsung dari $A \to Ba \mid c$, $B \to Ab \mid d$.
4. Lakukan left factoring pada $S \to \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{if}\ E\ \texttt{then}\ S$.
5. Lakukan left factoring pada primary → `ID` | `ID` "("args ")".
6. Jelaskan aturan else terdekat dan berikan contoh.
7. Mengapa left recursion tidak menjadi masalah pada parser bottom-up?
8. Periksa apakah grammar hasil transformasi masih menghasilkan bahasa yang sama.

Refleksi

Refleksi

1. Transformasi mana yang paling mudah Anda pahami?
2. Apakah Anda dapat menjelaskan perbedaan ambiguitas dan konflik FIRST?
3. Mengapa grammar yang baik untuk manusia belum tentu baik untuk parser?

Asesmen

Asesmen Bab IX menjadi bagian Tugas Teori (8%) dan UTS (20%). Fokusnya adalah kemampuan menganalisis dan mentransformasi grammar.

Tabel 9.4. Instrumen asesmen Bab IX

Instrumen	Indikator	Bobot
Tugas Teori	Ambiguitas, left recursion, left factoring	8%
UTS	Analisis grammar dan transformasi	20%
P2-B	Grammar LL(1) dan bukti kesetaraan	bagian P2

Kriteria kelulusan: mahasiswa dapat mengidentifikasi ambiguitas dan konflik, menghapus left recursion, melakukan left factoring, serta menjelaskan dampaknya pada parsing.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan ambiguitas dengan contoh.

☐ Saya dapat menjelaskan dangling else dan aturan resolusi.

☐ Saya dapat menjelaskan precedence dan associativity.

☐ Saya dapat menghapus left recursion langsung.

☐ Saya dapat menghapus left recursion tidak langsung.

☐ Saya dapat melakukan left factoring.

☐ Saya dapat mentransformasi grammar Subset-C.

☐ Saya sudah menyelesaikan P2-B.

Rangkuman

Pada bab ini telah dibahas:

- ambiguitas grammar dan dangling else;
- precedence dan associativity;
- penghapusan left recursion langsung dan tidak langsung;
- left factoring;
- transformasi grammar Subset-C menjadi bentuk LL(1).

Bab X akan memakai grammar hasil transformasi untuk menghitung FIRST dan FOLLOW, membangun tabel LL(1), dan mengimplementasikan recursive descent parser.

Evaluasi

1. Definisikan tata bahasa ambigu dan berikan satu contoh.
2. Jelaskan dua pohon urai pada dangling else.
3. Tuliskan aturan else terdekat.
4. Hilangkan left recursion dari $A \to Aa \mid Ab \mid c$.
5. Hilangkan left recursion tidak langsung dari $A \to Ba \mid c$, $B \to Ab \mid d$.
6. Lakukan left factoring pada $S \to \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{if}\ E\ \texttt{then}\ S$.
7. Jelaskan mengapa parser top-down tidak dapat menangani left recursion.
8. Mengapa grammar hasil transformasi perlu diuji kesetaraannya?

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan konsep (70%), contoh (30%).
- Soal 4–6: kebenaran transformasi (80%), kelengkapan (20%).
- Soal 7–8: penjelasan (60%), argumentasi (40%).

BAB X

Top-Down Parsing: Recursive Descent, LL(1), FIRST, dan FOLLOW

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 3.3, Sub-CPMK 4.1

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan prinsip recursive descent dan predictive parsing;
2. menghitung himpunan FIRST dan FOLLOW;
3. membangun tabel parsing LL(1);
4. mengimplementasikan recursive descent parser yang sinkron dengan tabel;
5. menangani kesalahan sintaksis secara informatif;
6. menyelesaikan M2: parser untuk Subset-C.

Materi Pokok

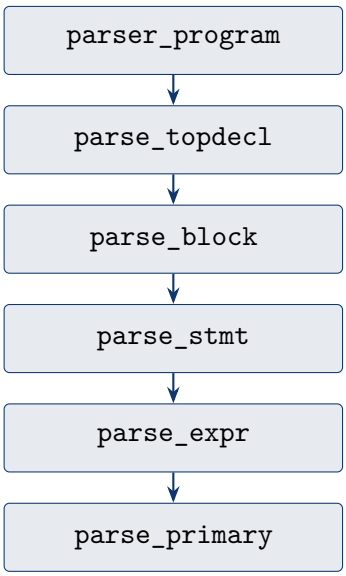
- Recursive descent: satu fungsi per nonterminal.
- Himpunan FIRST dan FOLLOW.
- Tabel parsing LL(1) dan predictive parser.
- Konstruksi recursive descent yang sinkron dengan tabel.
- Penanganan kesalahan sintaksis dan pemulihan.

10.1 Recursive Descent: Satu Fungsi per Nonterminal

10.1.1 Prinsip

Recursive descent adalah teknik parsing top-down yang mengimplementasikan satu fungsi untuk setiap nonterminal. Fungsi `parseE` bertanggung jawab mengenali string yang

diturunkan dari E; fungsi `parseT` untuk T, dan seterusnya. Setiap fungsi memutuskan aturan produksi berdasarkan token saat ini.



Gambar 10.1. Rantai pemanggilan utama recursive descent parser

10.1.2 Struktur Parser

Parser menyimpan lexer, token saat ini, dan status kesalahan. Antarmukanya didefinisikan pada `parser.h`, seperti ditunjukkan pada Kode Program 10.1.

```
// Berkas: include/parser.h
// Recursive descent parser untuk Subset-C.

#ifndef SUBSETC_PARSER_H
#define SUBSETC_PARSER_H

#include "lexer.h"
#include "ast.h"

typedef struct {
    Lexer lexer;
    Token current;
    int had_error;
} Parser;

void parser_init(Parser *parser, const char *source,
                 const char *filename);
AstNode *parser_program(Parser *parser);
```

Kode Program 10.1. Antarmuka `parser.h`

Fungsi `advance` mengambil token berikutnya dari lexer. Fungsi `check` menguji jenis token tanpa mengonsumsinya, dan `match` mengonsumsi token jika jenisnya sesuai. Ketiga fungsi ini menjadi primitif seluruh parser.

10.1.3 Satu Fungsi per Nonterminal

Sebagai contoh, `parse_primary` menangani literal, identifier, pemanggilan fungsi, dan ekspresi dalam kurung. Implementasinya ditunjukkan pada Kode Program 10.2.

```
97  static AstNode *parse_primary(Parser *parser) {
98      Token token = parser->current;
99
100     if (check(parser, TOK_INT_LIT) || check(parser, TOK_FLOAT_LIT) ||
101         check(parser, TOK_CHAR_LIT) || check(parser, TOK_STRING_LIT)) {
102         AstKind kind = AST_INT_LIT;
103         if (token.type == TOK_FLOAT_LIT) {
104             kind = AST_FLOAT_LIT;
105         } else if (token.type == TOK_CHAR_LIT) {
106             kind = AST_CHAR_LIT;
107         } else if (token.type == TOK_STRING_LIT) {
108             kind = AST_STRING_LIT;
109         }
110         advance(parser);
111         return ast_new(kind, token);
112     }
113
114     if (check(parser, TOK_IDENT)) {
115         AstNode *node;
116         advance(parser);
117         if (match(parser, TOK_LPAREN)) {
118             node = ast_new(AST_CALL, token);
119             if (!check(parser, TOK_RPAREN)) {
120                 ast_add_child(node, parse_expr(parser));
121                 while (match(parser, TOK_COMMA)) {
122                     ast_add_child(node, parse_expr(parser));
123                 }
124             }
125             expect(parser, TOK_RPAREN, "diharapkan ')'");
126             return node;
127         }
128         return ast_new(AST_IDENT, token);
129     }
130
131     if (match(parser, TOK_LPAREN)) {
132         AstNode *node = parse_expr(parser);
```

```
133         expect(parser, TOK_RPAREN, "diharapkan ')'");
134         return node;
135     }
136
137     error(parser, "diharapkan ekspresi");
138     advance(parser);
139     return ast_new(AST_IDENT, token);
140 }
```

Kode Program 10.2. Fungsi `parse_primary`

Setiap cabang memeriksa token saat ini dan memilih aturan yang sesuai. Jika tidak ada cabang yang cocok, parser melaporkan kesalahan dan mengembalikan simpul pengganti agar analisis dapat dilanjutkan.

10.1.4 Keuntungan dan Keterbatasan

Recursive descent mudah ditulis, mudah di-debug, dan menghasilkan pesan kesalahan yang baik. Keterbatasannya adalah tidak semua grammar dapat diurai secara top-down. Left recursion harus dihapus, dan grammar harus difaktorkan agar parser dapat memilih aturan berdasarkan satu token lookahead.

Informasi

Recursive descent adalah bentuk khusus predictive parsing. Jika grammar memenuhi syarat LL(1), recursive descent dapat dibangun langsung dari tabel parsing. Jika tidak, parser mungkin memerlukan lookahead tambahan atau backtracking terbatas.

10.2 Himpunan FIRST dan FOLLOW

10.2.1 Definisi FIRST

Definisi 10.1 (FIRST)**.** FIRST(α) adalah himpunan terminal yang dapat muncul sebagai simbol pertama pada string yang diturunkan dari α. Jika α dapat menurunkan ε, maka $\varepsilon \in \text{FIRST}(\alpha)$.

Aturan perhitungan:

1. jika X terminal, $\text{FIRST}(X) = \{X\}$;
2. jika $X \to \varepsilon$, tambahkan ε;
3. jika $X \to Y_1 Y_2 \cdots Y_k$, tambahkan $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$; jika Y_1 nullable, lanjut ke Y_2, dan seterusnya;
4. ulangi sampai tidak ada perubahan.

10.2.2 Definisi FOLLOW

Definisi 10.2 (FOLLOW)**.** FOLLOW(A) untuk nonterminal A adalah himpunan terminal yang dapat muncul tepat setelah A pada suatu bentuk kalimat. Simbol akhir masukan \$ ditambahkan ke FOLLOW simbol awal.

Aturan perhitungan:

1. tambahkan \$ ke FOLLOW(S);
2. untuk aturan $A \to \alpha B \beta$, tambahkan FIRST(β) $\setminus \{\varepsilon\}$ ke FOLLOW(B);
3. jika β nullable atau kosong, tambahkan FOLLOW(A) ke FOLLOW(B);
4. ulangi sampai stabil.

10.2.3 Contoh

Untuk grammar

$$\begin{aligned} &E \to TE', && E' \to +TE' \mid \varepsilon, \\ &T \to FT', && T' \to *FT' \mid \varepsilon, \\ &F \to (E) \mid \texttt{id}, \end{aligned}$$

hasil perhitungan adalah:

Tabel 10.1. FIRST dan FOLLOW grammar ekspresi

Nonterminal	FIRST	FOLLOW
E	$\{\texttt{id}, (\}$	$\{\$,)\}$
E'	$\{+, \varepsilon\}$	$\{\$,)\}$
T	$\{\texttt{id}, (\}$	$\{\$,), +\}$
T'	$\{*, \varepsilon\}$	$\{\$,), +\}$
F	$\{\texttt{id}, (\}$	$\{\$,), *, +\}$

10.2.4 Alat Bantu

Perhitungan manual rawan kesalahan. Alat `first_follow.py` menghitung FIRST dan FOLLOW secara otomatis. Potongan fungsi `compute_first` ditunjukkan pada Kode Program 10.3.

```
61 def compute_first(productions, nonterminals):
62     first = {symbol: set() for symbol in nonterminals}
63     changed = True
64
```

```
    while changed:
        changed = False
        for lhs, alternatives in productions.items():
            for alternative in alternatives:
                if not alternative or alternative == [EPSILON]:
                    if EPSILON not in first[lhs]:
                        first[lhs].add(EPSILON)
                        changed = True
                    continue

                nullable = True
                for symbol in alternative:
                    if symbol not in nonterminals:
                        if symbol not in first[lhs]:
                            first[lhs].add(symbol)
                            changed = True
                        nullable = False
                        break

                    before = len(first[lhs])
                    first[lhs] |= first[symbol] - {EPSILON}
                    if len(first[lhs]) != before:
                        changed = True

                    if EPSILON not in first[symbol]:
                        nullable = False
                        break

                if nullable and EPSILON not in first[lhs]:
                    first[lhs].add(EPSILON)
                    changed = True

    return first
```

Kode Program 10.3. Fungsi `compute_first` pada `first_follow.py`

Alat ini membaca grammar dalam format sederhana dan mencetak FIRST/FOLLOW. Gunakan alat untuk memeriksa hasil manual sebelum membangun tabel LL(1).

10.2.5 Kondisi LL(1)

Sebuah grammar bersifat LL(1) jika untuk setiap nonterminal A dan setiap pasangan aturan $A \to \alpha$ dan $A \to \beta$ berlaku:

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing,$$

dan jika salah satu nullable, maka

$$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing.$$

Jika kondisi ini dilanggar, tabel parsing akan memiliki sel ganda dan grammar bukan LL(1).

10.3 Tabel Parsing LL(1) dan Predictive Parser

10.3.1 Konstruksi Tabel

Tabel parsing $M[A, a]$ berisi aturan produksi untuk nonterminal A dan token lookahead a. Konstruksinya:

1. untuk setiap aturan $A \to \alpha$ dan setiap $a \in \text{FIRST}(\alpha)$, isi $M[A, a] = A \to \alpha$;
2. jika $\varepsilon \in \text{FIRST}(\alpha)$, untuk setiap $b \in \text{FOLLOW}(A)$, isi $M[A, b] = A \to \alpha$;
3. sel yang tidak terisi berarti kesalahan sintaksis.

Tabel 10.2. Tabel LL(1) untuk grammar ekspresi

	`id`	`+`	`*`	`$`
E	$E \to TE'$			
E'		$E' \to +TE'$		$E' \to \varepsilon$
T	$T \to FT'$			
T'		$T' \to \varepsilon$	$T' \to *FT'$	$T' \to \varepsilon$
F	$F \to$ `id`			

Tabel pada Tabel 10.2 belum memuat kolom `(` dan `)`. Untuk kelengkapan, sel E dan T pada kolom `(` diisi aturan yang sama, sedangkan sel E' dan T' pada kolom `)` diisi ε.

10.3.2 Algoritma Predictive Parser

Predictive parser menggunakan stack eksplisit:

1. dorong \$ dan simbol awal ke stack;
2. a adalah token saat ini;
3. selama stack tidak kosong:
   (a) X adalah puncak stack;

(b) jika X terminal atau \$, cocokkan dengan a; jika cocok, pop dan majukan a;

(c) jika X nonterminal, lihat $M[X, a]$;

(d) jika aturan $X \to Y_1 \cdots Y_k$ ada, pop X dan dorong $Y_k, \ldots, Y_1$;

(e) jika sel kosong, laporkan kesalahan.

10.3.3 Contoh Penelusuran

Untuk masukan `id` + `id` ∗ `id` \$, penelusuran stack adalah:

Tabel 10.3. Penelusuran predictive parser

Stack	Masukan	Aksi
$\$E$	`id` + `id` ∗ `id` \$	$E \to TE'$
$\$E'T$	`id` + `id` ∗ `id` \$	$T \to FT'$
$\$E'T'F$	`id` + `id` ∗ `id` \$	$F \to$ `id`
$\$E'T'$`id`	`id` + `id` ∗ `id` \$	cocokkan `id`
$\$E'T'$	+`id` ∗ `id` \$	$T' \to \varepsilon$
$\$E'$	+`id` ∗ `id` \$	$E' \to +TE'$
$\$E'T+$	+`id` ∗ `id` \$	cocokkan +
$\$E'T$	`id` ∗ `id` \$	$T \to FT'$
$\$E'T'F$	`id` ∗ `id` \$	$F \to$ `id`
$\$E'T'$`id`	`id` ∗ `id` \$	cocokkan `id`
$\$E'T'$	∗`id` \$	$T' \to *FT'$
$\$E'T'F*$	∗`id` \$	cocokkan ∗
$\$E'T'F$	`id` \$	$F \to$ `id`
$\$E'T'$`id`	`id` \$	cocokkan `id`
$\$E'T'$	\$	$T' \to \varepsilon$
$\$E'$	\$	$E' \to \varepsilon$
\$	\$	terima

Penelusuran pada Tabel 10.3 menunjukkan bahwa parser tidak pernah mundur. Setiap keputusan diambil dari tabel berdasarkan token saat ini, sehingga waktu parsing linear terhadap panjang masukan.

10.4 Konstruksi Recursive Descent yang Sinkron dengan Tabel

10.4.1 Dari Tabel ke Fungsi

Setiap nonterminal menjadi satu fungsi. Isi fungsi mengikuti aturan pada tabel LL(1):

- untuk aturan $A \to X_1 \cdots X_k$, panggil `parseX1`, `parseX2`, dan seterusnya;
- untuk terminal, panggil `match`;
- untuk ε, tidak ada aksi;
- untuk sel kosong, laporkan kesalahan.

Tabel berikut memetakan nonterminal ke fungsi parser:

Tabel 10.4. Pemetaan nonterminal ke fungsi parser

Nonterminal	Fungsi	Tanggung jawab
`program`	`parser_program`	seluruh berkas
`topdecl`	`parse_topdecl`	deklarasi fungsi
`block`	`parse_block`	daftar pernyataan
`stmt`	`parse_stmt`	satu pernyataan
`expr–primary`	`parse_expr`, ...	ekspresi berprioritas

10.4.2 Pemilihan Berdasarkan Token

Fungsi `parse_stmt` memilih cabang berdasarkan token awal. Potongan implementasinya ditunjukkan pada Kode Program 10.4.

```
286 static AstNode *parse_stmt(Parser *parser) {
287     Token token = parser->current;
288 
289     if (is_type(token.type)) {
290         AstNode *node = parse_decl(parser);
291         expect(parser, TOK_SEMI, "diharapkan ';' setelah deklarasi");
292         return node;
293     }
294 
295     if (check(parser, TOK_LBRACE)) {
296         return parse_block(parser);
297     }
298 
299     if (check(parser, TOK_IF)) {
300         return parse_if(parser);
301     }
```

```
302
303     if (check(parser, TOK_WHILE)) {
304         return parse_while(parser);
305     }
306
307     if (check(parser, TOK_FOR)) {
308         return parse_for(parser);
309     }
310
311     if (check(parser, TOK_RETURN)) {
312         AstNode *node = ast_new(AST_RETURN, token);
313         advance(parser);
314         if (!check(parser, TOK_SEMI)) {
315             ast_add_child(node, parse_expr(parser));
316         }
317         expect(parser, TOK_SEMI, "diharapkan ';' setelah return");
318         return node;
319     }
320
321     if (check(parser, TOK_BREAK)) {
322         AstNode *node = ast_new(AST_BREAK, token);
323         advance(parser);
324         expect(parser, TOK_SEMI, "diharapkan ';' setelah break");
325         return node;
326     }
327
328     if (check(parser, TOK_CONTINUE)) {
329         AstNode *node = ast_new(AST_CONTINUE, token);
330         advance(parser);
```

Kode Program 10.4. Bagian awal `parse_stmt`

Setiap cabang memeriksa jenis token dan memanggil fungsi yang sesuai. Jika tidak ada cabang yang cocok, parser melaporkan kesalahan dan mengembalikan simpul pengganti.

10.4.3 Sinkronisasi

Setelah kesalahan, parser harus kembali ke jalur yang dapat dilanjutkan. Fungsi `synchronize` memajukan token sampai titik aman, misalnya titik koma atau kurung tutup. Implementasinya ditunjukkan pada Kode Program 10.5.

```
42 static void synchronize(Parser *parser) {
43     while (!check(parser, TOK_EOF) &&
44            !check(parser, TOK_SEMI) &&
45            !check(parser, TOK_RBRACE)) {
46         advance(parser);
```

```
47      }
48      if (check(parser, TOK_SEMI)) {
49          advance(parser);
50      }
51  }
52
53  static int is_type(TokenType type) {
54      return type == TOK_INT || type == TOK_FLOAT ||
55             type == TOK_CHAR || type == TOK_VOID;
```

Kode Program 10.5. Fungsi `synchronize`

Titik sinkronisasi yang umum adalah `;`, `}`, dan awal pernyataan berikutnya. Pemilihan titik sinkronisasi memengaruhi jumlah kesalahan yang dilaporkan: terlalu agresif dapat menghasilkan laporan palsu, terlalu konservatif dapat melewatkan kesalahan.

10.4.4 Verifikasi dengan Tabel

Setelah implementasi, bandingkan perilaku parser dengan tabel LL(1):

1. untuk setiap sel tabel, uji masukan yang memicu aturan tersebut;
2. pastikan parser tidak memilih aturan yang salah;
3. uji sel kosong dan pastikan kesalahan dilaporkan.

Verifikasi ini penting karena recursive descent yang ditulis manual dapat menyimpang dari tabel tanpa terlihat. Pengujian sistematis membantu menemukan perbedaan tersebut.

10.5 Penanganan Kesalahan Sintaksis

10.5.1 Jenis Kesalahan

Kesalahan sintaksis terjadi ketika aliran token tidak sesuai dengan tata bahasa. Contoh umum:

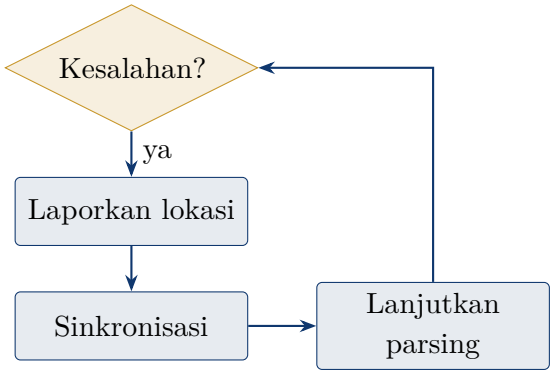
- titik koma hilang;
- kurung tidak seimbang;
- kata kunci yang salah;
- ekspresi tidak lengkap;
- deklarasi fungsi tanpa nama.

Kesalahan sintaksis berbeda dari kesalahan leksikal. Lexer hanya melihat karakter, sedangkan parser melihat token dan struktur. Karena itu, parser dapat memberikan pesan yang lebih kontekstual.

10.5.2 Strategi Pemulihan

Tiga strategi pemulihan yang umum:

1. **panic mode**: lewati token sampai titik sinkronisasi;
2. **phrase-level recovery**: perbaiki token lokal, misalnya sisipkan titik koma yang hilang;
3. **error production**: tambahkan aturan yang mengenali kesalahan umum dan menghasilkan pesan khusus.



Gambar 10.2. Alur panic mode pada parser

10.5.3 Pesan Kesalahan yang Baik

Pesan kesalahan sebaiknya memuat:

- lokasi: baris dan kolom;
- harapan: token apa yang diharapkan;
- temuan: token apa yang ditemukan;
- saran: perbaikan yang mungkin.

Tabel 10.5. Contoh pesan kesalahan sintaksis

Masukan	Pesan
`int a = 1`	`baris 1, kolom 10: diharapkan ';' setelah deklarasi, ditemukan 'return'`
`if (a`	`baris 1, kolom 6: diharapkan ')' setelah kondisi`
`return ;`	`baris 1, kolom 8: ekspresi diharapkan setelah return`
`int (`	`baris 1, kolom 5: nama fungsi diharapkan`

10.5.4 Batas Pemulihan

Parser tidak dapat memperbaiki semua kesalahan. Jika masukan terlalu rusak, parser sebaiknya berhenti setelah sejumlah kesalahan dan melaporkan ringkasan. Strategi ini mencegah ledakan pesan dan membantu pengguna fokus pada masalah utama.

Informasi

Kualitas pesan kesalahan sering menjadi pembeda antara compiler yang menyenangkan dan yang menyulitkan. Investasi pada pesan yang baik memberi manfaat besar bagi pengguna.

Contoh

Contoh 10.3 (Menghitung FIRST)**.** Untuk grammar

$$E \to TE', \quad E' \to +TE' \mid \varepsilon, \quad T \to FT', \quad T' \to *FT' \mid \varepsilon, \quad F \to (E) \mid \texttt{id},$$

$\text{FIRST}(F) = \{\texttt{id}, (\}$. Karena $T \to FT'$ dan F tidak nullable, $\text{FIRST}(T) = \text{FIRST}(F)$. Demikian pula $\text{FIRST}(E) = \text{FIRST}(T)$. Untuk E', $\text{FIRST}(E') = \{+, \varepsilon\}$.

Contoh 10.4 (Menghitung FOLLOW)**.** $\text{FOLLOW}(E) = \{\$,)\}$ karena E adalah simbol awal dan muncul di dalam kurung. $\text{FOLLOW}(E')$ sama dengan $\text{FOLLOW}(E)$. $\text{FOLLOW}(T) = \{\$,), +\}$ karena T diikuti E', dan $\text{FIRST}(E')$ memuat $+$. $\text{FOLLOW}(F) = \{\$,), *, +\}$ karena F diikuti T', dan $\text{FIRST}(T')$ memuat $*$.

Contoh 10.5 (Mengisi tabel LL(1))**.** Aturan $E \to TE'$ diisikan pada kolom $\text{FIRST}(T) = \{\texttt{id}, (\}$. Aturan $E' \to \varepsilon$ diisikan pada kolom $\text{FOLLOW}(E') = \{\$,)\}$. Aturan $T' \to *FT'$ diisikan pada kolom $*$. Sel yang tidak terisi menandakan kesalahan sintaksis.

Contoh 10.6 (Penelusuran stack)**.** Untuk masukan $\texttt{id} + \texttt{id}$ \$, stack dimulai dengan $\$E$. Aturan $E \to TE'$ menghasilkan stack $\$E'T$. Aturan $T \to FT'$ menghasilkan $\$E'T'F$. Aturan $F \to \texttt{id}$ menghasilkan $\$E'T'\texttt{id}$, lalu `id` dicocokkan dan di-pop. Setelah $T' \to \varepsilon$, $E' \to +TE'$, dan seterusnya, parser menerima masukan ketika stack hanya berisi \$.

Contoh 10.7 (Pemulihan titik koma)**.** Untuk masukan

```
int a = 1
return 0;
```

parser melaporkan `baris 1, kolom 10: diharapkan ';' setelah deklarasi, ditemukan 'return'`. Setelah itu, parser menyinkronkan ke titik koma berikutnya dan melanjutkan. Program tetap dapat dianalisis lebih lanjut meskipun mengandung kesalahan.

Aktivitas Pembelajaran dan Praktikum

Tabel 10.6. Rencana aktivitas Bab X

Sesi	Bentuk	Kegiatan
1	Ceramah	Recursive descent dan predictive parsing
2	Latihan	Menghitung FIRST dan FOLLOW
3	Lokakarya	Membangun tabel LL(1) dan menelusuri stack
4	Praktikum	M2: parser dan pengujian

M2 / Praktikum 2: Parser

Tujuan. Membangun recursive descent parser untuk Subset-C yang menghasilkan AST dan menangani kesalahan sintaksis.

Keluaran.

1. Parser untuk ekspresi, deklarasi, dan pernyataan kontrol.
2. AST minimal untuk ekspresi dan pernyataan.
3. Minimal 20 kasus uji: valid dan tidak valid.
4. Diagnostik sintaksis dengan lokasi dan harapan token.
5. Laporan singkat.

Langkah kerja.

1. Hitung FIRST dan FOLLOW untuk grammar hasil Bab IX.
2. Bangun tabel LL(1) dan periksa konflik.
3. Implementasikan fungsi parser per nonterminal.

4. Bangun AST saat mengurangi aturan.
5. Tambahkan sinkronisasi dan pesan kesalahan.
6. Uji dengan program valid dan tidak valid.

Rubrik.

Tabel 10.7. Rubrik Praktikum 2

Aspek	Indikator	Bobot
Kebenaran	Parser menerima program valid, menolak yang tidak valid	40%
Kualitas kode	Struktur fungsi, penamaan, komentar	30%
Kelengkapan	Ekspresi, pernyataan, AST, diagnostik	20%
Pengujian	Kasus normal, batas, dan kesalahan	10%

Definisi selesai.

☐ Parser lulus 20 kasus uji.

☐ AST terbentuk untuk program valid.

☐ Diagnostik menyebut lokasi dan harapan.

☐ Laporan diserahkan.

Latihan dan Refleksi

Latihan

1. Hitung FIRST untuk setiap nonterminal pada grammar ekspresi.
2. Hitung FOLLOW untuk setiap nonterminal pada grammar ekspresi.
3. Bangun tabel LL(1) untuk grammar tersebut.
4. Telusuri stack untuk masukan `id * id + id $`.
5. Jelaskan mengapa grammar dengan left recursion tidak dapat dipakai untuk recursive descent.
6. Rancang fungsi `parse_if` dan jelaskan pemulihannya.
7. Berikan contoh kesalahan sintaksis dan pesan yang baik.
8. Jelaskan perbedaan panic mode dan phrase-level recovery.

Refleksi

Refleksi

1. Bagian mana dari perhitungan FIRST/FOLLOW yang paling sulit?
2. Apakah Anda dapat menjelaskan mengapa parser LL(1) tidak pernah mundur?
3. Bagaimana kualitas pesan kesalahan memengaruhi pengalaman pengguna?

Asesmen

Bab X memuat Praktikum 2 (8%) dan bagian UTS (20%). Fokusnya adalah FIRST/FOLLOW, LL(1), dan implementasi parser.

Tabel 10.8. Instrumen asesmen Bab X

Instrumen	Indikator	Bobot
Praktikum 2	Parser, AST, 20 tes, diagnostik	8%
UTS	FIRST/FOLLOW, tabel LL(1), penelusuran	20%
Kuis	Recursive descent dan pemulihan	formatif

Kriteria kelulusan: parser menerima program valid, menolak program tidak valid dengan pesan yang informatif, dan membangun AST yang sesuai dengan struktur program.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan prinsip recursive descent.

☐ Saya dapat menghitung FIRST.

☐ Saya dapat menghitung FOLLOW.

☐ Saya dapat membangun tabel LL(1).

☐ Saya dapat menelusuri predictive parser.

☐ Saya dapat mengimplementasikan parser per nonterminal.

☐ Saya dapat menangani kesalahan sintaksis.

☐ Saya dapat membangun AST minimal.

☐ Saya sudah menyelesaikan M2/Praktikum 2.

Rangkuman

Pada bab ini telah dibahas:

- recursive descent dan satu fungsi per nonterminal;
- himpunan FIRST dan FOLLOW serta cara menghitungnya;
- tabel parsing LL(1) dan predictive parser;
- konstruksi recursive descent yang sinkron dengan tabel;
- penanganan kesalahan sintaksis dan pemulihan.

Bab XI akan memperkenalkan parsing bottom-up, dimulai dari model shift-reduce dan operator precedence. Pendekatan ini menangani left recursion secara alami dan menjadi dasar parser LR pada Bab XII.

Evaluasi

1. Jelaskan prinsip recursive descent.
2. Hitung FIRST dan FOLLOW untuk grammar $S \to AB$, $A \to a \mid \varepsilon$, $B \to b$.
3. Bangun tabel LL(1) untuk grammar pada Soal 2.
4. Telusuri stack untuk masukan `a b $`.
5. Mengapa grammar dengan left recursion harus ditransformasi?
6. Jelaskan perbedaan panic mode dan error production.
7. Rancang pesan kesalahan untuk deklarasi tanpa titik koma.
8. Jelaskan hubungan antara tabel LL(1) dan kode recursive descent.

Kunci dan Rubrik Singkat

- Soal 1–4: ketepatan perhitungan (70%), penelusuran (30%).
- Soal 5–6: pemahaman konsep (70%), contoh (30%).
- Soal 7–8: kualitas pesan dan penjelasan (60%), argumentasi (40%).

BAB XI

Bottom-Up Parsing: Shift-Reduce dan Operator Precedence

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 4.2

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan model shift-reduce dan peran stack;
2. mengidentifikasi handle dan viable prefix;
3. menjelaskan konflik shift-reduce dan reduce-reduce;
4. membangun tabel operator precedence;
5. membandingkan parsing top-down dan bottom-up.

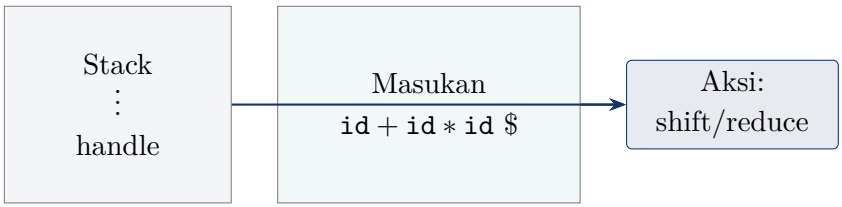
Materi Pokok

- Model shift-reduce, stack, dan handle.
- Viable prefix dan konflik.
- Operator precedence grammar.
- Tabel dan algoritma operator precedence.
- Perbandingan top-down dan bottom-up.

11.1 Model Shift-Reduce, Stack, dan Handle

11.1.1 Ide Dasar

Parser bottom-up membangun pohon urai dari daun ke akar. Pada setiap langkah, parser membaca token dari kiri ke kanan dan memutuskan apakah akan *shift* (mendorong token ke stack) atau *reduce* (mengganti ruas kanan aturan dengan nonterminal). Parser menerima masukan ketika stack berisi simbol awal dan seluruh masukan telah dibaca.



Gambar 11.1. Model shift-reduce dengan stack dan masukan

11.1.2 Contoh Tata Bahasa

Sepanjang bab ini digunakan tata bahasa ekspresi:

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to \texttt{id}.$$

Tata bahasa ini left-recursive, sehingga cocok untuk bottom-up tetapi tidak cocok untuk top-down tanpa transformasi.

11.1.3 Handle

Definisi 11.1 (Handle)**.** Handle adalah substring pada bentuk kalimat yang cocok dengan ruas kanan suatu aturan produksi dan dapat direduksi menjadi nonterminal sebelah kiri.

Pada bentuk kalimat $\texttt{id} + \texttt{id} * \texttt{id}$, handle pertama adalah $\texttt{id}$ yang direduksi menjadi F, lalu T, lalu E. Pemilihan handle yang tepat menentukan keberhasilan parsing.

11.1.4 Penelusuran Shift-Reduce

Untuk masukan $\texttt{id} + \texttt{id} * \texttt{id}$ \$:

Tabel 11.1. Penelusuran shift-reduce

Stack	Masukan	Aksi
	$\texttt{id} + \texttt{id} * \texttt{id}\ \$$	shift
$\texttt{id}$	$+\texttt{id} * \texttt{id}\ \$$	reduce $F \to \texttt{id}$
F	$+\texttt{id} * \texttt{id}\ \$$	reduce $T \to F$
T	$+\texttt{id} * \texttt{id}\ \$$	reduce $E \to T$
E	$+\texttt{id} * \texttt{id}\ \$$	shift $+$
$E+$	$\texttt{id} * \texttt{id}\ \$$	shift $\texttt{id}$
$E + \texttt{id}$	$*\texttt{id}\ \$$	reduce $F \to \texttt{id}$
$E + F$	$*\texttt{id}\ \$$	reduce $T \to F$
$E + T$	$*\texttt{id}\ \$$	shift $*$
$E + T*$	$\texttt{id}\ \$$	shift $\texttt{id}$
$E + T * \texttt{id}$	$\$$	reduce $F \to \texttt{id}$
$E + T * F$	$\$$	reduce $T \to T * F$
$E + T$	$\$$	reduce $E \to E + T$
E	$\$$	terima

Penelusuran pada Tabel 11.1 menunjukkan bahwa parser tidak perlu melihat seluruh masukan. Keputusan diambil berdasarkan isi stack dan token saat ini.

11.1.5 Keuntungan

Bottom-up parsing dapat menangani left recursion secara alami, mendukung grammar yang lebih luas, dan menjadi dasar parser LR. Keterbatasannya adalah tabel dan algoritma yang lebih kompleks daripada recursive descent.

11.2 Viable Prefix dan Konflik

11.2.1 Viable Prefix

Definisi 11.2 (Viable prefix)**.** Viable prefix adalah prefiks dari bentuk kalimat yang dapat muncul pada stack parser shift-reduce tanpa melewati handle.

Viable prefix menentukan apakah stack masih dapat diperluas menjadi bentuk kalimat yang valid. Jika stack bukan viable prefix, parser tidak mungkin mencapai simbol awal.

11.2.2 Konflik Shift-Reduce

Konflik shift-reduce terjadi ketika parser dapat memilih antara shift dan reduce untuk stack dan token yang sama. Konflik ini sering muncul pada grammar ambigu, misalnya dangling else atau ekspresi tanpa tingkat prioritas.

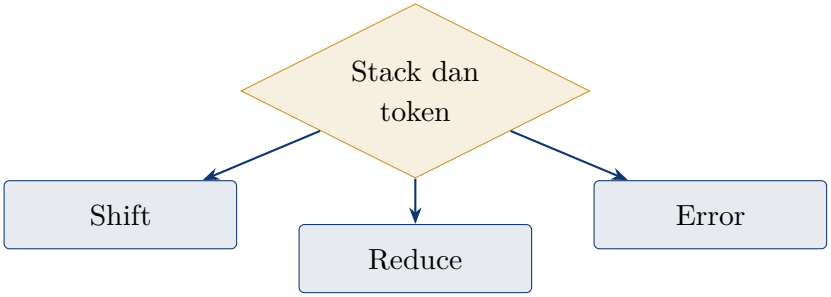
Contoh 11.3 (Konflik shift-reduce)**.** Pada grammar

$$E \to E + E \mid E * E \mid \texttt{id},$$

setelah stack berisi $E + E$ dan token berikutnya $*$, parser dapat mengurangi $E + E$ menjadi E atau menggeser $*$ untuk membentuk $E * E$. Pilihan ini menentukan apakah penjumlahan atau perkalian yang lebih dulu dikelompokkan.

11.2.3 Konflik Reduce-Reduce

Konflik reduce-reduce terjadi ketika dua aturan berbeda dapat diterapkan pada handle yang sama. Konflik ini biasanya menunjukkan bahwa grammar terlalu ambigu atau memiliki aturan yang tumpang tindih.



Gambar 11.2. Tiga kemungkinan aksi pada parser shift-reduce

11.2.4 Menyelesaikan Konflik

Konflik dapat diselesaikan dengan:

- mengubah grammar (precedence, associativity, factoring);
- menambahkan aturan prioritas pada parser generator;
- memilih shift atau reduce berdasarkan kebijakan tetap.

Pemilihan kebijakan harus didokumentasikan. Parser generator seperti Bison secara default memilih shift, tetapi keputusan ini dapat menyembunyikan bug jika tidak disengaja.

11.3 Operator Precedence Grammar

11.3.1 Definisi

Operator precedence grammar adalah grammar yang memenuhi:

1. tidak memiliki produksi ε;
2. tidak memiliki dua nonterminal bersebelahan pada ruas kanan.

Sifat ini memungkinkan parser menentukan hubungan precedence antara terminal yang berdekatan tanpa membangun tabel LR lengkap.

11.3.2 Relasi Precedence

Tiga relasi khusus digunakan:

- $a \lessdot b$: a memiliki precedence lebih rendah daripada b;
- $a \doteq b$: a dan b berada pada tingkat yang sama;
- $a \gtrdot b$: a memiliki precedence lebih tinggi daripada b.

Simbol $<$, $=$, dan $>$ di sini bukan operator aritmetika, melainkan relasi precedence.

11.3.3 Tabel Operator Precedence

Untuk grammar ekspresi dengan terminal $\{\texttt{id}, +, *, \$, (,)\}$, tabelnya berbentuk:

Tabel 11.2. Tabel operator precedence sederhana

	id	+	*	(	)
id		$\gtrdot$	$\gtrdot$		$\gtrdot$
+	$\lessdot$	$\gtrdot$	$\lessdot$	$\lessdot$	$\gtrdot$
*	$\lessdot$	$\gtrdot$	$\gtrdot$	$\lessdot$	$\gtrdot$
(	$\lessdot$	$\lessdot$	$\lessdot$	$\lessdot$	$\doteq$
)		$\gtrdot$	$\gtrdot$		$\gtrdot$

Sel kosong menandakan kesalahan sintaksis. Tabel ini lebih kecil daripada tabel LR karena hanya memuat terminal, bukan himpunan item.

11.3.4 Contoh Relasi

Untuk ekspresi $\texttt{id} + \texttt{id} * \texttt{id}$:

- $\texttt{id}\cdot > +$: `id` harus direduksi sebelum $+$;
- $+ < \cdot\texttt{id}$: $+$ menunggu operand;
- $+ < \cdot *$: $*$ memiliki precedence lebih tinggi;
- $*\cdot > \texttt{id}$: $*$ menunggu operand.

Relasi ini menentukan kapan parser harus shift dan kapan harus reduce. Parser membandingkan terminal teratas stack dengan token saat ini.

11.3.5 Keterbatasan

Operator precedence hanya menangani bahasa dengan struktur operator yang sederhana. Grammar dengan konstruksi seperti dangling else, deklarasi, atau pemanggilan fungsi memerlukan parser LR yang lebih umum. Karena itu, operator precedence lebih cocok untuk ekspresi daripada seluruh bahasa pemrograman.

11.4 Tabel dan Algoritma Operator Precedence

11.4.1 Algoritma

Parser operator precedence menggunakan stack yang menyimpan terminal dan nonterminal. Pada setiap langkah:

1. cari terminal teratas pada stack, sebut a;
2. bandingkan a dengan token saat ini b;
3. jika $a < \cdot b$, shift b;
4. jika $a \doteq b$, shift atau tangani kurung;
5. jika $a\cdot > b$, reduce handle teratas;
6. jika sel kosong, laporkan kesalahan.

Handle adalah substring pada stack yang cocok dengan ruas kanan aturan. Setelah reduce, parser membandingkan kembali terminal teratas dengan token saat ini.

11.4.2 Contoh Penelusuran

Untuk masukan `id` + `id` ∗ `id`:

1. shift `id`, reduce ke F, lalu T, lalu E;
2. shift $+$;
3. shift `id`, reduce ke F, lalu T;
4. bandingkan $+$ dengan $*$: $+ < \cdot *$, shift $*$;
5. shift `id`, reduce ke F;
6. bandingkan $*$ dengan \$: $* \cdot > \$$, reduce $T \to T * F$;
7. bandingkan $+$ dengan \$: $+ \cdot > \$$, reduce $E \to E + T$;
8. terima.

11.4.3 Implementasi Sederhana

Kalkulator shift-reduce pada `shift_reduce.c` mengilustrasikan stack nilai dan operator. Program ini bukan parser Subset-C lengkap, tetapi menunjukkan mekanisme shift dan reduce. Potongan fungsinya ditunjukkan pada Kode Program 11.1.

```
27      }
28  }
29
30  static void reduce_top(int *values, int *vtop,
31                         char *ops, int *otop) {
32      int right = values[(*vtop)--];
33      int left = values[(*vtop)--];
34      char op = ops[(*otop)--];
35      values[++(*vtop)] = apply(left, op, right);
36      printf("  reduce: %d %c %d -> %d\n",
37             left, op, right, values[*vtop]);
38  }
39
40  int main(void) {
```

Kode Program 11.1. Fungsi `reduce_top` pada `shift_reduce.c`

Program membaca ekspresi digit tunggal, mendorong nilai dan operator, lalu mengurangi sesuai precedence. Keluaran program menunjukkan urutan shift dan reduce, sehingga mahasiswa dapat membandingkannya dengan penelusuran manual.

11.4.4 Deteksi Kesalahan

Kesalahan terdeteksi ketika relasi precedence tidak terdefinisi. Sebagai contoh, pada masukan `id + +id`, setelah shift + kedua, tabel tidak memiliki relasi yang valid antara + dan +. Parser melaporkan kesalahan dan dapat melakukan pemulihan dengan melewati token sampai titik aman.

Peringatan

Operator precedence tidak menangani semua konflik. Jika grammar memiliki produksi ε atau dua nonterminal bersebelahan, tabel precedence tidak dapat dibangun. Gunakan parser LR untuk grammar yang lebih umum.

11.5 Perbandingan Top-Down dan Bottom-Up

11.5.1 Perbandingan

Tabel 11.3. Perbandingan parsing top-down dan bottom-up

Aspek	Top-down	Bottom-up
Arah	Akar ke daun	Daun ke akar
Left recursion	Harus dihapus	Dapat ditangani
Grammar	LL(1) dan variannya	LR, SLR, LALR, precedence
Tabel	Satu entri per nonterminal	Satu entri per state
Pesan kesalahan	Mudah dibuat kontekstual	Lebih sulit
Implementasi	Recursive descent	Stack dan tabel
Contoh	Parser LL(1)	Bison, Yacc, operator precedence

11.5.2 Kapan Memilih Top-Down

Top-down cocok ketika:

- grammar relatif sederhana dan dapat difaktorkan;
- pesan kesalahan yang baik menjadi prioritas;
- parser ditulis tangan untuk kontrol penuh;
- pembelajaran konsep parsing menjadi tujuan.

11.5.3 Kapan Memilih Bottom-Up

Bottom-up cocok ketika:

- grammar besar dan kompleks;
- left recursion alami dan tidak ingin diubah;
- parser generator tersedia;
- kinerja dan cakupan grammar lebih penting daripada kesederhanaan implementasi.

11.5.4 Jembatan ke LR

Operator precedence adalah bentuk sederhana dari parsing bottom-up. Bab XII akan memperkenalkan LR parsing yang lebih umum: item LR(0), SLR(1), canonical LR(1), dan LALR(1). LR menggabungkan kekuatan bottom-up dengan tabel yang dapat dibangun secara sistematis.

Informasi

Banyak compiler produksi memakai parser generator LR karena mampu menangani grammar bahasa nyata tanpa banyak transformasi. Namun, memahami shift-reduce dan precedence tetap penting untuk membaca konflik dan pesan diagnostik yang dihasilkan generator.

Contoh

Contoh 11.4 (Menemukan handle)**.** Pada bentuk kalimat $\texttt{id} + \texttt{id} * \texttt{id}$, handle pertama adalah `id` paling kiri. Setelah direduksi menjadi F, T, dan E, handle berikutnya adalah `id` kedua. Pemilihan handle yang salah, misalnya mereduksi seluruh $\texttt{id} + \texttt{id}$ terlebih dahulu, menghasilkan pohon yang tidak sesuai precedence.

Contoh 11.5 (Relasi precedence)**.** Untuk $\texttt{id} + \texttt{id} * \texttt{id}$:

$$\texttt{id} \cdot > +, \quad + < \cdot \texttt{id}, \quad + < \cdot *, \quad * \cdot > \texttt{id}.$$

Relasi ini memaksa perkalian direduksi sebelum penjumlahan.

Contoh 11.6 (Unary minus)**.** Operator unary minus memiliki precedence lebih tinggi daripada binary minus. Pada ekspresi $-a * b$, unary minus diterapkan pada a terlebih dahulu, lalu hasilnya dikalikan b. Grammar harus membedakan aturan unary dan binary, atau parser generator harus mendeklarasikan precedence unary secara eksplisit.

Contoh 11.7 (Kesalahan pada operator precedence)**.** Pada masukan $\texttt{id} + +\texttt{id}$, setelah shift $+$ kedua, tabel tidak memiliki relasi antara $+$ dan $+$. Parser melaporkan kesalahan pada token kedua. Pemulihan dapat dilakukan dengan melewati token sampai operator atau operand berikutnya.

Contoh 11.8 (Membaca keluaran kalkulator)**.** Program `shift_reduce.c` mencetak urutan shift dan reduce. Untuk masukan `2+3*4`, program mencetak shift angka 2, shift operator +, shift angka 3, shift operator *, shift angka 4, lalu reduce 3*4, reduce 2+12, dan hasil 14. Urutan ini menunjukkan bahwa perkalian direduksi lebih dahulu.

Aktivitas Pembelajaran dan Praktikum

Tabel 11.4. Rencana aktivitas Bab XI

Sesi	Bentuk	Kegiatan
1	Ceramah	Model shift-reduce dan handle
2	Latihan	Menelusuri stack untuk ekspresi
3	Lokakarya	Membangun tabel operator precedence
4	Praktikum	Kalkulator operator precedence

Lokakarya: Penelusuran dan Kalkulator

Tujuan. Memahami shift-reduce melalui penelusuran manual dan implementasi kalkulator sederhana.

Langkah kerja.

1. Telusuri shift-reduce untuk `id` $*$ `id` $+$ `id`.
2. Tandai handle pada setiap langkah reduce.
3. Bangun tabel operator precedence untuk operator $+$, $-$, $*$, $/$, dan tanda kurung.
4. Modifikasi `shift_reduce.c` agar mendukung operator $-$, $/$, dan tanda kurung.
5. Uji dengan ekspresi yang mengandung precedence dan associativity berbeda.

Definisi selesai.

☐ Penelusuran manual benar.

☐ Tabel precedence lengkap.

☐ Kalkulator mendukung empat operator dan kurung.

☐ Minimal 10 ekspresi uji.

Bab ini tidak menambahkan milestone proyek baru. Latihan menjadi bagian dari Praktikum 3 (7%) yang dilaporkan pada Bab XIII.

Latihan dan Refleksi

Latihan

1. Jelaskan perbedaan shift dan reduce.
2. Definisikan handle dan berikan contoh.
3. Telusuri shift-reduce untuk `id` $*$ `id` $+$ `id`.
4. Jelaskan viable prefix.
5. Berikan contoh konflik shift-reduce.
6. Berikan contoh konflik reduce-reduce.
7. Bangun tabel operator precedence untuk $+$, $*$, dan `id`.
8. Bandingkan top-down dan bottom-up dari sisi grammar yang didukung.

Refleksi

Refleksi

1. Apakah Anda dapat menjelaskan mengapa bottom-up menangani left recursion?
2. Bagian mana dari penelusuran shift-reduce yang paling sulit?
3. Mengapa konflik perlu diselesaikan secara eksplisit?

Asesmen

Asesmen Bab XI menjadi bagian UTS (20%) dan Praktikum 3 (7%). Fokusnya adalah penelusuran shift-reduce dan operator precedence.

Tabel 11.5. Instrumen asesmen Bab XI

Instrumen	Indikator	Bobot
UTS	Penelusuran shift-reduce dan konflik	20%
Lokakarya	Tabel precedence dan kalkulator	bagian P3
Kuis	Handle dan viable prefix	formatif

Kriteria kelulusan: mahasiswa dapat menelusuri shift-reduce, menjelaskan handle dan konflik, serta membangun tabel operator precedence sederhana.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan model shift-reduce.

☐ Saya dapat mengidentifikasi handle.

☐ Saya dapat menjelaskan viable prefix.

☐ Saya dapat menjelaskan konflik shift-reduce dan reduce-reduce.

☐ Saya dapat membangun tabel operator precedence.

☐ Saya dapat menelusuri parser operator precedence.

☐ Saya dapat membandingkan top-down dan bottom-up.

☐ Saya sudah menyelesaikan lokakarya kalkulator.

Rangkuman

Pada bab ini telah dibahas:

- model shift-reduce, stack, dan handle;
- viable prefix dan konflik shift-reduce/reduce-reduce;
- operator precedence grammar dan tabelnya;
- algoritma operator precedence dan deteksi kesalahan;
- perbandingan parsing top-down dan bottom-up.

Bab XII akan memperluas bottom-up parsing menjadi LR parsing: item LR(0), SLR(1), canonical LR(1), dan LALR(1). LR menyediakan tabel sistematis dan menangani grammar yang lebih luas daripada operator precedence.

Evaluasi

1. Jelaskan perbedaan shift dan reduce.
2. Definisikan handle dan viable prefix.
3. Telusuri shift-reduce untuk `id` + `id` ∗ `id`.
4. Jelaskan konflik shift-reduce dengan contoh.
5. Jelaskan konflik reduce-reduce dengan contoh.
6. Bangun tabel operator precedence untuk +, ∗, `id`, dan $.

7. Mengapa operator precedence tidak cocok untuk seluruh bahasa?
8. Bandingkan kelebihan top-down dan bottom-up.

Kunci dan Rubrik Singkat

- Soal 1–2: ketepatan definisi (70%), contoh (30%).
- Soal 3–5: kebenaran penelusuran dan konflik (70%), penjelasan (30%).
- Soal 6–8: ketepatan tabel dan analisis (60%), argumentasi (40%).

BAB XII

LR Parsing: Canonical LR, SLR, LALR, dan Konflik

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 4.2

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan item LR(0), closure, dan goto;
2. membangun automaton LR(0) dan tabel SLR(1);
3. menjelaskan canonical LR(1) dan LALR(1);
4. menganalisis konflik shift-reduce dan reduce-reduce;
5. memilih strategi parsing LL(k) atau LR(k) sesuai kebutuhan.

Materi Pokok

- Item LR(0), closure, dan goto.
- Automaton LR(0) dan tabel SLR(1).
- Canonical LR(1) dan LALR(1).
- Konflik dan penyelesaiannya.
- Perbandingan LL(k) dan LR(k).

12.1 Item LR(0), Closure, dan Goto

12.1.1 Item LR(0)

Definisi 12.1 (Item LR(0))**.** Item LR(0) adalah aturan produksi dengan sebuah titik yang menunjukkan posisi parser. Item $[A \to \alpha \cdot \beta]$ berarti parser telah mengenali α dan mengharapkan β.

Sebagai contoh, aturan $E \to E + T$ memiliki empat item:

$$[E \to \cdot E + T], \quad [E \to E \cdot + T], \quad [E \to E + \cdot T], \quad [E \to E + T \cdot].$$

Titik di awal berarti belum ada yang dikenali; titik di akhir berarti seluruh ruas kanan sudah dikenali dan aturan siap direduksi.

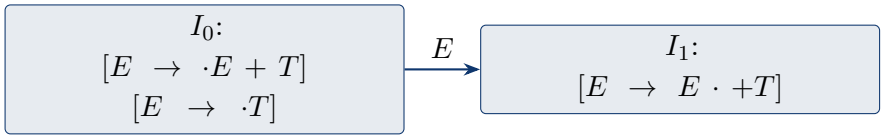
12.1.2 Closure

Jika item $[A \to \alpha \cdot B\beta]$ berada dalam himpunan item dan B adalah nonterminal, maka semua aturan $B \to \gamma$ harus ditambahkan dalam bentuk $[B \to \cdot\gamma]$. Proses ini disebut closure.

Contoh 12.2 (Closure)**.** Untuk item $[E \to \cdot T]$ dan aturan $T \to T * F \mid F$, closure menambahkan $[T \to \cdot T * F]$ dan $[T \to \cdot F]$. Karena F juga nonterminal, closure menambahkan aturan F pula.

12.1.3 Goto

goto(I, X) adalah closure dari himpunan item yang titiknya bergeser melewati simbol X. Jika I memuat $[A \to \alpha \cdot X\beta]$, maka goto$(I, X)$ memuat $[A \to \alpha X \cdot \beta]$.



Gambar 12.1. Item dan transisi goto pada simbol E

12.1.4 Koleksi Kanonik

Koleksi kanonik LR(0) dibangun dengan:

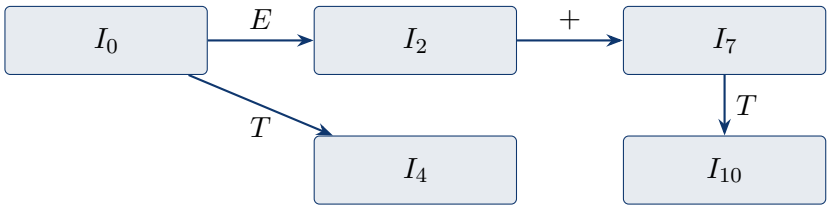
1. mulai dari closure item awal $[S' \to \cdot S]$;
2. untuk setiap state dan simbol X, hitung goto(I, X);
3. jika hasilnya belum ada, tambahkan sebagai state baru;
4. ulangi sampai tidak ada state baru.

Jumlah state bergantung pada grammar. Untuk grammar ekspresi pada `grammar-lr.bnf`, koleksi kanonik memiliki 12 state.

12.2 Automaton LR(0) dan SLR(1)

12.2.1 Automaton LR(0)

Automaton LR(0) adalah graf berarah dengan state berupa himpunan item dan transisi berupa goto. State awal adalah closure item awal. State penerima adalah state yang memuat item $[S' \to S\cdot]$.



Gambar 12.2. Bagian automaton LR(0) untuk grammar ekspresi

12.2.2 Tabel SLR(1)

SLR(1) menggunakan automaton LR(0) dan himpunan FOLLOW untuk memutuskan reduksi:

1. jika item $[A \to \alpha \cdot a\beta]$ dan a terminal, isi ACTION$[I, a]$ = shift goto(I, a);
2. jika item $[A \to \alpha\cdot]$ dan $A \neq S'$, untuk setiap $a \in$ FOLLOW(A), isi ACTION$[I, a]$ = reduce $A \to \alpha$;
3. jika item $[S' \to S\cdot]$, isi ACTION$[I, \$]$ = accept;
4. untuk nonterminal A, isi GOTO$[I, A]$ = goto(I, A).

Tabel 12.1. Potongan tabel SLR(1) untuk grammar ekspresi

State	`id`	`+`	`*`	`$`
0	s5			
1				
2		s7		acc
3		r($T \to F$)	r($T \to F$)	r($T \to F$)
4		r($E \to T$)	s8	r($E \to T$)
5		r($F \to$ `id`)	r($F \to$ `id`)	r($F \to$ `id`)

12.2.3 Alat Bantu

Skrip `slr_table.py` membangun koleksi kanonik dan tabel SLR(1). Fungsi `closure` dan `goto` ditunjukkan pada Kode Program 12.1.

```
def closure(items, productions):
    result = set(items)
    changed = True

    while changed:
        changed = False
        for lhs, rhs, dot in list(result):
            if dot < len(rhs) and rhs[dot] in productions:
                symbol = rhs[dot]
                for alternative in productions[symbol]:
                    item = (symbol, tuple(alternative), 0)
                    if item not in result:
                        result.add(item)
                        changed = True

    return frozenset(result)


def goto(items, symbol, productions):
    moved = set()

    for lhs, rhs, dot in items:
        if dot < len(rhs) and rhs[dot] == symbol:
            moved.add((lhs, rhs, dot + 1))

    if not moved:
        return None

    return closure(moved, productions)
```

Kode Program 12.1. Fungsi `closure` dan `goto`

Fungsi `build_slr_table` memakai FOLLOW untuk mengisi aksi reduce. Potongannya ditunjukkan pada Kode Program 12.2.

```
def build_slr_table(productions, nonterminals, start_symbol):
    states, transitions, augmented = canonical_collection(
        productions, start_symbol
    )
    first = compute_first(augmented, set(augmented.keys()))
    follow = compute_follow(
```

```
        augmented, set(augmented.keys()), list(augmented.keys()), first
    )

    actions = {}
    gotos = {}
    conflicts = []

    for state_index, state in enumerate(states):
        for lhs, rhs, dot in state:
            if dot < len(rhs):
                symbol = rhs[dot]
                if symbol in nonterminals:
                    target = transitions.get((state_index, symbol))
                    if target is not None:
                        gotos[(state_index, symbol)] = target
                else:
                    target = transitions.get((state_index, symbol))
                    if target is not None:
                        actions.setdefault(
                            (state_index, symbol), set()
                        ).add(("shift", target))
            else:
                if lhs == START_PRIME:
                    actions.setdefault(
                        (state_index, "$"), set()
                    ).add(("accept", None))
                else:
                    for terminal in follow.get(lhs, set()):
                        actions.setdefault(
                            (state_index, terminal), set()
                        ).add(("reduce", (lhs, tuple(rhs))))

    for key, values in actions.items():
        if len(values) > 1:
            conflicts.append((key, values))

    return states, transitions, actions, gotos, conflicts, follow
```

Kode Program 12.2. Fungsi `build_slr_table`

Alat ini juga melaporkan konflik. Untuk grammar ekspresi, hasilnya adalah 12 state dan tidak ada konflik.

12.3 Canonical LR(1) dan LALR(1)

12.3.1 Item LR(1)

Item LR(1) menambahkan lookahead:

$$[A \to \alpha \cdot \beta,\ a],$$

dengan a terminal atau \$. Lookahead menunjukkan token yang boleh muncul setelah reduksi. Informasi ini membuat LR(1) lebih presisi daripada SLR(1), yang memakai FOLLOW secara global.

12.3.2 Closure LR(1)

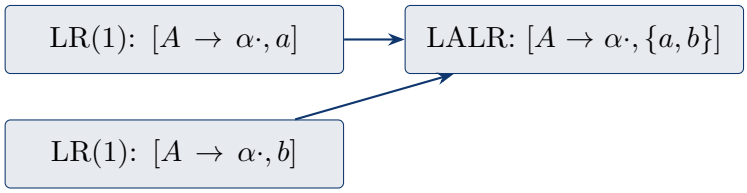
Jika item $[A \to \alpha \cdot B\beta,\ a]$ berada dalam himpunan, maka untuk setiap aturan $B \to \gamma$ dan setiap $b \in \text{FIRST}(\beta a)$, tambahkan $[B \to \cdot\gamma,\ b]$. Perhatikan bahwa lookahead dihitung dari βa, bukan hanya dari FOLLOW(B).

12.3.3 Canonical LR(1)

Koleksi kanonik LR(1) dibangun seperti LR(0), tetapi state memuat item dengan lookahead. Jumlah state LR(1) biasanya jauh lebih besar daripada LR(0). Untuk grammar bahasa nyata, ledakan state dapat menjadi masalah.

12.3.4 LALR(1)

LALR(1) menggabungkan state LR(1) yang memiliki inti (*core*) sama, yaitu himpunan item tanpa lookahead. Penggabungan ini mengurangi jumlah state mendekati LR(0), tetapi mempertahankan sebagian presisi LR(1).



Gambar 12.3. Penggabungan state LR(1) dengan inti sama menjadi LALR(1)

12.3.5 Perbandingan

Tabel 12.2. Perbandingan SLR, LR(1), dan LALR

Aspek	**SLR(1)**	**LR(1)**	**LALR(1)**
Lookahead	FOLLOW global	Per item	Per inti state
Jumlah state	Kecil	Besar	Sedang
Kekuatan	Paling lemah	Paling kuat	Hampir LR(1)
Konflik	Lebih mungkin	Paling sedikit	Sedikit
Praktik	Jarang	Jarang	Umum (Bison/Yacc)

Bison dan Yacc secara default membangun tabel LALR(1). Jika konflik masih muncul, opsi LR(1) kanonik dapat digunakan, tetapi dengan biaya ukuran tabel yang lebih besar.

12.4 Konflik Shift-Reduce dan Reduce-Reduce

12.4.1 Konflik Shift-Reduce

Konflik shift-reduce terjadi ketika satu sel tabel memiliki aksi shift dan reduce. Contoh klasik adalah dangling else:

$$\text{stmt} \to \text{if } E \text{ then stmt} \mid \text{if } E \text{ then stmt else stmt}.$$

Setelah parser melihat `if E then stmt` dan token berikutnya `else`, ia dapat menggeser `else` atau mengurangi `stmt` terlebih dahulu.

12.4.2 Konflik Reduce-Reduce

Konflik reduce-reduce terjadi ketika dua aturan dapat direduksi pada sel yang sama. Contohnya, jika dua nonterminal memiliki ruas kanan yang sama dan FOLLOW yang tumpang tindih:

$$A \to x, \qquad B \to x.$$

Parser tidak dapat menentukan apakah x harus direduksi menjadi A atau B.

12.4.3 Resolusi

Konflik dapat diselesaikan dengan:

1. mengubah grammar agar tidak ambigu;
2. menambahkan precedence dan associativity;

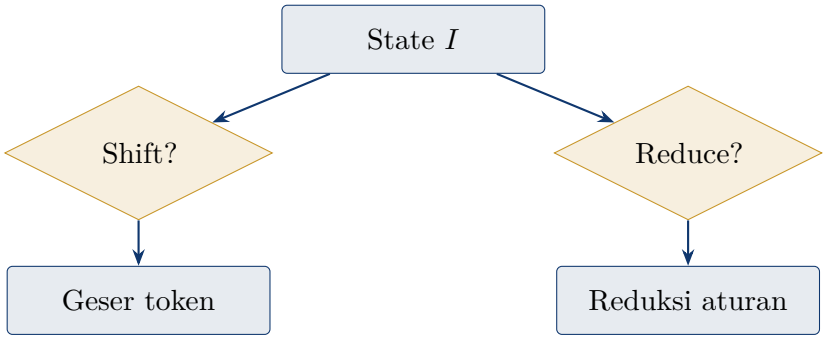
3. memilih aksi default secara eksplisit;

4. memisahkan aturan yang tumpang tindih.

Bison menyediakan deklarasi precedence:

```
%left OR
%left AND
%left EQ NE
%left LT LE GT GE
%left PLUS MINUS
%left STAR SLASH PERCENT
%right NOT UMINUS
```

Deklarasi ini memberitahu Bison cara menyelesaikan konflik shift-reduce berdasarkan prioritas operator.



Gambar 12.4. Konflik shift-reduce: dua aksi untuk satu sel tabel

12.4.4 Membaca Laporan Konflik

Parser generator melaporkan konflik beserta state dan token pemicu. Laporan ini harus dibaca, bukan diabaikan. Konflik yang tidak disengaja sering menandakan grammar yang terlalu ambigu atau aturan yang tumpang tindih.

Peringatan

Jangan hanya menaikkan kapasitas parser atau memilih aksi default untuk menyembunyikan konflik. Pahami penyebabnya terlebih dahulu. Konflik yang tersembunyi dapat menghasilkan pohon urai yang salah.

12.5 LL(k) versus LR(k) dan Pemilihan Strategi

12.5.1 Perbandingan

Tabel 12.3. Perbandingan keluarga LL(k) dan LR(k)

Aspek	**LL(k)**	**LR(k)**
Arah	Kiri ke kanan, derivasi kiri	Kiri ke kanan, derivasi kanan
Lookahead	k token	k token
Left recursion	Tidak didukung	Didukung
Grammar	Harus difaktorkan	Lebih alami
Tabel	Relatif kecil	Lebih besar
Kesalahan	Mudah dilokalisasi	Perlu strategi tambahan
Implementasi	Recursive descent	Stack dan tabel

12.5.2 Pemilihan

LL(k) cocok untuk:

- parser tulisan tangan;
- grammar yang dapat difaktorkan;
- prioritas pada pesan kesalahan.

LR(k) cocok untuk:

- grammar besar dan kompleks;
- parser generator;
- left recursion alami;
- cakupan bahasa yang lebih luas.

12.5.3 Di Balik Layar Bison

Bison membangun tabel LALR(1) dari aturan grammar. Pengguna menulis aturan, deklarasi token, dan semantic action. Bison menghasilkan fungsi `yyparse` yang memakai tabel tersebut. Jika terjadi konflik, Bison melaporkan jumlah dan lokasinya.

Pada Bab XIII, kita akan menulis grammar Bison untuk Subset-C, menambahkan semantic action, dan membangun AST. Pemahaman item dan konflik pada bab ini membantu membaca laporan Bison dan memperbaiki grammar.

Informasi

Tidak ada strategi yang selalu terbaik. Pilihan bergantung pada ukuran bahasa, kebutuhan diagnostik, ketersediaan alat, dan keterampilan tim. Yang penting adalah memahami trade-off dan mendokumentasikan keputusan.

Contoh

Contoh 12.3 (Menghitung closure)**.** Diberikan item $[E \to \cdot T]$ dan aturan $T \to T * F \mid F$. Closure menambahkan $[T \to \cdot T * F]$ dan $[T \to \cdot F]$. Karena F adalah nonterminal dengan aturan $F \to (E) \mid \texttt{id}$, closure juga menambahkan $[F \to \cdot(E)]$ dan $[F \to \cdot \texttt{id}]$.

Contoh 12.4 (Menghitung goto)**.** Untuk state yang memuat $[E \to E \cdot +T]$, goto pada simbol $+$ menghasilkan item $[E \to E + \cdot T]$, lalu closure menambahkan aturan untuk T. State baru ini kemudian memiliki transisi pada simbol-simbol yang dapat mengikuti T.

Contoh 12.5 (Mengisi tabel SLR)**.** State yang memuat $[E \to E + T\cdot]$ menghasilkan aksi reduce $E \to E + T$ pada semua token di FOLLOW(E), yaitu $\{\$,), +\}$. State yang memuat $[T \to T \cdot *F]$ menghasilkan aksi shift pada $*$.

Contoh 12.6 (Konflik dangling else)**.** Pada state setelah `if E then stmt`, token `else` dapat memicu shift (memasangkan else dengan if dalam) atau reduce (mengurangi if luar). Tanpa aturan prioritas, tabel memiliki dua aksi pada sel yang sama. Bison biasanya memilih shift, yang sesuai dengan aturan else terdekat.

Contoh 12.7 (Penggabungan LALR)**.** Dua state LR(1) dengan inti yang sama, misalnya $\{[A \to \alpha\cdot, a]\}$ dan $\{[A \to \alpha\cdot, b]\}$, digabung menjadi satu state LALR $\{[A \to \alpha\cdot, \{a, b\}]\}$. Penggabungan mengurangi jumlah state, tetapi dapat menimbulkan konflik reduce-reduce yang tidak ada pada LR(1) kanonik.

Aktivitas Pembelajaran dan Praktikum

Tabel 12.4. Rencana aktivitas Bab XII

Sesi	Bentuk	Kegiatan
1	Ceramah	Item LR(0), closure, dan goto
2	Latihan	Membangun automaton LR(0)
3	Lokakarya	Tabel SLR dan analisis konflik
4	Praktikum	Verifikasi tabel dan persiapan Bison

Lokakarya: Verifikasi Tabel LR

Tujuan. Membandingkan tabel yang dibangun manual dengan keluaran `slr_table.py`.

Langkah kerja.

1. Bangun koleksi kanonik LR(0) untuk grammar ekspresi secara manual.
2. Jalankan `slr_table.py` pada `spec/grammar-lr.bnf`.
3. Bandingkan jumlah state, isi item, dan tabel ACTION/GOTO.
4. Identifikasi perbedaan dan jelaskan penyebabnya.
5. Uji grammar yang mengandung dangling else dan catat konfliknya.

Definisi selesai.

☐ Koleksi kanonik manual benar.

☐ Tabel manual cocok dengan alat.

☐ Perbedaan terdokumentasi.

☐ Konflik dangling else dianalisis.

Hasil lokakarya menjadi bagian Praktikum 3 (7%) yang dilaporkan pada Bab XIII.

Latihan dan Refleksi

Latihan

1. Definisikan item LR(0) dan berikan contoh.
2. Hitung closure dari $[E \to \cdot T]$.
3. Hitung goto dari state yang memuat $[E \to E \cdot +T]$ pada simbol $+$.
4. Bangun koleksi kanonik LR(0) untuk grammar $S \to aSb \mid \varepsilon$.
5. Bangun tabel SLR untuk grammar pada Latihan 4.
6. Jelaskan perbedaan SLR, LR(1), dan LALR.
7. Berikan contoh konflik shift-reduce.
8. Berikan contoh konflik reduce-reduce.

Refleksi

Refleksi

1. Apakah Anda dapat menjelaskan mengapa LR lebih kuat daripada SLR?
2. Bagian mana dari konstruksi tabel yang paling menantang?
3. Bagaimana Anda akan membaca laporan konflik dari Bison?

Asesmen

Asesmen Bab XII menjadi bagian UTS (20%) dan Praktikum 3 (7%). Fokusnya adalah item LR(0), tabel SLR, dan analisis konflik.

Tabel 12.5. Instrumen asesmen Bab XII

Instrumen	Indikator	Bobot
UTS	Item LR(0), tabel SLR, konflik	20%
Lokakarya	Verifikasi tabel dengan alat	bagian P3
Kuis	Closure, goto, dan perbedaan LR	formatif

Kriteria kelulusan: mahasiswa dapat membangun item LR(0), tabel SLR(1), menjelaskan konflik, dan memilih strategi parsing yang sesuai.

Checklist Pencapaian Kompetensi

☐ Saya dapat mendefinisikan item LR(0).

☐ Saya dapat menghitung closure.

☐ Saya dapat menghitung goto.

☐ Saya dapat membangun koleksi kanonik LR(0).

☐ Saya dapat membangun tabel SLR(1).

☐ Saya dapat menjelaskan perbedaan SLR, LR(1), dan LALR.

☐ Saya dapat menganalisis konflik.

☐ Saya dapat memilih strategi LL atau LR.

☐ Saya sudah menyelesaikan lokakarya verifikasi tabel.

Rangkuman

Pada bab ini telah dibahas:

- item LR(0), closure, dan goto;
- automaton LR(0) dan tabel SLR(1);
- canonical LR(1) dan LALR(1);
- konflik shift-reduce dan reduce-reduce;
- perbandingan LL(k) dan LR(k).

Bab XIII akan memakai parser generator Bison untuk membangun parser Subset-C, menambahkan semantic action, dan menghasilkan AST. Kita akan melihat bagaimana teori LR diterapkan dalam alat nyata.

Evaluasi

1. Definisikan item LR(0) dan jelaskan arti posisi titik.
2. Hitung closure dari $[A \to \cdot BC]$ dengan aturan $B \to b$.
3. Hitung goto dari state yang memuat $[A \to a \cdot B]$ pada simbol B.
4. Bangun tabel SLR untuk grammar $S \to A$, $A \to aA \mid b$.
5. Jelaskan perbedaan SLR dan LALR.
6. Berikan contoh konflik shift-reduce dan cara menyelesaikannya.
7. Mengapa Bison memilih LALR sebagai default?
8. Kapan Anda memilih LL(k) daripada LR(k)?

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan definisi dan perhitungan (70%), penjelasan (30%).
- Soal 4: kebenaran tabel (70%), kelengkapan (30%).
- Soal 5–8: pemahaman konsep (60%), argumentasi (40%).

BAB XIII

Parser Generator dan Semantic Actions

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 4.3, Sub-CPMK 4.4

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan arsitektur parser generator;
2. menulis deklarasi, aturan, dan semantic action Bison;
3. membangun AST dari semantic action;
4. mengintegrasikan Flex dan Bison dalam satu build;
5. membandingkan Bison, ANTLR, dan parser tulisan tangan.

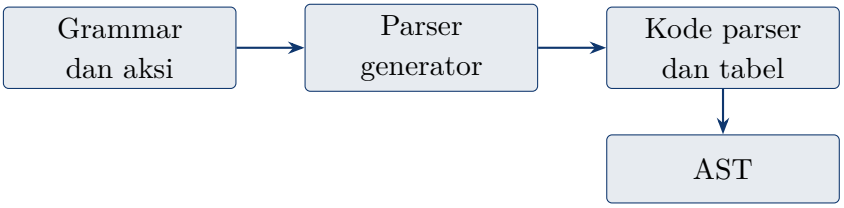
Materi Pokok

- Arsitektur parser generator.
- Deklarasi, aturan, dan konflik Bison.
- Semantic action dan konstruksi AST.
- Integrasi Flex–Bison dan build.
- ANTLR dan perbandingan pendekatan.

13.1 Arsitektur Parser Generator

13.1.1 Ide Dasar

Parser generator menerima spesifikasi grammar dan menghasilkan kode parser secara otomatis. Pengguna menulis aturan produksi dan semantic action; generator membangun tabel LR atau LL dan menghasilkan fungsi parser. Pendekatan ini memisahkan deskripsi bahasa dari detail algoritma parsing.



Gambar 13.1. Alur kerja parser generator

13.1.2 Keuntungan

Parser generator memberikan beberapa keuntungan:

- mengurangi kode yang ditulis tangan;
- tabel dibangun secara sistematis sehingga mengurangi kesalahan;
- perubahan grammar mudah diterapkan;
- mendukung grammar yang lebih besar daripada recursive descent.

Keterbatasannya adalah ketergantungan pada alat, pesan kesalahan yang kadang kurang kontekstual, dan kurva belajar untuk membaca laporan konflik.

13.1.3 Alat yang Umum

Alat parser generator yang populer:

- **Bison/Yacc**: grammar LALR(1), aksi C/C++;
- **ANTLR**: grammar LL(*), aksi Java/C++/Python;
- **JavaCC**: grammar LL(k), aksi Java;
- **Menhir**: varian Yacc untuk OCaml.

Buku ini memakai Bison karena banyak dipakai di industri dan terintegrasi baik dengan C. ANTLR dibahas sebagai pembanding.

13.2 Bison: Deklarasi, Aturan, dan Konflik

13.2.1 Struktur Berkas Bison

Berkas Bison (`.y`) terdiri atas tiga bagian:

1. **deklarasi**: token, tipe, precedence, dan kode C;
2. **aturan**: grammar dan semantic action;
3. **kode pengguna**: fungsi bantu dan `yyerror`.

13.2.2 Deklarasi

Bagian deklarasi mendefinisikan token, tipe nilai, dan precedence. Potongan `parser.y` ditunjukkan pada Kode Program 13.1.

```
%{
/* Berkas: src/parser.y
 * Grammar Bison untuk Subset-C (versi inti).
 * Membangun AST melalui semantic action.
 *
 * Build:
 *   bison -d -o build/parser.tab.c src/parser.y
 *   gcc -Iinclude -Ibuild build/parser.tab.c src/token.c \
 *       src/error.c src/lexer.c src/ast.c src/main_bison.c \
 *       -o build/scc
 */
#include <stdio.h>
#include <stdlib.h>
#include "ast.h"

void yyerror(const char *message);
int yylex(void);
%}

%code requires {
#include "ast.h"
}

%union {
    Token token;
    AstNode *node;
}

%token <token> T_INT T_FLOAT T_CHAR T_VOID
%token <token> T_IF T_ELSE T_WHILE T_FOR T_RETURN T_BREAK T_CONTINUE
    T_PRINT
%token <token> T_IDENT T_INT_LIT T_FLOAT_LIT T_CHAR_LIT T_STRING_LIT
%token <token> T_PLUS T_MINUS T_STAR T_SLASH T_PERCENT
%token <token> T_LT T_LE T_GT T_GE T_EQ T_NE T_AND T_OR T_NOT T_ASSIGN
%token <token> T_LPAREN T_RPAREN T_LBRACE T_RBRACE T_SEMI T_COMMA
%token <token> T_EOF

%type <node> program stmt_list stmt block assign_or_call
%type <node> expr logor logand equality relation additive term unary
    primary
```

```
%type <node> opt_expr args args_tail

%left T_OR
%left T_AND
%left T_EQ T_NE
%left T_LT T_LE T_GT T_GE
%left T_PLUS T_MINUS
%left T_STAR T_SLASH T_PERCENT
%right T_NOT UMINUS

%start program
```

Kode Program 13.1. Deklarasi `parser.y`

`%union` mendefinisikan tipe nilai semantik. Setiap token memiliki nilai bertipe `Token`, sedangkan nonterminal memiliki nilai bertipe `AstNode *`. Deklarasi `%type` menghubungkan nonterminal dengan tipe tersebut.

13.2.3 Aturan dan Aksi

Aturan grammar ditulis dengan simbol `:` dan `|`. Setiap alternatif dapat memiliki blok aksi C. Potongan aturan `program` dan `stmt_list` ditunjukkan pada Kode Program 13.2.

```
program
    : stmt_list T_EOF
        {
            $$ = ast_new(AST_PROGRAM, $2.token);
            AstNode *child = $1;
            while (child) {
                AstNode *next = child->next;
                child->next = NULL;
                ast_add_child($$, child);
                child = next;
            }
        }
    ;

stmt_list
    : %empty
        { $$ = NULL; }
    | stmt_list stmt
        {
            if ($1) {
                AstNode *last = $1;
                while (last->next) {
                    last = last->next;
```

```
76                  }
77                  last->next = $2;
78                  $$ = $1;
79              } else {
80                  $$ = $2;
81              }
82          }
```

Kode Program 13.2. Aturan `program` dan `stmt_list`

Aksi pada `program` membuat simpul `AST_PROGRAM` dan memindahkan setiap pernyataan dari daftar sementara ke daftar anak simpul. Pemindahan ini diperlukan karena Bison membangun daftar melalui `next`, sedangkan AST menyimpan anak melalui `children`.

13.2.4 Konflik

Bison melaporkan konflik shift-reduce dan reduce-reduce [7]. Precedence dideklarasikan dengan:

```
%left T_OR
%left T_AND
%left T_EQ T_NE
%left T_LT T_LE T_GT T_GE
%left T_PLUS T_MINUS
%left T_STAR T_SLASH T_PERCENT
%right T_NOT UMINUS
```

Deklarasi ini menyelesaikan konflik ekspresi tanpa mengubah grammar. Untuk dangling else, Bison secara default memilih shift, yang sesuai dengan aturan else terdekat.

Peringatan

Konflik yang tidak disengaja harus diperbaiki, bukan disembunyikan. Periksa keluaran Bison dengan opsi `-v` untuk melihat `parser.output` dan lokasi konflik.

13.3 Semantic Actions dan Konstruksi AST

13.3.1 Nilai Semantik

Bison menyediakan variabel khusus:

- `$$`: nilai hasil nonterminal kiri;
- `$1`, `$2`, ...: nilai simbol pada ruas kanan;
- `$<token>1`: akses anggota union tertentu.

Aksi dijalankan ketika aturan direduksi. Pada titik itu, semua nilai simbol ruas kanan sudah tersedia, sehingga AST dapat dibangun dari bawah ke atas.

13.3.2 Membangun AST

Aturan `assign_or_call` dan `block` menunjukkan pola pembangunan AST. Potongannya ditunjukkan pada Kode Program 13.3.

```
assign_or_call
    : T_ASSIGN expr
        {
            $$ = ast_new(AST_ASSIGN, $1.token);
            ast_add_child($$, $2);
        }
    | T_LPAREN args T_RPAREN
        {
            $$ = ast_new(AST_CALL, $1.token);
            AstNode *arg = $2;
            while (arg) {
                AstNode *next = arg->next;
                arg->next = NULL;
                ast_add_child($$, arg);
                arg = next;
            }
        }
    ;

block
    : T_LBRACE stmt_list T_RBRACE
        {
            $$ = ast_new(AST_BLOCK, $1.token);
            AstNode *child = $2;
            while (child) {
                AstNode *next = child->next;
                child->next = NULL;
                ast_add_child($$, child);
                child = next;
            }
        }
    ;
```

Kode Program 13.3. Aksi untuk assignment, call, dan block

Perhatikan bahwa aksi tidak hanya membuat simpul, tetapi juga memindahkan anak dari daftar `next` ke daftar `children`. Pemisahan ini menjaga agar AST memiliki struktur pohon yang bersih.

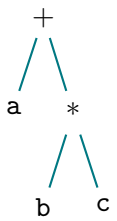
13.3.3 Contoh AST

Aturan `primary` membangun simpul literal, identifier, dan call. Potongannya ditunjukkan pada Kode Program 13.4.

```
primary : T_INT_LIT     { $$ = ast_new(AST_INT_LIT, $1.token); }
        | T_FLOAT_LIT   { $$ = ast_new(AST_FLOAT_LIT, $1.token); }
        | T_CHAR_LIT    { $$ = ast_new(AST_CHAR_LIT, $1.token); }
        | T_STRING_LIT  { $$ = ast_new(AST_STRING_LIT, $1.token); }
        | T_IDENT       { $$ = ast_new(AST_IDENT, $1.token); }
        | T_IDENT T_LPAREN args T_RPAREN
            {
                $$ = ast_new(AST_CALL, $1.token);
                AstNode *arg = $3;
                while (arg) {
                    AstNode *next = arg->next;
                    arg->next = NULL;
                    ast_add_child($$, arg);
                    arg = next;
                }
            }
        | T_LPAREN expr T_RPAREN
            { $$ = $2; }
        ;
```

Kode Program 13.4. Aksi untuk `primary`

Untuk ekspresi $a + b * c$, aksi `term` membangun simpul `*` dengan anak `b` dan `c`; aksi `additive` membangun simpul `+` dengan anak `a` dan simpul `*`. Hasilnya adalah AST yang mencerminkan precedence.



Gambar 13.2. AST untuk $a + b * c$

13.3.4 Location Tracking

Bison dapat melacak lokasi token dan aturan. Dengan `%locations`, setiap simbol memiliki `yylloc` yang dapat disalin ke simpul AST. Lokasi ini berguna untuk diagnostik semantik pada Bab XIV–XVI.

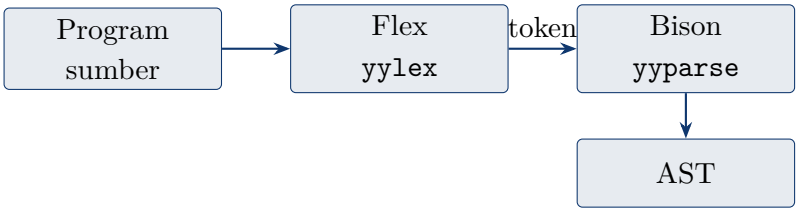
Informasi

Semantic action sebaiknya singkat dan fokus pada pembangunan AST. Logika kompleks sebaiknya dipindahkan ke fungsi bantu agar grammar tetap mudah dibaca.

13.4 Integrasi Flex–Bison dan Build

13.4.1 Antarmuka `yylex`

Bison memanggil `yylex` untuk mendapatkan token berikutnya. Fungsi ini harus mengisi `yylval` dan mengembalikan kode token Bison. Karena `lexer_next` mengembalikan `Token`, driver perlu memetakan jenis token ke kode Bison.



Gambar 13.3. Integrasi Flex dan Bison

13.4.2 Langkah Build

Build lengkap:

```
flex -o build/lexer_flex.c src/lexer_flex.l
bison -d -o build/parser.tab.c src/parser.y
gcc -Iinclude -Ibuild build/lexer_flex.c build/parser.tab.c \
    src/token.c src/error.c src/ast.c src/main_bison.c \
    -o build/scc
```

Perintah pertama menghasilkan lexer C. Perintah kedua menghasilkan parser dan header token. Perintah ketiga mengompilasi semuanya menjadi executable `scc`.

13.4.3 Driver

Driver membuka berkas, menyiapkan lexer global, lalu memanggil `yyparse`. Jika parsing berhasil dan tidak ada kesalahan leksikal, driver mencetak AST atau melanjutkan ke analisis semantik.

```
int main(int argc, char **argv) {
    if (argc != 2) {
        fprintf(stderr, "penggunaan: %s berkas.sc\n", argv[0]);
        return 2;
    }
```

```
    if (!lexer_open(argv[1])) {
        return 2;
    }
    int status = yyparse();
    lexer_close();
    return status;
}
```

13.4.4 Catatan Lingkungan

Lingkungan penyusunan buku ini tidak memiliki Flex, Bison, dan GCC. Kode pada bab ini disusun sebagai artefak yang dapat dibangun pada lingkungan dengan toolchain lengkap. Mahasiswa harus menjalankan build dan tes sendiri, lalu mencatat versi alat yang dipakai.

Peringatan

Perbedaan versi Bison dapat mengubah jumlah konflik dan nama simbol yang dihasilkan. Selalu catat versi Bison pada laporan praktikum.

13.5 ANTLR dan Perbandingan Pendekatan

13.5.1 ANTLR

ANTLR adalah parser generator yang menghasilkan parser LL(*) dengan dukungan *visitor* dan *listener* [2]. Grammar ditulis dalam berkas `.g4`. Contoh ringkas:

```
grammar Expr;
prog   : stat+ ;
stat   : expr NEWLINE ;
expr   : expr ('*'|'/') expr
       | expr ('+'|'-') expr
       | INT
       | '(' expr ')'
       ;
INT    : [0-9]+ ;
NEWLINE: '\r'? '\n' ;
WS     : [ \t]+ -> skip ;
```

ANTLR membangun pohon parse otomatis. Pengguna dapat menulis *visitor* untuk mengubah pohon menjadi AST atau melakukan analisis. ANTLR juga menyediakan pesan kesalahan dan pemulihan yang cukup baik.

13.5.2 Perbandingan

Tabel 13.1. Perbandingan Bison, ANTLR, dan parser tangan

Aspek	Bison	ANTLR	Tangan
Algoritma	LALR(1)	LL(*)	Recursive descent
Bahasa aksi	C/C++	Java, C++, Python	Bahasa pilihan
Pohon	Dibangun manual	Otomatis	Dibangun manual
Kontrol	Tinggi	Sedang	Penuh
Diagnostik	Perlu kode tambahan	Bawaan baik	Penuh, tetapi manual
Kurva belajar	Sedang	Rendah–sedang	Rendah

13.5.3 Kapan Memilih

Bison cocok untuk proyek C/C++ yang membutuhkan kontrol dan kinerja. ANTLR cocok untuk prototipe cepat, bahasa dengan grammar kompleks, dan tim yang bekerja di ekosistem Java/Python. Parser tangan cocok untuk bahasa kecil, pembelajaran, dan kebutuhan diagnostik yang sangat spesifik.

Informasi

Pilihan alat bukan penentu keberhasilan. Yang lebih penting adalah grammar yang benar, pengujian yang memadai, dan pemahaman tentang konflik serta pemulihan kesalahan.

Contoh

Contoh 13.1 (Aturan ekspresi Bison)**.** Aturan

```
expr : expr T_PLUS expr
     | expr T_STAR expr
     | T_INT_LIT
     ;
```

dapat menimbulkan konflik shift-reduce karena precedence tidak dideklarasikan. Dengan `%left T_PLUS` dan `%left T_STAR`, Bison menyelesaikan konflik sesuai prioritas perkalian.

Contoh 13.2 (Membangun simpul AST)**.** Aksi

```
expr : expr T_PLUS expr
     {
         $$ = ast_new(AST_BINARY, $2.token);
         $$->left = $1;
```

```
5           $$->right = $3;
6       }
7       ;
```

membuat simpul `BINARY` dengan token operator dari `$2`, anak kiri dari `$1`, dan anak kanan dari `$3`.

Contoh 13.3 (AST untuk $a + b * c$)**.** Aksi `term` membangun simpul `*` untuk $b * c$. Aksi `additive` membangun simpul `+` dengan anak `a` dan simpul `*`. Hasilnya adalah AST pada Gambar 13.2.

Contoh 13.4 (Precedence dan associativity)**.** Deklarasi

```
1 %left T_PLUS T_MINUS
2 %left T_STAR T_SLASH T_PERCENT
```

membuat `*` lebih tinggi daripada `+`. Keduanya kiri-asosiatif, sehingga $a - b - c$ diurai sebagai $(a - b) - c$. Untuk operator kanan-asosiatif, gunakan `%right`.

Contoh 13.5 (Location tracking)**.** Dengan `%locations`, Bison menyediakan `@1`, `@2`, dan `@$` untuk lokasi simbol. Aksi dapat menyalin `@$.first_line` ke simpul AST, sehingga diagnostik semantik dapat menyebut baris yang tepat.

Aktivitas Pembelajaran dan Praktikum

Tabel 13.2. Rencana aktivitas Bab XIII

Sesi	Bentuk	Kegiatan
1	Ceramah	Arsitektur parser generator dan Bison
2	Live coding	Menulis grammar dan semantic action
3	Latihan	Membaca laporan konflik Bison
4	Praktikum	M3: Bison + AST + integrasi

M3 / Praktikum 3: Bison dan AST

Tujuan. Membangun parser Bison untuk Subset-C dan menghasilkan AST lengkap.

Keluaran.

1. Grammar Bison untuk ekspresi, pernyataan, dan deklarasi.
2. Semantic action yang membangun AST.
3. Integrasi Flex–Bison dan driver.

4. Minimal 20 kasus uji: valid dan tidak valid.

5. Perbandingan dengan recursive descent.

Langkah kerja.

1. Lengkapi `src/parser.y`.

2. Definisikan `%union`, `%token`, dan `%type`.

3. Tulis semantic action untuk setiap aturan.

4. Integrasikan `yylex` dari lexer.

5. Jalankan Bison dan perbaiki konflik.

6. Uji dan bandingkan hasilnya dengan parser tangan.

Rubrik.

Tabel 13.3. Rubrik Praktikum 3

Aspek	Indikator	Bobot
Kebenaran	Grammar dan AST benar	40%
Kualitas kode	Aksi ringkas, penamaan, komentar	30%
Kelengkapan	Ekspresi, pernyataan, deklarasi	20%
Pengujian	Kasus normal, batas, kesalahan	10%

Definisi selesai.

☐ Grammar Bison tanpa konflik tak terduga.

☐ AST terbentuk untuk program valid.

☐ 20 kasus uji tersedia.

☐ Perbandingan dengan recursive descent terdokumentasi.

Latihan dan Refleksi

Latihan

1. Jelaskan peran `%token`, `%type`, dan `%union`.

2. Tuliskan aturan Bison untuk ekspresi penjumlahan dan perkalian.

3. Jelaskan arti `$$` dan `$1`.

4. Bagaimana semantic action membangun AST?

5. Mengapa precedence dapat menyelesaikan konflik?

6. Jelaskan integrasi `yylex` dan `yyparse`.

7. Bandingkan Bison dan ANTLR.

8. Kapan parser tangan lebih tepat daripada parser generator?

Refleksi

Refleksi

1. Apakah Anda dapat membaca laporan konflik Bison?

2. Bagian mana dari semantic action yang paling menantang?

3. Alat mana yang akan Anda pilih untuk proyek nyata? Mengapa?

Asesmen

Bab XIII memuat Praktikum 3 dengan bobot 7% sesuai RPS. Asesmen mencakup grammar, AST, integrasi, dan pengujian.

Tabel 13.4. Instrumen asesmen Bab XIII

Instrumen	Indikator	Bobot
Praktikum 3	Grammar Bison, AST, integrasi, tes	7%
Kuis	Semantic action dan konflik	formatif
Review kode	Kualitas grammar dan aksi	bagian P3

Kriteria kelulusan: grammar Bison membangun AST yang benar, konflik ditangani secara eksplisit, dan integrasi dengan lexer berjalan.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan arsitektur parser generator.

☐ Saya dapat menulis deklarasi Bison.

☐ Saya dapat menulis aturan dan semantic action.

☐ Saya dapat membangun AST dari aksi.

☐ Saya dapat menangani konflik dengan precedence.

☐ Saya dapat mengintegrasikan Flex dan Bison.

☐ Saya dapat membandingkan Bison, ANTLR, dan parser tangan.

☐ Saya sudah menyelesaikan M3/Praktikum 3.

Rangkuman

Pada bab ini telah dibahas:

- arsitektur parser generator dan keuntungannya;
- deklarasi, aturan, dan konflik Bison;
- semantic action dan konstruksi AST;
- integrasi Flex–Bison dan build;
- ANTLR dan perbandingan pendekatan.

Bab XIV akan beralih ke analisis semantik: attribute grammar, syntax-directed definition, dan translation scheme. AST yang dihasilkan pada bab ini menjadi input untuk analisis tersebut.

Evaluasi

1. Jelaskan alur kerja parser generator.
2. Tuliskan deklarasi token dan tipe untuk ekspresi.
3. Jelaskan peran `%left` dan `%right`.
4. Tuliskan semantic action untuk aturan penjumlahan.
5. Bagaimana AST dibangun dari aksi?
6. Jelaskan integrasi `yylex` dan `yyparse`.
7. Bandingkan Bison dan ANTLR dari sisi kontrol dan produktivitas.
8. Mengapa laporan konflik harus dibaca?

Kunci dan Rubrik Singkat

- Soal 1–4: ketepatan deklarasi dan aksi (70%), kelengkapan (30%).
- Soal 5–6: pemahaman integrasi (60%), contoh (40%).
- Soal 7–8: analisis (60%), argumentasi (40%).

BAB XIV

Analisis Semantik: Attribute Grammar, SDD, dan Translation Schemes

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 1.5

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan ruang lingkup analisis semantik;
2. membedakan atribut synthesized dan inherited;
3. menulis syntax-directed definition (SDD);
4. menjelaskan translation scheme dan penerapannya;
5. melakukan inferensi tipe untuk ekspresi Subset-C.

Materi Pokok

- Tugas dan ruang lingkup analisis semantik.
- Attribute grammar: synthesized dan inherited.
- Syntax-directed definition.
- Translation scheme.
- Studi kasus inferensi tipe.

14.1 Tugas dan Ruang Lingkup Analisis Semantik

14.1.1 Di Luar Sintaksis

Analisis sintaksis memastikan program memiliki bentuk yang benar. Analisis semantik memastikan program memiliki makna yang masuk akal. Kesalahan seperti variabel yang tidak dideklarasikan, tipe yang tidak cocok, atau pemanggilan fungsi dengan jumlah argumen yang salah tidak dapat dideteksi oleh tata bahasa bebas konteks.

Tabel 14.1. Tugas utama analisis semantik

Tugas	Pertanyaan yang dijawab
Resolusi nama	Apakah identifier sudah dideklarasikan?
Pemeriksaan tipe	Apakah operand memiliki tipe yang kompatibel?
Pemeriksaan fungsi	Apakah jumlah dan tipe argumen sesuai?
Kontrol alur	Apakah `break` berada di dalam loop?
Inisialisasi	Apakah variabel dipakai sebelum diberi nilai?
Keterjangkauan	Apakah semua jalur fungsi mengembalikan nilai?

14.1.2 Informasi yang Dibutuhkan

Analisis semantik membutuhkan:

- AST atau pohon urai yang telah dibangun;
- tabel simbol dengan informasi scope dan tipe;
- aturan tipe untuk setiap operator dan konstruksi;
- mekanisme pelaporan kesalahan.

Tanpa tabel simbol, analisis hanya dapat memeriksa struktur lokal. Tanpa aturan tipe, analisis tidak dapat menentukan kompatibilitas operand.

14.1.3 Urutan Analisis

Urutan yang umum:

1. bangun tabel simbol dan resolusi nama;
2. tentukan tipe setiap ekspresi;
3. periksa kecocokan tipe pada assignment dan pemanggilan;
4. periksa aturan kontrol alur;
5. laporkan kesalahan dan lanjutkan.

Analisis dapat dilakukan dalam satu lintasan AST atau beberapa lintasan. Beberapa lintasan memudahkan pemisahan tugas, tetapi memerlukan AST yang stabil.

Informasi

Analisis semantik tidak harus menghasilkan kode. Tujuannya adalah memastikan program valid dan mengumpulkan informasi yang dibutuhkan fase berikutnya, seperti tipe dan alamat simbol.

14.2 Attribute Grammar: Synthesized dan Inherited

14.2.1 Atribut

Definisi 14.1 (Attribute grammar)**.** Attribute grammar adalah tata bahasa bebas konteks yang setiap simbolnya memiliki atribut dan setiap produksinya memiliki aturan semantik untuk menghitung atribut tersebut.

Atribut dapat berupa tipe, nilai, alamat, kode, atau informasi lain. Setiap atribut memiliki domain nilai dan aturan evaluasi.

14.2.2 Atribut Synthesized

Atribut synthesized dihitung dari atribut anak. Contoh klasik adalah atribut `type` pada simpul ekspresi:

$$E.type = \text{rule}(E_1.type, E_2.type).$$

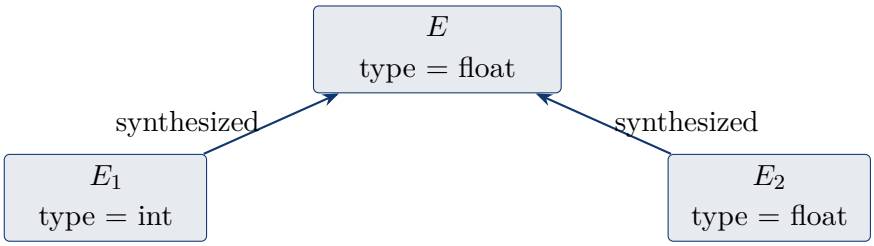
Nilai mengalir dari daun ke akar. SDD yang hanya memakai atribut synthesized disebut *S-attributed*.

14.2.3 Atribut Inherited

Atribut inherited dihitung dari orang tua atau saudara. Contohnya adalah atribut `env` yang membawa lingkungan simbol ke anak:

$$\text{stmt}.env = \text{block}.env.$$

Nilai mengalir dari akar ke daun atau antar-saudara. SDD yang memakai inherited disebut *L-attributed* jika ketergantungannya mengikuti urutan kiri-ke-kanan.



Gambar 14.1. Atribut synthesized mengalir dari anak ke orang tua

14.2.4 Dependency Graph

Ketergantungan antar-atribut membentuk graf berarah. Evaluasi atribut harus mengikuti urutan topologis graf tersebut. Jika graf memiliki siklus, evaluasi tidak dapat dilakukan tanpa iterasi atau pendekatan khusus.

Peringatan

Atribut inherited dapat menimbulkan siklus jika aturan saling bergantung. Periksa graf ketergantungan sebelum mengimplementasikan evaluasi.

14.3 Syntax-Directed Definition

14.3.1 Definisi

Definisi 14.2 (SDD)**.** Syntax-directed definition (SDD) adalah tata bahasa dengan atribut dan aturan semantik. Setiap produksi $A \to \alpha$ memiliki aturan untuk menghitung atribut simbol-simbolnya.

SDD mendeskripsikan *apa* yang dihitung, bukan *kapan* perhitungan dilakukan. Urutan evaluasi ditentukan oleh ketergantungan antar-atribut.

14.3.2 Contoh SDD untuk Ekspresi

Tabel 14.2. SDD sederhana untuk tipe ekspresi

Produksi	Aturan semantik
$E \to E_1 + E_2$	$E.type = \text{arith}(E_1.type, E_2.type)$
$E \to E_1 * E_2$	$E.type = \text{arith}(E_1.type, E_2.type)$
$E \to$ `id`	$E.type = \text{lookup}(\texttt{id}.name)$
$E \to$ `INT_LIT`	$E.type = \text{int}$
$E \to$ `FLOAT_LIT`	$E.type = \text{float}$

Aturan `arith` mengembalikan `float` jika salah satu operand `float`, `int` jika keduanya `int` atau `char`, dan `error` jika operand tidak numerik.

14.3.3 S-Attributed dan L-Attributed

SDD S-attributed hanya memakai atribut synthesized, sehingga dapat dievaluasi dengan traversal post-order. SDD L-attributed memakai atribut inherited yang hanya bergantung pada simbol di kiri atau orang tua, sehingga dapat dievaluasi dengan traversal kiri-ke-kanan.

Tabel 14.3. Perbandingan S-attributed dan L-attributed

Aspek	S-attributed	L-attributed
Atribut	Synthesized	Synthesized dan inherited
Evaluasi	Post-order	Kiri-ke-kanan
Implementasi	Mudah	Perlu konteks
Contoh	Tipe ekspresi	Lingkungan scope

14.3.4 Evaluasi

Evaluasi SDD dapat dilakukan:

- saat parsing, jika aturan sudah cukup;
- setelah AST terbentuk, dengan traversal;
- dengan alat khusus yang membangun dependency graph.

Pada proyek Subset-C, evaluasi dilakukan setelah AST terbentuk. Pendekatan ini memisahkan parsing dari analisis dan memudahkan pengujian.

14.4 Translation Schemes

14.4.1 Definisi

Translation scheme adalah tata bahasa dengan potongan program semantik yang disisipkan di tengah ruas kanan. Berbeda dari SDD yang mendeskripsikan aturan, translation scheme mendeskripsikan urutan eksekusi.

Definisi 14.3 (Translation scheme)**.** Translation scheme adalah tata bahasa dengan aksi semantik yang ditempatkan pada posisi tertentu dalam produksi. Aksi dijalankan ketika posisi tersebut dicapai selama parsing.

14.4.2 Contoh

Untuk aturan

$$E \to E_1 + E_2$$

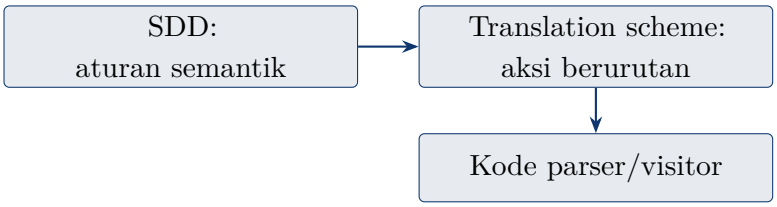
translation scheme dapat ditulis:

$$E \to E_1 + \{E.type = \text{arith}(E_1.type, E_2.type);\}\ E_2.$$

Aksi dijalankan setelah E_1 dan sebelum E_2 diproses.

14.4.3 Dari SDD ke Translation Scheme

SDD S-attributed dapat diubah menjadi translation scheme dengan menempatkan aksi di akhir produksi. SDD L-attributed memerlukan penempatan aksi yang lebih hati-hati agar inherited attribute sudah tersedia.



Gambar 14.2. Dari SDD ke translation scheme ke implementasi

14.4.4 Contoh pada Bison

Semantic action Bison adalah bentuk translation scheme:

```
expr : expr T_PLUS expr
     {
         $$ = ast_new(AST_BINARY, $2.token);
         $$->left = $1;
         $$->right = $3;
     }
     ;
```

Aksi dijalankan saat aturan direduksi, setelah kedua operand tersedia. Ini adalah pola S-attributed yang khas.

14.4.5 Kapan Memilih

SDD lebih mudah dibaca dan dianalisis. Translation scheme lebih dekat ke implementasi dan mudah dipasang pada parser. Untuk dokumentasi, SDD lebih baik; untuk implementasi, translation scheme lebih praktis.

Informasi

Jangan mencampur logika parsing dan logika semantik yang kompleks dalam satu aksi. Pindahkan perhitungan ke fungsi bantu agar grammar tetap ringkas dan dapat diuji.

14.5 Studi Kasus: Inferensi Tipe Subset-C

14.5.1 Modul Tipe

Modul `types.h` mendefinisikan jenis tipe dan operasi dasar. Potongannya ditunjukkan pada Kode Program 14.1.

```c
// Berkas: include/types.h
// Representasi tipe untuk analisis semantik Subset-C.

#ifndef SUBSETC_TYPES_H
#define SUBSETC_TYPES_H

typedef enum {
    TYPE_INT,
    TYPE_FLOAT,
    TYPE_CHAR,
    TYPE_VOID,
    TYPE_STRING,
    TYPE_BOOL,
    TYPE_ERROR
} TypeKind;

typedef struct {
    TypeKind kind;
} Type;

const char *type_name(TypeKind kind);
int type_is_numeric(TypeKind kind);
TypeKind type_arith(TypeKind left, TypeKind right);
int type_assignable(TypeKind target, TypeKind source);
```

Kode Program 14.1. Antarmuka `types.h`

Tipe `TYPE_ERROR` dipakai sebagai nilai propagasi. Jika satu operand error, hasil operasi juga error, sehingga kesalahan tidak berlipat ganda.

14.5.2 Aturan Tipe

Tabel 14.4. Aturan tipe untuk operator Subset-C

Konstruksi	Tipe hasil	Catatan
`int + int`	`int`	
`int + float`	`float`	promosi
`char + int`	`int`	promosi
`float + float`	`float`	
`void + int`	`error`	operand tidak valid
`a < b`	`bool`	operand numerik
`!a`	`bool`	operand bool
`a = b`	tipe `a`	harus assignable

14.5.3 Contoh Inferensi

Untuk program

```
1 int a = 2;
2 float b = 3.5;
3 float c = a + b;
```

tipe `a` adalah `int`, tipe `b` adalah `float`, dan tipe `a + b` adalah `float`. Assignment ke `c` valid karena `float` assignable ke `float`. Jika `c` bertipe `int`, assignment memerlukan cast eksplisit yang belum didukung v1.0.

14.5.4 Propagasi Error

Jika sebuah identifier tidak ditemukan, tipe ekspresinya menjadi `error`. Operator yang menerima `error` mengembalikan `error` tanpa melaporkan kesalahan baru. Strategi ini mencegah satu kesalahan awal menghasilkan puluhan pesan turunan.

Peringatan

Jangan melaporkan kesalahan tipe berulang untuk ekspresi yang sama. Gunakan nilai `error` sebagai peredam dan laporkan hanya kesalahan akar.

Contoh

Contoh 14.4 (Atribut synthesized)**.** Untuk ekspresi `2 + 3.5`, tipe anak kiri adalah `int` dan anak kanan `float`. Aturan `arith(int, float)` menghasilkan `float`. Atribut tipe simpul akar dihitung dari tipe anak, sehingga merupakan atribut synthesized.

Contoh 14.5 (Atribut inherited)**.** Pada blok

```
{
    int x = 1;
    {
        int y = x + 1;
    }
}
```

lingkungan scope dalam mewarisi tabel simbol scope luar. Atribut `env` pada blok dalam dihitung dari blok luar. Ini adalah atribut inherited karena nilainya berasal dari orang tua.

Contoh 14.6 (SDD ke translation scheme)**.** Aturan SDD

$$E \to E_1 + E_2 \quad \{E.type = \text{arith}(E_1.type, E_2.type)\}$$

dapat ditulis sebagai aksi di akhir produksi:

```
expr : expr T_PLUS expr
     { $$.type = type_arith($1.type, $3.type); }
     ;
```

Aksi dijalankan saat reduksi, setelah tipe kedua operand diketahui.

Contoh 14.7 (Inferensi tipe campuran)**.** Untuk `int a; float b; char c;`, ekspresi `a + b * c` memiliki tipe:

1. `b * c`: `arith(float, char)` = `float`;
2. `a + float`: `arith(int, float)` = `float`.

Hasil akhirnya `float`.

Contoh 14.8 (Propagasi error)**.** Jika identifier `q` tidak dideklarasikan, tipe `q` adalah `error`. Ekspresi `q + 1` menghasilkan `error` tanpa pesan tambahan. Hanya kesalahan "identifier q tidak dideklarasikan" yang dilaporkan.

Aktivitas Pembelajaran dan Praktikum

Tabel 14.5. Rencana aktivitas Bab XIV

Sesi	Bentuk	Kegiatan
1	Ceramah	Ruang lingkup analisis semantik
2	Latihan	Menulis SDD untuk tipe ekspresi
3	Diskusi	Synthesized versus inherited
4	Praktikum	P4-A: inferensi tipe ekspresi

P4-A: Inferensi Tipe Ekspresi

Tujuan. Mengimplementasikan inferensi tipe untuk ekspresi Subset-C.

Langkah kerja.

1. Lengkapi `include/types.h` dan `src/types.c`.
2. Tulis fungsi `infer_expr` yang menerima simpul AST ekspresi dan tabel simbol.
3. Terapkan aturan tipe pada Tabel 14.4.
4. Tangani propagasi `TYPE_ERROR`.
5. Uji dengan minimal 15 ekspresi, termasuk campuran tipe dan identifier yang tidak dideklarasikan.

Definisi selesai.

☐ Aturan tipe diimplementasikan.

☐ Promosi `int` ke `float` benar.

☐ Error dipropagasikan tanpa pesan berulang.

☐ 15 kasus uji tersedia.

P4-A menjadi bagian Praktikum 4 (8%) yang dilaporkan pada Bab XV.

Latihan dan Refleksi

Latihan

1. Jelaskan perbedaan analisis sintaksis dan semantik.
2. Definisikan atribut synthesized dan inherited.
3. Tuliskan SDD untuk tipe ekspresi penjumlahan dan perkalian.
4. Jelaskan perbedaan SDD dan translation scheme.
5. Berikan contoh atribut inherited pada scope.
6. Tentukan tipe `int + float * char`.
7. Jelaskan strategi propagasi error.
8. Mengapa analisis semantik memerlukan tabel simbol?

Refleksi

Refleksi

1. Apakah Anda dapat membedakan SDD dan translation scheme?
2. Bagian mana dari inferensi tipe yang paling menantang?
3. Mengapa satu kesalahan tidak boleh menghasilkan banyak pesan?

Asesmen

Asesmen Bab XIV menjadi bagian Praktikum 4 (8%) dan Tugas Teori (8%). Fokusnya adalah SDD, translation scheme, dan inferensi tipe.

Tabel 14.6. Instrumen asesmen Bab XIV

Instrumen	Indikator	Bobot
P4-A	Inferensi tipe ekspresi	bagian P4
Tugas Teori	SDD dan translation scheme	8%
Kuis	Atribut synthesized/inherited	formatif

Kriteria kelulusan: mahasiswa dapat menulis SDD, menjelaskan perbedaan synthesized dan inherited, serta mengimplementasikan inferensi tipe dasar.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan ruang lingkup analisis semantik.

☐ Saya dapat membedakan atribut synthesized dan inherited.

☐ Saya dapat menulis SDD.

☐ Saya dapat menjelaskan translation scheme.

☐ Saya dapat menentukan tipe ekspresi campuran.

☐ Saya dapat mempropagasikan error.

☐ Saya dapat mengimplementasikan inferensi tipe dasar.

☐ Saya sudah menyelesaikan P4-A.

Rangkuman

Pada bab ini telah dibahas:

- tugas dan ruang lingkup analisis semantik;
- attribute grammar dengan atribut synthesized dan inherited;
- syntax-directed definition (SDD);
- translation scheme dan penerapannya;
- studi kasus inferensi tipe Subset-C.

Bab XV akan membahas tabel simbol, manajemen scope, dan pemeriksaan tipe secara lebih rinci. Implementasi tabel simbol menjadi fondasi bagi analisis semantik yang lengkap.

Evaluasi

1. Jelaskan tiga tugas analisis semantik.
2. Definisikan atribut synthesized dan inherited.
3. Tuliskan SDD untuk tipe ekspresi.
4. Jelaskan perbedaan SDD dan translation scheme.
5. Tentukan tipe `int + float * char`.
6. Jelaskan strategi propagasi `TYPE_ERROR`.
7. Mengapa tabel simbol diperlukan?
8. Berikan contoh kesalahan semantik yang tidak dapat dideteksi parser.

Kunci dan Rubrik Singkat

- Soal 1–4: ketepatan konsep (70%), contoh (30%).
- Soal 5–6: kebenaran aturan dan propagasi (70%), penjelasan (30%).
- Soal 7–8: analisis (60%), argumentasi (40%).

BAB XV

Symbol Table, Scope Management, dan Type Checking

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 1.6

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. menjelaskan struktur dan operasi tabel simbol;
2. mengimplementasikan tabel simbol dengan scope bersarang;
3. menjelaskan shadowing dan resolusi nama;
4. menerapkan aturan tipe pada ekspresi dan pernyataan;
5. menangani konversi tipe dan pelaporan kesalahan;
6. menyelesaikan M4: symbol table dan type checker.

Materi Pokok

- Struktur dan operasi tabel simbol.
- Implementasi hash table dan chaining.
- Nested scope, shadowing, dan resolusi nama.
- Sistem tipe dan pemeriksaan tipe.
- Konversi tipe dan pelaporan kesalahan.

15.1 Struktur dan Operasi Symbol Table

15.1.1 Atribut Simbol

Tabel simbol memetakan nama ke informasi tentang nama tersebut. Atribut yang umum:

- nama dan panjangnya;

- jenis simbol: variabel, fungsi, atau parameter;
- tipe;
- scope asal;
- lokasi deklarasi (baris, kolom);
- alamat memori atau indeks register (untuk fase berikutnya).

Definisi 15.1 (Tabel simbol)**.** Tabel simbol adalah struktur data yang mendukung operasi `define`, `lookup`, dan `scope management` untuk seluruh identifier dalam program.

15.1.2 Operasi Dasar

Tabel 15.1. Operasi tabel simbol

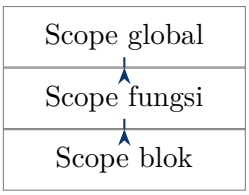
Operasi	Deskripsi
`define`	Menambahkan simbol ke scope saat ini
`lookup`	Mencari simbol dari scope dalam ke luar
`lookup_current`	Mencari simbol hanya di scope saat ini
`enter_scope`	Mendorong scope baru
`leave_scope`	Melepas scope teratas

15.1.3 Representasi

Tabel simbol dapat diimplementasikan sebagai:

- daftar tertaut sederhana;
- hash table dengan chaining;
- pohon pencarian seimbang;
- kombinasi hash table dan stack scope.

Untuk proyek Subset-C, setiap scope memiliki daftar simbol tertaut. Implementasi ini sederhana dan cukup untuk program perkuliahan. Untuk program besar, hash table diperlukan agar `lookup` tetap cepat.



Gambar 15.1. Stack scope: pencarian bergerak dari dalam ke luar

15.1.4 Resolusi Nama

Saat `lookup` dipanggil, tabel mencari di scope saat ini. Jika tidak ditemukan, pencarian berlanjut ke scope induk, lalu ke scope global. Jika tetap tidak ditemukan, identifier dianggap tidak dideklarasikan.

Urutan ini penting untuk shadowing: identifier di scope dalam menutupi identifier dengan nama sama di scope luar. Shadowing diizinkan, sedangkan deklarasi ganda dalam scope yang sama merupakan kesalahan.

15.2 Implementasi Hash Table di C

15.2.1 Mengapa Hash Table

Pencarian linear pada daftar simbol memiliki kompleksitas $O(n)$ per lookup. Untuk program dengan ribuan identifier, biaya ini menjadi signifikan. Hash table menurunkan kompleksitas rata-rata menjadi $O(1)$ jika fungsi hash dan ukuran tabel dipilih dengan baik.

15.2.2 Fungsi Hash

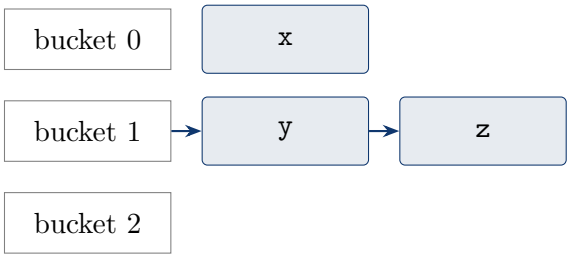
Fungsi hash yang sederhana untuk string:

$$h(s) = \left(\sum_{i=0}^{n-1} s_i \cdot 31^{n-1-i} \right) \bmod m,$$

dengan m ukuran tabel. Perkalian dengan 31 menyebar karakter dengan cukup baik dan mudah diimplementasikan sebagai `h = h * 31 + c`.

15.2.3 Chaining

Setiap bucket berisi daftar simbol dengan hash yang sama. Operasi `define` menambahkan ke depan daftar; `lookup` menelusuri daftar bucket. Ukuran tabel sebaiknya bilangan prima atau pangkat dua dengan fungsi hash yang sesuai.



Gambar 15.2. Hash table dengan chaining

15.2.4 Implementasi Proyek

Pada proyek Subset-C, tabel simbol diimplementasikan sebagai daftar tertaut per scope. Struktur ini mudah dipahami dan cukup untuk program kecil. Mahasiswa dapat menggantinya dengan hash table sebagai pengayaan.

Antarmuka tabel simbol ditunjukkan pada Kode Program 15.1.

```
// Berkas: include/symtab.h
// Tabel simbol dengan scope bersarang.

#ifndef SUBSETC_SYMTAB_H
#define SUBSETC_SYMTAB_H

#include "types.h"

typedef enum {
    SYM_VAR,
    SYM_FUNC,
    SYM_PARAM
} SymbolKind;

typedef struct Symbol {
    char *name;
    TypeKind type;
    SymbolKind kind;
    int line;
    int column;
    struct Symbol *next;
} Symbol;

typedef struct Scope {
    Symbol *symbols;
    struct Scope *parent;
} Scope;

typedef struct {
    Scope *current;
} SymTab;

void symtab_init(SymTab *table);
void symtab_free(SymTab *table);

void symtab_enter_scope(SymTab *table);
void symtab_leave_scope(SymTab *table);

```

```
Symbol *symtab_define(SymTab *table, const char *name, int length,
                      TypeKind type, SymbolKind kind,
                      int line, int column);
Symbol *symtab_lookup(const SymTab *table, const char *name,
                      int length);
Symbol *symtab_lookup_current(const SymTab *table, const char *name,
                              int length);
```

Kode Program 15.1. Antarmuka `symtab.h`

Fungsi `symtab_define` dan `symtab_lookup` menjadi inti implementasi. Keduanya menangani nama yang tidak selalu null-terminated dengan menerima panjang nama secara eksplisit.

15.3 Nested Scopes dan Shadowing

15.3.1 Scope Bersarang

Setiap blok dan fungsi memiliki scope sendiri. Saat parser memasuki blok, tabel simbol mendorong scope baru. Saat blok selesai, scope dilepas dan simbol-simbolnya tidak lagi terlihat.

Implementasi `symtab_enter_scope` dan `symtab_leave_scope` ditunjukkan pada Kode Program 15.2.

```
void symtab_enter_scope(SymTab *table) {
    Scope *scope = (Scope *)calloc(1, sizeof(Scope));
    if (!scope) {
        fprintf(stderr, "kehabisan memori pada tabel simbol\n");
        exit(1);
    }
    scope->symbols = NULL;
    scope->parent = table->current;
    table->current = scope;
}

void symtab_leave_scope(SymTab *table) {
    Scope *scope = table->current;
    Symbol *symbol;
    Symbol *next;

    if (!scope || !scope->parent) {
        return;
    }

```

```
    symbol = scope->symbols;
    while (symbol) {
        next = symbol->next;
        free(symbol->name);
        free(symbol);
        symbol = next;
    }

    table->current = scope->parent;
    free(scope);
}
```

Kode Program 15.2. Mendorong dan melepas scope

Setiap scope menyimpan pointer ke scope induk. Saat scope dilepas, memori simbol dan scope dibebaskan, lalu pointer kembali ke induk.

15.3.2 Shadowing

Definisi 15.2 (Shadowing)**.** Shadowing terjadi ketika identifier pada scope dalam memiliki nama yang sama dengan identifier pada scope luar. Identifier dalam menutupi identifier luar selama scope dalam aktif.

Contoh 15.3 (Shadowing)

```
int x = 1;
{
    int x = 2;
    print(x);
}
print(x);
```

Di dalam blok, `x` merujuk ke variabel lokal bernilai 2. Setelah blok selesai, `x` kembali merujuk ke variabel global bernilai 1.

15.3.3 Lookup

Fungsi `symtab_lookup` menelusuri scope dari dalam ke luar. Implementasinya ditunjukkan pada Kode Program 15.3.

```
static Symbol *lookup_in_scope(Scope *scope, const char *name,
                               int length) {
    Symbol *symbol;
    for (symbol = scope ? scope->symbols : NULL;
         symbol; symbol = symbol->next) {
        if ((int)strlen(symbol->name) == length &&
            strncmp(symbol->name, name, (size_t)length) == 0) {
            return symbol;
```

```
        }
    }
    return NULL;
}

Symbol *symtab_lookup_current(const SymTab *table, const char *name,
                              int length) {
    return lookup_in_scope(table->current, name, length);
}

Symbol *symtab_lookup(const SymTab *table, const char *name,
                      int length) {
    Scope *scope;
    for (scope = table->current; scope; scope = scope->parent) {
        Symbol *symbol = lookup_in_scope(scope, name, length);
        if (symbol) {
            return symbol;
        }
    }
    return NULL;
}

void symtab_free(SymTab *table) {
    while (table->current) {
```

Kode Program 15.3. Lookup pada scope bersarang

Fungsi `lookup_in_scope` membandingkan nama dan panjangnya. Perbandingan panjang lebih dahulu menghindari biaya `strncmp` yang tidak perlu dan menangani nama yang bukan null-terminated.

15.3.4 Deklarasi Ganda

Deklarasi ganda dalam scope yang sama merupakan kesalahan. Parser semantik harus memanggil `lookup_current` sebelum `define`. Jika simbol sudah ada, laporkan kesalahan dan jangan menambahkan simbol baru.

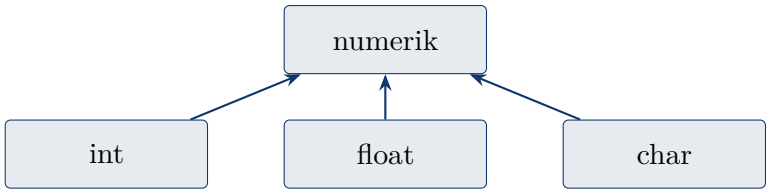
Informasi

Shadowing berbeda dari deklarasi ganda. Shadowing melibatkan scope berbeda; deklarasi ganda melibatkan scope yang sama. Keduanya harus dibedakan dalam pesan kesalahan.

15.4 Sistem Tipe dan Pemeriksaan Tipe

15.4.1 Sistem Tipe

Sistem tipe menentukan aturan pembentukan tipe dan kompatibilitas. Pada Subset-C, tipe dasar adalah `int`, `float`, `char`, `void`, `string`, dan `bool`. Tipe `error` dipakai untuk propagasi.



Gambar 15.3. Hierarki tipe numerik Subset-C

15.4.2 Aturan Kompatibilitas

Aturan assignment:

- tipe sama selalu kompatibel;
- `int` dan `char` dapat dipromosikan ke `float`;
- `char` dapat dipromosikan ke `int`;
- `float` tidak dapat ditetapkan ke `int` tanpa cast.

Aturan aritmetika:

- jika salah satu operand `float`, hasilnya `float`;
- jika keduanya `int` atau `char`, hasilnya `int`;
- operand non-numerik menghasilkan `error`.

15.4.3 Traversal AST

Pemeriksaan tipe dilakukan dengan traversal AST. Fungsi `check_expr` mengembalikan tipe simpul dan melaporkan kesalahan. Potongannya ditunjukkan pada Kode Program 15.4.

```
55 static TypeKind check_expr(SymTab *table, AstNode *node) {
56     if (!node) {
57         return TYPE_ERROR;
58     }
59
60     switch (node->kind) {
61     case AST_IDENT: {
```

```
62          Symbol *symbol = symtab_lookup(table, node->token.lexeme,
63                                         node->token.length);
64          if (!symbol) {
65              char message[256];
66              snprintf(message, sizeof(message),
67                       "identifier '%.*s' tidak dideklarasikan",
68                       node->token.length, node->token.lexeme);
69              report(node, message);
70              return TYPE_ERROR;
71          }
72          return symbol->type;
73      }
74
75      case AST_INT_LIT:    return TYPE_INT;
76      case AST_FLOAT_LIT:  return TYPE_FLOAT;
77      case AST_CHAR_LIT:   return TYPE_CHAR;
78      case AST_STRING_LIT: return TYPE_STRING;
79
80      case AST_BINARY:
81          return check_binary(table, node);
82
83      case AST_UNARY: {
84          TypeKind operand = check_expr(table, node->right);
85          if (node->token.type == TOK_NOT) {
86              return TYPE_BOOL;
87          }
88          if (!type_is_numeric(operand)) {
89              report(node, "operator unary memerlukan operand numerik");
90              return TYPE_ERROR;
91          }
92          return operand;
93      }
94
95      case AST_CALL: {
96          Symbol *symbol = symtab_lookup(table, node->token.lexeme,
97                                         node->token.length);
98          check_children_expr(table, node);
99          if (!symbol) {
100             char message[256];
101             snprintf(message, sizeof(message),
102                      "fungsi '%.*s' tidak dideklarasikan",
103                      node->token.length, node->token.lexeme);
104             report(node, message);
105             return TYPE_ERROR;
```

```
106         }
107         if (node->token.length == 5 &&
108             strncmp(node->token.lexeme, "print", 5) == 0) {
109             return TYPE_VOID;
110         }
111         return symbol->type;
112     }
113
114     case AST_ASSIGN: {
115         Symbol *symbol = symtab_lookup(table, node->token.lexeme,
116                                        node->token.length);
117         TypeKind value = check_expr(table, node->left);
118         if (!symbol) {
119             char message[256];
120             snprintf(message, sizeof(message),
121                      "identifier '%.*s' tidak dideklarasikan",
122                      node->token.length, node->token.lexeme);
123             report(node, message);
124             return TYPE_ERROR;
125         }
126         if (!type_assignable(symbol->type, value)) {
127             char message[256];
128             snprintf(message, sizeof(message),
129                      "tidak dapat menetapkan %s ke %s",
130                      type_name(value), type_name(symbol->type));
131             report(node, message);
132             return TYPE_ERROR;
133         }
134         return symbol->type;
135     }
136
137     default:
138         return TYPE_ERROR;
139     }
140 }
```

Kode Program 15.4. Fungsi `check_expr`

Setiap cabang menangani satu jenis simpul. Untuk identifier, pemeriksaan dilakukan melalui tabel simbol. Untuk ekspresi biner, tipe anak dihitung lebih dahulu, lalu aturan operator diterapkan.

15.4.4 Blok dan Deklarasi

Fungsi `check_block` mendorong scope, memproses deklarasi, lalu melepas scope. Potongannya ditunjukkan pada Kode Program 15.5.

```
static void check_block(SymTab *table, AstNode *node) {
    AstNode *child;
    symtab_enter_scope(table);
    for (child = node->children; child; child = child->next) {
        if (child->kind == AST_DECL) {
            TypeKind init_type = TYPE_ERROR;
            if (child->children) {
                init_type = check_expr(table, child->children);
            }
            if (symtab_lookup_current(table, child->token.lexeme,
                                      child->token.length)) {
                char message[256];
                snprintf(message, sizeof(message),
                         "identifier '%.*s' sudah dideklarasikan",
                         child->token.length, child->token.lexeme);
                report(child, message);
            } else {
                symtab_define(table, child->token.lexeme,
                              child->token.length, child->decl_type,
                              SYM_VAR, child->token.line,
                              child->token.column);
            }
            if (child->children &&
                !type_assignable(child->decl_type, init_type)) {
                char message[256];
                snprintf(message, sizeof(message),
                         "tidak dapat menetapkan %s ke %s",
                         type_name(init_type),
                         type_name(child->decl_type));
                report(child, message);
            }
        } else if (child->kind == AST_BLOCK) {
            check_block(table, child);
        } else {
            check_expr(table, child);
            check_children_expr(table, child);
        }
    }
    symtab_leave_scope(table);
}
```

Kode Program 15.5. Fungsi `check_block`

Deklarasi diperiksa terhadap `lookup_current` sebelum ditambahkan. Inisialisasi diperiksa terhadap tipe deklarasi. Jika inisialisasi tidak kompatibel, kesalahan dilaporkan pada lokasi deklarasi.

15.4.5 Fungsi dan Parameter

Fungsi didefinisikan pada scope global. Parameter didefinisikan pada scope fungsi, lalu blok tubuh diperiksa. Tipe kembalian fungsi disimpan pada simbol fungsi dan dipakai saat memeriksa pemanggilan.

15.5 Konversi Tipe dan Pelaporan Kesalahan

15.5.1 Konversi Implisit

Konversi implisit (*coercion*) terjadi ketika tipe sumber dapat dipromosikan ke tipe target tanpa kehilangan informasi:

- `int` ke `float`;
- `char` ke `int`;
- `char` ke `float`.

Konversi sebaliknya memerlukan cast eksplisit. Pada Subset-C v1.0, cast belum didukung, sehingga `float` ke `int` merupakan kesalahan.

15.5.2 Konversi Eksplisit

Cast eksplisit ditulis `(int) ekspresi`. Cast memberi tahu compiler bahwa kehilangan presisi disengaja. Jika cast belum didukung, compiler harus melaporkan kesalahan dengan jelas, bukan diam-diam melakukan konversi.

Tabel 15.2. Aturan konversi Subset-C v1.0

Dari	Ke	Implisit?	Catatan
`int`	`float`	ya	promosi
`char`	`int`	ya	promosi
`char`	`float`	ya	promosi
`float`	`int`	tidak	memerlukan cast
`int`	`char`	tidak	berpotensi kehilangan data
`void`	tipe lain	tidak	tidak valid

15.5.3 Pesan Kesalahan

Pesan kesalahan tipe harus menyebut:

- lokasi ekspresi atau deklarasi;
- tipe yang diharapkan;
- tipe yang ditemukan;
- saran perbaikan bila memungkinkan.

Tabel 15.3. Contoh pesan kesalahan tipe

Masukan	Pesan
`int x = 3.5;`	`baris 1, kolom 9: tidak dapat menetapkan float ke int`
`int x; x + "halo";`	`baris 1, kolom 12: operator aritmetika memerlukan operand numerik`
`return q;`	`baris 1, kolom 8: identifier 'q' tidak dideklarasikan`
`int x; int x;`	`baris 1, kolom 8: identifier 'x' sudah dideklarasikan`

15.5.4 Propagasi Error

Nilai `TYPE_ERROR` meredam kesalahan turunan. Jika satu operand error, hasil operasi juga error tanpa pesan baru. Strategi ini menjaga laporan tetap fokus pada akar masalah.

Peringatan

Jangan menampilkan semua kesalahan sekaligus tanpa prioritas. Kelompokkan berdasarkan akar masalah dan urutkan berdasarkan lokasi agar pengguna dapat memperbaiki secara sistematis.

Contoh

Contoh 15.4 (Lookup pada scope bersarang)**.** Pada program

```
int x = 1;
{
    int y = 2;
    print(x + y);
}
```

saat memeriksa `x + y`, tabel mencari `x` di scope blok (tidak ada), lalu di scope global (ditemukan, tipe `int`). `y` ditemukan di scope blok (tipe `int`). Hasil `x + y` bertipe `int`.

Contoh 15.5 (Shadowing)

```
int x = 1;
{
    int x = 2;
    print(x);
}
print(x);
```

`print(x)` di dalam blok mencetak 2; `print(x)` di luar blok mencetak 1. Tabel simbol menyimpan dua simbol `x` pada scope berbeda.

Contoh 15.6 (Pemeriksaan tipe ekspresi)**.** Untuk `int a; float b; a + b`, `check_expr` pada identifier `a` mengembalikan `int`, `b` mengembalikan `float`. Aturan `type_arith(int, float)` mengembalikan `float`.

Contoh 15.7 (Deklarasi ganda)

```
int main() {
    int x = 1;
    int x = 2;
    return 0;
}
```

Saat deklarasi kedua diproses, `lookup_current` menemukan `x` yang sudah ada. Pemeriksa melaporkan `identifier 'x' sudah dideklarasikan` dan tidak menambahkan simbol baru.

Contoh 15.8 (Kesalahan konversi)

```
int main() {
    int x = 3.5;
```

```
    return 0;
}
```

Tipe inisialisasi adalah `float`, sedangkan tipe deklarasi adalah `int`. Karena `float` tidak assignable ke `int` tanpa cast, pemeriksa melaporkan `tidak dapat menetapkan float ke int`.

Aktivitas Pembelajaran dan Praktikum

Tabel 15.4. Rencana aktivitas Bab XV

Sesi	Bentuk	Kegiatan
1	Ceramah	Tabel simbol dan scope
2	Live coding	Implementasi `symtab.c`
3	Latihan	Pemeriksaan tipe ekspresi
4	Praktikum	M4: symbol table dan type checker

M4 / Praktikum 4: Symbol Table dan Type Checker

Tujuan. Mengimplementasikan tabel simbol bersarang dan pemeriksa tipe untuk Subset-C.

Keluaran.

1. `symtab.h` dan `symtab.c`.
2. `types.h`, `types.c`, dan `typecheck.c`.
3. Driver `main_semantic.c`.
4. Minimal 20 kasus uji semantik.
5. Laporan singkat.

Langkah kerja.

1. Implementasikan `define`, `lookup`, dan scope.
2. Implementasikan aturan tipe.
3. Traversal AST untuk deklarasi dan ekspresi.
4. Tangani `TYPE_ERROR`.

5. Uji program valid dan tidak valid.

Rubrik.

Tabel 15.5. Rubrik Praktikum 4

Aspek	Indikator	Bobot
Kebenaran	Scope, lookup, dan tipe benar	40%
Kualitas kode	Struktur, penamaan, komentar	30%
Kelengkapan	Deklarasi, ekspresi, fungsi	20%
Pengujian	Kasus normal, shadowing, error	10%

Definisi selesai.

☐ Tabel simbol mendukung scope bersarang.

☐ Pemeriksa tipe menangani promosi dan error.

☐ 20 kasus uji tersedia.

☐ Laporan diserahkan.

Latihan dan Refleksi

Latihan

1. Sebutkan atribut yang disimpan tabel simbol.
2. Jelaskan perbedaan `lookup` dan `lookup_current`.
3. Mengapa scope perlu didorong dan dilepas?
4. Jelaskan shadowing dengan contoh.
5. Rancang hash function sederhana untuk identifier.
6. Tentukan tipe `int + char * float`.
7. Jelaskan aturan konversi implisit.
8. Rancang pesan kesalahan untuk `int x = 3.5;`.

Refleksi

Refleksi

1. Bagian mana dari tabel simbol yang paling menantang?
2. Apakah Anda dapat menjelaskan mengapa `lookup` dimulai dari scope dalam?
3. Bagaimana Anda memastikan pesan kesalahan tidak berulang?

Asesmen

Bab XV memuat Praktikum 4 dengan bobot 8% sesuai RPS. Asesmen mencakup tabel simbol, scope, dan pemeriksaan tipe.

Tabel 15.6. Instrumen asesmen Bab XV

Instrumen	Indikator	Bobot
Praktikum 4	Symbol table dan type checker	8%
Kuis	Scope, shadowing, dan aturan tipe	formatif
Review kode	Kualitas implementasi	bagian P4

Kriteria kelulusan: tabel simbol mendukung scope bersarang, pemeriksa tipe menangani promosi dan error, dan 20 kasus uji lulus.

Checklist Pencapaian Kompetensi

☐ Saya dapat menjelaskan struktur tabel simbol.

☐ Saya dapat mengimplementasikan `define` dan `lookup`.

☐ Saya dapat mengelola scope bersarang.

☐ Saya dapat menjelaskan shadowing.

☐ Saya dapat menerapkan aturan tipe.

☐ Saya dapat menangani konversi implisit.

☐ Saya dapat menghasilkan pesan kesalahan tipe.

☐ Saya sudah menyelesaikan M4/Praktikum 4.

Rangkuman

Pada bab ini telah dibahas:

- struktur dan operasi tabel simbol;
- implementasi hash table dan chaining;
- nested scope, shadowing, dan resolusi nama;
- sistem tipe dan pemeriksaan tipe;
- konversi tipe dan pelaporan kesalahan.

Bab XVI akan membahas AST, validasi kode, dan penanganan error secara menyeluruh. Kita akan menggabungkan parser, tabel simbol, dan pemeriksa tipe menjadi satu validator yang memberikan diagnostik informatif.

Evaluasi

1. Sebutkan lima atribut simbol.
2. Jelaskan perbedaan `lookup` dan `lookup_current`.
3. Jelaskan shadowing dengan contoh.
4. Rancang hash table untuk 100 identifier.
5. Tentukan tipe `int + char * float`.
6. Jelaskan aturan konversi implisit.
7. Rancang pesan kesalahan untuk deklarasi ganda.
8. Mengapa propagasi error penting?

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan konsep (70%), contoh (30%).
- Soal 4–6: rancangan dan aturan (70%), penjelasan (30%).
- Soal 7–8: kualitas pesan dan argumentasi (60%), kelengkapan (40%).

BAB XVI

Abstract Syntax Tree, Validasi Kode, dan Penanganan Error

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: Sub-CPMK 4.4, Sub-CPMK 1.7

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. merancang dan membangun AST yang sesuai;
2. melakukan traversal AST dengan pola visitor;
3. menerapkan aturan validasi kode secara multi-pass;
4. mengklasifikasikan dan memulihkan kesalahan;
5. menghasilkan diagnostik yang informatif;
6. menyelesaikan M5: validator dan proyek akhir.

Materi Pokok

- Desain dan konstruksi AST.
- Traversal AST dan visitor di C.
- Validasi kode: aturan dan multi-pass.
- Klasifikasi, pemulihan, dan pelaporan kesalahan.
- Diagnostik informatif dan pengujian.

16.1 Desain dan Konstruksi AST

16.1.1 Prinsip Desain

AST yang baik memenuhi beberapa prinsip:

- setiap simpul merepresentasikan satu konstruksi bahasa;

- informasi sintaksis yang tidak bermakna dihilangkan;
- simpul menyimpan lokasi untuk diagnostik;
- struktur mendukung traversal yang mudah;
- tipe dan informasi semantik dapat ditambahkan tanpa mengubah bentuk dasar.

16.1.2 Struktur Generik

Proyek Subset-C memakai simpul generik:

```
typedef struct AstNode {
    AstKind kind;
    Token token;
    TypeKind decl_type;
    struct AstNode *left;
    struct AstNode *right;
    struct AstNode *children;
    struct AstNode *next;
} AstNode;
```

Struktur ini menyimpan:

- `kind`: jenis simpul;
- `token`: token representatif untuk lokasi dan lexeme;
- `decl_type`: tipe deklarasi (untuk deklarasi/fungsi);
- `left` dan `right`: operand biner/unary;
- `children` dan `next`: daftar anak.

Antarmuka lengkap AST ditunjukkan pada Kode Program 16.1.

```
// Berkas: include/ast.h
// Abstract Syntax Tree sederhana untuk Subset-C.

#ifndef SUBSETC_AST_H
#define SUBSETC_AST_H

#include <stdio.h>
#include "token.h"
#include "types.h"

typedef enum {
    AST_PROGRAM,
```

```
    AST_FUNC,
    AST_PARAM,
    AST_BLOCK,
    AST_DECL,
    AST_ASSIGN,
    AST_CALL,
    AST_IF,
    AST_WHILE,
    AST_FOR,
    AST_RETURN,
    AST_BREAK,
    AST_CONTINUE,
    AST_PRINT,
    AST_BINARY,
    AST_UNARY,
    AST_INT_LIT,
    AST_FLOAT_LIT,
    AST_CHAR_LIT,
    AST_STRING_LIT,
    AST_IDENT,
    AST_EXPR_STMT,
    AST_EMPTY
} AstKind;

typedef struct AstNode {
    AstKind kind;
    Token token;
    TypeKind decl_type;
    struct AstNode *left;
    struct AstNode *right;
    struct AstNode *children;
    struct AstNode *next;
} AstNode;

AstNode *ast_new(AstKind kind, Token token);
void ast_add_child(AstNode *parent, AstNode *child);
const char *ast_kind_name(AstKind kind);
void ast_print(FILE *out, const AstNode *node, int indent);
void ast_free(AstNode *node);

#endif /* SUBSETC_AST_H */
```

Kode Program 16.1. Antarmuka `ast.h`

16.1.3 Generik versus Spesifik

Simpul generik lebih mudah diimplementasikan dan dipelajari, tetapi kurang type-safe. Simpul spesifik, misalnya `BinaryExpr` dan `IfStmt`, memberikan kejelasan dan mencegah akses field yang salah. Untuk proyek besar, simpul spesifik biasanya lebih baik.

Informasi

Pilihan struktur AST memengaruhi seluruh fase berikutnya. Luangkan waktu untuk merancangnya sebelum menulis parser. Perubahan struktur setelah parser selesai biasanya mahal.

16.1.4 Membangun AST

AST dibangun saat parsing:

- aturan yang hanya mengatur sintaksis tidak membuat simpul;
- aturan operator membuat simpul biner dengan dua anak;
- aturan daftar membuat simpul dan menambahkan anak;
- deklarasi menyimpan tipe pada `decl_type`.

Pendekatan ini menghasilkan AST yang ringkas dan langsung mencerminkan struktur program.

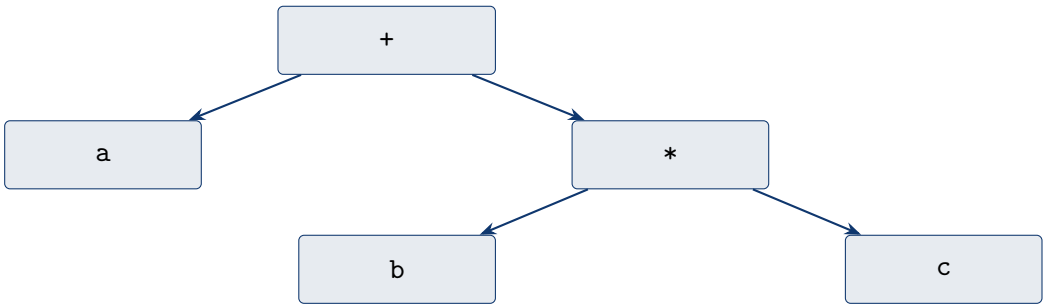
16.2 Traversal AST dan Visitor di C

16.2.1 Traversal Rekursif

Traversal AST adalah proses mengunjungi simpul-simpulnya. Dua urutan umum:

- **pre-order**: kunjungi simpul, lalu anak-anak;
- **post-order**: kunjungi anak-anak, lalu simpul.

Pre-order cocok untuk analisis yang membutuhkan konteks dari orang tua, misalnya validasi loop. Post-order cocok untuk sintesis, misalnya perhitungan tipe.



Gambar 16.1. Urutan pre-order: $+$, a, $*$, b, c

16.2.2 Pola Visitor

Pola visitor memisahkan operasi dari struktur data. Pada C, visitor dapat diimplementasikan dengan:

- fungsi yang menerima simpul dan konteks;
- tabel fungsi per jenis simpul;
- `switch` pada `kind`.

Proyek Subset-C memakai `switch` karena sederhana dan mudah dibaca. Setiap operasi baru ditambahkan sebagai fungsi traversal terpisah, misalnya `typecheck` dan `validate`.

16.2.3 Contoh Traversal

Fungsi `visit_children` mengunjungi semua anak:

```
static void visit_children(Context *context, AstNode *node) {
    AstNode *child;
    for (child = node->children; child; child = child->next) {
        visit(context, child);
    }
}
```

Fungsi `visit` menangani jenis simpul dan memanggil `visit_children` bila perlu. Pola ini menghindari duplikasi kode traversal.

16.2.4 Konteks Traversal

Traversal sering membutuhkan konteks, misalnya kedalaman loop atau tipe kembalian fungsi. Konteks diteruskan sebagai parameter dan diperbarui saat masuk/keluar simpul. Pada validator, konteks berisi `loop_depth`, `return_type`, dan `has_return`.

16.3 Validasi Kode: Aturan dan Multi-Pass

16.3.1 Aturan Validasi

Validasi memeriksa aturan yang tidak tercakup oleh tipe:

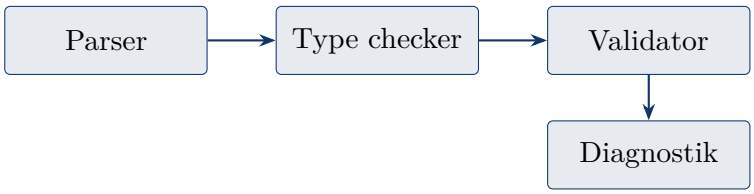
1. `break` dan `continue` hanya di dalam loop;
2. fungsi non-`void` harus mengembalikan nilai;
3. `main` harus `int main()` tanpa parameter;
4. `return` hanya di dalam fungsi;
5. pernyataan setelah `return` tidak terjangkau;
6. pemanggilan fungsi sesuai signature.

Aturan dapat diperluas sesuai kebutuhan bahasa. Yang penting adalah setiap aturan memiliki pesan kesalahan yang jelas dan lokasi yang tepat.

16.3.2 Multi-Pass

Validasi dilakukan setelah analisis semantik:

1. parsing menghasilkan AST;
2. analisis semantik membangun tabel simbol dan memeriksa tipe;
3. validasi memeriksa aturan kontrol dan kelengkapan;
4. pelaporan mengumpulkan dan mengurutkan diagnostik.



Gambar 16.2. Pipeline validasi multi-pass

16.3.3 Implementasi Validator

Validator menggunakan traversal dengan konteks. Potongan fungsi `visit` ditunjukkan pada Kode Program 16.2.

```
31 static void visit(Context *context, AstNode *node) {
32     if (!node) {
33         return;
34     }
35 
36     switch (node->kind) {
37     case AST_FUNC: {
38         Context inner;
39         inner.loop_depth = 0;
40         inner.return_type = node->decl_type;
41         inner.has_return = 0;
42         inner.had_error = 0;
43 
44         visit_children(&inner, node);
45 
46         if (node->decl_type != TYPE_VOID && !inner.has_return) {
47             report(node, context,
48                    "fungsi non-void harus mengembalikan nilai");
49         }
50         if (inner.had_error) {
51             context->had_error = 1;
52         }
53         break;
54     }
55 
56     case AST_BLOCK:
57         visit_children(context, node);
58         break;
59 
60     case AST_WHILE:
61     case AST_FOR:
62         context->loop_depth++;
63         visit_children(context, node);
64         context->loop_depth--;
65         break;
66 
67     case AST_BREAK:
68         if (context->loop_depth == 0) {
69             report(node, context,
70                    "break hanya valid di dalam loop");
71         }
72         break;
73 
74     case AST_CONTINUE:
```

```
75          if (context->loop_depth == 0) {
76              report(node, context,
77                     "continue hanya valid di dalam loop");
78          }
79          break;
80
81      case AST_RETURN:
82          context->has_return = 1;
83          visit_children(context, node);
84          break;
85
86      default:
87          visit_children(context, node);
88          break;
89      }
90  }
```

Kode Program 16.2. Fungsi `visit` pada `validate.c`

Setiap simpul fungsi memulai konteks baru dengan `return_type` dan `loop_depth` yang sesuai. Simpul loop menaikkan kedalaman sebelum mengunjungi anak dan menurunkannya setelah selesai. Simpul `break` dan `continue` memeriksa kedalaman loop.

16.3.4 Validasi Fungsi

Fungsi `validate_program` menjadi titik masuk. Potongannya ditunjukkan pada Kode Program 16.3.

```
92  int validate_program(AstNode *root) {
93      Context context;
94      context.loop_depth = 0;
95      context.return_type = TYPE_VOID;
96      context.has_return = 0;
97      context.had_error = 0;
98
99      visit_children(&context, root);
100     return context.had_error;
101 }
```

Kode Program 16.3. Fungsi `validate_program`

Validator mengembalikan jumlah kesalahan. Driver menggabungkan hasil parser, type checker, dan validator untuk menentukan status akhir.

16.4 Klasifikasi, Pemulihan, dan Pelaporan Kesalahan

16.4.1 Klasifikasi Kesalahan

Tabel 16.1. Klasifikasi kesalahan dan fase pendeteksinya

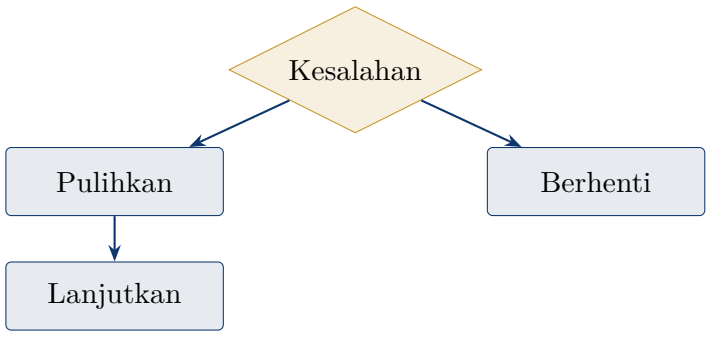
Kelas	Fase	Contoh
Leksikal	Lexer	karakter ilegal, string tidak ditutup
Sintaksis	Parser	titik koma hilang, kurung tidak seimbang
Semantik	Type checker	identifier tidak dideklarasikan, tipe salah
Validasi	Validator	`break` di luar loop, return hilang
Run-time	Eksekusi	pembagian nol, stack overflow

16.4.2 Pemulihan

Setiap fase memiliki strategi pemulihan:

- lexer: lewati karakter ilegal;
- parser: sinkronisasi ke titik koma atau kurung tutup;
- type checker: gunakan `TYPE_ERROR` dan lanjut;
- validator: catat kesalahan dan lanjut ke simpul berikutnya.

Pemulihan memungkinkan compiler melaporkan beberapa kesalahan dalam satu proses. Namun, pemulihan yang agresif dapat menghasilkan pesan palsu. Keseimbangan perlu diuji.



Gambar 16.3. Keputusan pemulihan setelah kesalahan

16.4.3 Format Diagnostik

Format yang konsisten memudahkan pembacaan dan integrasi dengan editor:

berkas : baris : kolom : error : pesan.

Contoh:

```
tests/invalid/break_outside.sc:2:5: error: break hanya valid di dalam loop
```

16.4.4 Batas Pelaporan

Setelah sejumlah kesalahan, compiler sebaiknya berhenti dan melaporkan ringkasan. Batas ini mencegah ledakan pesan dan membantu pengguna fokus. Nilai batas dapat dikonfigurasi, misalnya 20 kesalahan.

Tabel 16.2. Contoh pesan dari setiap fase

Fase	Pesan
Lexer	`baris 1, kolom 1: string tidak ditutup`
Parser	`baris 2, kolom 10: diharapkan ';' setelah deklarasi`
Type checker	`baris 3, kolom 9: tidak dapat menetapkan float ke int`
Validator	`baris 2, kolom 5: break hanya valid di dalam loop`

16.5 Diagnostik Informatif dan Pengujian

16.5.1 Kualitas Pesan

Pesan yang baik memuat:

- lokasi yang tepat;
- penyebab yang spesifik;
- harapan atau aturan yang dilanggar;
- saran perbaikan bila memungkinkan.

Bandingkan:

- buruk: `error`;
- cukup: `baris 4: type error`;
- baik: `baris 4, kolom 9: tidak dapat menetapkan float ke int; gunakan cast eksplisit`.

16.5.2 Kode Diagnostik

Untuk proyek besar, setiap kesalahan dapat memiliki kode, misalnya `E001` untuk identifier tidak dideklarasikan. Kode memudahkan dokumentasi, pencarian, dan pengujian otomatis.

16.5.3 Pengujian

Pengujian validator mencakup:

- program valid harus diterima;
- program tidak valid harus ditolak;
- pesan kesalahan harus menyebut lokasi yang benar;
- jumlah kesalahan harus wajar, tidak berulang.

Direktori `tests/invalid` memuat program untuk menguji aturan validasi. Setiap program memiliki satu kesalahan utama, sehingga hasilnya mudah diverifikasi.

16.5.4 Driver Final

Driver final menggabungkan seluruh fase. Potongannya ditunjukkan pada Kode Program 16.4.

```
int main(int argc, char **argv) {
    char *source;
    Parser parser;
    AstNode *program;
    SymTab table;
    int semantic_errors;
    int validation_errors;

    if (argc != 2) {
        fprintf(stderr, "penggunaan: %s berkas.sc\n", argv[0]);
        return 2;
    }

    source = read_file(argv[1]);
    if (!source) {
        fprintf(stderr, "tidak dapat membaca %s\n", argv[1]);
        return 2;
    }

    parser_init(&parser, source, argv[1]);
```

```
60      program = parser_program(&parser);
61
62      if (parser.had_error || parser.lexer.had_error) {
63          ast_free(program);
64          free(source);
65          return 1;
66      }
67
68      symtab_init(&table);
69      semantic_errors = typecheck_program(&table, program);
70      validation_errors = validate_program(program);
71
72      if (!semantic_errors && !validation_errors) {
73          printf("OK\n");
74          ast_print(stdout, program, 0);
75      }
76
77      symtab_free(&table);
78      ast_free(program);
79      free(source);
80      return (semantic_errors || validation_errors) ? 1 : 0;
```

Kode Program 16.4. Driver final `main.c`

Jika tidak ada kesalahan, driver mencetak `OK` dan AST. Jika ada kesalahan, driver mengembalikan status bukan nol. Status ini dapat digunakan oleh skrip pengujian atau sistem build.

16.5.5 M5: Proyek Akhir

M5 menggabungkan seluruh milestone:

- lexer (M1);
- parser dan AST (M2);
- Bison dan AST lengkap (M3);
- symbol table dan type checker (M4);
- validator dan diagnostik (M5).

Keluaran akhir adalah CLI `scc` yang menerima satu berkas Subset-C, menjalankan seluruh pipeline, dan melaporkan hasilnya.

Tabel 16.3. Artefak M5

Artefak	Status	Keterangan
`validate.c`	selesai	aturan validasi
`main.c`	selesai	CLI final
`tests/invalid`	selesai	tes validasi
`run-final-tests.bat`	selesai	skrip tes
Laporan akhir	wajib	dokumentasi dan refleksi

Contoh

Contoh 16.1 (Dump AST)**.** Untuk program

```
int main() {
    int a = 2 + 3;
    return 0;
}
```

dump AST kira-kira:

```
PROGRAM
  FUNC main
    BLOCK
      DECL a
        BINARY +
          INT_LIT 2
          INT_LIT 3
      RETURN
        INT_LIT 0
```

Setiap simpul mencerminkan konstruksi program tanpa detail titik koma atau kurung.

Contoh 16.2 (Break di luar loop)

```
int main() {
    break;
    return 0;
}
```

Validator memasuki fungsi dengan `loop_depth = 0`. Saat mengunjungi `BREAK`, kedalaman masih nol, sehingga validator melaporkan `break hanya valid di dalam loop`.

Contoh 16.3 (Fungsi tanpa return)

```
int hitung(int x) {
    int y = x + 1;
}
```

Fungsi bertipe `int` tetapi tidak memiliki `return`. Validator melaporkan `fungsi non-void harus mengembalikan nilai` pada lokasi nama fungsi.

Contoh 16.4 (Format diagnostik)**.** Keluaran

```
tests/invalid/break_outside.sc:2:5: error: break hanya valid di dalam loop
```

memuat berkas, baris, kolom, tingkat, dan pesan. Editor dapat melompat ke lokasi berdasarkan format ini.

Contoh 16.5 (Menguji program tidak valid)**.** Setiap berkas di `tests/invalid` memiliki satu kesalahan utama. Skrip `run-final-tests.bat` memastikan program valid diterima dan program tidak valid ditolak. Pendekatan satu-kesalahan-per-berkas memudahkan isolasi dan verifikasi.

Aktivitas Pembelajaran dan Praktikum

Tabel 16.4. Rencana aktivitas Bab XVI

Sesi	Bentuk	Kegiatan
1	Ceramah	Desain AST dan traversal
2	Live coding	Validator dan diagnostik
3	Diskusi	Kualitas pesan kesalahan
4	Praktikum	M5: proyek akhir dan demo

M5 / Project Final: Mini Compiler Front-End

Tujuan. Menggabungkan lexer, parser, AST, type checker, dan validator menjadi CLI yang bekerja.

Keluaran.

1. CLI `scc` yang menerima satu berkas Subset-C.
2. Pipeline lengkap: lexer, parser, type checker, validator.
3. Minimal 40 kasus uji: valid dan tidak valid.
4. Laporan proyek dan demo.

Langkah kerja.

1. Integrasikan seluruh modul.

2. Tambahkan aturan validasi yang belum tercakup.
3. Uji dengan program valid dan tidak valid.
4. Ukur waktu dan penggunaan memori.
5. Tulis laporan dan siapkan demo.

Rubrik.

Tabel 16.5. Rubrik Project Final

Aspek	Indikator	Bobot
Fungsionalitas	Pipeline bekerja untuk program valid	30%
Integrasi	Modul terhubung dan konsisten	20%
Kualitas kode	Struktur, penamaan, komentar	15%
Pengujian	40 kasus uji, kasus batas	15%
Dokumentasi	Laporan dan README	10%
Presentasi	Demo dan penjelasan	10%

Definisi selesai.

☐ CLI memproses program valid tanpa kesalahan.

☐ CLI menolak program tidak valid dengan pesan jelas.

☐ 40 kasus uji tersedia.

☐ Laporan dan demo selesai.

Latihan dan Refleksi

Latihan

1. Jelaskan prinsip desain AST.
2. Bandingkan simpul generik dan spesifik.
3. Jelaskan pre-order dan post-order untuk AST.
4. Rancang aturan validasi untuk `return`.
5. Rancang aturan validasi untuk `main`.
6. Jelaskan strategi pemulihan di setiap fase.
7. Tuliskan format diagnostik yang baik.
8. Mengapa jumlah pesan kesalahan perlu dibatasi?

Refleksi

Refleksi

1. Bagian mana dari validator yang paling menantang?
2. Apakah pesan kesalahan proyek Anda sudah informatif?
3. Apa yang akan Anda perbaiki jika memulai proyek dari awal?

Asesmen

Bab XVI memuat Project Final dengan bobot 20% sesuai RPS. Asesmen mencakup fungsionalitas, integrasi, kualitas kode, pengujian, dokumentasi, dan presentasi.

Tabel 16.6. Instrumen asesmen Bab XVI

Instrumen	Indikator	Bobot
Project Final	Pipeline lengkap dan tes	20%
Laporan	Dokumentasi desain dan hasil	bagian proyek
Demo	Presentasi dan penjelasan	bagian proyek

Kriteria kelulusan: CLI memproses program valid, menolak program tidak valid dengan diagnostik informatif, dan seluruh modul terintegrasi dengan baik.

Checklist Pencapaian Kompetensi

☐ Saya dapat merancang AST.

☐ Saya dapat melakukan traversal AST.

☐ Saya dapat menerapkan aturan validasi.

☐ Saya dapat mengklasifikasikan kesalahan.

☐ Saya dapat memulihkan dan melanjutkan analisis.

☐ Saya dapat menghasilkan diagnostik informatif.

☐ Saya dapat menguji program valid dan tidak valid.

☐ Saya sudah menyelesaikan M5/Project Final.

Rangkuman

Pada bab ini telah dibahas:

- desain dan konstruksi AST;
- traversal AST dan pola visitor di C;
- validasi kode dengan aturan dan multi-pass;
- klasifikasi, pemulihan, dan pelaporan kesalahan;
- diagnostik informatif, pengujian, dan proyek akhir.

Bab XVII akan menutup perjalanan dengan evaluasi, refleksi, dan integrasi kompetensi. Kita akan memetakan kembali seluruh Sub-CPMK, membahas blueprint ujian, dan merencanakan tindak lanjut.

Evaluasi

1. Jelaskan prinsip desain AST.
2. Bandingkan simpul generik dan spesifik.
3. Jelaskan pre-order dan post-order.
4. Rancang aturan validasi untuk `break`.
5. Rancang aturan validasi untuk fungsi non-void.
6. Jelaskan strategi pemulihan parser dan type checker.
7. Tuliskan format diagnostik dan komponennya.
8. Mengapa pengujian program tidak valid penting?

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan konsep (70%), contoh (30%).
- Soal 4–6: rancangan aturan dan pemulihan (70%), penjelasan (30%).
- Soal 7–8: kualitas format dan argumentasi (60%), kelengkapan (40%).

BAB XVII

Evaluasi, Refleksi, dan Integrasi Kompetensi

Outcome Pembelajaran dan Sub-CPMK

Sub-CPMK: integrasi seluruh Sub-CPMK 1.1–Sub-CPMK 4.4

Setelah menyelesaikan bab ini, mahasiswa mampu:

1. memetakan pencapaian seluruh Sub-CPMK;
2. memahami blueprint kuis, UTS, UAS, dan proyek akhir;
3. menggunakan rubrik komprehensif untuk menilai kinerja;
4. menyusun refleksi dan portofolio belajar;
5. merencanakan tindak lanjut studi dan karier.

Materi Pokok

- Peta pencapaian 19 Sub-CPMK.
- Blueprint kuis, UTS, UAS, dan proyek akhir.
- Rubrik komprehensif dan penilaian.
- Refleksi, jurnal, dan portofolio.
- Tindak lanjut dan topik lanjutan.

17.1 Peta Pencapaian 19 Sub-CPMK

17.1.1 Rekapitulasi

Seluruh sembilan belas Sub-CPMK telah dibahas pada bab-bab inti. Tabel berikut merangkum bab utama, asesmen, dan bukti belajar yang diharapkan.

Tabel 17.1. Peta pencapaian Sub-CPMK

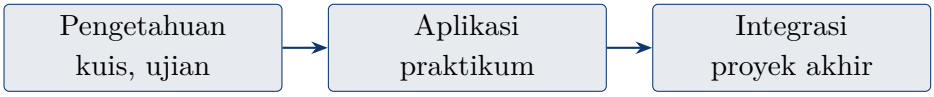
Kode	Kompetensi	Bab	Bukti belajar
1.1	Membedakan interpreter, compiler, dan translator	III	Kuis, ujian tulis
1.2	Menjelaskan fase kompilasi, tabel simbol, dan error handling	III	Kuis, diagram pipeline
1.3	Menganalisis one-pass/multi-pass dan run-time environment	III	Tugas teori, analisis kasus
1.4	Menjelaskan hierarki Chomsky dan automata	IV	Tugas klasifikasi, UTS
1.5	Menjelaskan attribute grammar, SDD, dan translation scheme	XIV	Tugas teori, P4
1.6	Mengimplementasikan symbol table dan type checking	XV	P4, kode dan tes
1.7	Menangani semantic error dengan pesan informatif	XVI	P4, proyek akhir
2.1	Menulis ekspresi reguler untuk token	V	Tugas teori, P1-A
2.2	Mengimplementasikan NFA/DFA untuk token recognition	VI	P1-B, kode
2.3	Menjelaskan ekivalensi NFA-DFA dan minimalisasi	VI	Tugas teori, UTS
2.4	Mengimplementasikan lexical analyzer	VII	P1, kode dan tes
3.1	Menulis CFG, BNF/EBNF, derivasi, dan parse tree	VIII	P2-A, ujian tulis
3.2	Menganalisis ambiguitas, left recursion, left factoring	IX	Tugas teori, UTS
3.3	Menghitung FIRST/FOLLOW dan menjelaskan LL(1)/LR(1)	X	UTS, P2
3.4	Merancang CFG untuk bahasa sederhana	VIII	P2-A, proyek
4.1	Membangun recursive descent dan predictive parser	X	P2, kode dan tes
4.2	Menerapkan bottom-up parsing dan menganalisis konflik	XI–XII	UTS, P3
4.3	Menggunakan parser generator dan semantic actions	XIII	P3, kode
4.4	Membangun AST dan memvalidasi kode	XIII, XVI	P3, proyek akhir

17.1.2 Analisis Ketercapaian

Ketercapaian dapat dianalisis pada tiga tingkat:

1. **pengetahuan**: mahasiswa dapat menjelaskan konsep;
2. **aplikasi**: mahasiswa dapat menerapkan teknik;
3. **integrasi**: mahasiswa dapat menggabungkan teknik dalam proyek.

Nilai kuis dan ujian mengukur tingkat pengetahuan dan aplikasi. Praktikum dan proyek mengukur aplikasi dan integrasi. Kombinasi keduanya memberikan gambaran kompetensi yang lebih lengkap.



Gambar 17.1. Tiga tingkat ketercapaian kompetensi

17.2 Blueprint Kuis, UTS, UAS, dan Proyek Akhir

17.2.1 Komponen dan Bobot

Tabel 17.2. Blueprint asesmen dan cakupan CPMK

Komponen	CPMK	Fokus	Bobot
Kuis	01, 02	Konsep kompilasi, token, automata	5%
Tugas Teori	01–03	Grammar, automata, parsing	8%
Praktikum 1	02	Lexer	8%
Praktikum 2	03, 04	Parser	8%
Praktikum 3	04	Parser generator dan AST	7%
Praktikum 4	01	Symbol table dan semantik	8%
UTS	01–03	Teori bahasa dan parsing	20%
Project Final	02–04	Mini compiler front-end	20%
UAS	01–04	Komprehensif	16%

17.2.2 UTS

UTS menekankan konsep teori dan analisis:

1. klasifikasi tata bahasa dan automata (C2–C4);

2. derivasi, pohon urai, dan ambiguitas (C3–C4);
3. FIRST/FOLLOW dan tabel LL(1) (C3–C4);
4. analisis shift-reduce dan konflik (C4–C5);
5. soal koding singkat (C3).

17.2.3 UAS

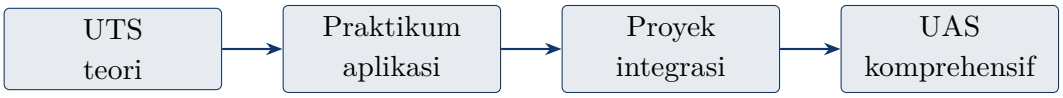
UAS menekankan integrasi dan analisis:

1. arsitektur kompilator dan lingkungan run-time (C2–C4);
2. analisis semantik dan type checking (C3–C4);
3. validasi AST dan diagnostik (C4–C5);
4. studi kasus program dengan kesalahan (C4–C5);
5. refleksi dan desain solusi (C5–C6).

17.2.4 Proyek Akhir

Proyek akhir menilai kemampuan integrasi:

- pipeline lexer–parser–AST–type checker–validator;
- kualitas kode dan pengujian;
- dokumentasi dan demo;
- kemampuan menjelaskan keputusan desain.



Gambar 17.2. Alur asesmen dari teori menuju integrasi

17.2.5 Contoh Soal UTS

1. Diberikan tata bahasa, tentukan tipe Chomsky dan jelaskan alasannya.
2. Buat derivasi leftmost dan pohon urai untuk string tertentu.
3. Hitung FIRST dan FOLLOW, lalu bangun tabel LL(1).
4. Telusuri shift-reduce dan tandai handle pada setiap langkah.
5. Jelaskan konflik shift-reduce dan cara menyelesaikannya.

17.2.6 Contoh Soal UAS

1. Jelaskan perbedaan one-pass dan multi-pass dengan contoh.
2. Tentukan tipe ekspresi campuran dan jelaskan promosi.
3. Diberikan AST, tuliskan aturan validasi yang dilanggar.
4. Rancang pesan kesalahan untuk tiga kasus berbeda.
5. Usulkan perbaikan desain untuk program yang gagal validasi.

17.3 Rubrik Komprehensif dan Penilaian

17.3.1 Dimensi Penilaian

Rubrik komprehensif menilai lima dimensi:

1. **pengetahuan**: ketepatan konsep dan istilah;
2. **aplikasi**: kemampuan menerapkan teknik;
3. **analisis**: kemampuan menganalisis masalah dan trade-off;
4. **integrasi**: kemampuan menggabungkan komponen;
5. **komunikasi**: kejelasan laporan dan presentasi.

Tabel 17.3. Rubrik komprehensif

Dimensi	Indikator	Bobot
Pengetahuan	Konsep benar, istilah tepat	20%
Aplikasi	Teknik diterapkan dengan benar	25%
Analisis	Trade-off dan konflik dianalisis	20%
Integrasi	Komponen bekerja bersama	25%
Komunikasi	Laporan dan demo jelas	10%

17.3.2 Level Kinerja

Setiap dimensi dinilai pada empat level:

- **sangat baik**: lengkap, tepat, dan mendalam;
- **baik**: tepat dengan sedikit kekurangan;

- **cukup**: memenuhi kebutuhan dasar;
- **kurang**: ada kesalahan konsep atau bagian hilang.

17.3.3 Konversi Nilai

Tabel 17.4. Konversi nilai akhir

Rentang	Nilai
85–100	A
70–84	B
60–69	C
50–59	D
<50	E

17.3.4 Menghindari Perhitungan Ganda

Nilai praktikum dan proyek akhir harus dihitung secara terpisah. Praktikum menilai milestone; proyek menilai integrasi akhir. Jika mahasiswa menggunakan kode yang sama untuk keduanya, penilai harus memastikan tidak ada pengulangan bobot.

Peringatan

Dokumentasikan pemetaan nilai dari setiap komponen ke CPMK. Tanpa pemetaan yang jelas, sulit membuktikan ketercapaian kompetensi secara akuntabel.

17.4 Refleksi, Jurnal, dan Portofolio

17.4.1 Siklus Refleksi

Refleksi menggunakan siklus *what, so what, now what*:

1. **what**: apa yang terjadi?
2. **so what**: mengapa itu penting?
3. **now what**: apa yang akan dilakukan berikutnya?

Siklus ini membantu mahasiswa mengubah pengalaman menjadi pembelajaran yang terarah.

17.4.2 Jurnal Mingguan

Jurnal mingguan memuat:

- materi yang dipelajari;
- kesulitan yang dihadapi;
- strategi yang dicoba;
- hasil dan rencana perbaikan.

Jurnal tidak perlu panjang. Yang penting adalah konsistensi dan kejujuran. Jurnal menjadi bukti proses belajar, bukan hanya hasil.

17.4.3 Portofolio

Portofolio memuat bukti kompetensi:

- kode proyek dan tes;
- laporan praktikum;
- diagram dan catatan desain;
- refleksi dan evaluasi diri;
- presentasi atau demo.

Portofolio dinilai berdasarkan kelengkapan, kualitas, dan refleksi. Bukan jumlah dokumen, tetapi kedalaman bukti.

Tabel 17.5. Rubrik portofolio

Aspek	Indikator	Bobot
Kelengkapan	Semua milestone terwakili	30%
Kualitas	Kode, tes, dan laporan bermutu	30%
Refleksi	Analisis mendalam dan jujur	25%
Presentasi	Portofolio mudah dipahami	15%

17.4.4 Evaluasi Diri

Mahasiswa diminta menilai dirinya pada setiap CPMK dengan skala 1–4, disertai bukti. Evaluasi diri dibandingkan dengan penilaian dosen untuk menemukan kesenjangan persepsi.

Refleksi

Jawab:

1. Kompetensi apa yang paling berkembang selama satu semester?
2. Kompetensi apa yang masih perlu diperkuat?
3. Bukti apa yang menunjukkan kemajuan tersebut?
4. Apa rencana konkret untuk semester berikutnya?

17.5 Tindak Lanjut dan Topik Lanjutan

17.5.1 Setelah Front-End

Buku ini berfokus pada front-end. Topik lanjutan yang dapat dipelajari:

- **representasi antara**: SSA, three-address code, LLVM IR;
- **optimasi**: constant folding, dead code elimination, loop optimization, data-flow analysis;
- **pembangkitan kode**: instruction selection, register allocation, calling convention;
- **sistem tipe**: polimorfisme, inferensi tipe, dependent types;
- **analisis program**: static analysis, abstract interpretation, model checking;
- **pengujian compiler**: fuzzing, differential testing, metamorphic testing;
- **bahasa domain khusus**: desain dan implementasi DSL.

17.5.2 Jalur Belajar

Tabel 17.6. Jalur belajar lanjutan

Minat	Topik	Sumber
Compiler engineering	LLVM, codegen, optimasi	`llvm.org`
Bahasa pemrograman	sistem tipe, semantik formal	buku TAPL
Analisis program	static analysis, abstract interpretation	riset dan alat open source
Pengujian	fuzzing, differential testing	`oss-fuzz`
DSL	parser generator, interpreter	ANTLR, Xtext

17.5.3 Karier

Kompetensi front-end berguna pada:

- pengembangan compiler dan alat bahasa;
- static analysis dan linter;
- query optimizer dan mesin data;
- pengembangan DSL dan konfigurasi;
- keamanan aplikasi (analisis kode);
- AI untuk kode (parsing, transpiling, verifikasi).

17.5.4 Saran Akhir

Terus berlatih dengan proyek nyata. Baca kode compiler open source, tetapi mulai dari modul kecil. Dokumentasikan keputusan desain dan uji secara sistematis. Kemampuan membaca spesifikasi, memodelkan bahasa, dan memberikan diagnostik yang baik akan berguna jauh di luar mata kuliah ini.

Informasi

Compiler adalah sistem besar yang dibangun dari komponen kecil yang saling terhubung. Keterampilan yang Anda latih di sini—membaca spesifikasi, menulis parser, memeriksa tipe, dan menguji—berlaku pada banyak sistem perangkat lunak lain.

Contoh

Contoh 17.1 (Soal UTS dan jawaban)**. Soal.** Diberikan tata bahasa

$$S \to aSb \mid \varepsilon.$$

Tentukan tipe Chomsky, buat derivasi untuk $aabb$, dan jelaskan apakah tata bahasa ini LL(1).

Jawaban. Tata bahasa bertipe 2 (bebas konteks) karena aturan berbentuk $A \to \alpha$. Derivasi:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb.$$

Tata bahasa ini bukan LL(1) karena aturan $S \to aSb$ dan $S \to \varepsilon$ memiliki FIRST yang tumpang tindih dengan FOLLOW(S). Parser top-down memerlukan lookahead atau transformasi.

Contoh 17.2 (Soal UAS dan jawaban)**. Soal.** Diberikan program

```
int main() {
    int x = 3.5;
    break;
    return 0;
}
```

Sebutkan kesalahan semantik dan validasi, lalu tuliskan pesan yang informatif.

Jawaban. Kesalahan semantik: `float` tidak dapat ditetapkan ke `int` tanpa cast. Kesalahan validasi: `break` di luar loop. Pesan:

```
baris 1, kolom 9: tidak dapat menetapkan float ke int
baris 2, kolom 5: break hanya valid di dalam loop
```

Contoh 17.3 (Pertanyaan demo proyek)**.** 1. Mengapa Anda memilih recursive descent atau Bison?

2. Bagaimana parser menangani dangling else?

3. Di mana tabel simbol menyimpan informasi scope?

4. Bagaimana Anda menguji program tidak valid?

5. Apa keterbatasan proyek Anda saat ini?

Jawaban yang baik menunjukkan pemahaman trade-off, bukan sekadar menyebut fitur.

Contoh 17.4 (Kutipan esai refleksi)**.** "Pada awal semester, saya menganggap parsing hanya soal mencocokkan token. Setelah mengerjakan proyek, saya memahami bahwa keputusan desain grammar memengaruhi kualitas pesan kesalahan dan kemudahan pengujian. Saya belajar memisahkan logika parsing dari logika semantik dan menulis tes untuk setiap aturan."

Contoh 17.5 (Analisis ketercapaian)**.** Jika nilai kuis tinggi tetapi proyek rendah, kemungkinan mahasiswa memahami konsep tetapi kesulitan integrasi. Jika proyek tinggi tetapi ujian rendah, kemungkinan mahasiswa mahir praktik tetapi lemah teori. Analisis ini membantu menentukan tindak lanjut yang tepat.

Aktivitas Pembelajaran dan Praktikum

Tabel 17.7. Rencana aktivitas Bab XVII

Sesi	Bentuk	Kegiatan
1	Diskusi	Peta pencapaian dan analisis hasil
2	Latihan	Soal UAS dan studi kasus
3	Presentasi	Demo proyek akhir
4	Refleksi	Peer review dan retrospektif

Demo Akhir dan Peer Review

Tujuan. Mempresentasikan proyek, menerima umpan balik, dan menyusun rencana perbaikan.

Langkah kerja.

1. Siapkan demo 10 menit: arsitektur, fitur, dan keterbatasan.
2. Tunjukkan satu program valid dan satu program tidak valid.
3. Jelaskan keputusan desain utama.
4. Minta teman menguji dengan program baru.
5. Catat umpan balik dan susun rencana perbaikan.

Definisi selesai.

☐ Demo berjalan tanpa kesalahan.

☐ Program valid dan tidak valid ditunjukkan.

☐ Umpan balik tercatat.

☐ Rencana perbaikan disusun.

Demo dan peer review menjadi bagian dari Project Final (20%).

Latihan dan Refleksi

Latihan

1. Petakan lima Sub-CPMK ke bukti belajar Anda.

2. Analisis kekuatan dan kelemahan proyek Anda.
3. Rancang satu soal UAS beserta rubriknya.
4. Jelaskan tiga topik lanjutan yang ingin dipelajari.
5. Susun rencana belajar satu bulan setelah semester.
6. Berikan umpan balik konstruktif untuk proyek teman.
7. Identifikasi satu keputusan desain yang akan Anda ubah.
8. Tuliskan ringkasan portofolio dalam satu halaman.

Refleksi

Refleksi

1. Kompetensi apa yang paling berkembang?
2. Kompetensi apa yang masih perlu diperkuat?
3. Apa bukti konkret dari kemajuan tersebut?
4. Apa langkah berikutnya?

Asesmen

Bab XVII tidak memiliki bobot tersendiri. Asesmennya terintegrasi dalam UAS (16%) dan Project Final (20%). Fokusnya adalah kemampuan menghubungkan konsep, menganalisis, dan merefleksikan.

Tabel 17.8. Instrumen asesmen integratif

Instrumen	Indikator	Bobot
UAS	Integrasi konsep dan studi kasus	16%
Project Final	Pipeline lengkap, demo, laporan	20%
Portofolio	Bukti belajar dan refleksi	bagian proyek

Kriteria kelulusan: mahasiswa dapat menjelaskan seluruh pipeline, menganalisis kesalahan, mempertahankan keputusan desain, dan menyusun rencana pengembangan diri.

Checklist Pencapaian Kompetensi

☐ Saya dapat memetakan 19 Sub-CPMK ke bukti belajar.

☐ Saya memahami blueprint UTS dan UAS.

☐ Saya memahami rubrik proyek akhir.

☐ Saya dapat menganalisis hasil belajar saya.

☐ Saya memiliki portofolio yang lengkap.

☐ Saya dapat menjelaskan keputusan desain proyek.

☐ Saya dapat memberikan umpan balik konstruktif.

☐ Saya memiliki rencana tindak lanjut.

Rangkuman

Pada bab ini telah dibahas:

- peta pencapaian 19 Sub-CPMK;
- blueprint kuis, UTS, UAS, dan proyek akhir;
- rubrik komprehensif dan penilaian;
- refleksi, jurnal, dan portofolio;
- tindak lanjut dan topik lanjutan.

Buku ini berakhir, tetapi perjalanan belajar berlanjut. Front-end kompilator adalah fondasi untuk memahami bahasa, alat, dan sistem yang lebih besar. Teruslah berlatih, membaca kode, dan menguji ide baru.

Evaluasi

1. Petakan lima Sub-CPMK ke bab dan asesmen yang sesuai.
2. Jelaskan komponen blueprint UTS.
3. Jelaskan komponen blueprint UAS.
4. Rancang satu soal studi kasus dan rubriknya.
5. Jelaskan rubrik komprehensif lima dimensi.
6. Bagaimana menghindari perhitungan ganda?

7. Susun kerangka portofolio satu halaman.
8. Pilih dua topik lanjutan dan jelaskan alasannya.

Kunci dan Rubrik Singkat

- Soal 1–3: ketepatan pemetaan dan blueprint (70%), kelengkapan (30%).
- Soal 4–5: kualitas rancangan dan rubrik (70%), kejelasan (30%).
- Soal 6–8: analisis dan argumentasi (60%), relevansi (40%).

LAMPIRAN A

Rubrik Penilaian Lengkap

Lampiran ini memuat rubrik penilaian yang dipakai sepanjang semester. Bobot total seluruh komponen adalah 100% dan mengikuti RPS.

A.1 Bobot Komponen Penilaian

Tabel A.1. Komponen penilaian dan bobotnya

Komponen	Cakupan	Bobot
Kuis	Sub-CPMK 1.1–1.3	5%
Tugas Teori	Grammar, automata, parsing	8%
Praktikum 1	Lexer	8%
Praktikum 2	Parser	8%
Praktikum 3	Parser generator dan AST	7%
Praktikum 4	Symbol table dan analisis semantik	8%
UTS	CPMK 01–03	20%
Project Final	Mini compiler front-end	20%
UAS	CPMK 01–04	16%
Total		**100%**

A.2 Rubrik Praktikum

Tabel A.2. Rubrik umum Praktikum 1–4

Aspek	Indikator	Bobot
Kebenaran	Program memenuhi spesifikasi dan lulus tes	40%
Kualitas kode	Keterbacaan, struktur, penamaan, komentar	30%
Kelengkapan	Semua fitur wajib diimplementasikan	20%
Pengujian	Tes mencakup kasus normal dan kasus batas	10%

A.3 Konversi Nilai

Tabel A.3. Konversi nilai akhir

Rentang	Nilai
85–100	A
70–84	B
60–69	C
50–59	D
<50	E

LAMPIRAN B

Contoh Tugas, Portofolio, dan Rubrik Refleksi

Lampiran ini memuat contoh lembar tugas, template laporan, dan rubrik refleksi yang dapat langsung dipakai atau disesuaikan.

B.1 Lembar Tugas Praktikum

Tabel B.1. Format lembar tugas praktikum

Bagian	Isi
Identitas	Nama, NIM, kelas, tanggal
Tujuan	Sub-CPMK yang diukur
Deskripsi	Konteks dan masalah yang diselesaikan
Keluaran	Berkas, laporan, demo
Kriteria	Rubrik dan bobot
Batas waktu	Tanggal dan waktu pengumpulan
Integritas	Pernyataan orisinalitas

B.2 Template Laporan Proyek

Laporan proyek memuat:

1. **Pendahuluan**: tujuan, ruang lingkup, dan batasan.
2. **Spesifikasi bahasa**: token, grammar, dan aturan semantik.
3. **Arsitektur**: modul, alur data, dan keputusan desain.
4. **Implementasi**: lexer, parser, AST, type checker, validator.
5. **Pengujian**: kasus uji, hasil, dan cakupan.
6. **Keterbatasan**: fitur yang belum didukung.

7. **Refleksi**: pembelajaran dan rencana perbaikan.

8. **Lampiran**: kode, keluaran, dan tautan repositori.

B.3 Template Jurnal Refleksi

Tabel B.2. Template jurnal mingguan

Pertanyaan	Jawaban
Apa yang dipelajari?	
Apa yang berhasil?	
Apa yang sulit?	
Apa yang akan dilakukan?	
Bukti/tautan	

B.4 Rubrik Refleksi

Tabel B.3. Rubrik penilaian refleksi

Aspek	Indikator	Bobot
Kedalaman	Menjelaskan penyebab, bukan hanya kejadian	30%
Keterkaitan	Menghubungkan pengalaman dengan konsep	25%
Kejujuran	Mengakui kesulitan dan kesalahan	20%
Rencana	Tindakan konkret dan terukur	25%

B.5 Contoh Entri Portofolio

Studi Kasus

Judul: Menangani dangling else pada parser Subset-C.
Masalah: Parser awal memasangkan `else` ke `if` luar, sehingga program `if (a) if (b) x; else y;` berperilaku salah.
Tindakan: Mengubah grammar dan menambahkan uji kasus. Parser memilih shift pada token `else`, sesuai aturan else terdekat.
Bukti: Berkas uji `if_else.sc` dan keluaran AST.
Refleksi: Ambiguitas tidak cukup diselesaikan dengan intuisi; perlu aturan eksplisit dan pengujian.

LAMPIRAN C

Glosarium

Tabel C.1. Istilah penting dalam buku ini

Istilah	Definisi
Alfabet	Himpunan berhingga simbol yang membentuk string.
Ambiguitas	Keadaan ketika satu string memiliki lebih dari satu pohon urai.
Assembler	Translator dari bahasa assembly ke kode mesin.
Associativity	Aturan pengelompokan operator dengan precedence sama.
AST	*Abstract Syntax Tree*; representasi pohon yang menghilangkan detail sintaksis.
Atribut	Informasi tambahan pada simpul tata bahasa, misalnya tipe atau nilai.
Attribute grammar	Tata bahasa dengan atribut dan aturan semantik.
Back end	Bagian kompilator yang menghasilkan kode target.
BNF	*Backus-Naur Form*; notasi tata bahasa bebas konteks.
Bootstrap	Proses membangun compiler menggunakan bahasa yang dikompilasi.
CFG	*Context-Free Grammar*; tata bahasa bebas konteks.
Chomsky	Hierarki kelas tata bahasa dan bahasa.
Closure	Himpunan item yang ditambahkan secara transitif.
Coercion	Konversi tipe implisit.
Compiler	Translator dari bahasa tingkat tinggi ke bahasa target.
Cross-compiler	Compiler yang menghasilkan kode untuk platform berbeda.
Dangling else	Ambiguitas pemasangan `else` dengan `if`.
Derivasi	Urutan penerapan aturan produksi.
DFA	*Deterministic Finite Automaton*.
Diagnostik	Pesan kesalahan atau peringatan dengan lokasi.
EBNF	*Extended BNF*; BNF dengan operator pengulangan dan opsional.
Epsilon closure	Himpunan state yang dicapai melalui transisi epsilon.
Error recovery	Strategi melanjutkan analisis setelah kesalahan.
FIRST	Himpunan terminal yang dapat muncul pertama dari suatu simbol.
FOLLOW	Himpunan terminal yang dapat muncul setelah suatu nonterminal.

Istilah	**Definisi**
Front end	Bagian kompilator yang menganalisis program sumber.
Grammar	Empat-tupel nonterminal, terminal, aturan, dan simbol awal.
Handle	Substring yang dapat direduksi pada shift-reduce.
Inherited	Atribut yang dihitung dari orang tua atau saudara.
Interpreter	Program yang mengeksekusi program sumber secara langsung.
IR	*Intermediate Representation*; representasi antara.
Item	Aturan produksi dengan titik posisi.
Kata kunci	Identifier yang dicadangkan oleh bahasa.
Kleene star	Operasi pengulangan nol atau lebih.
LALR	*Look-Ahead LR*; LR(1) dengan penggabungan state.
Left factoring	Mengeluarkan prefiks umum dari alternatif aturan.
Left recursion	Aturan yang memulai ruas kanan dengan nonterminal kiri.
Lexeme	String konkret yang cocok dengan token.
Lexer	Fase yang mengubah karakter menjadi token.
LL(1)	Parser top-down dengan satu token lookahead.
LR	Parser bottom-up dengan lookahead.
L-value	Ekspresi yang dapat menjadi target assignment.
Minimalisasi	Menggabungkan state DFA yang ekuivalen.
NFA	*Nondeterministic Finite Automaton*.
Nonterminal	Simbol yang dapat diturunkan lebih lanjut.
One-pass	Kompilator yang membaca masukan satu kali.
Operator precedence	Parser bottom-up berbasis relasi precedence.
Parse tree	Pohon yang merepresentasikan derivasi lengkap.
Parser	Fase yang memeriksa struktur sintaksis.
Parser generator	Alat yang menghasilkan parser dari grammar.
Pattern	Aturan yang mendefinisikan bentuk lexeme.
PDA	*Pushdown Automaton*; otomata dengan stack.
Precedence	Tingkat pengikatan operator.
Predictive parser	Parser top-down tanpa backtracking.
Produksi	Aturan penulisan ulang pada tata bahasa.
Recursive descent	Parser top-down dengan satu fungsi per nonterminal.
Reduce	Mengganti handle dengan nonterminal.
Regular expression	Notasi untuk bahasa regular.
Scope	Wilayah keberlakuan deklarasi.
SDD	*Syntax-Directed Definition*.
Semantic action	Kode yang dijalankan saat aturan direduksi.
Sentential form	String campuran terminal dan nonterminal dalam derivasi.

Istilah	Definisi
Shadowing	Identifier dalam menutupi identifier luar.
Shift	Mendorong token ke stack.
SLR	*Simple LR*.
State	Konfigurasi mesin abstrak.
Subset construction	Konversi NFA ke DFA.
Symbol table	Struktur yang memetakan nama ke informasi simbol.
Synthesized	Atribut yang dihitung dari anak.
Tata bahasa	Aturan pembentuk bahasa.
Terminal	Simbol yang tidak dapat diturunkan lebih lanjut.
Thompson	Konstruksi NFA dari ekspresi reguler.
Token	Kategori leksikal hasil lexer.
Translation scheme	Tata bahasa dengan aksi semantik tersisip.
Type checking	Pemeriksaan kompatibilitas tipe.
Viable prefix	Prefiks stack yang masih dapat menjadi bentuk kalimat.
Visitor	Pola traversal yang memisahkan operasi dari struktur.

LAMPIRAN D

Referensi Tambahan dan Sumber Belajar

D.1 Referensi RPS

- [1] **Thain (2024)**: pengantar compiler modern dengan proyek B-Minor; sangat baik untuk scanning, parsing, dan AST.
- [2] **Mogensen (2010)**: teori ringkas dan padat; kuat untuk automata, parsing, dan type checking.
- [3] **Bergmann (2024)**: teori, alat, dan contoh; berguna untuk latihan terstruktur.
- [4] **Bornat (2008)**: panduan praktis menulis compiler; berguna untuk perspektif implementasi.
- [5] **Critchlow dan Eck (2024)**: dasar teori komputasi dan automata.
- [6] **Srikant**: catatan NPTEL; ringkas dan berorientasi ujian.
- [7] **MIT OCW 6.035**: kuliah dan tugas compiler.
- [8] **Stanford CS143**: slide dan handout parsing.
- [9] **Saarland**: struktur compiler.
- [10] **McGill CS520**: handout top-down parsing.
- [11] **Edinburgh**: compiling techniques.
- [12] **Flex Manual**: referensi lexer generator.
- [13] **GNU Bison Manual**: referensi parser generator.
- [14] **ANTLR 4**: dokumentasi parser generator.
- [15] **Wikibooks Compiler Construction**: ringkasan praktis.
- [16] **Wikibooks Formal Languages**: teori bahasa dan automata.

D.2 Karya Lokal

- Nystrom (2021), *Crafting Interpreters*: implementasi interpreter dan VM, sangat baik untuk scanning dan parsing.
- Guntoro (2024), *Teknik Kompilasi*: istilah Indonesia, penanganan kesalahan, dan tabel informasi.
- Juarna, *Catatan Kuliah Teknik Kompilasi*: ringkasan leksikal, sintaksis, semantik, dan kode antara.

D.3 Alat dan Sumber Daring

- Compiler Explorer (`godbolt.org`): melihat kode assembly dari berbagai compiler.
- regex101 (`regex101.com`): menguji ekspresi reguler.
- Flex manual (`westes.github.io/flex/manual/`).
- GNU Bison manual (`gnu.org/software/bison/manual/`).
- ANTLR Lab (`lab.antlr.org`).
- LLVM Kaleidoscope (`llvm.org/docs/tutorial/`).
- TinyCC (`bellard.org/tcc/`).

D.4 Jalur Bacaan

Tabel D.1. Jalur bacaan menurut minat

Minat	Bacaan
Dasar	Thain Bab 1–5; Mogensen Bab 1–3
Lexer	Thain Bab 3; Flex manual
Parser	Thain Bab 4–5; Bison manual; Stanford CS143
Semantik	Thain Bab 7; Mogensen Bab 6
Praktik	Nystrom; Juarna; Guntoro
Teori	Critchlow dan Eck; Wikibooks Formal Languages

D.5 Peta Sitasi

Tabel D.2. Kunci bibliografi dan nomor RPS

Kunci	Nomor RPS
`thain2024`	[1]
`mogensen2010`	[2]
`bergmann2024`	[3]
`bornat2008`	[4]
`critchlow2024`	[5]
`srikant`	[6]
`mitocw2010`	[7]
`stanfordcs143`	[8]
`saarland2013`	[9]
`mcgill2026`	[10]
`edinburgh2024`	[11]
`flexmanual`	[12]
`bisonmanual`	[13]
`antlr4`	[14]
`wikibookscc`	[15]
`wikibookstfl`	[16]

Daftar Pustaka

[1] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson, second edition.

[2] ANTLR Project (2024). *ANTLR 4 Documentation*. https://www.antlr.org/.

[3] Bergmann, S. D. (2024). *Compiler Design: Theory, Tools, and Examples*. Rowan University. https://protect.bju.edu/cps/docs/cps450/BergmannCompilerDesign.pdf.

[4] Bornat, R. (2008). *Understanding and Writing Compilers: A Do-It-Yourself Guide*. Middlesex University, internet edition. https://www.eis.mdx.ac.uk/staffpages/r_bornat/books/compiling.pdf.

[5] Critchlow, C. and Eck, D. (2024). *Foundations of Computation*. Open Textbook Library, 2.3.2 edition. https://math.hws.edu/FoundationsOfComputation/FoundationsOfComputation_2.3.2_8.5x11.pdf.

[6] Estes, W., Paxson, V., and Millaway, J. (2017). *Lexical Analysis With Flex*, flex 2.6.2 edition. https://westes.github.io/flex/manual/.

[7] GNU Project (2024). *GNU Bison Manual*. https://www.gnu.org/software/bison/manual/.

[8] Guntoro (2024). *Teknik Kompilasi*. LPPM Universitas Lancang Kuning. ISBN 978-623-8358-38-0.

[Juarna] Juarna, A. Catatan kuliah teknik kompilasi. Catatan kuliah, tidak diterbitkan. Materi leksikal, sintaksis, semantik, dan kode antara.

[10] McGill University (2026). CS 520: Compiler design – top-down parsing. https://cs.mcgill.ca/~cs520/2026/slides/parsing-handout.pdf.

[11] MIT OpenCourseWare (2010). 6.035 computer language engineering: Lecture notes. https://ocw.mit.edu/courses/6-035-computer-language-engineering-spring-2010/pages/lecture-notes/.

[12] Mogensen, T. Æ. (2010). *Basics of Compiler Design*. University of Copenhagen, anniversary edition. https://hjemmesider.diku.dk/~torbenm/Basics/basics_lulu2.pdf.

[13] Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning. https://craftinginterpreters.com/.

[Srikant] Srikant, Y. N. Principles of compiler design: NPTEL lecture notes. Indian Institute of Science Bangalore. https://nitsri.asia/Department/Computer%20Science%20%26%20Engineering/CD-LEC-NOTES.pdf.

[15] Stanford University (2024). CS143: Compilers. https://web.stanford.edu/class/cs143/.

[16] Thain, D. (2024). *Introduction to Compilers and Language Design*. University of Notre Dame, second edition. https://dthain.github.io/books/compiler/compilerbook.pdf.

[17] Universität des Saarlandes (2013). The structure of compilers. https://compilers.cs.uni-saarland.de/teaching/cc/2013/slides/structure.pdf.

[18] University of Edinburgh (2024). Compiling techniques: The structure of compilers. https://opencourse.inf.ed.ac.uk/sites/default/files/2025-01/ct-lecture2-35000feet.pdf.

[19] Wikibooks (2024a). Compiler construction. https://en.wikibooks.org/wiki/Compiler_Construction.

[20] Wikibooks (2024b). Theory of formal languages, automata, and computation. https://en.wikibooks.org/wiki/Theory_of_Formal_Languages,_Automata,_and_Computation.

Tentang Penulis

Cecep Suwanda, S.Si., M.Kom. adalah dosen pada Program Studi Teknik Informatika, Fakultas Teknologi Informasi, Universitas Bale Bandung. Ia mengampu mata kuliah Teknik Kompilasi serta mata kuliah lain di bidang rekayasa perangkat lunak dan sistem komputasi.

Penulis menyusun buku ajar ini untuk mendukung pembelajaran berbasis *Outcome-Based Education* (OBE) dan mendekatkan teori bahasa formal dengan praktik pembangunan front-end kompilator. Buku ini dikembangkan dari pengalaman mengajar, diskusi dengan mahasiswa, dan kajian sumber-sumber terbuka yang tercantum pada daftar pustaka.

Saran, koreksi, dan masukan untuk perbaikan edisi berikutnya dapat disampaikan melalui Program Studi Teknik Informatika, Fakultas Teknologi Informasi, Universitas Bale Bandung.